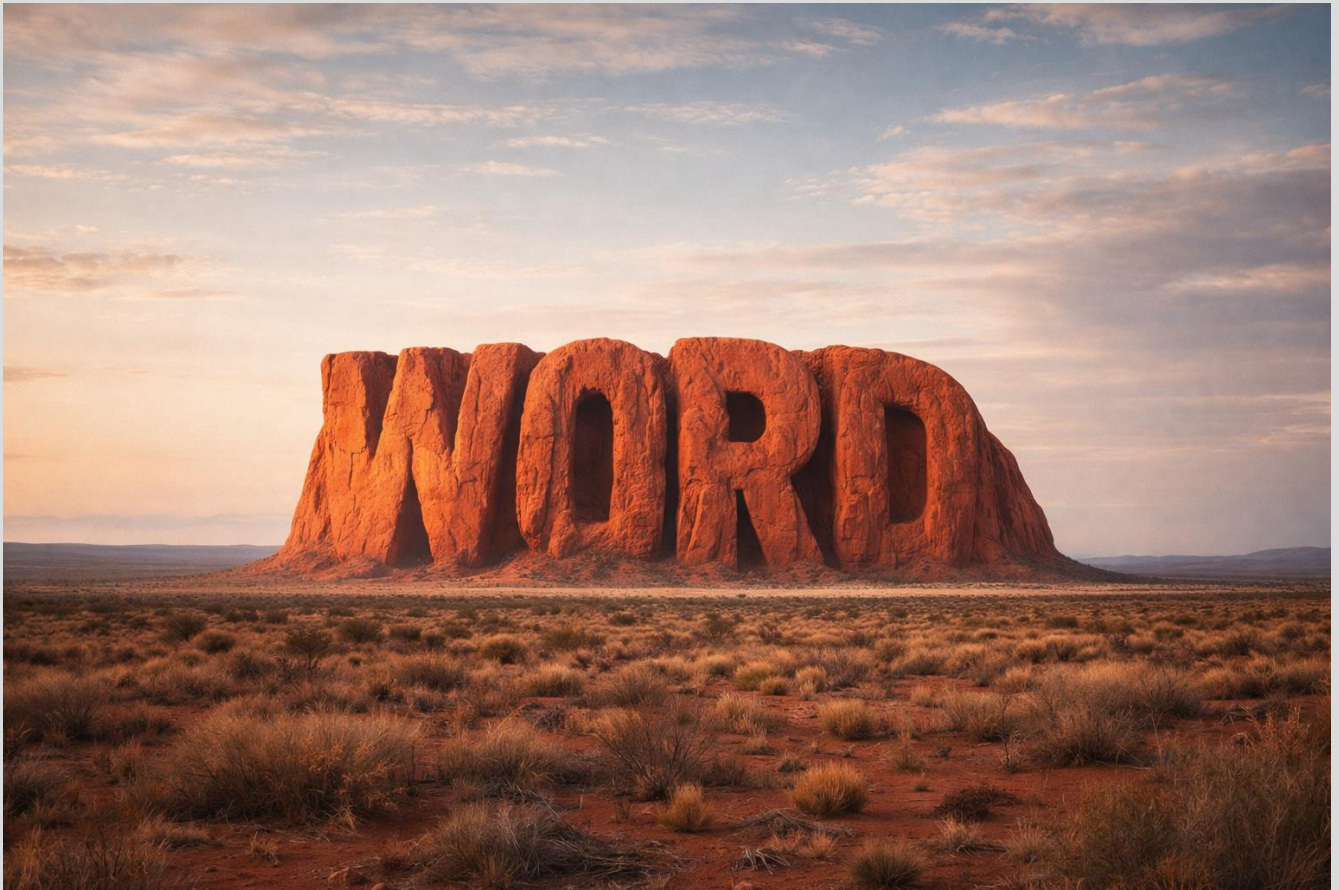




Introduction to Configuration of Microsoft Word

Maciej Słojewski

Published without authorization of Microsoft Corporation.

Published by Maciej Słojewski. For contact with author search the [LinkedIn](#) profile of the author.

Copyright © 2025 by Maciej Słojewski

Copyright: not all rights are reserved. [CC BY-NC-SA](#) = Creative Commons (CC) Attribution (BY) Non-commercial (NC) SA (Share-alike).

No patent liability is assumed with respect to the use of the information contained herein. Although every precaution has been taken in the preparation of this book, the publisher and author assume no responsibility for errors or omissions. Nor is any liability assumed resulting from the use of the information contained herein.

ISBN: 978-83-980485-0-7

Trademarks

Microsoft and the trademarks listed at <http://www.microsoft.com> on the "Trademarks" webpage are trademarks of the Microsoft group of companies. All other marks are the property of their respective owners.

Warning and Disclaimer

Every effort has been made to make this book as complete and as accurate as possible, but no warranty or fitness is implied. The information provided is on an "as is" basis. The author, the publisher shall have neither liability nor responsibility to any person or entity with respect to any loss or damage arising from the information contained in this book or from the use of the programs accompanying it.

GitHub and source code license

The source code of macros discussed in this book is available at <https://github.com/mslonik/Microsoft-Word-Configuration>

The source code license is MIT.

Document content copyright © 2026 Maciej Słojewski

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Revisions

Revision	Date	Change
1	2026-03-15	The first issue of this document.

To my friends, with gratitude for your support:

Sylvia,

Ewa,

Damian,

Maciej & Marta and
Piotr

THIS PAGE IS LEFT INTENTIONALLY BLANK

List of Content

Revisions.....	2
List of Content	5
1. Introduction, why did I write this book?	8
1.1. Less is more.....	8
1.2. How to read this book.....	9
1.3. The main drawbacks of Microsoft Word	9
1.4. You've been warned.....	10
1.5. Tested on.....	10
1.6. Disclaimer	11
2. Installation.....	12
2.1. What you get.....	12
2.2. Quick installation	12
2.3. Slow installation.....	14
2.3.1. Normal.dotm	14
2.3.2. Macros_ms.dotm	15
2.3.3. BB_ms.dotx.....	17
2.3.4. Theme_ms.thmx.....	18
2.3.5. Excalifont-Regular.ttf.....	18
3. What is Microsoft Word?	19
4. What is Microsoft Word configuration?	21
4.1. Configuration file example	21
4.2. Application configuration example.....	21
5. Introduction to configuration files.....	22
5.1. Location of configuration files in file system	23
5.2. Folder A (templates) and folder B (add-ins) in User Interface.....	24
5.1. Folder B (ribbon menu files)	25
5.2. Folder C (Theme) in User Interface	25
5.3. External configuration files and folders	26
5.4. Environment, backup operation files and folders	28
5.5. Environment, Microsoft Word and OneDrive	28
6. Configuration files DOTM or DOTX.....	32
6.1. What kind of information categories may be stored within?.....	32
6.2. DOTM or DOTX as a template: attachment relationship.....	33
6.3. Normal.dotm template.....	37
6.4. DOTM or DOTX as an add-in.....	39
6.5. Template file as a content stencil.....	40
6.6. DOTX file as container for building blocks.....	41
6.6.1. How to clean-up the default <i>Built-In Building Blocks.dotx</i> ?.....	43
7. What are the theme files?	45
8. What are the menu files?	48
9. What are the fonts files?.....	50
10. What are the content files DOCX and DOCM?	52
10.1. What kind of information categories may be stored within?.....	52
10.2. Recommended information categories	53

11. Information categories. Working with Microsoft Word.....	55
11.1. Microsoft Word workflows.....	55
11.2. Keyboard shortcuts.....	55
11.2.1. What are the keyboard shortcuts?	55
11.2.2. Why keyboard shortcuts in Microsoft Word?.....	55
11.2.3. Is there an alternative approach?.....	57
11.2.4. What are advantages and disadvantages of keyboard shortcuts?.....	58
11.2.5. How to start working with shortcuts.....	58
11.2.6. What have I done to make our life easier?	58
11.3. Working with Content.....	61
11.3.1. Automatic content creation.....	61
11.3.2. Manual content creation	65
11.3.3. Customized styles, keyboard shortcuts	66
11.3.4. Moving paragraph.....	73
11.3.5. Customized building blocks, keyboard shortcuts.....	73
11.4. Customized document properties	80
11.5. Customized Document Variables	81
11.6. Fonts.....	82
11.6.1. How to embed fonts within your document?	82
11.6.2. How to check which fonts will be embedded in my DOCX file?	83
11.6.3. Is font embedding process reversable?.....	83
11.6.4. Can I check which fonts are embedded in my DOCX file?.....	83
11.7. Customized building blocks	83
11.7.1. How to store and manage collection of building blocks without using the <i>Building Block Organizer</i>	83
11.7.1. Calling of the building blocks with <i>AutoText</i> feature	84
11.7.2. Entire collection of building blocks in one file	85
11.8. Customized captions.....	86
11.9. Cross-References	86
11.10. PDF as an output.....	87
11.11. Printing enhancements.....	88
11.12. Closing file enhancements	89
12. Configuration of Microsoft Word User Interface.....	90
12.1. Introduction, Microsoft Word and size of 24 inches display	90
12.2. Ribbon menu.....	92
12.3. Status bar.....	92
12.4. Panes.....	94
12.5. CustomUI14.xml	95
13. Design	97
13.1. Macros (Macros_ms.dotm).....	97
13.1.1. My VBA practices.....	98
13.1.2. Structure.....	100
13.1.3. How to call macros?	103
13.1.4. How can I get a list of all customized macros in this project?	108
13.1.5. How macros are licensed?.....	108
13.1.6. How to run certain macro each time document is opened?	108
13.2. Page Layout.....	108
13.2.1. Document page areas	108
13.2.2. Make colored frame around document content area.....	109
13.2.3. Make colored background of header and footer area.....	111
13.2.4. Design of customized page layout.....	112

13.2.5. Setting the customized margins	112
13.2.6. Setting the header and footer height	114
13.2.7. The first conclusions.....	116
13.2.8. Tables in header and footer	116
13.2.9. Setup of header extra width	116
13.2.10. Next conclusions	121
13.2.11. Header and footer follow-up.....	122
13.2.12. Additional section of document, linked headers and footers.....	122
13.2.13. Blank pages.....	123
13.3. Design of content, requirements.....	123
13.4. Fonts	124
13.5. Dense, compacted content.....	125
13.5.1. Minimum space between lines of the same paragraphs	125
13.5.2. Hyphenation.....	126
13.6. Styles.....	127
13.7. Lists.....	128
13.8. Tables.....	129
13.9. Security issues	129
13.9.1. How do you inform user that content of this document is protected?.....	129
13.9.2. Which confidentiality level should be assigned to particular document?	130
13.9.3. In which form document should be sent to external party?	130
14. This book GitHub repository.....	131
15. How to search for official support?	132
15.1. Official Microsoft online help	132
15.2. Official Microsoft forum	132
15.3. How to issue a problem?	132
15.4. Official releases of Microsoft Word and known bugs	132
16. Web pages dedicated to Microsoft Word.....	133
16.1. Short URL list.....	133
16.2. List with illustrations	133
17. References	135
18. Epilog	137
List of Pictures	138
List of Tables.....	141

1. Introduction, why did I write this book?

Why did I write another book about Microsoft Word? What is so special about it to create next book?

I have years of practice in a corporate environment, where Microsoft Word was the only one tool available for authoring all kinds of documentation: technical documentation, formal documentation, user documentation. No matter how you would put division line between different kinds of documentation, only Microsoft Word was allowed to create files.

Microsoft Word is a so called "word processor", to distinguish it from ordinary text editors and underline it is a feature reach, more like desktop publishing program. Additionally, it is WYSIWYG tool. This long acronym stands for What You See Is What You Get. At the time unique feature which enabled printed out content look the same as on the screen on time of authoring.

Be warned: there are alternatives for authoring documentation. Perhaps more straightforward, with a smaller number of exceptions, more intuitive, more convenient to use. I did not have a choice. Over years I paid hundreds of hours to customize Microsoft Word to follow my requirements. And this book is about this customization.

To be fair, every tool you would like to use professionally for content authoring would require plenty of additional work dedicated to customization. I still wonder, if hundreds, or likely thousands of hours. During my professional work I couldn't find one complete introduction or manual dealing with at least main intricacies of Microsoft Word configuration. So, I decided to write one on my own. Of course, it is true that I did it for me. But why not share it, if it is ready?

There is more general question, why write a book about configuration of a specific application? Firstly, the content of a book would depend on changes to this application. In other words, it can rapidly become out of date after the next application update. Secondly, it is quite subjective, as configuration would be dedicated to specific needs or applications. From one perspective all software is just a configuration to hardware. Let's keep this idea in mind.

Why a configuration matters? Because at least in case of Microsoft Word it can significantly reduce time necessary in any way to make content look right. The previous sentence doesn't sound too substantial. For example, what does "significantly" mean? I mean multiple times faster: two times at least. Thanks to limited set of styles, and keyboard shortcuts assigned to styles, Microsoft Word can really shine. Of course, it requires some upfront work (investment) related to configuring a set of styles and keyboard shortcuts. It pays off, and you can even try to estimate it quantitatively, if you wish. And that example is just one of many you can find in this book.

1.1. Less is more

Microsoft Word offers a tremendous number of functions, options, choices. It is like Swiss pocketknife equipped in more than hundreds of simple blades or tools. And when you finally have found one necessary for the accomplishment of your job, you realize that to make that job convenient, you would like to separate it from the rest of your knife-set, but it is not possible. As a result, you have found the right tool, but there is no easy way to make use of it.

I propose in this book another approach: less is more. I try to cut off as much as possible all not relevant functions, elements of interfaces, styles or whatever, to leave you with something simple to do your job.

1.2. How to read this book

As you wish or as you find it comfortable. My idea was to show some structure of data stored within Microsoft Word files.

Content of this book is provided on purpose also in format of DOCX file. Thanks to that you can experiment with formatting, knowing that this file, and this book, are interlinked. Building block, macros, which are described in this book, were also used to format it.

But maybe it would be easier to you just familiarize with results of my work by experimenting? If this is your case, then jump directly to chapter 2. After that I strongly recommend you to look in chapter 11, where you can find some details related to usage of customized macros.

In chapters 3, 4 and 5 I describe my point of view to Microsoft Word configuration.

Chapter 6 is dedicated to the most crucial from my perspective, designed difference between templates and add-ins. The same DOTM or DOTX files, depending on context, may be used in both roles.

In chapters 7, 8, 9 and 10 and discuss different types of files which can be used with Microsoft Word.

Perhaps all that is interesting to you is some insight to basics of User Interface configuration? Then look in the chapter 12.

In chapter 13 you can find some of my subjective thought related to formatting design.

In chapter 14 you can find basic introduction to this book interconnected GitHub repository.

In chapter 15 I list places where officially you can find help if you struggle with this tool. And in chapter 16 I answer to the same challenge, but this time showing unofficial web pages dedicated to this subject.

Please note, that behind references, listed in chapter 17, there is also chapter 18, with epilog, my final thoughts.

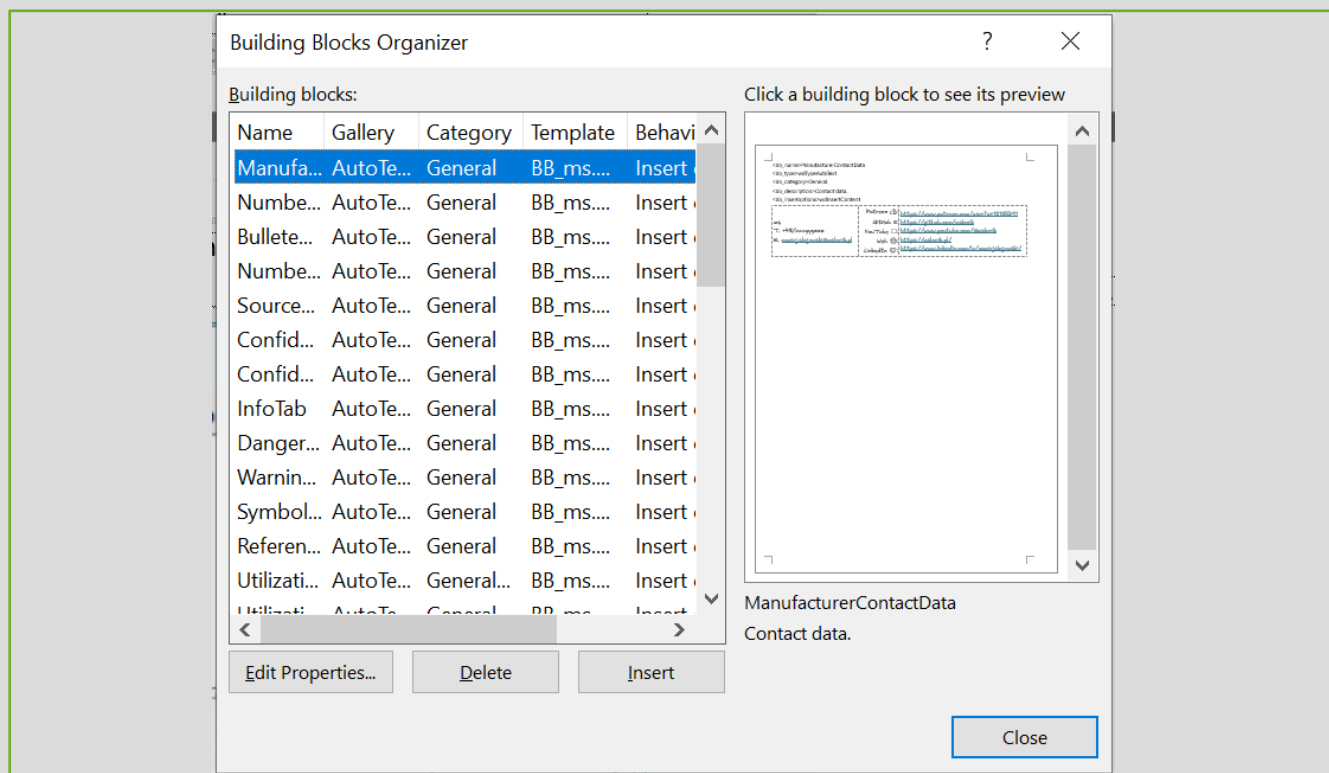
1.3. The main drawbacks of Microsoft Word

My main point: I was promised a possibility to manage this application from Visual Basic for Applications (VBA). But this promise is covered only partially. On the one hand it is a wonderful tool, on the other hand it is a curse, as code cannot be easily managed.

Below you can find my subjective list of main drawbacks of Microsoft Word:

- » By far the biggest is that tool is no longer maintained. Of course there are issued next revisions, but they fix completely unimportant issues, like shape of icons. Meanwhile many bugs and not finished subjects are left over. I'd like to say "Make Microsoft Word great again".
- » Many frequently used dialog windows are small, and you can't change its size. There is no window scaling (see [Pic. 1](#)).
- » There is no easy way to move in the content of long documents. Previous position of cursor is not remembered.
- » Many areas of Microsoft Word are not accessible from VBA code. Examples:
 - » status bar,
 - » some Microsoft Word configuration options
 - » Set Defaults tab of Manage Styles window
- » For many options there is no "default" setting which should enable us to restore initial configuration.

- » Shortcut management is cumbersome, to say at least.
- » AutoText definitions (text strings) cannot be applied to call a macro.
- » It is impossible to move some settings of the content, e.g. sizes of margins, distances to footer and header between different documents, without applying the VBA macros.
- » Be careful with field formulas. They are country specific. For example, some function names are localized (translated to specific language) and some not. Be especially careful for decimal point.
- » ...



Pic. 1. Example of small dialog box.

1.4. You've been warned...

If you don't need to use Microsoft Word, don't use it. There are alternatives, but... other applications would also require from you a lot of effort to prepare valid and useful configuration. Hopefully with less frustration.

And now, if you aren't yet discouraged or offended, or basically just curious what is in the content of this book, I sincerely invite you to read on.

1.5. Tested on

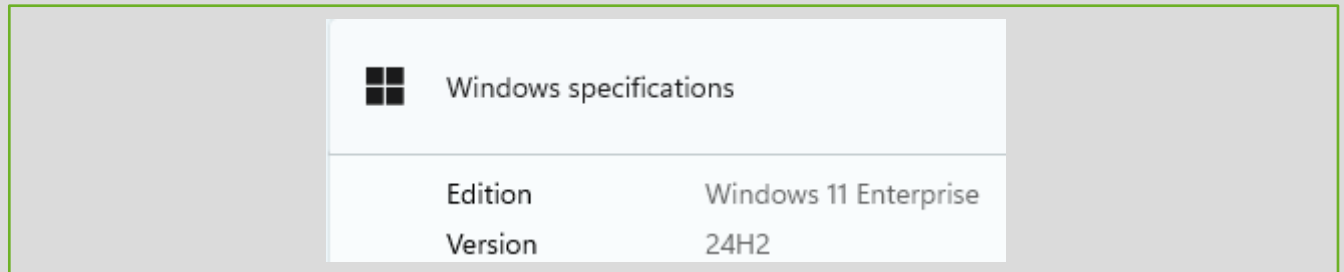
All the macros, as well as template files, have been tested on two PCs with slightly different configuration, as specified in [Tab. 1](#) and [Tab. 2](#).

Tab. 1. First tested setup.

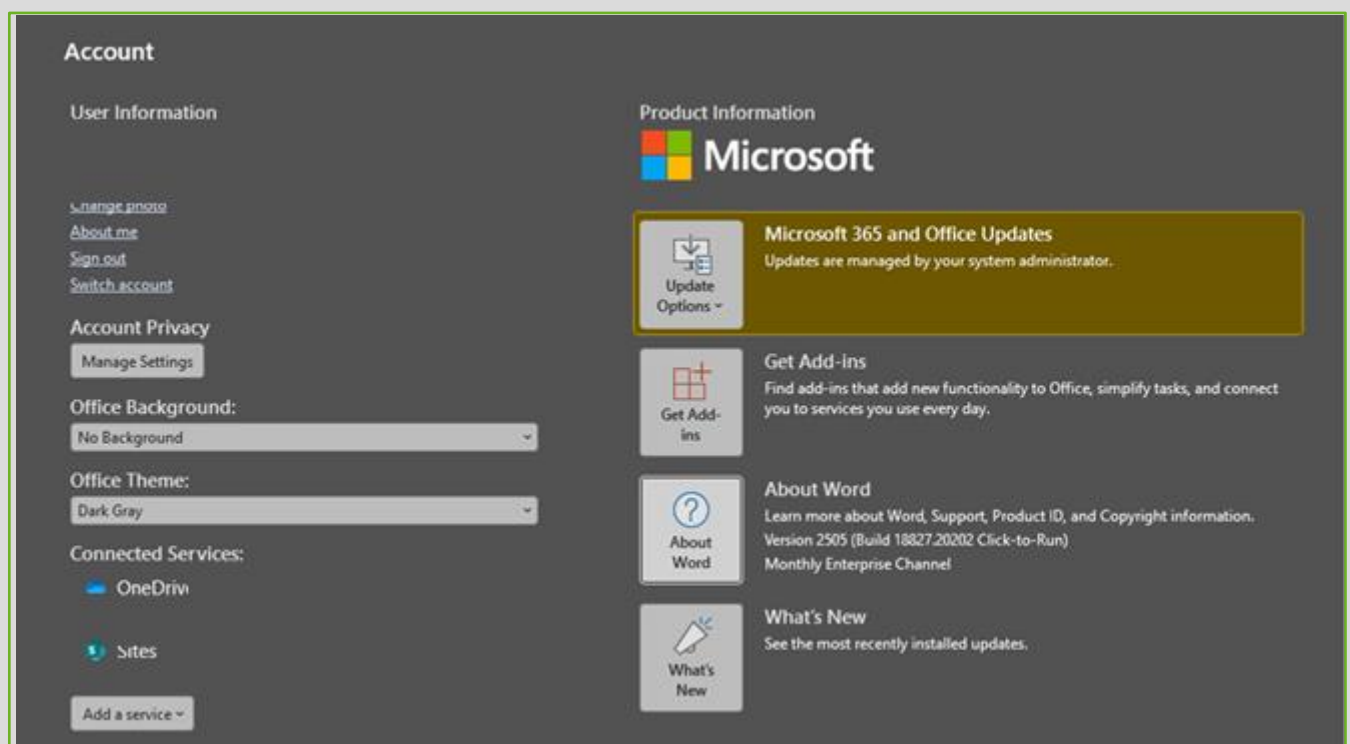
Feature	Microsoft Windows	Microsoft Word
Edition	Windows 10 Home	Microsoft® Word 2016 MSO 64-bit
Version	22H2	Version 2507
OS Build	19045.6216	Build 16.0.19029.20136

Tab. 2. Second tested setup.

Feature	Microsoft Windows	Microsoft Word
Edition	Windows 11 Enterprise	Microsoft® Word for Microsoft 365 MSO 64-bit
Version	24H2	2505
OS Build	26100.4652	16.0.18827.20102



Pic. 2. Microsoft Windows: Control Panel > System > About > Windows specifications.



Pic. 3. Microsoft Word: File > Account > About Word.

1.6. Disclaimer

This book is only an introduction to Microsoft Word configuration. It is also work in progress. I'm pretty sure you'll be able to improve it and adjust to your own needs.

No liabilities. Use "as is", including VBA source code.

2. Installation

All the files from [GitHub page](#) of this project are licensed according to MIT license [\[1\]](#). The files were used to prepare this book, and other way around, this book is about those files.

2.1. What you get

Clear separation between different information categories:

- » `BB_ms.dotx`
 - » Set of building block, including those applied in content of this book.
- » `Macros_ms.dotm`, among others:
 - » styles definitions in form of VBA code
 - » procedure to easily manage building block
 - » macros to manage other macros
 - » set of functions to easily manage references
 - » shortcuts and functions to manage shortcuts
 - » and many more
- » `Normal.dotm`
 - » almost clean, with no additional macros
 - » the only required setting is *Set Defaults*, compare chapter [13.2.1](#).
- » `Theme_ms.thmx`
 - » example theme definition, based on the `Excalifont-Regular.ttf`
- » `Excalifont-Regular.ttf`
 - » beautiful, hand-drawn font

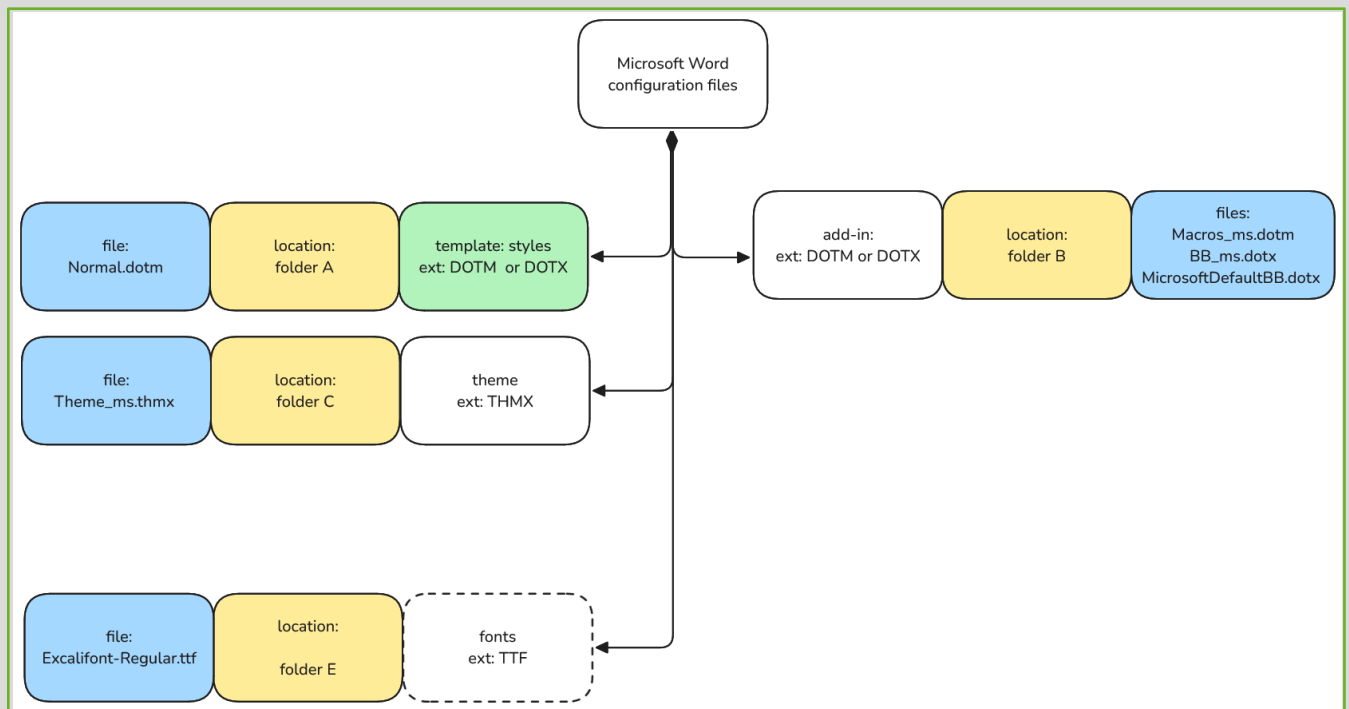
2.2. Quick installation

Assumptions:

- » full files are downloaded from GitHub repository
If you don't trust binary files such as DOTM or DOTX, refer to chapter [2.3](#).
- » you have access to your user profile in Microsoft Windows / Microsoft Word
- » you have default setup of configuration files in Microsoft Word.
If not, refer to chapter [5](#).

Let's start.

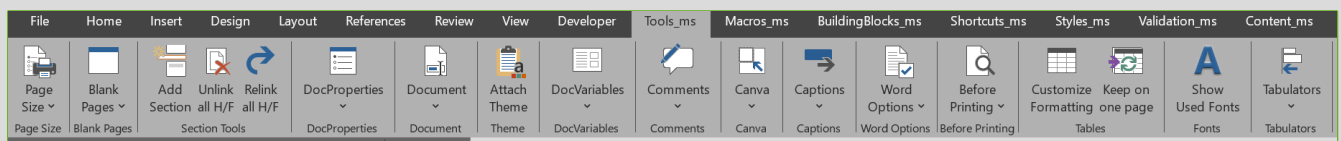
1. Download the files specified in [Pic. 4](#) from [GitHub page](#) of this project.
2. Make a backup to your current `Normal.dotm` file. For example, rename it to `Normal.back`.
3. Put the files in folders according to [Tab. 3](#).
4. Restart Microsoft Word.



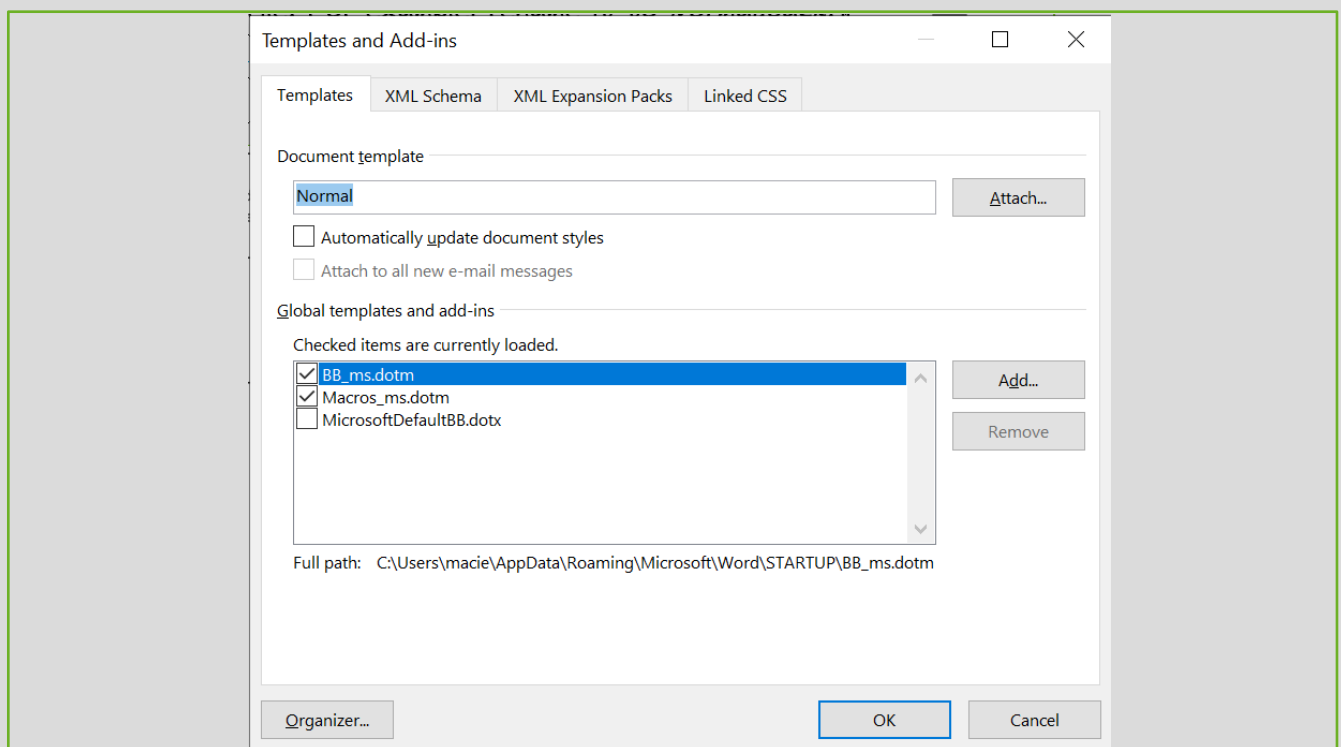
Pic. 4. Files necessary to be downloaded.

Make sure newly added add-ins are set up as in [Pic. 6](#).

If everything went well you should see now in menu new positions, like *Tools_ms*, *Macros_ms*, *BuildingBlocks_ms*, *Shortcuts_ms*, *Styles_ms*, *Validation_ms* and *Content_ms*, compare [Pic. 5](#).



Pic. 5. New menus.



Pic. 6. Set up of add-ins.

Tab. 3. The location of configuration files.

folder name	folder default location	comment
folder A	c:\Users\<UserName>\AppData\Roaming\Microsoft\Templates\	template files (DOTM or DOTX)
folder B	c:\Users\<UserName>\AppData\Roaming\Microsoft\Word\STARTUP\	add-in files (DOTM or DOTX) Microsoft Word automatically loads on startup any file from this folder in read-only mode.
folder C	c:\Users\<UserName>\AppData\Roaming\Microsoft\Templates\Document Themes\	theme files (THMX)
folder E	C:\Users\<UserName>\AppData\Local\Microsoft\Windows\Fonts	font files (TTF) C:\Windows\Fonts directory is the primary location for fonts that are available to all users on the computer. However, in more recent versions of Windows (Windows 10 and 11), a second location has become common for user-specific font installations. While C:\Windows\Fonts remains the classic and most important location for system fonts, it's not the only place where font files may be stored.

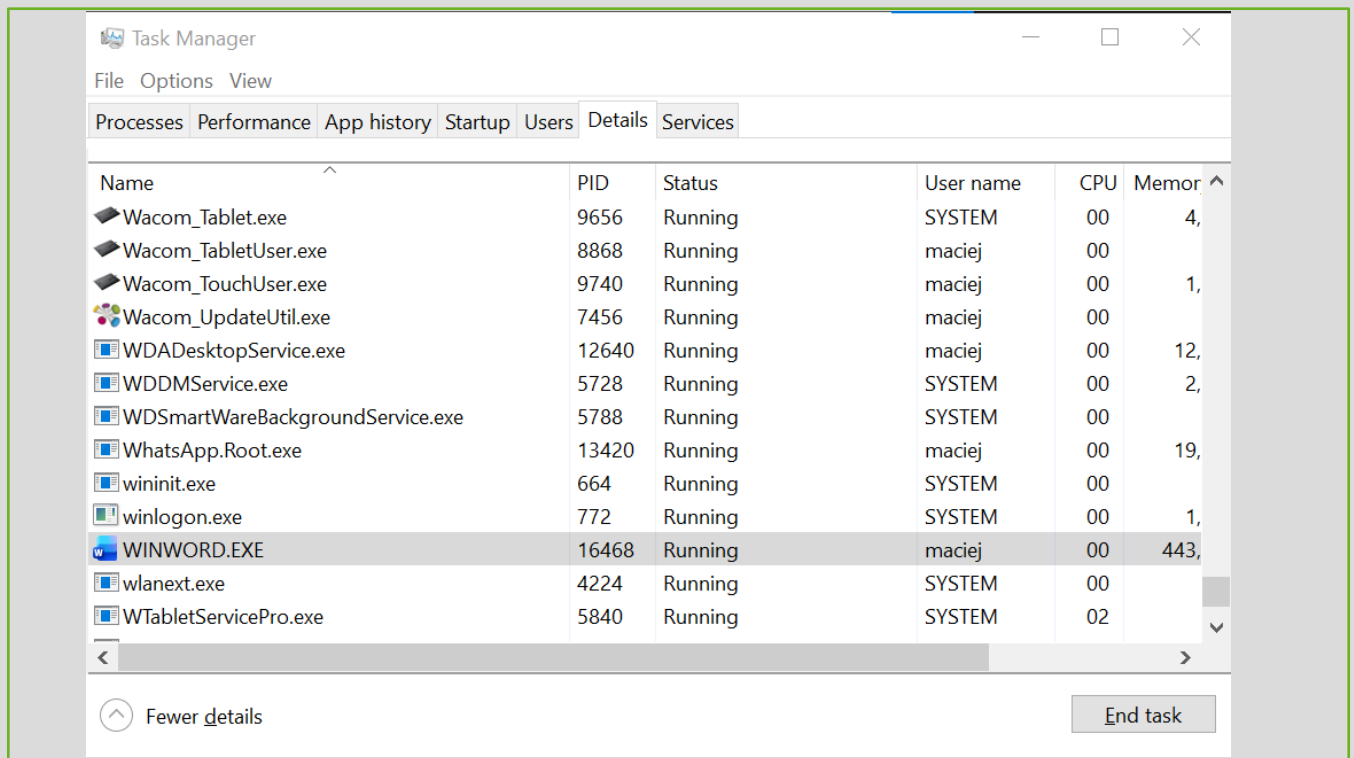
2.3. Slow installation

Nowadays downloading binary files directly from Internet, even from GitHub, could be harmful to your computer. In this chapter I will explain, how can you build all the required files by yourself.

2.3.1. Normal.dotm

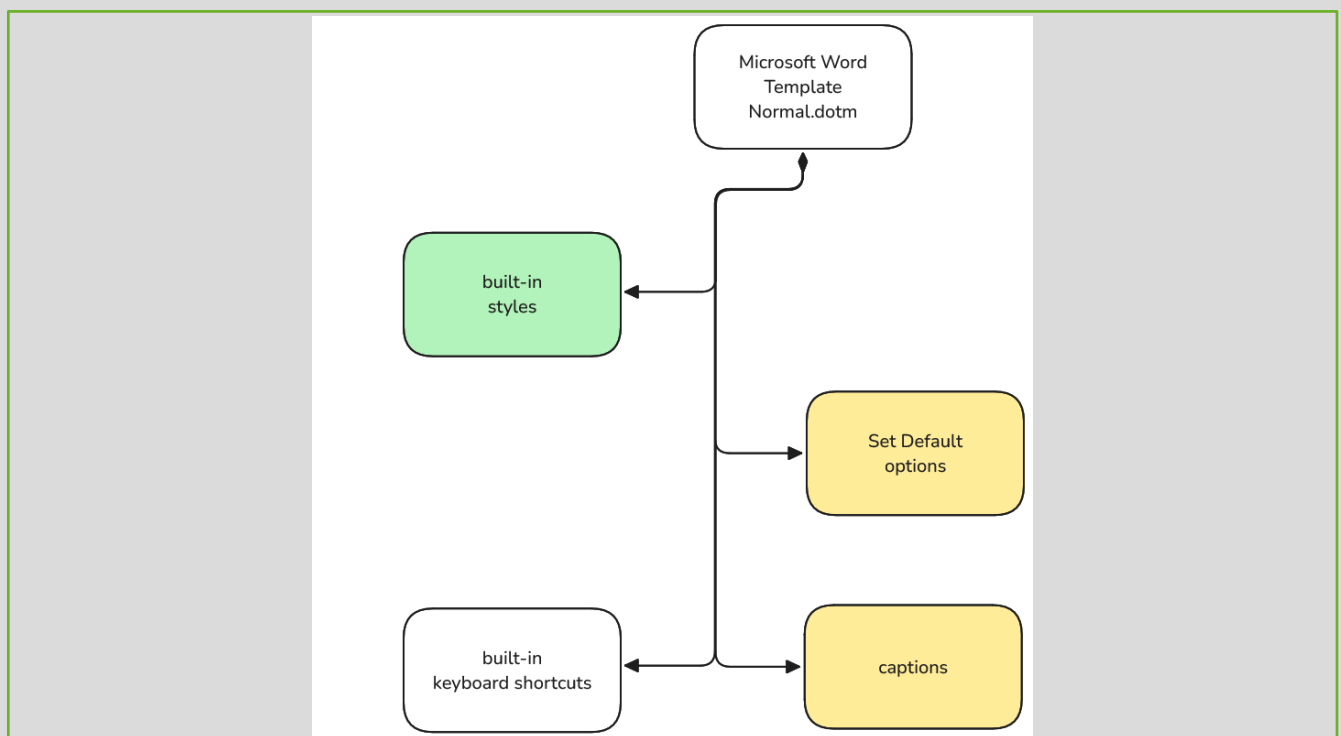
All we need from *Normal.dotm* is to be simple, clear and have specific *Set Default* settings.

1. Inform yourself, where according to your current configuration is stored *Normal.dotm*. By default, it is folder
c:\Users\<UserName>\AppData\Roaming\Microsoft\Templates\
If it is not the case, compare [Pic. 16](#).
2. Open the folder.
3. Close Microsoft Word. All instances. Make sure it is closed. Consult Task Manager if in doubts, compare [Pic. 7](#).
4. Make a backup of your current *Normal.dotm* by renaming it e.g. to *Normal.old*. Thanks to that, if something is wrong, you will have option to reverse all changes.
5. Run Microsoft Word.
6. Set up *Set Default*. Proceed according to chapter [6.3](#), [Pic. 35](#).
7. Close Microsoft Word. You should see now newly created *Normal.dotm*.



Pic. 7. Task Manager and Microsoft Word.

Regarding information categories (compare chapter 6.1) stored within *Normal.dotm*, compare Pic. 8. Yellow color shows unique information categories of *Normal.dotm*.



Pic. 8. *Normal.dotm*, information categories.

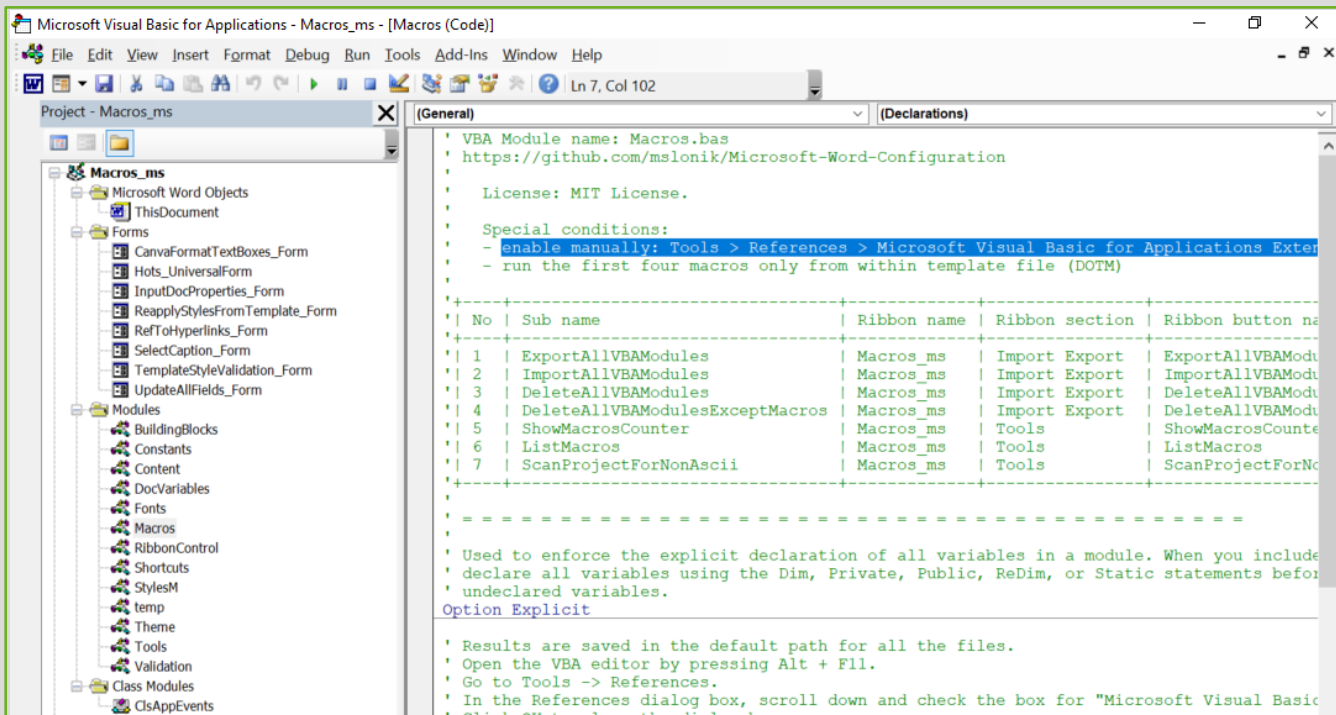
2.3.2. *Macros.ms.dotm*

We will import VBA source code manually to DOTM file.

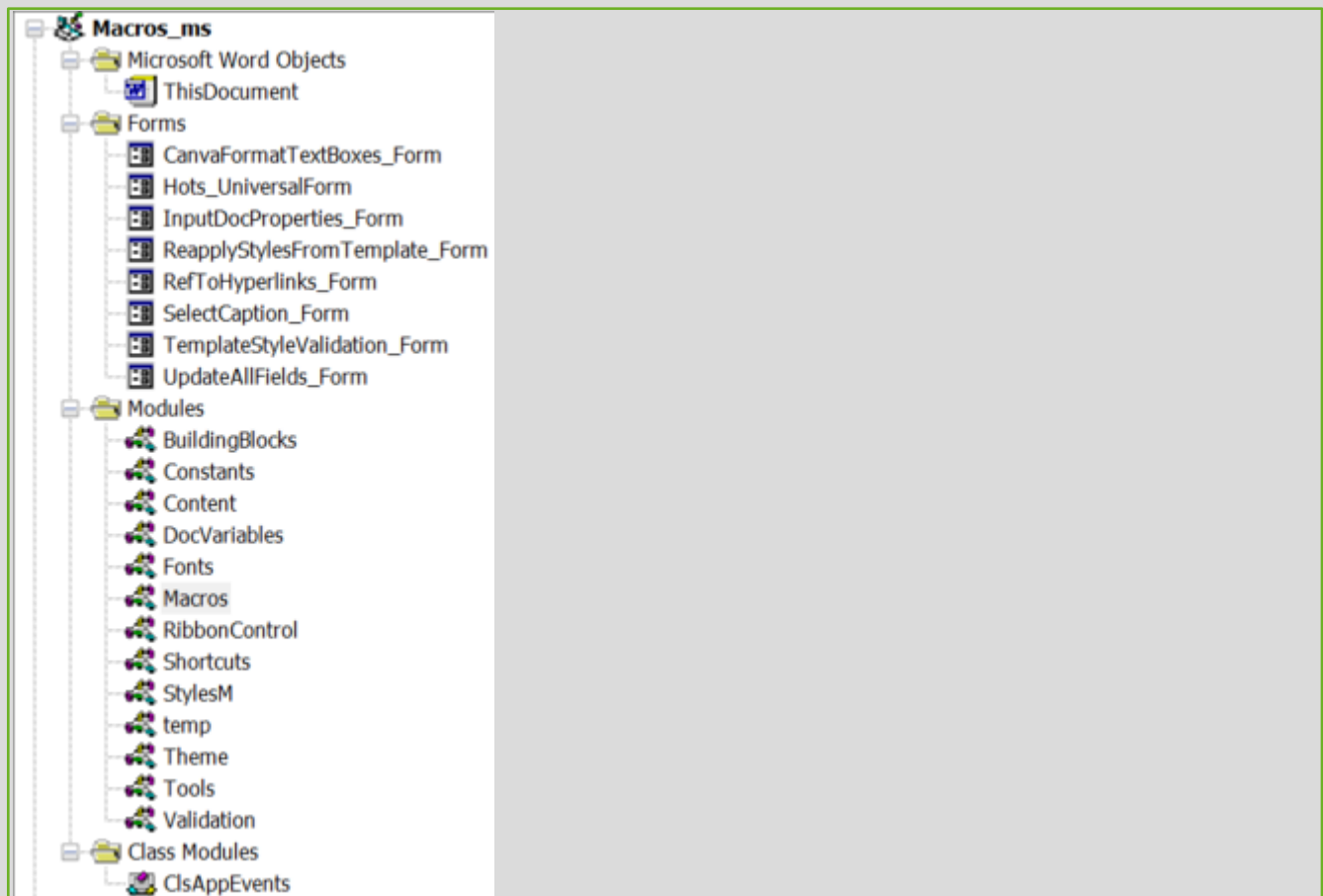
1. Copy the *Normal.dotm* created in previous step to the default *Startup* folder:
`c:\Users\<UserName>\AppData\Roaming\Microsoft\Word\STARTUP\`.
2. Rename *Normal.dotm* → *Macros.ms.dotm*.

3. Download all VBA modules from [GitHub repository \[15\]](#) in the same temporary folder. Consult structure of repository, see [Pic. 170](#), right part, macro modules. The macro files are text files with .bas, or .cls, or .frm extension. These are human readable files containing VBA source code. There is one exception: .frx files which are binary files and are also necessary to be downloaded.
4. Open *Macros.ms.dotm* for editing.
Do it on purpose: open Microsoft Word, select *File > Open* and navigate to *Macros.ms.dotm*.
5. Open VBE (Visual Basic Editor). Press *Alt + F11* or menu: *Developer > Visual Basic*, compare [Pic. 9](#).
6. Change project name, compare [Pic. 124](#).
7. Import your first module: *Macros.bas*.
File > Import File... (*Ctrl + M*), navigate to *Macros.bas*.
8. Enable manually in VBE: *Tools > References > Microsoft Visual Basic for Applications Extensibility 5.3*.
Now you can take a shortcut and try to import all remaining source code files by running the macro *ImportAllVBAModules*. Or you can import remaining files manually, one by one.

The expected result should be like shown in the following pictures:

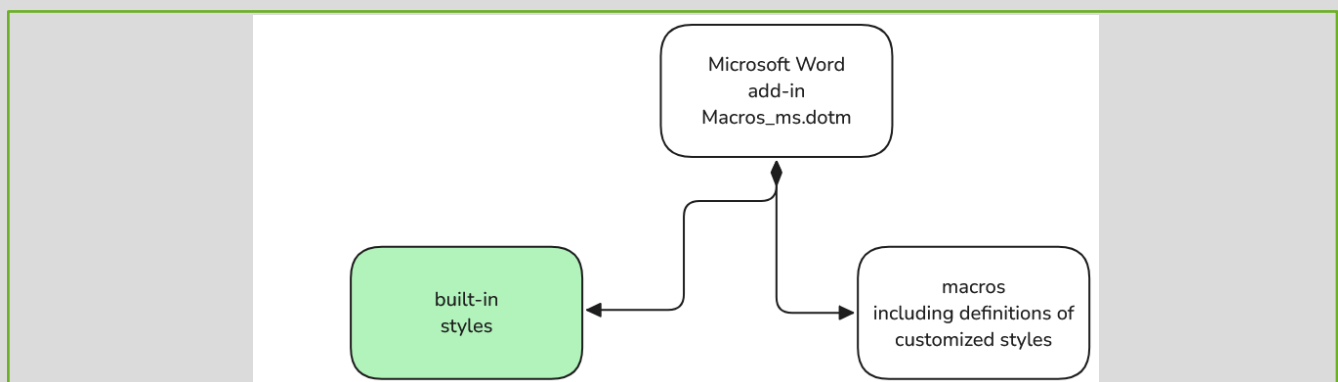


Pic. 9. VBE with imported all source files.



Pic. 10. VBE, enlarged structure of Macros.ms.

Regarding information categories (compare chapter 6.1) stored within *Macros.ms.dotm*, see Pic. 11.



Pic. 11. Macros.ms.dotm, information categories.

2.3.3. BB.ms.dotx

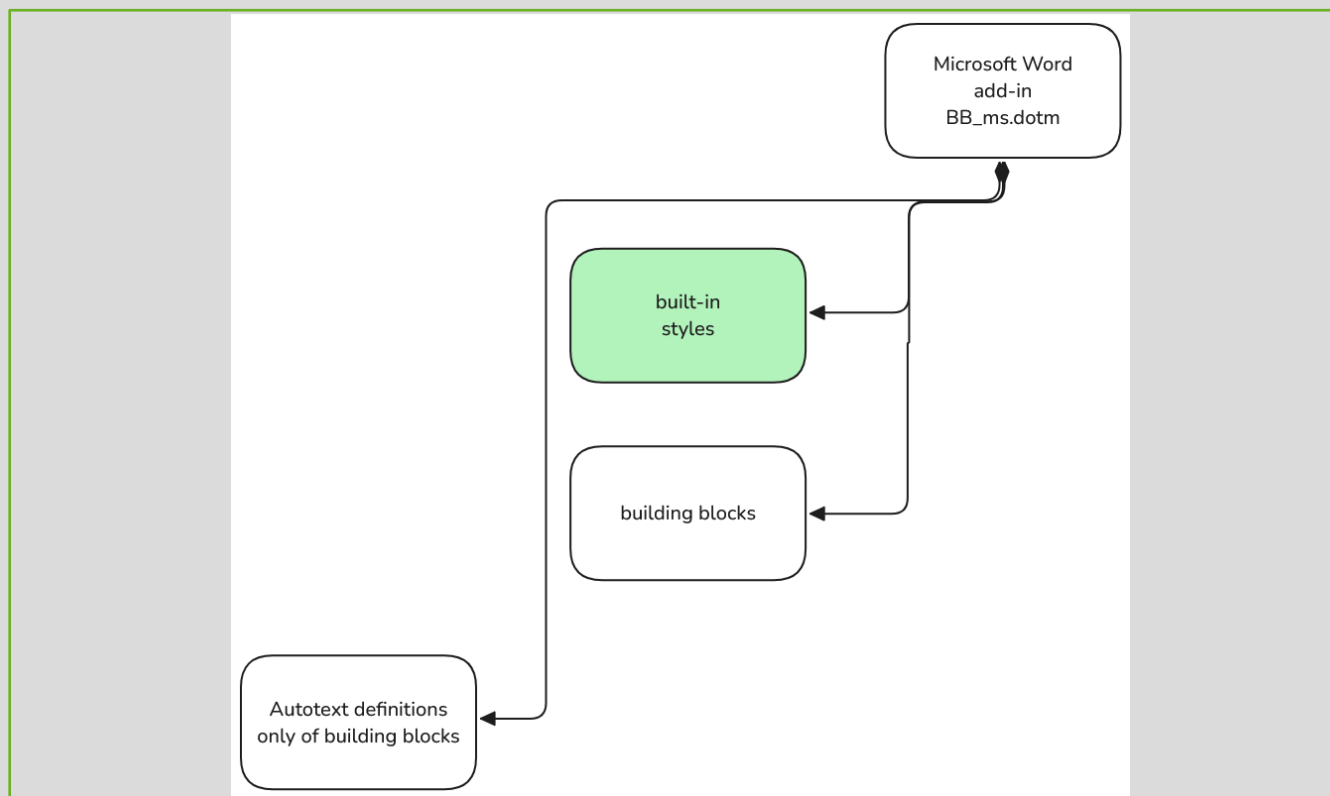
This file contains building block, including those used to compose this book. If you wish to follow examples from this book, it is important to keep naming convention and structure of building block. The file *BB.ms.dotx* do not contain any macros, so copying it into your environment has moderate or even low security risk.

You can manually build content of this file from the file *BB.ms_BB_Content.docx*, which you can find in the book [repository \[15\]](#), following method described in chapter 11.7.

The *BB.ms.dotx* can be even empty, but I strongly recommend to keep such file as one of the configuration files within your Microsoft Word environment. In other words, at least

create file of the same name and save it in the startup folder:
c:\Users\<UserName>\AppData\Roaming\Microsoft\Word\STARTUP\.

Regarding information categories (compare chapter 6.1) stored within *Normal.dotm*, see [Pic. 12](#).



Pic. 12. *BB_ms.dotx*, information categories.

2.3.4. Theme_ms.thmx

This is binary file. If you wish you can create it by yourself following steps described in chapter [7](#).

2.3.5. Excalifont-Regular.ttf

You can download the Excalifont-Regular.ttf from the book [repository \[15\]](#) and copy it to C:\Users\<UserName>\AppData\Local\Microsoft\Windows\Fonts

The Excalifont follows MIT license.

Or you can

1. enter the [Excalifont subpage of Excalidraw project](#)
2. download the Excalifont in [woff2](#) font format.
3. use for example [online file format converter](#) to change format from woff2 to TTF.

Side note: the Excalidraw editor [\[10\]](#) was used to prepare block diagrams in this book.

3. What is Microsoft Word?

Microsoft Word is so called *word processor* class application. The functions of the Microsoft Word program are between those of a simple text editor and a desktop publishing program. It is probably even simpler than that: editors are programs, where input and output is just text file. The word processor is program where output is sheet of paper, or a file which you can print out.

The WYSIWYG (What You See Is What You Get) idea was to represent content on the screen exactly in the same manner as it will look like after printing. The first word processors used to be a piece of hardware and software, where three devices were combined:

1. screen,
to display document layout
2. keyboard,
to input or edit content
3. printer
to output content

Additionally, there used to be also a processing unit to run *word processor* software and store input and output.

It could be hard to believe nowadays, that the most expensive device in the whole set used to be a printer. Please note vertical orientation of the screen to better mimic sheet of paper.

Later all three devices were changed into single computer program, which optionally can use external printer.



Pic. 13. Hardware oriented word processor devices [2].

The Microsoft Word to store content, including text, static pictures, equations and other static objects, uses dedicated format, named OOXML ([Office Open XML](#)) [16]. This format was standardized according to ISO/IEC 29500.

For me is somehow sad, that to represent text on sheet of paper, humans developed quite complex and unique set of text language, XML, which at the end is not readable by humans and is stored in binary format.

4. What is Microsoft Word configuration?

Configuration is a set of parameters to set up state of Microsoft Word, such as:

1. Configuration files,
files which store customized configuration. These files contain information important to both: formatting content and operation of Microsoft Word.
2. Microsoft Word application configuration,
stored within Microsoft Windows registry, can be saved in the form of files, but moving it to another computer requires additional precaution.

Unfortunately, Microsoft Word configuration is not ready to be easily moved from one PC to another. To make it possible I apply VBA macros stored in configuration files. Nevertheless, there are still strange quirks which in a specific situation require manual settings, as not all User Interface options are available from VBA code.

4.1. Configuration file example

The *Normal.dotm* file, named a template file, stores the default set of styles. The styles influence setup of look and feel of the content. No matter if you know about it, or consciously make a choice, any content present on a Microsoft Word screen, some styles are applied. By default, styles come from *Normal.dotm* file. It means, that by default, if no other template file is linked to a content file, that one is applied. The *Normal.dotm* file by default is stored within a folder:

C:\Users\<YourUserName>\AppData\Roaming\Microsoft\Templates\

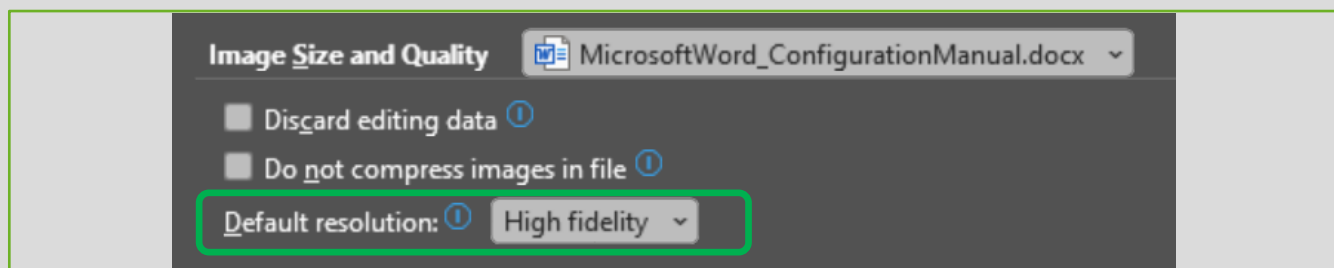
It can be copied to another PC, to another user profile.

4.2. Application configuration example

How is a graphical content displayed on a screen, you can see in *File > Options > Advanced > Image Size and Quality*. The *Default resolution* parameter cannot be set from VBA. Probably it is stored within Microsoft Windows registry:

HKEY_CURRENT_USER\Software\Microsoft\Office\16.0\Word

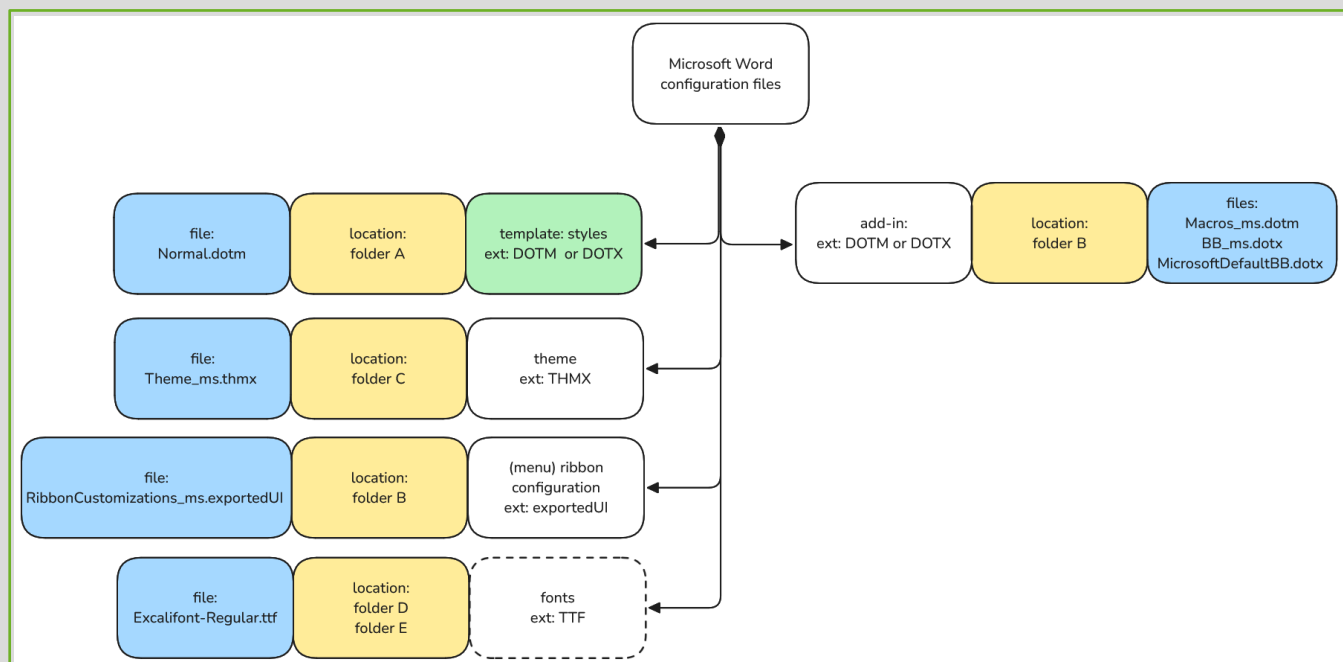
But where exactly, I cannot say as it is not documented.



Pic. 14. Microsoft Word application setting example, default resolution of images.

5. Introduction to configuration files

In this chapter we will meet configuration files, see in [Pic. 15](#). Whether we like it or not, the conclusion is there are several types and format of the file which can be involved in the process of configuring the Microsoft Word. These configuration files create an environment for Microsoft Word operation and your workbench for content authoring.



Pic. 15. Microsoft Word configuration files.

Legend:

- » The green color represents the only one mandatory category, the templates.
- » The dashed line represents files which are technically not configuration files, but may be helpful to get expected look and feel. The only practical case will be to integrate font files within DOCX file if it is edited on PC where specific fonts are not available. Then Microsoft Word enables integration of font files with DOCX file.
- » The *ext* stands for *file extension*.
- » The yellow color represents file location.
- » The blue color represents file name used in this book.

The file with extension DOTM or DOTX can be named as template or add-in. The distinguishing between these two is essential and "designed" by creators of Microsoft Word, but not obvious and somehow hidden. To some degree you can recognize "which is which" by its location in file system. More on that later in this book.

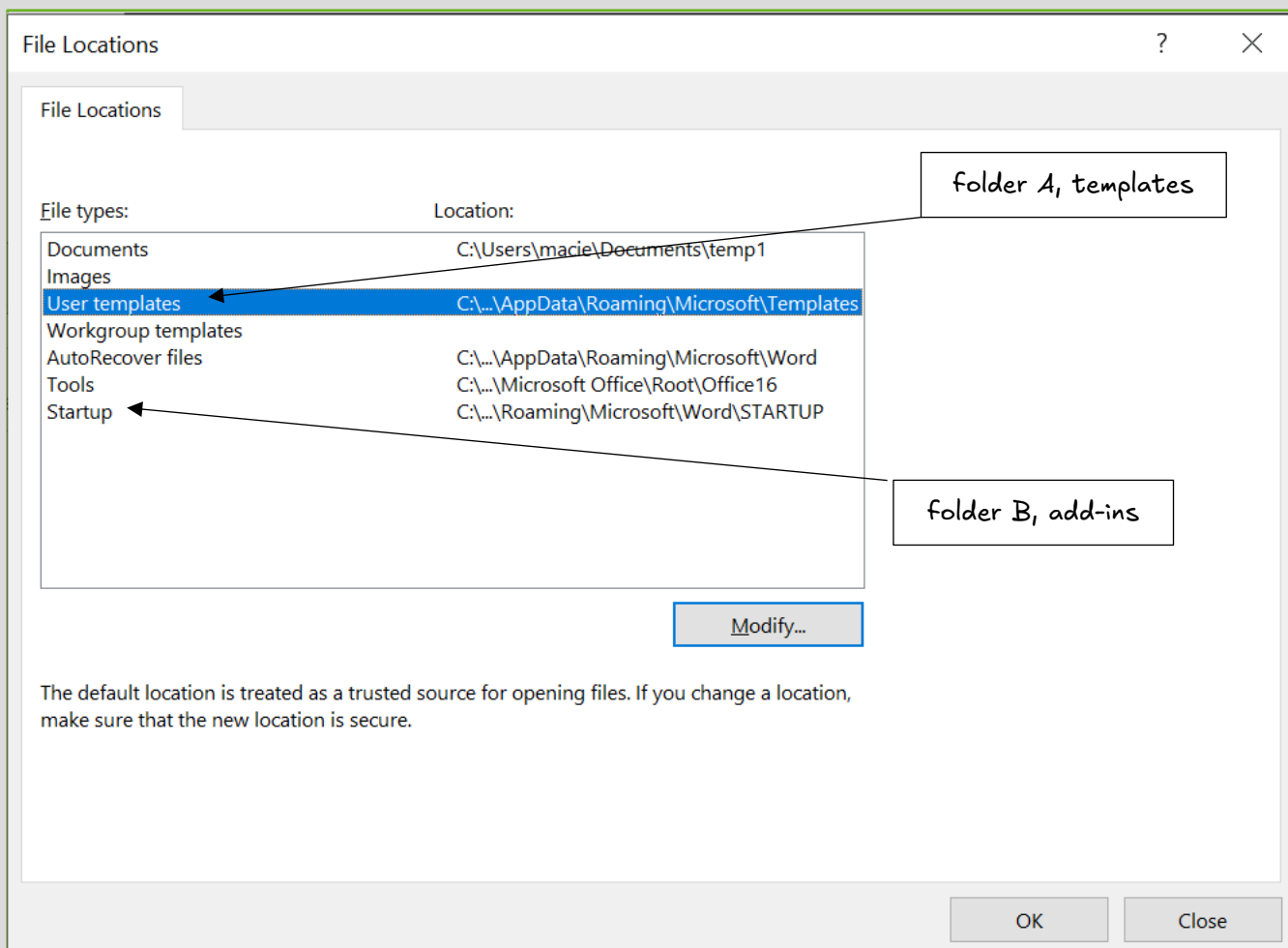
5.1. Location of configuration files in file system

Tab. 4. The default location of configuration files.

folder name	folder default location	comment
folder A	c:\Users\<UserName>\AppData\Roaming\Microsoft\Templates\	template files (DOTM or DOTX)
folder B	c:\Users\<UserName>\AppData\Roaming\Microsoft\Word\STARTUP\	add-in files (DOTM or DOTX) Microsoft Word automatically loads on startup any file from this folder in read-only mode.
		(menu) ribbon configuration files (exportedUI)
folder C	c:\Users\<UserName>\AppData\Roaming\Microsoft\Templates\Document Themes\	theme files (THMX)
folder D	C:\Windows\Fonts	font files (TTF) C:\Windows\Fonts directory is the primary location for fonts that are available to all users on the computer. However, in more recent versions of Windows (Windows 10 and 11), a second location has become common for user-specific font installations.
folder E	C:\Users\<UserName>\AppData\Local\Microsoft\Windows\Fonts	So, while C:\Windows\Fonts remains the classic and most important location for system fonts, it's not the only place where font files may be stored.

Location of folder A and folder B can be changed in Microsoft Word application. *Word > File > Options > Advanced > General > File locations...* (see [Fig. 16](#)).

I recommend keeping the default locations.

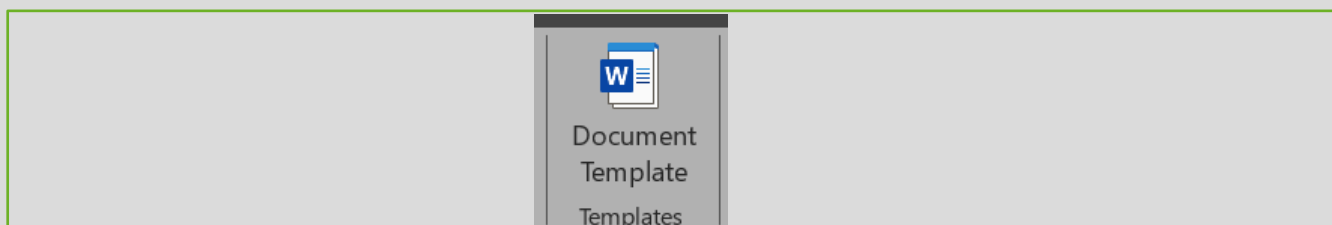


Pic. 16. Folders where Microsoft Word configuration is stored.

5.2. Folder A (templates) and folder B (add-ins) in User Interface

The files stored in the specified folders A and B can be visible on time of Microsoft Word operation in menu *Developer > Document Template* (compare [Pic. 17](#), [Pic. 18](#)):

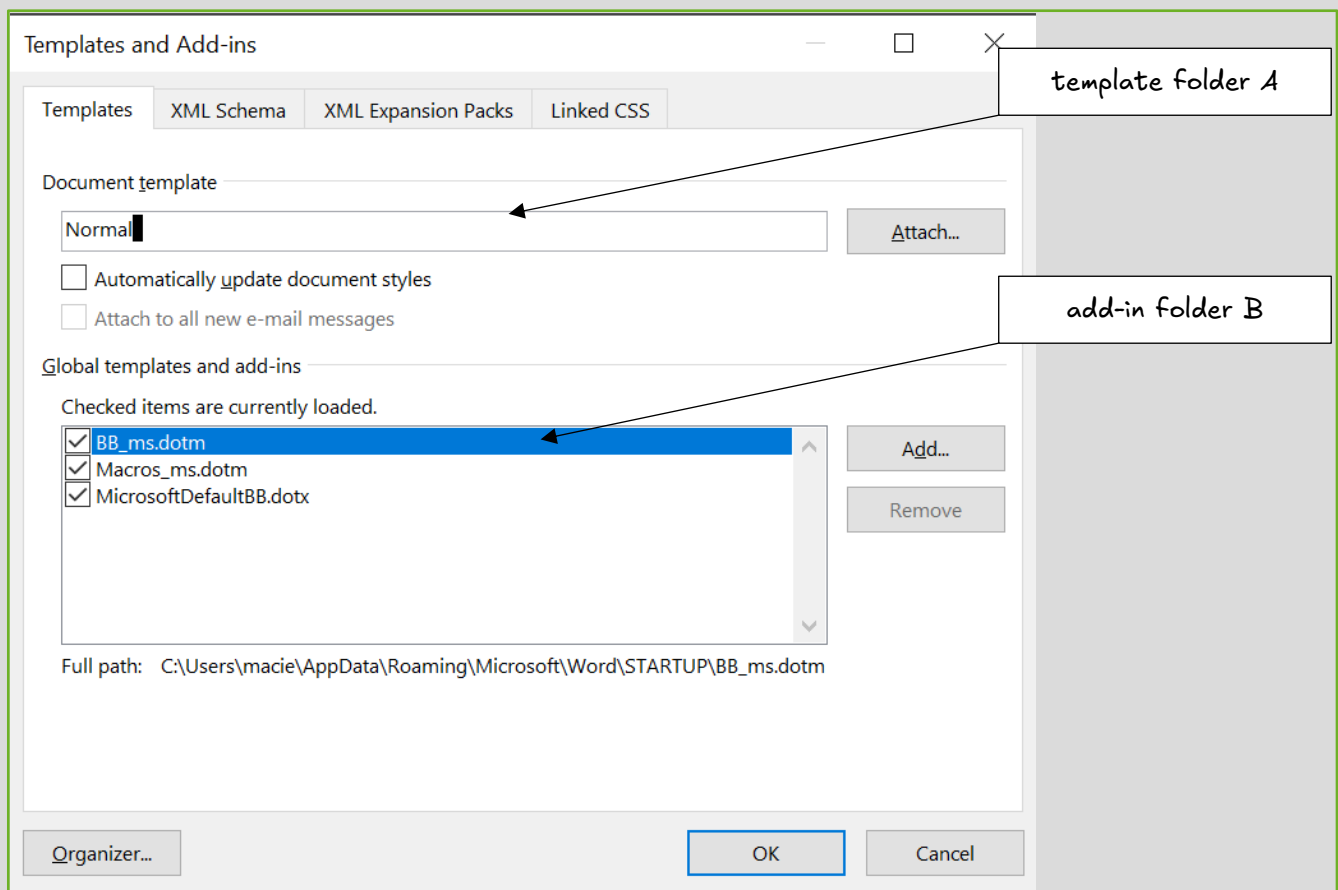
- » When the button *Attach...* is pressed, content of folder A is visible and one of the files from that folder can be selected.
- » When button *Add...* is pressed, content of folder B is visible and any of the files from that folder can be selected.



Pic. 17. Document Template button.

Info

The menu section *Developer > Document Template* is surprisingly not visible by default, I mean, when Microsoft Word is installed. To enable it select *File > Options > Customize Ribbon*. Select from the right *Main Tabs > Design* and enable it ☒.



Pic. 18. Templates and add-ins ver. folder A and folder B.

As you can see, only one file from folder A can be attached to currently edited file. On the same time all DOTM and DOTX files are shown from folder B. The User Interface enables to switch on ☒ or off ☐ whichever add-in file on time of editing, what makes Microsoft Word configuration flexible. You can for example keep separate sets of building block, dedicated to specific subject, in the separate files and enable or disable them only when required.

5.1. Folder B (ribbon menu files)

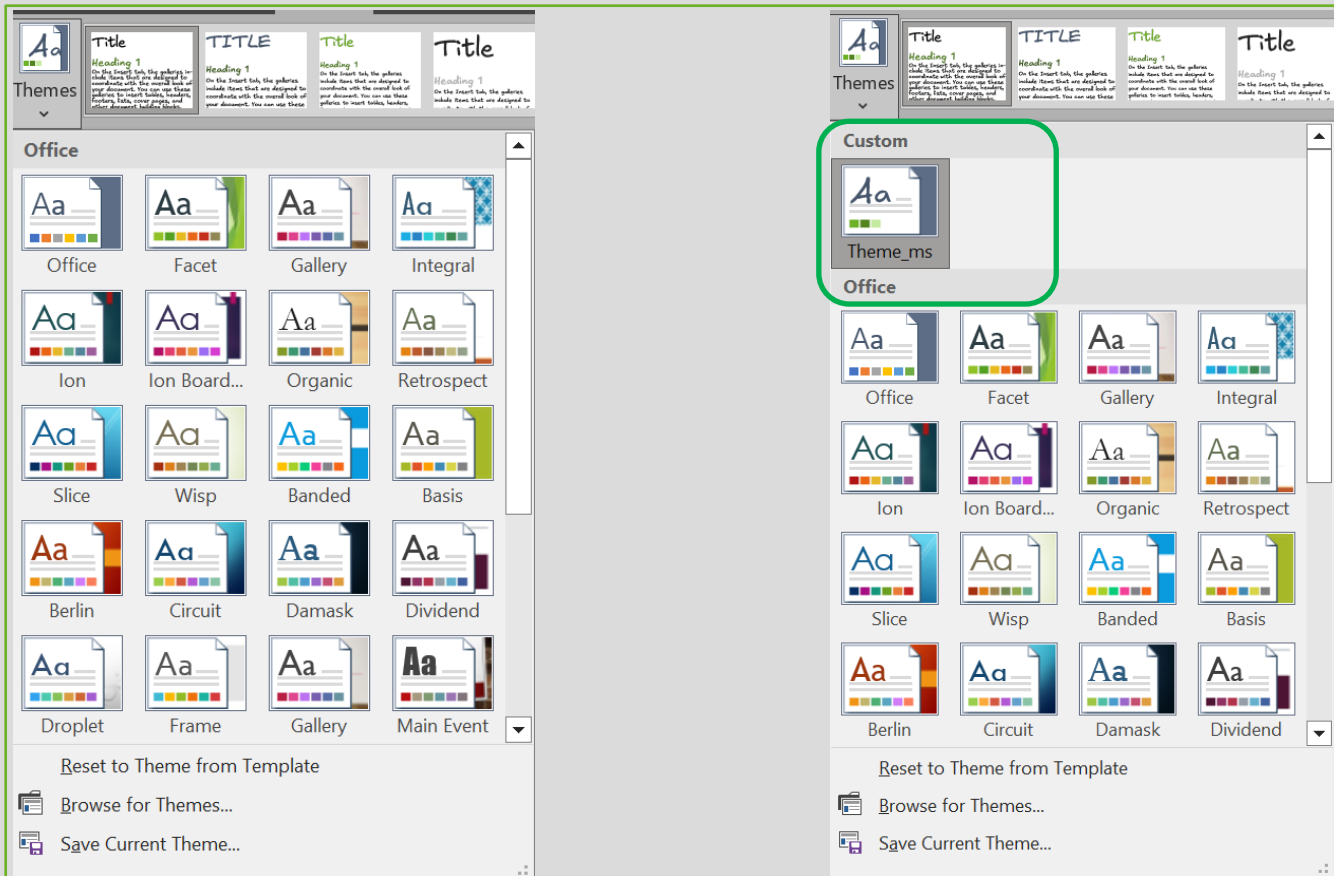
I store my menu ribbon files .exportedUI in folder B just by choice. It means I could store them elsewhere, but folder B is convenient, easy to remember location to me. Location of those files aren't visible in Microsoft Word User Interface.

5.2. Folder C (Theme) in User Interface

The location of folder C (Theme) cannot be changed by user.

Content of this folder is visible in User Interface. Select from ribbon menu *Design > Themes*:

- » If this folder is empty and there is no customized theme file, the menu *Design > Themes* contains only default themes, compare [Pic. 19](#), the left part.
- » If this folder is not empty and there is at least one customized theme file, the menu *Design > Themes* shows also the *Custom* section, compare [Pic. 19](#), the right part.

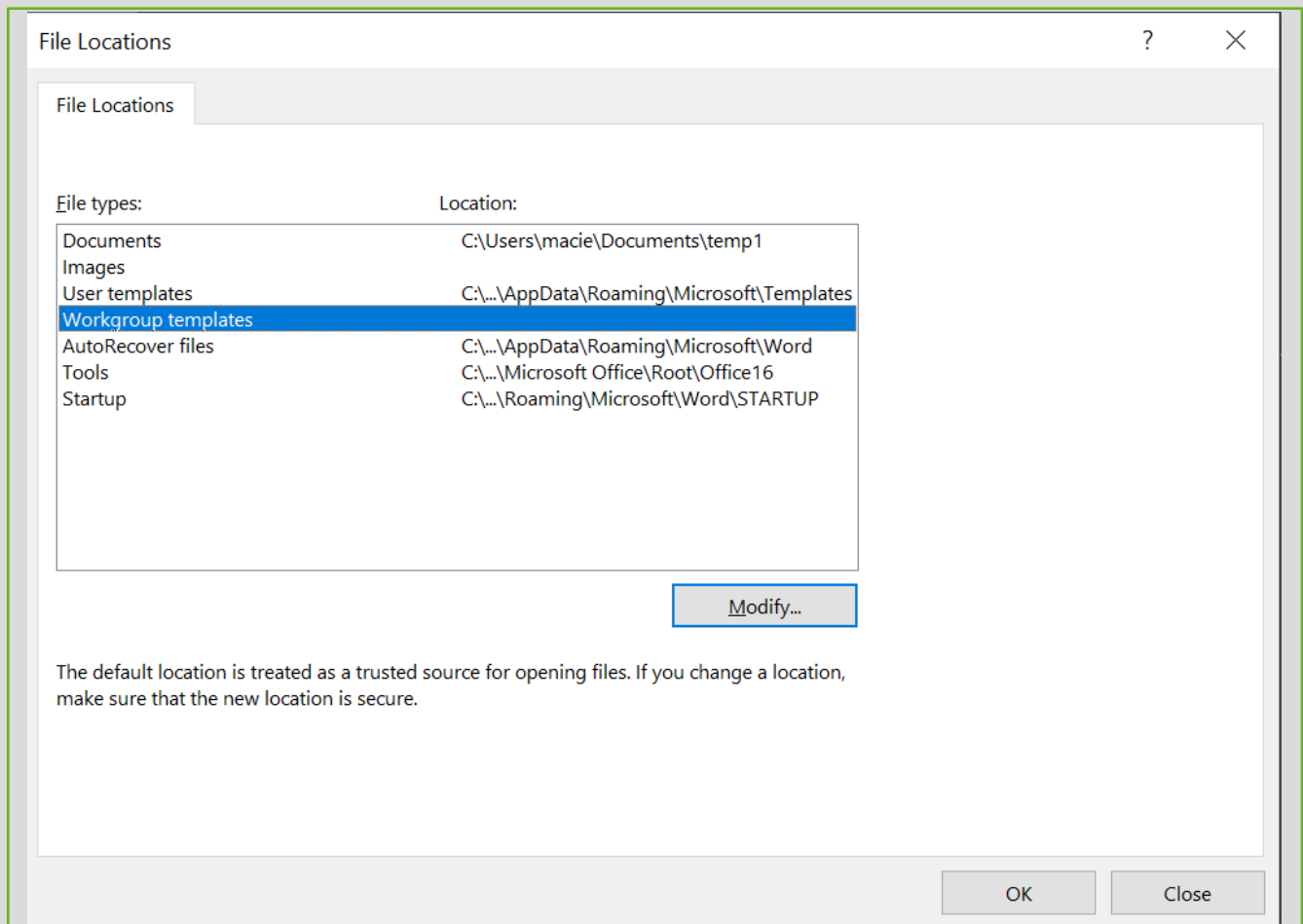


Pic. 19. Location of theme file.

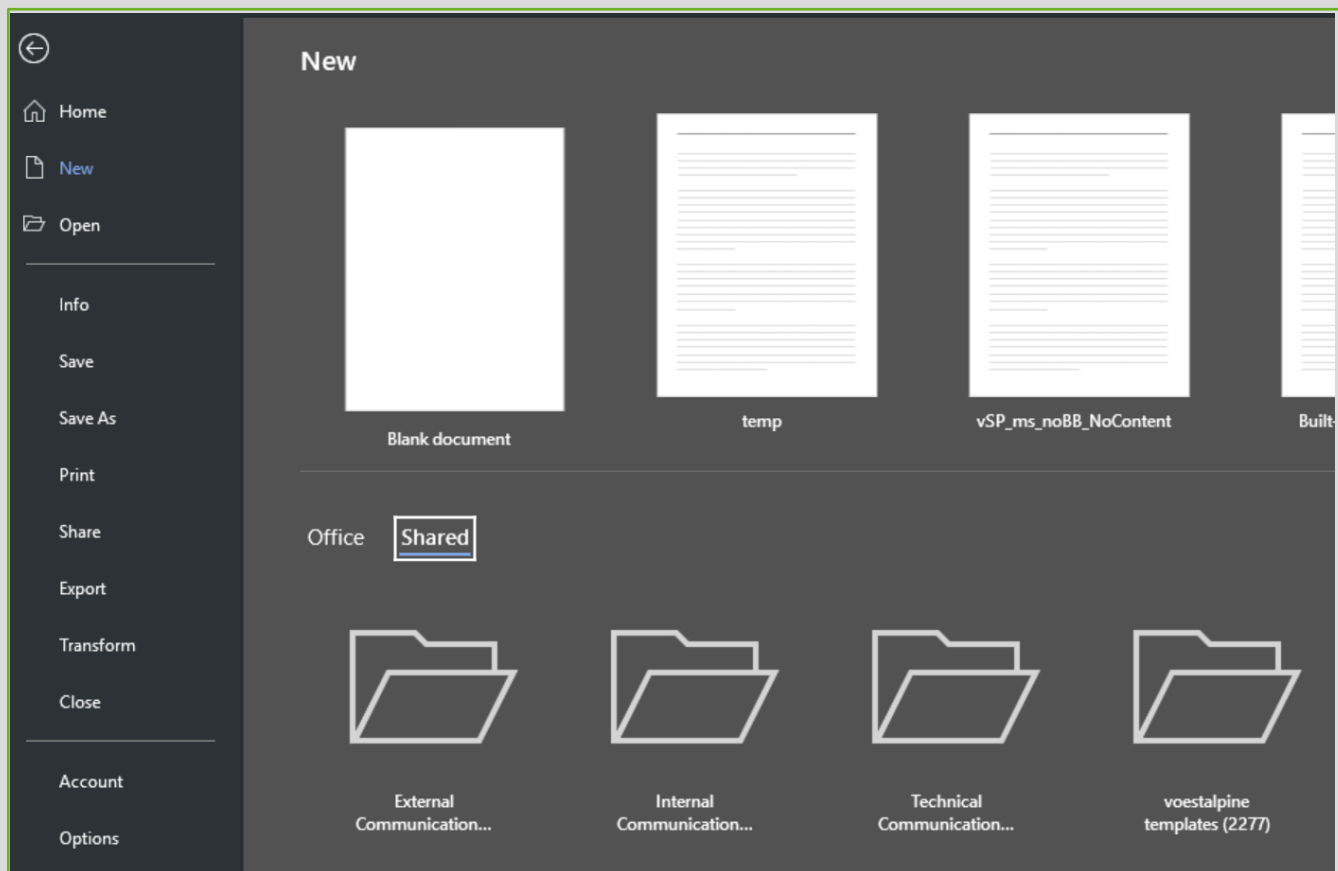
5.3. External configuration files and folders

After entering *File > Options > Advanced > General* and pressing *File locations* it is possible to set up the *Workgroup templates* folder where additional template configuration files might be located (compare [Pic. 20](#)).

This folder is empty by default. In your company environment it can be used to place there official company's templates. If it is the case, then the user on time of creating a new file will have option to choose any of the templates from User Interface: *File > New > Shared* (compare [Pic. 21](#)). It enables quicker choice of template from the predefined set.



Pic. 20. Workgroup templates.



Pic. 21. Access to workgroup templates.

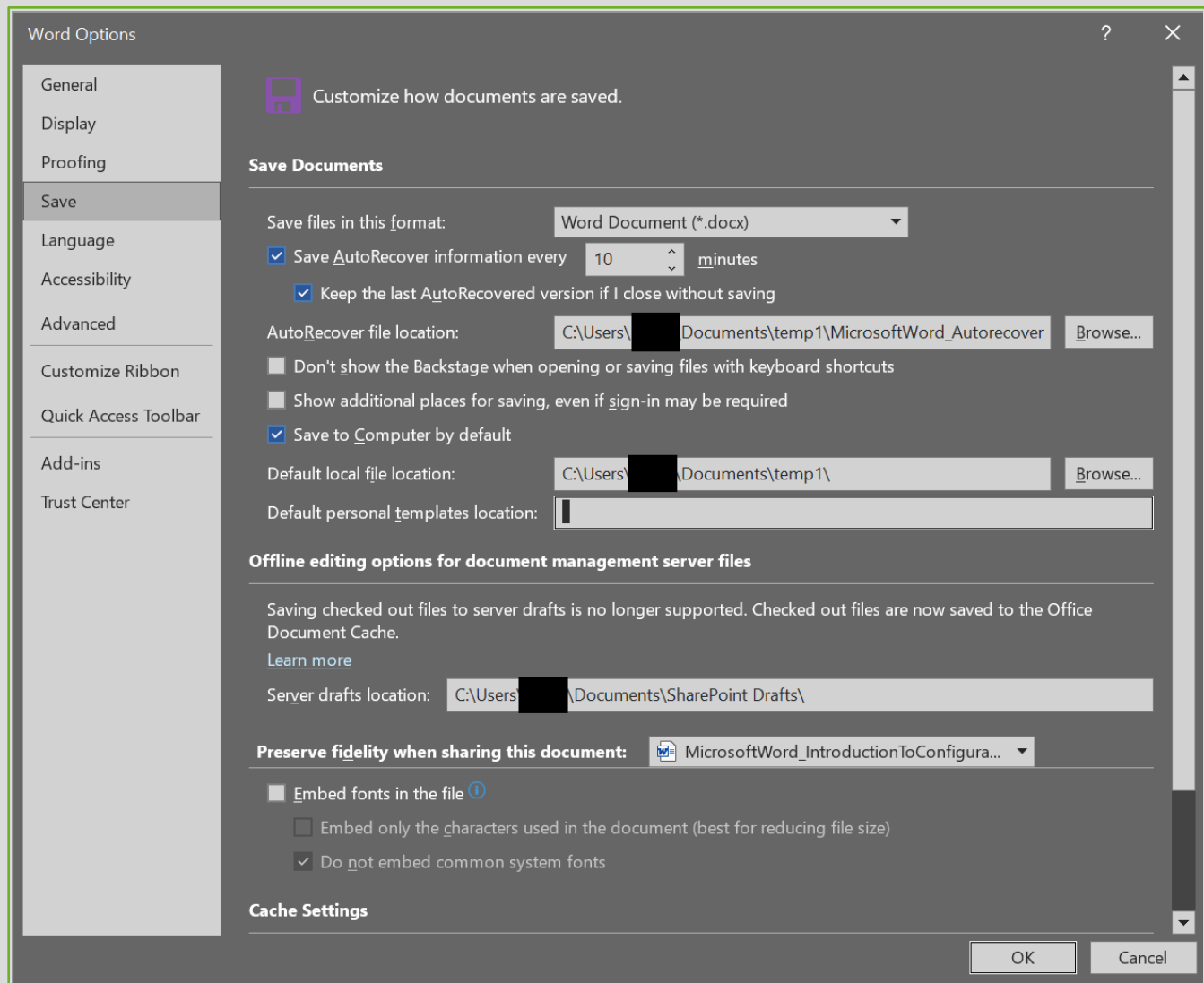
5.4. Environment, backup operation files and folders

Further folder and file settings are available in Microsoft Word after selection of *File > Option > Save*. There is some inconsequence, as some settings are doubled and could be selected also from *Word > File > Options > Advanced > General > File locations...*, see content of previous subchapters for comparison.

My personal choice for *AutoRecover* file location is temporary folder where I keep all other files. I also set for *Default file location* my preferred folder. It is used by many of my macros as the default output.

The *Server drafts* location is not relevant from my point of view, so I'll not touch it in this document.

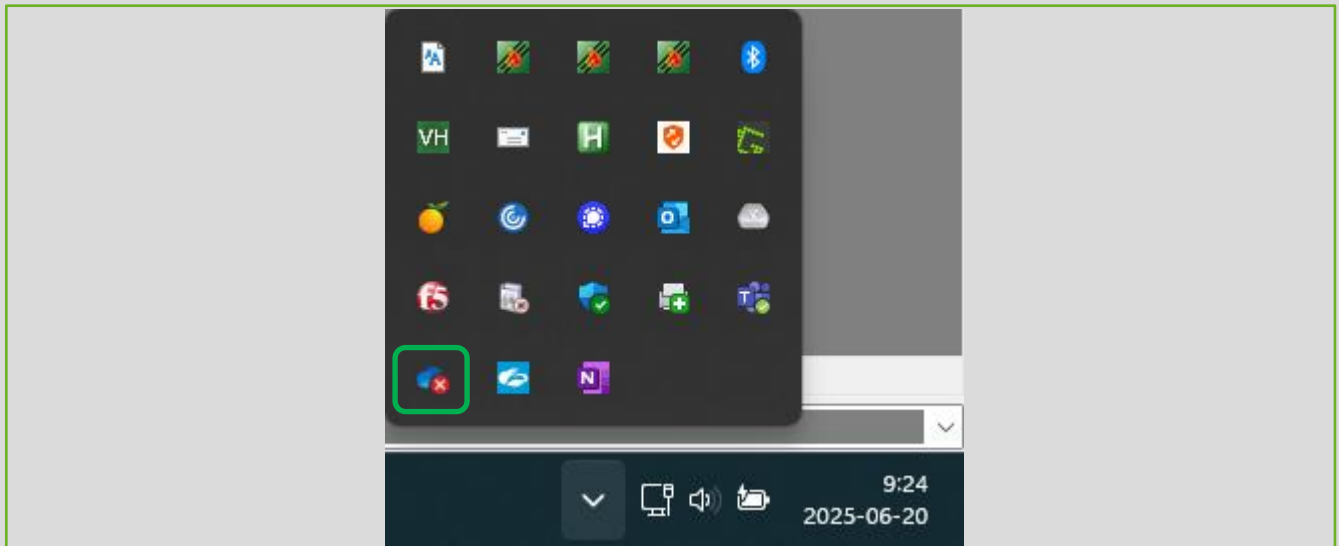
By the way I strongly recommend checking (☒) option *Save AutoRecover*.



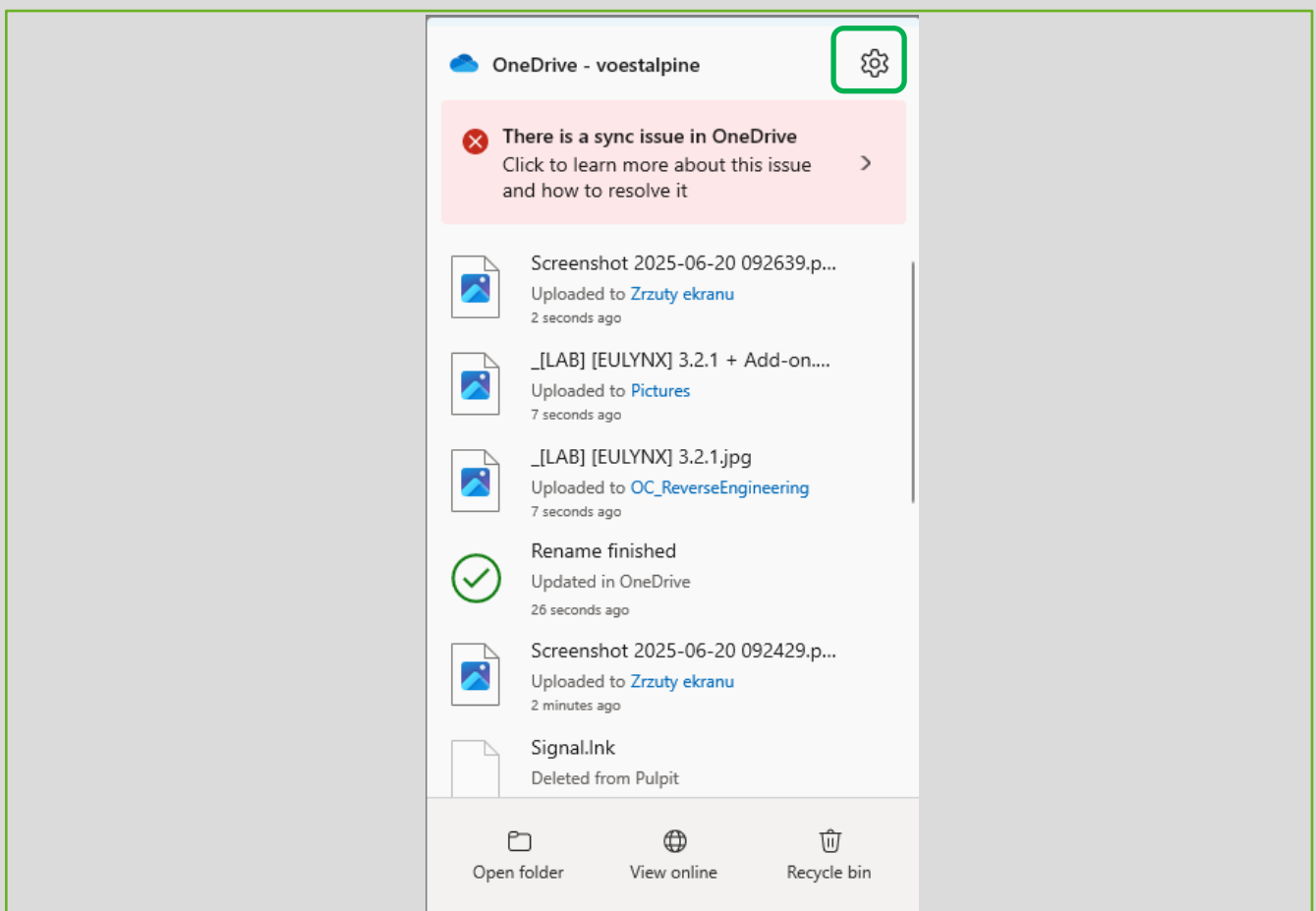
Pic. 22. File > Option > Save.

5.5. Environment, Microsoft Word and OneDrive

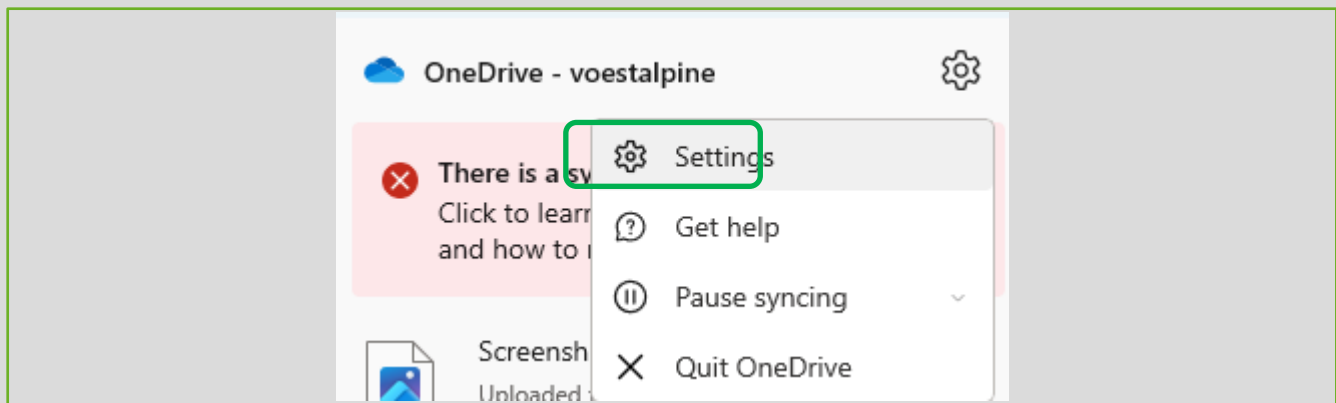
Because Microsoft Word could use (Microsoft) OneDrive as a file service, it is good to recognize relationship between these two applications. Observe green rectangle in the following pictures.



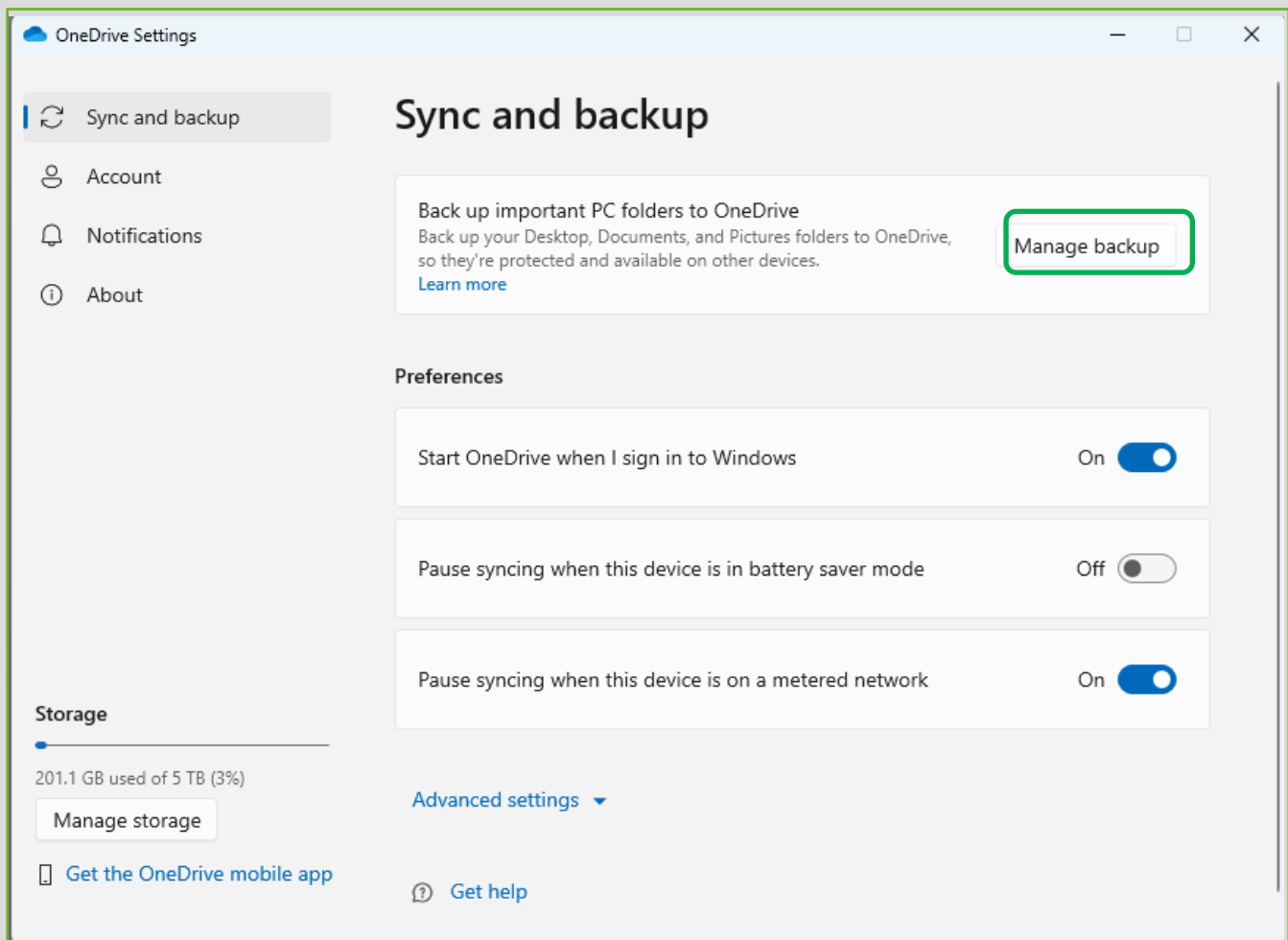
Pic. 23. Find OneDrive icon in SystemTray.



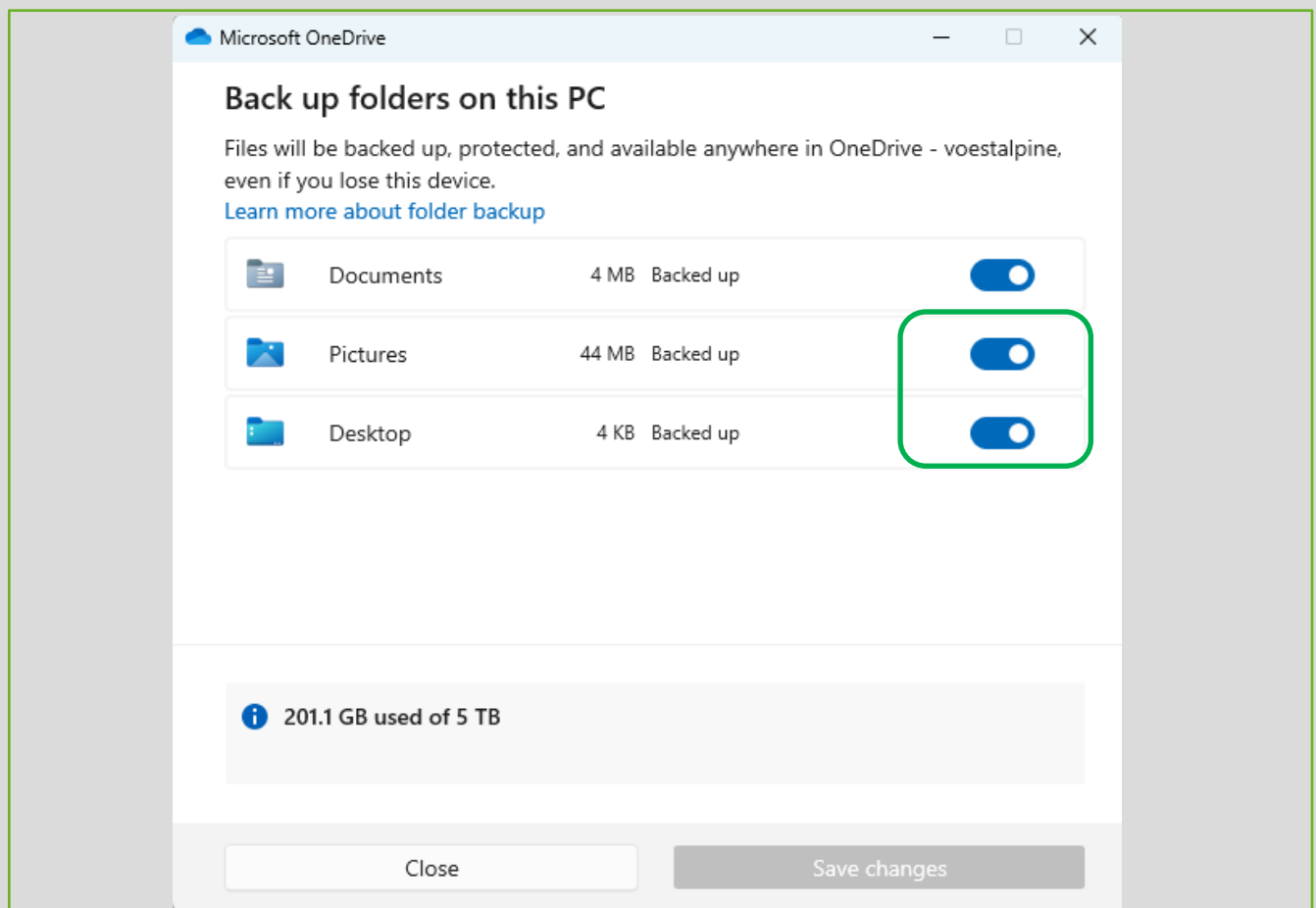
Pic. 24. Select settings (⚙️ symbol).



Pic. 25. Select settings again (⚙️ symbol).



Pic. 26. Select Manage backup button.



Pic. 27. Consider switching off.

Info

I recommend switching off *Pictures* and *Desktop* folders.

6. Configuration files DOTM or DOTX

What are the configuration files DOTM or DOTX? There is no simple answer to that question. From my perspective I imagine them are like [containers](#) or even [intermodal containers](#) in abstract form. They are like big boxes for holding different small boxes aka categories of information inside.



Pic. 28. DOTM or DOTX as containers.

DOTM or DOTX can be both, a template file or add-in. It depends on context, understood as file location.

For me traditionally the "template" word has two meanings:

- » traditional, equal to stencil:
set of rules dedicated e.g. to **content**,
e.g. you get a sheet of paper where only a few blank areas are ready to be filled in.
- » new,
a set of **configuration** settings.

In this book the word template is applied in this second meaning. Actually, Microsoft Word template files can be both at the same time (technically you can store content within template files), but I decided to separate them from content, at least here, to better grasp the idea.

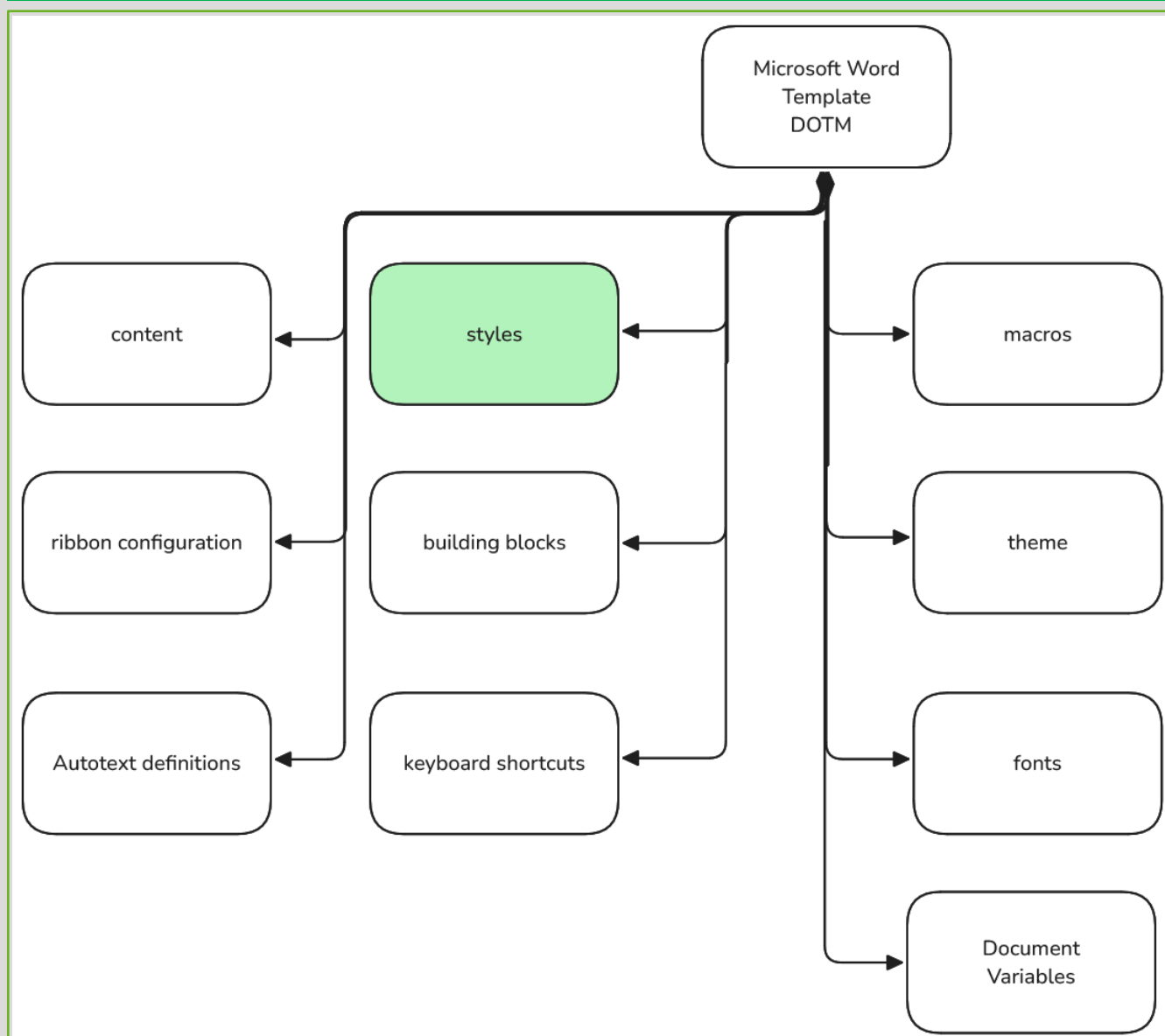
The DOTM or DOTX files are binary. You can manually change the extension of such files to ZIP. Then you can see what information is stored inside. Don't be surprised not seeing any of the subfolders or files from [Pic. 29](#). The internal organization of files within ZIP folder isn't so easy and straightforward.

6.1. What kind of information categories may be stored within?

Let's see what boxes are available within our containers.

Legend:

- » green color shows ever existing modules,
- » an arrow, which finishing end is diamond shaped and pointing end is a full black arrow, should be read as "is made of".



Pic. 29. Configuration categories in DOTM file.

The DOTM file from [Pic. 29](#) is a container which always contains just one module: *styles*. On the other hand, it can contain up to all 10 different information categories.

There is the second type of file, DOTX. Files with this extension will not contain the *macros* category. Be aware that it is not enough just to change file extension. You must use Microsoft Word for that purpose and *Save As* function to change from one file category to another."

6.2. DOTM or DOTX as a template: attachment relationship

What happens when you start the Microsoft Word application which is not yet altered or re-configured?

When Microsoft Word is started by clicking icon "Word" or making deliberate choice from within Start Menu, and application configuration wasn't altered in any strange way, it opens *Document1*, what is visible in the middle of Microsoft Word main window top bar, see [Pic. 30](#). In the meantime, Microsoft Word already attached the default template file *Normal.dotm* and copied all built-in styles (> 350 items) to this file.

Pic. 30. Example.

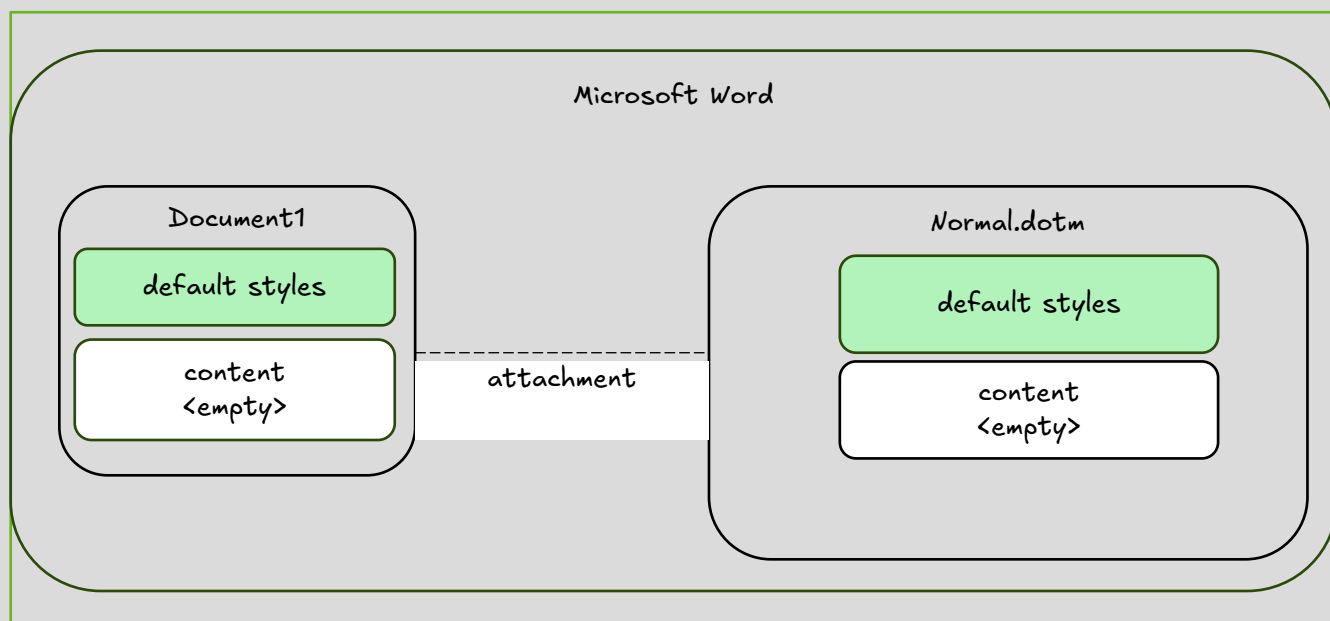
There is alternative way of running Microsoft Word, preferred by many users. They locate default Microsoft Word template file authored by company, e.g. *MyCompanyTemplate-File.dotm* and "run it" as if it was an executive file (EXE), by double clicking or pressing enter upon file name. Then Microsoft Word starts and again *Document1* is opened, but this time list of styles contains: all built-in styles (> 350 items) plus specific styles defined in this *MyCompanyTemplateFile.dotm*.

In other words, no matter which way you start Microsoft Word, the attachment relationship is established automatically between your content file (*Document1*) and one of the DOTM files, which act as a template file, by default the *Normal.dotm*.

The only way to at least have a feeling what is going on in User Interface with attached template file is to investigate ribbon menu *Developer > Templates > Document Template* button (see [Pic. 17](#)).

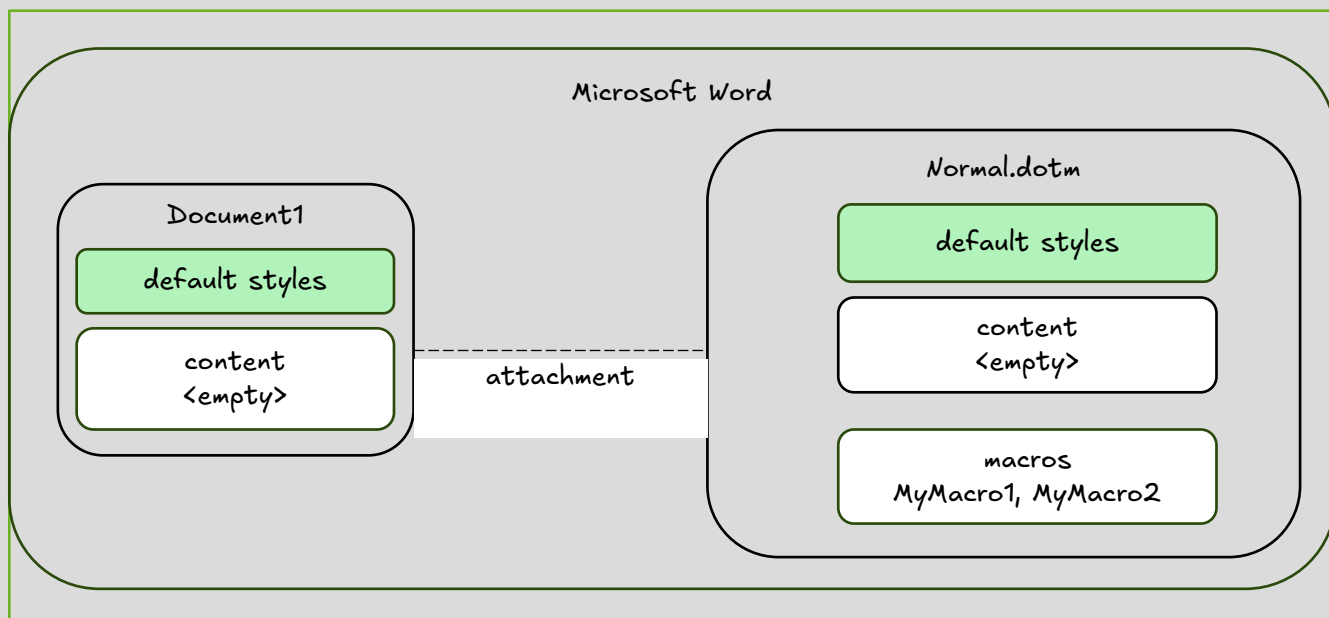
Besides copying of styles, thanks to template file, your *Document1* gets access to all modules of the *Normal.dotm* container. Again, by default the *Normal.dotm* contains just predefined styles, but it might contain much more. This attachment relationship lets the file *Document1* to store just content and styles, and no other information..

Please also note that by default both files: *Document1* and *Normal.dotm* contain the same set of styles. Those styles are automatically copied from the *Normal.dotm* to *Document1* on time of creation of *Document1*.



Pic. 31. Attachment relationship.

Let's assume for a moment that we have already modified the template *Normal.dotm* by adding few macros, compare [Pic. 32](#).



Pic. 32. Attachment: Document1 has access to Normal.dotm containers.

Now the Document1 will have access to the macros MyMacro1 and MyMacro2 only if template file Normal.dotm is attached to it. If you send Document1.docx itself to any other user and she / he would like to run any of those macros, it won't be possible. In other words, some categories of information are separated or accessible only if specific relationship is established.

On the other hand, all content files which had attachment relationship to the Normal.dotm in the past and Normal.dotm from that moment is stored in the same place, would have access to the macros.

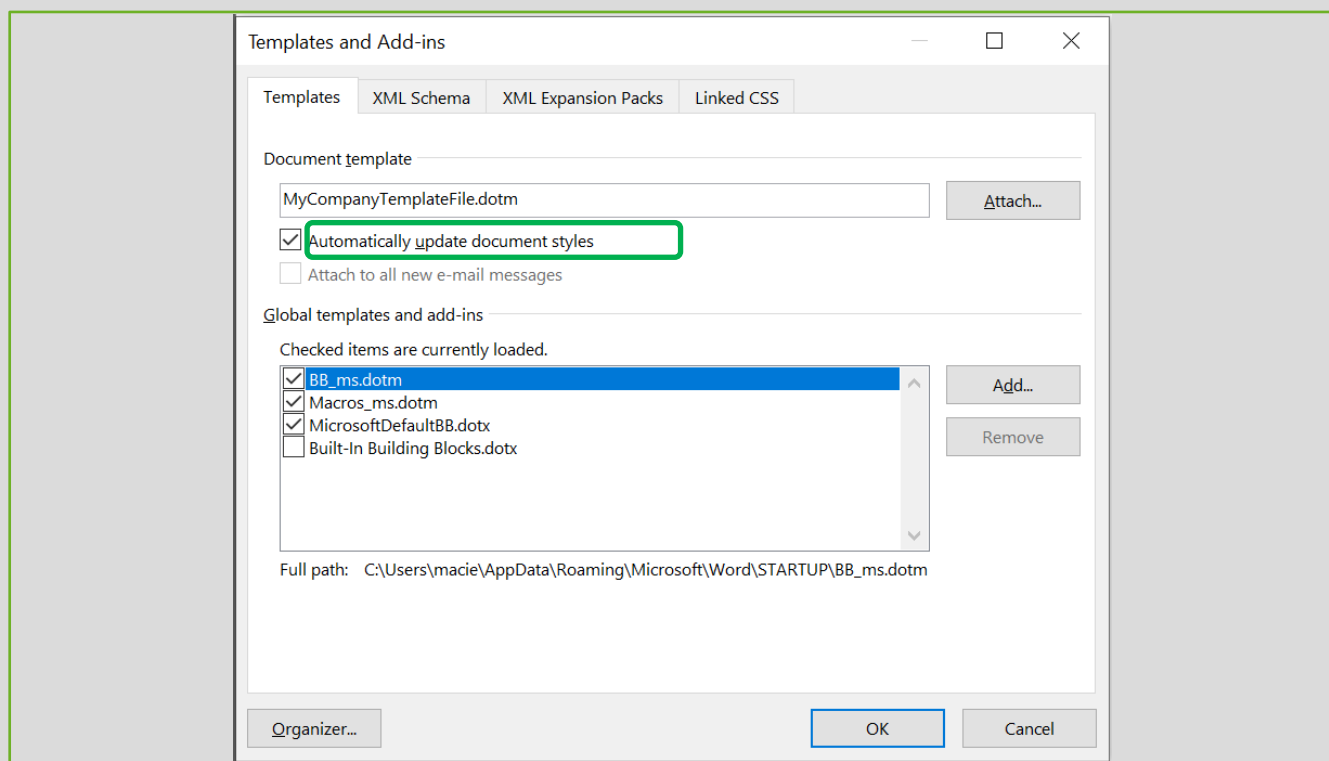
The attachment relationship is quite special, because first it is styles oriented. You can attach to your Document1 any other DOTM or DOTX file, e.g. MyCompanyTemplateFile.dotm, making it the template file. Then you can use special tick ☒ named *Automatically update document styles* and press OK, see [Pic. 33](#) to copy customized styles from MyCompanyTemplateFile.dotm to Document1, see [Pic. 34](#). The document Document1 stores the full path to the template.

The tick named *Automatically update document styles* is quite unfortunate and causes a lot of confusion. It should be represented in a User Interface rather as a button to show that action of copying styles from template file to content file takes place after activating this button. Practice shows you should mark this tick ☒, then click OK button to deliberately copy the styles. Next open the same window again and unmark this tick ☐. If you won't do that, then each time you click OK button, the styles will be silently copied again to your content file. You have been warned. It could be the case, that styles definition in template file have meanwhile meaningfully changed, and then when you reopen your file, are silently updated, and your content may be a bit differently formatted.

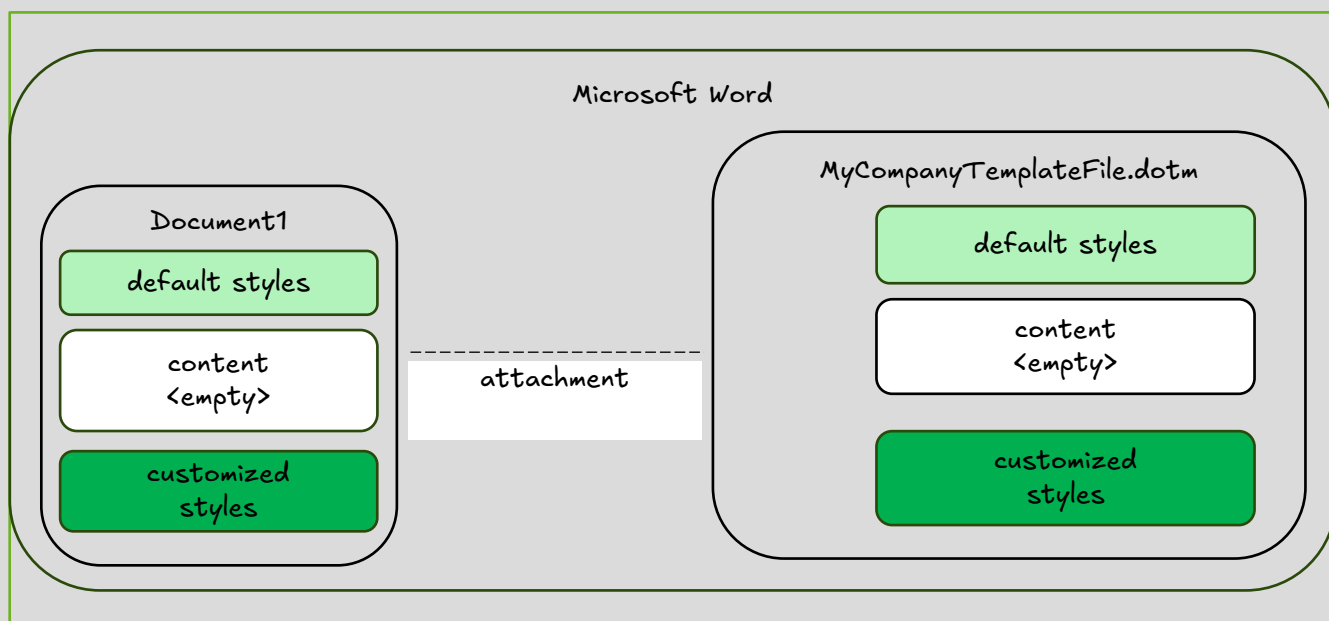
Another caveat is that user is never explicitly informed about copying of styles. In my opinion the idea of automatic (silent) update of styles definition is quite strange, as users don't get feedback about what styles and when have been updated. I don't like it, and I don't recommend enabling it. My preference is to copy styles definitions consciously. The set of styles definitions should remain stable, so there is most of the time no reason to update them.

After styles are copied from template file to content file, they are stored in that file. In other words, they are not dynamically updated; if definition of style is changed in template file, it is not automatically updated in content file, unless the tick ☒ *Automatically update document styles* is not set.

To conclude: if we take into consideration styles transfer, the attachment relationship should be temporary. This is why it is marked with dashed line in block diagrams.



Pic. 33. Tick *Automatically update document styles*.



Pic. 34. Attachment: *Document1* has got styles from *MyCompanyTemplateFile.dotm*.

Summary of attachment relationship:

- » **Importing of styles:**
After template is attached to document, styles defined in template become available in document. It means, that all unique styles which are present in the template, are now available also in the document.
- » **Styles replacement:**
If in document are present styles named the same as styles in the template, attachment relationship silently modifies styles in this document.
- » **No dynamic link:**
Microsoft Word do not keep dynamic link between the template file and document.

After importing of styles, the styles stay embedded in document till they are not attached again. If styles are not protected, then can be modified, edited in this document, independently from the template file.

6.3. Normal.dotm template

Formatting of the content is influenced, almost as everything else, by applied template file. By default, this would be the *Normal.dotm* file, but even if this is not the case and you've chosen another template file for your content document, e.g. *MyTemplate.dotm*, the *Normal.dotm* will still influence your content document, including headers and footers.

In this chapter I will show you how to prepare a minimum configuration of *Normal.dotm* for your environment. I strongly recommend you to follow the described steps.

I want to be sure that *Normal.dotm* is clear of any changes added by user or Microsoft Word application. The *Normal.dotm* could be seen as set of default settings for all future templates and documents. This file default settings are stored withing Microsoft Word itself. If this file is missing (e.g. was shifted elsewhere or deleted), it is recreated (respawned). Let's create completely clean *Normal.dotm*.

Check where your *Normal.dotm* could be found. By default it is in *User templates* path:

c:\Users\<UserName>\AppData\Roaming\Microsoft\Templates\

Keep this path in a clipboard for a minute.

Close Microsoft Word and make sure it is not running. Tip: run operating system Task Manager (Ctrl + Shift + Esc), look to the details section and check there is no *Windowrd.exe* running (compare [Pic. 7](#)). If it does, stop it manually.

Run a file manager of your choice. Navigate to *User templates* path. Rename or backup to a path of your liking the *Normal.dotm* from that folder. Now, this path should be free of *Normal.dotm*. Run Microsoft Word and let it open a default *Document1* file. Look again into *User templates* path. It should still be empty. If you now close again Microsoft Word, the new, pristine *Normal.dotm* will be saved in *User templates* folder.

Open again Microsoft Word and while *Document1* is opened navigate to default settings of the styles: ribbon menu > *Home* > *Styles*, click on diagonal arrow ↘ in bottom-right corner (or Ctrl + Shift + Alt + S), and click the button *Manage Styles*. (compare [Pic. 35](#), left, red color).

Enter the *Set Defaults* tab. Observe the settings, in particular paragraph spacing after equal to 8 pt and line spacing equal to multiple at 1.16 (compare [Pic. 35](#), right).

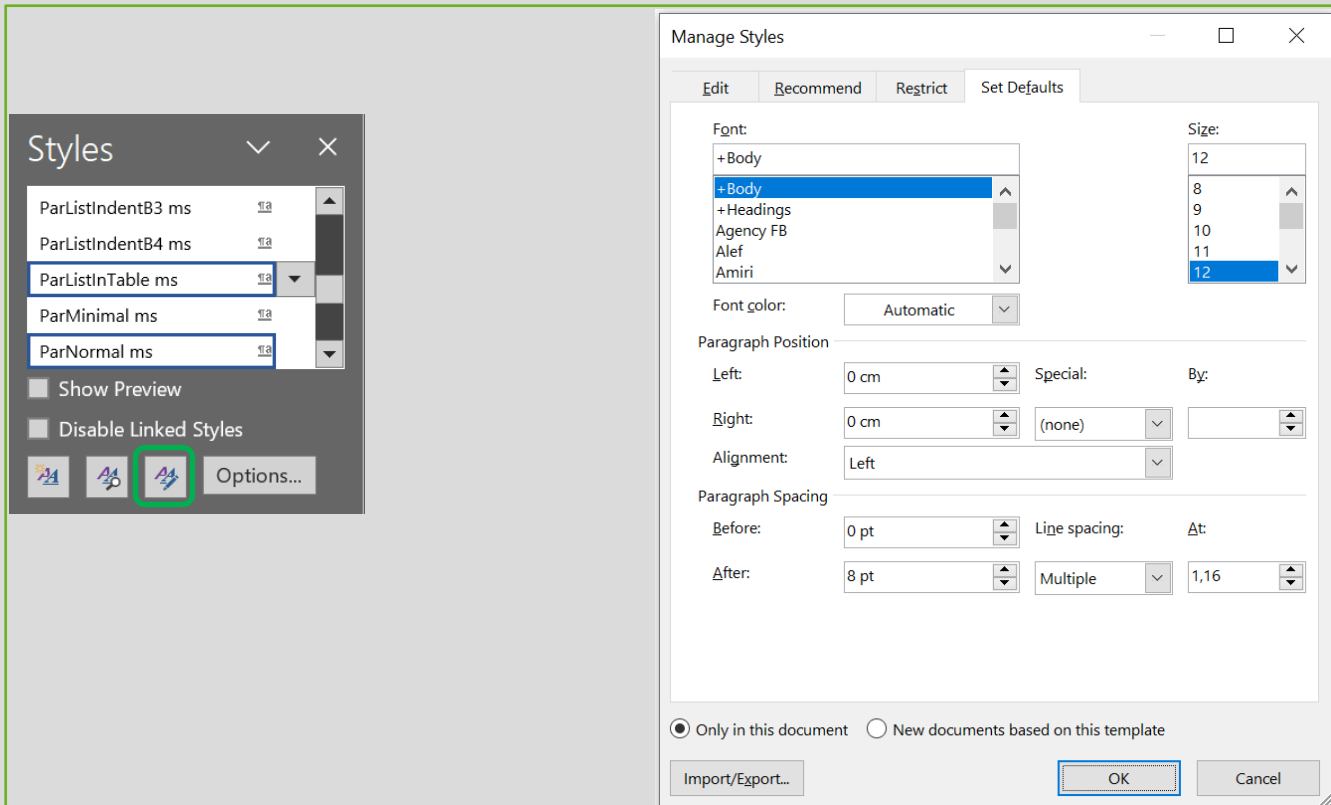
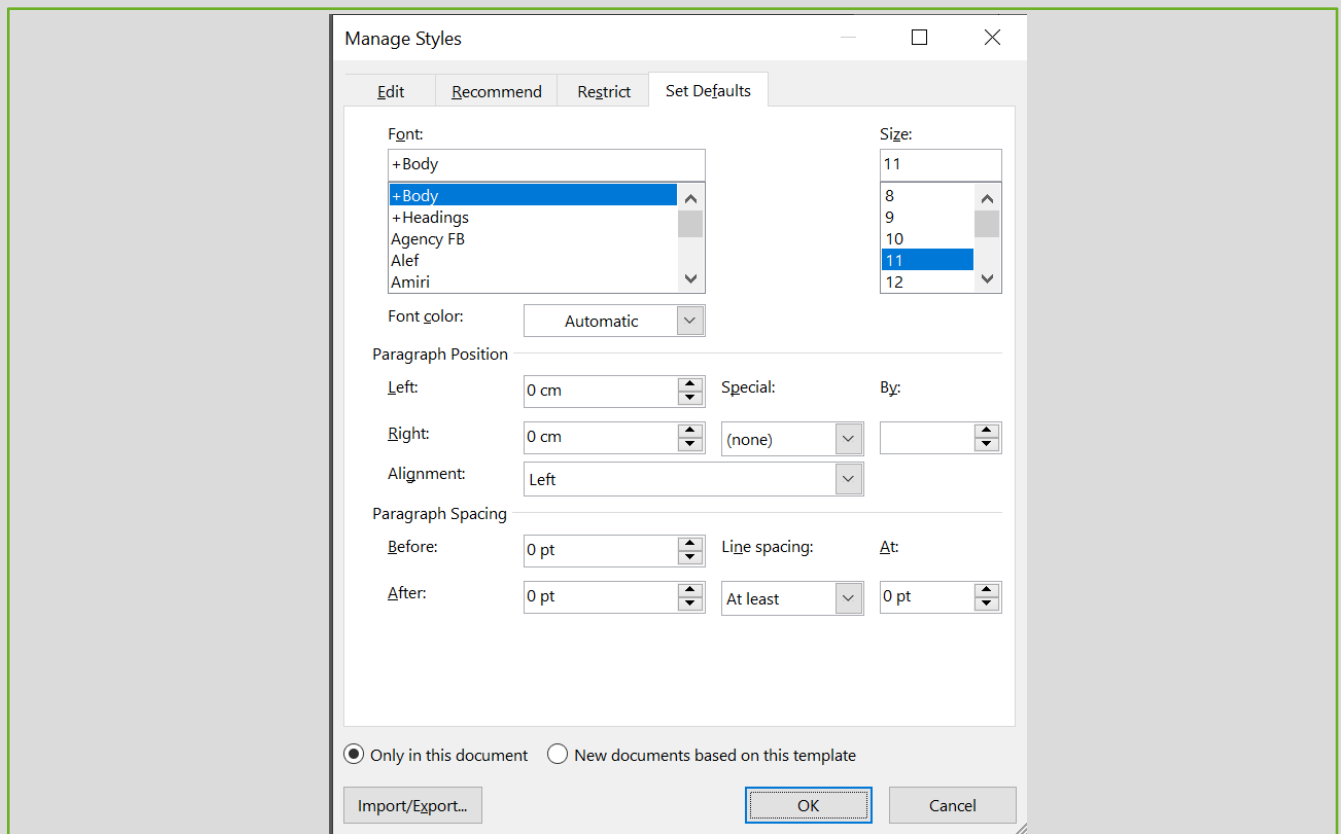


Fig. 35. Manage Styles button, Set Defaults tab.

Those settings have been copied to *Document1* from *Normal.dotm*. They influence style *Normal* and all styles based on the *Normal*, so by default all the styles, because all the styles in Microsoft Word are based on the style *Normal*.

I strongly recommend to change *Set Defaults* to values from the [Fig. 36](#) and save them in *Normal.dotm*.

1. open the *Normal.dotm* for editing
2. navigate to default settings of the styles: ribbon menu > *Home* > *Styles*, click on diagonal arrow ↘ in bottom-right corner (or Ctrl + Shift + Alt + S), and click the button *Manage Styles*. (compare [Fig. 35](#), left, red color).
3. Enter the *Set Defaults* tab and set them as in the [Fig. 36](#).



Pic. 36. Recommended Set Defaults.

Surprisingly:

- » There is no way to set up the *Set Defaults* settings from VBA code, at least on 2025-07-25. In other words, you must change them manually and cannot automatize this task.
- » Those settings have influence to subtle formatting of the content, such as header and footer distances between paragraphs, vertical distances within tables, to name a few.
- » For newly created file, settings of the *Set Defaults* are imported from *Normal.dotm* > *Set Defaults*.
- » This is one of the biggest and the most annoying quirks in Microsoft Word. You've been warned.

For example, if you change line spacing from *at least 0 pt* to *single*, your existing document formatting may change radically. Settings of all styles can be affected. Restoring previous settings without styles definitions written in VBA can then be laborious.

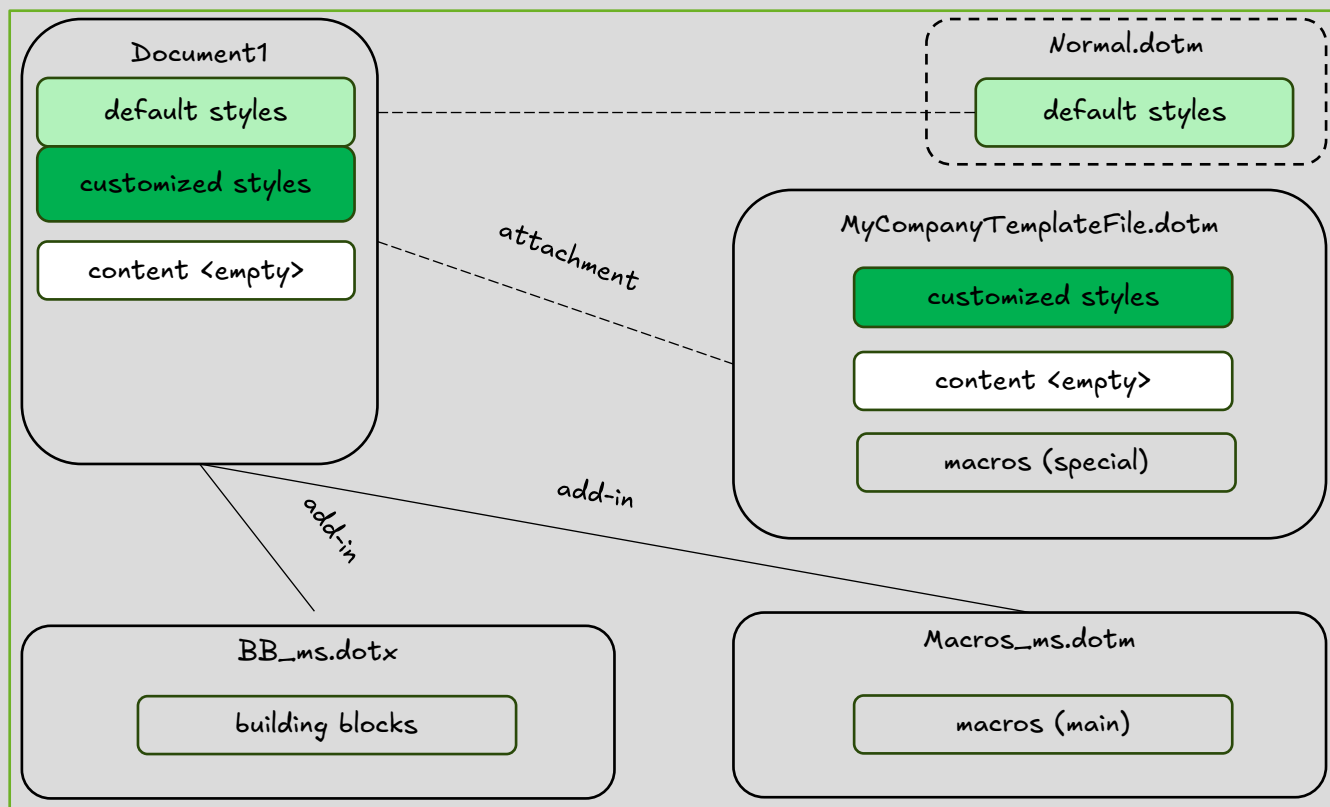
6.4. DOTM or DOTX as an add-in

Besides the *attachment* relationship there is also *add-in* relationship. A single content document can have only one *attachment* relationship and numerous *add-in* relationships.

If a DOTM or DOTX file is in the dedicated Microsoft Word startup folder, it becomes an *add-in*. The *add-in* word can be understood as plugin. The same file, depending on context (file folder), can be a *template* file or *add-in* file.

The difference between *template* file and *add-in* is that *add-in* can be easily switched on or off on time of work. For example, if user wishes to get access to building blocks from specific *add-in*, she / he can do that from *Templates and Add-in* window.

It is quite convenient to store sets of macros and / or building blocks in separate files and attach them temporarily to content file.



Pic. 37. My setup.

Please compare [Pic. 37](#) and [Pic. 34](#). For clarity the [Pic. 37](#) do not contain the Microsoft Word frame. It shows what happens when new *Document1* is created with *attachment* relationship to *MyCompanyTemplateFile.dotm*:

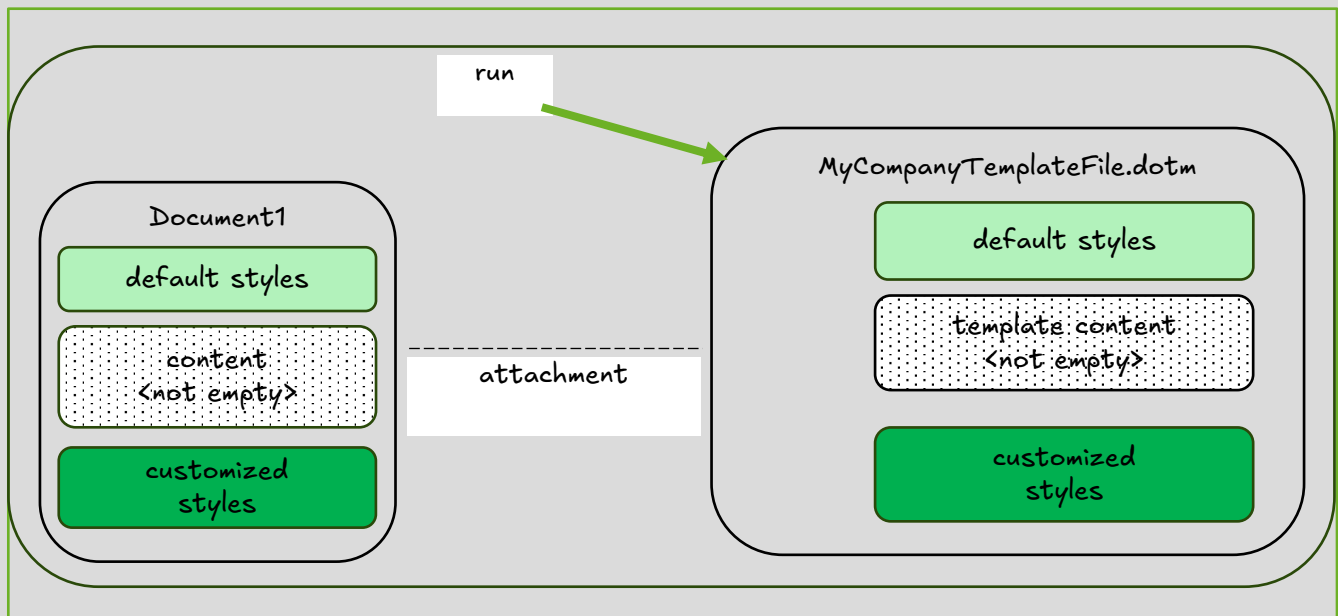
- » One-time automatic action:
Copy of default styles from *Normal.dotm* to *Document1*.
- » One-time manual action:
Copy of customized styles from *MyCompanyTemplateFile.dotm* to *Document1*.

The *Document1* has now access to all information categories from all three files: *MyCompanyTemplateFile.dotm*, *Macros_ms.dotm*, *BB_ms.dotx*.

6.5. Template file as a content stencil

The file DOTM or DOTX may contain content. Then if the user activates such file (e.g. by double clicking or pressing enter on it), Microsoft Word starts and newly created file *Document1* gets not only set extra set of customized styles (extra to default styles), but also copy of the template content, compare [Pic. 38](#).

The file DOTM or DOTX, when used to run Microsoft Word, acts as a *template* file. When it stores also content, it can be used as a content stencil.



Pic. 38. Attachment: *Document1* has got styles and content from *MyCompanyTemplateFile.dotm*.

Again: when it is attached, not "run", then it can be used to automatically copy styles, but not the content.

The copying of content, as described above, is a bit controversial to me. If you wish to create a set of repeatable content, like in a document stencil, you may use for that purpose building blocks and compose a new content file out of existing building blocks. This is the recommended and preferred solution. I opt for clear separation of configuration and the content.

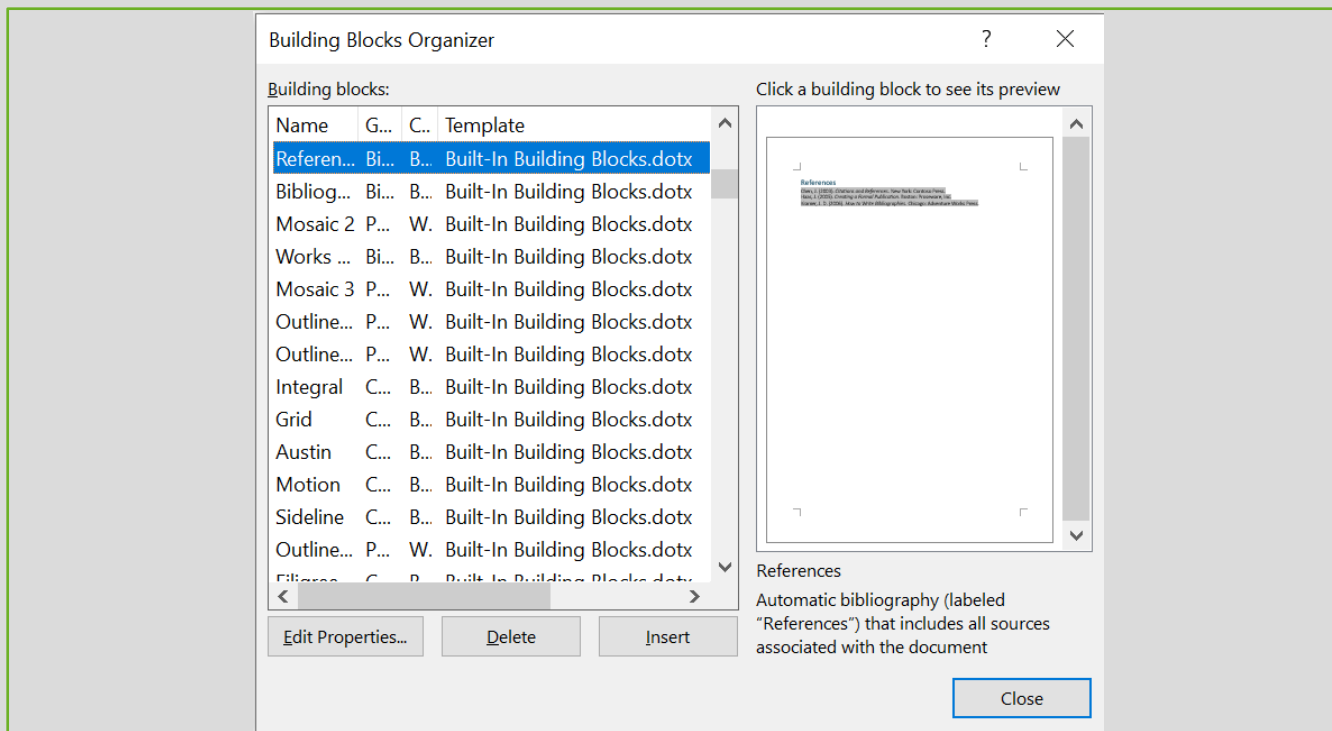
Of course, for the same purpose you can use any content file: just duplicate it, rename a copy and continue with content creation.

6.6. DOTX file as container for building blocks

Building blocks are sets of reusable content. Each time you find in your work couple of sentences or paragraphs, which you think it might be useful in future content creation, it is worth to think if not define a new building block. The Microsoft Word building block keeps formatting, understood as set of styles applied to content on time of defining building block.

I strongly recommend you to make use of building blocks in your work. Unfortunately, usage of User Interface is far from optimal.

Microsoft Word by default comes with huge set of predefined building blocks. You can find them navigating in menu to *Insert > Quick Parts > Building Blocks Organizer...*



Pic. 39. Building Blocks Organizer window.

The content visible in the picture [Pic. 39](#) comes from the default file *Built-In Building Blocks.dotx*, which you can find by default in the folder:

c:\Users\<UserName>\AppData\Roaming\Microsoft\Document Building Blocks\1033\16\

The exact folder name may vary from above. It depends on your localized version of Microsoft Word. Mine is 1033 what equals to US English. The 16 stays for version of my Microsoft Word.

The file *Built-In Building Blocks.dotx* is part of Microsoft Word itself. If it is deleted, after restart it will be recreated.

Whenever you add a new building block, add it to another DOTX file, not the default one. In other words, don't store your building blocks in the default file *Built-In Building Blocks.dotx*. Create your own file for that purpose, for example *BB_ms.dotx* and store it as another add-in file in the Microsoft Word startup folder:

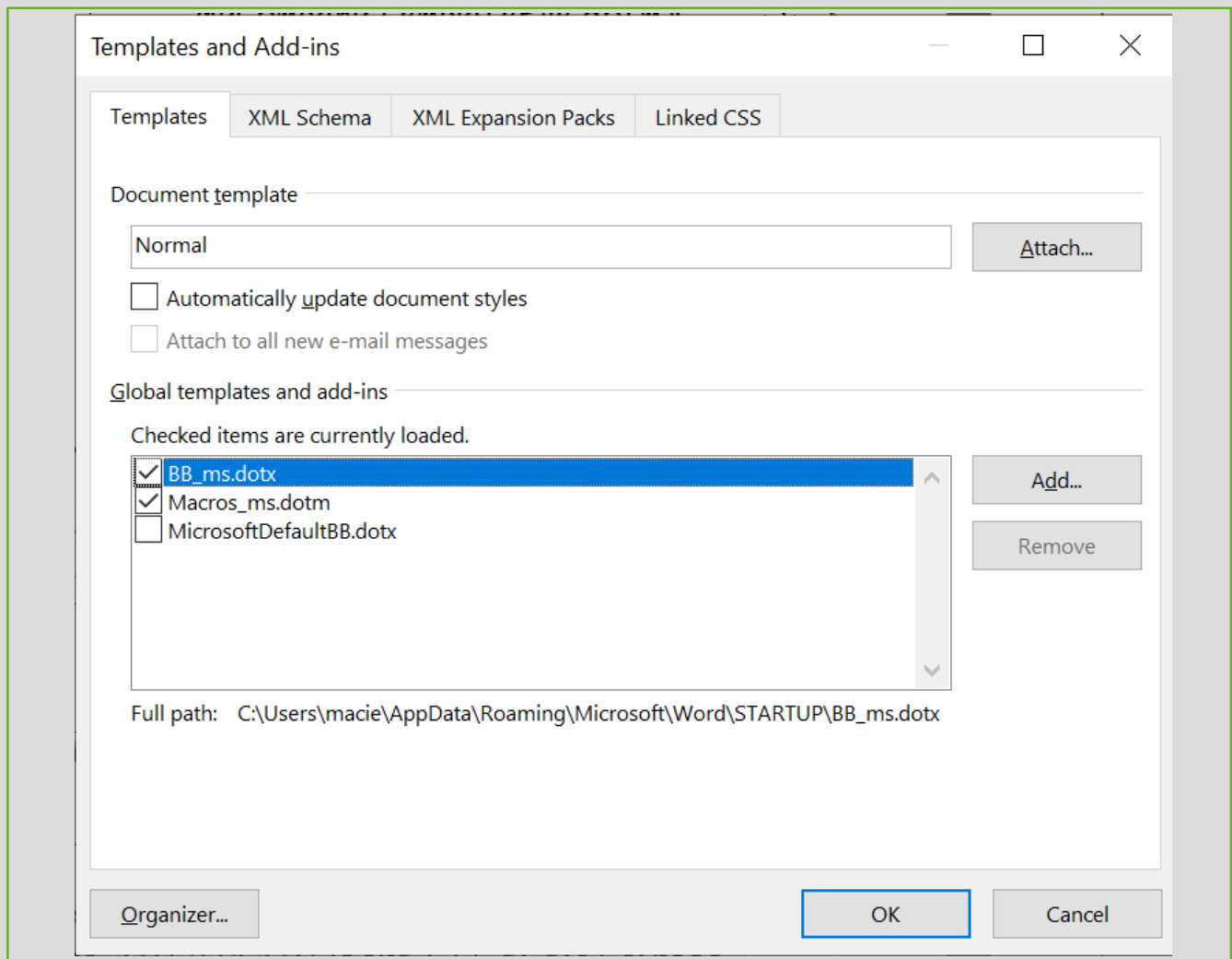
c:\Users\macie\AppData\Roaming\Microsoft\Word\STARTUP\

Thanks to that you will have access to your building blocks whenever Microsoft Word starts and you will also have an option to switch off unwanted set of building block. This way you can store different sets of building block in different files and use them in flexible way.

With one exception: the default set of building blocks stored by Microsoft in *Built-In Building Blocks.dotx* is visible all the time, what makes filtering in small, not-scalable *Building Block Organizer* window at least cumbersome.

There is however a set of tricks, which makes life easier:

1. We can clean-up content of the default *Built-In Building Blocks.dotx* without losing access to default set of building block, but making access to them an option. This way we can overcome to some degree limitation of *Building Block Organizer*.
2. We can store and manage our collection of building blocks without using the *Building Block Organizer*.
3. We can combine calling of the ours building blocks with *AutoText* feature of Microsoft Word.



Pic. 40. Modularization of building blocks (BB files).

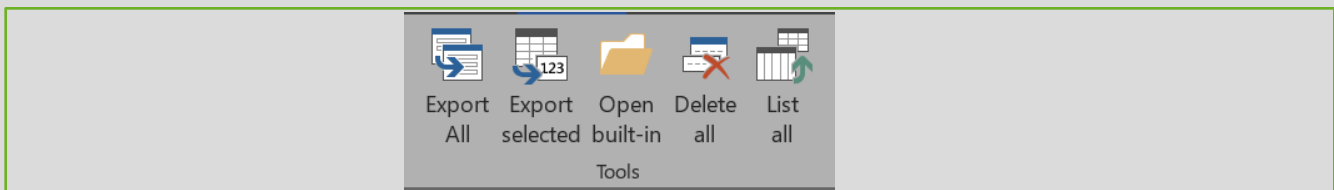
6.6.1. How to clean-up the default *Built-In Building Blocks.dotx*?

Assumption: you have already installed and configured my set of macros from *Macros_ms.dotm*.

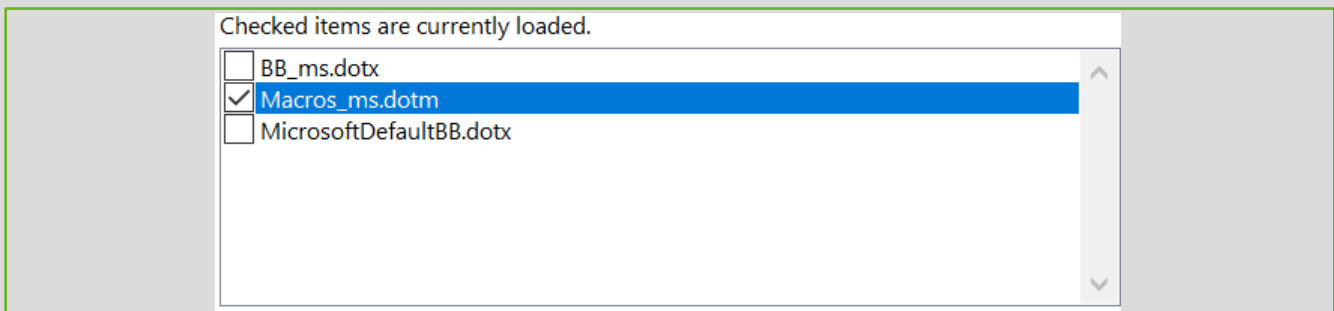
1. Close Microsoft Word.
2. Open the folder where *Built-In Building Blocks.dotx* is located, e.g.
`c:\Users\<UserName>\AppData\Roaming\Microsoft\Document Building Blocks\1033\16\`
3. Make a copy of this file and rename it to *Built-In Building Blocks.old*
 It can be useful for future references. For example, if you had stored your building blocks in the past in that default file, you will still have a chance to save them.
4. Make another copy of this file and rename it to *MicrosoftDefaultBB.dotx*.
5. Move the *MicrosoftDefaultBB.dotx* to
`c:\Users\macie\AppData\Roaming\Microsoft\Word\STARTUP\`
 This way you will have a chance to still have access to built-in building blocks, but this time on purpose.
6. Open Microsoft Word. From customized menu select *BuildingBlocks_ms > Open built-in*.
 Make sure the correct file is opened. If it is not, you can open it manually.
 After opening you should not see any content on the screen.
7. From menu select *BuildingBlocks_ms > Delete all*.
8. Restart Microsoft Word.
 This step is not mandatory, I suppose.

To verify your actions:

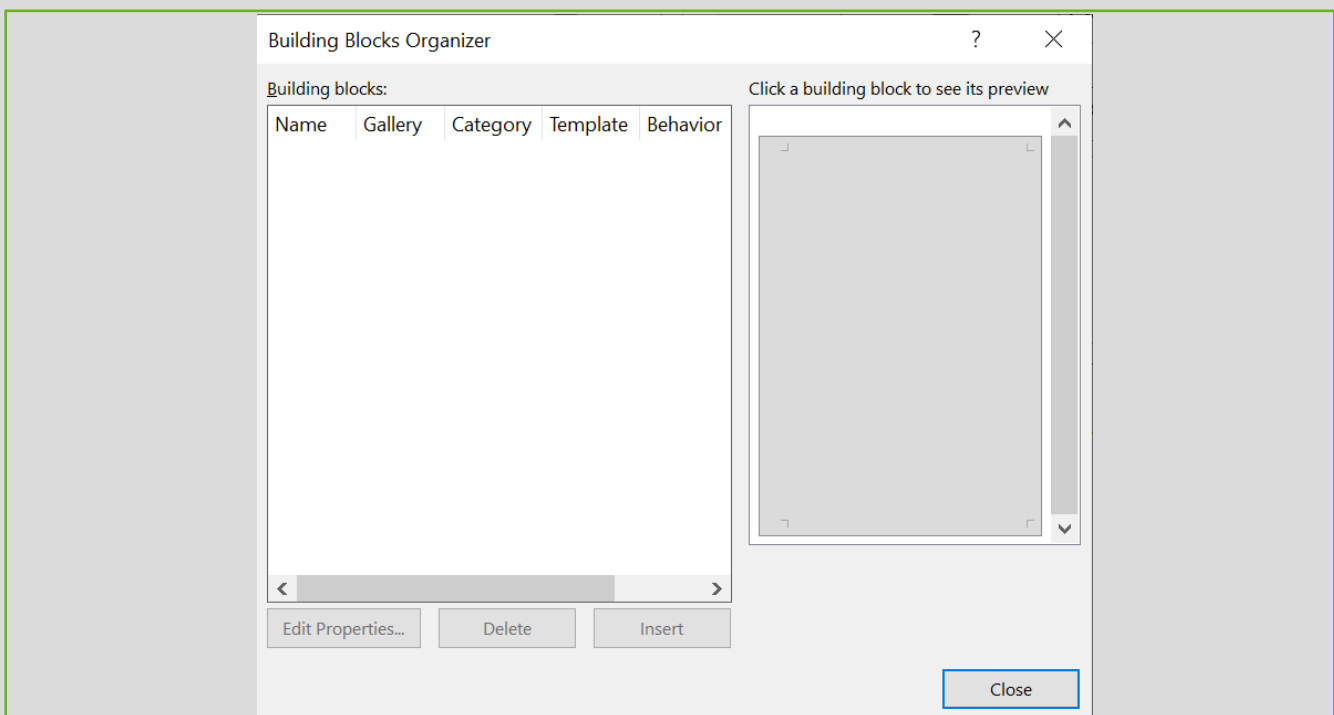
1. Open *Templates and Add-ins* and deselect all building block related files, compare [Pic. 42](#).
2. Open *Building Block Organizer* to check if you see any built-in building blocks, compare [Pic. 43](#).



Pic. 41. BuildingBlocks_ms buttons.



Pic. 42. Settings of building block related files.



Pic. 43. Empty Building Block Organizer.

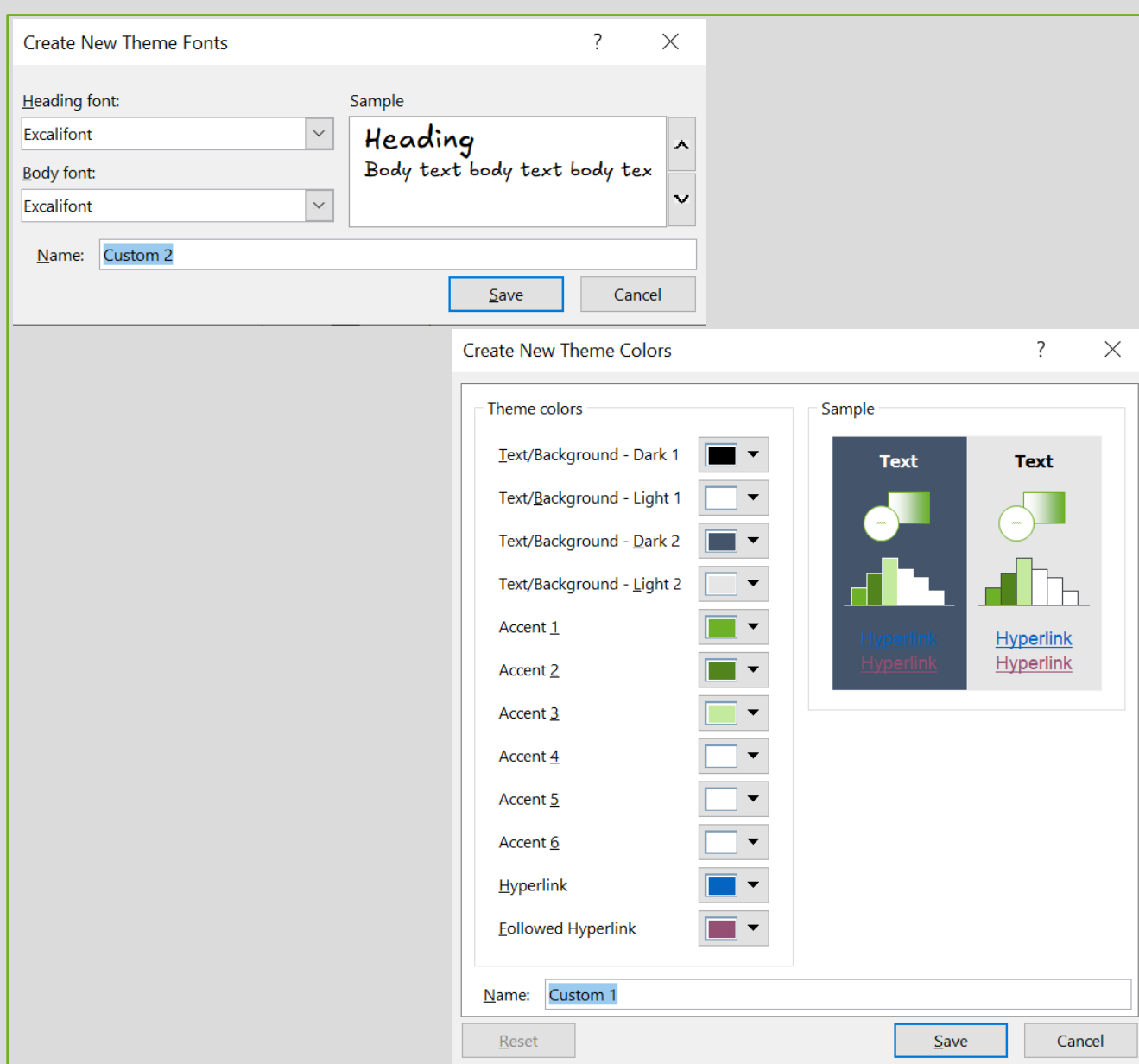
7. What are the theme files?

Theme is abstract concept of Microsoft Word related to content formatting. No matter if you know about this concept or not, it is always applied to your content. It is better from my perspective to know it and apply it consciously.

By default, to your content is applied the *Office* theme. You can check your document current selection from menu *Design > Themes*.

Theme is a set of specific formatting features, such as:

- a. two types of fonts:
 - i. Heading font
 - ii. Body font
- b. colors:
 - i. 4× text / background
 - ii. 6× accent



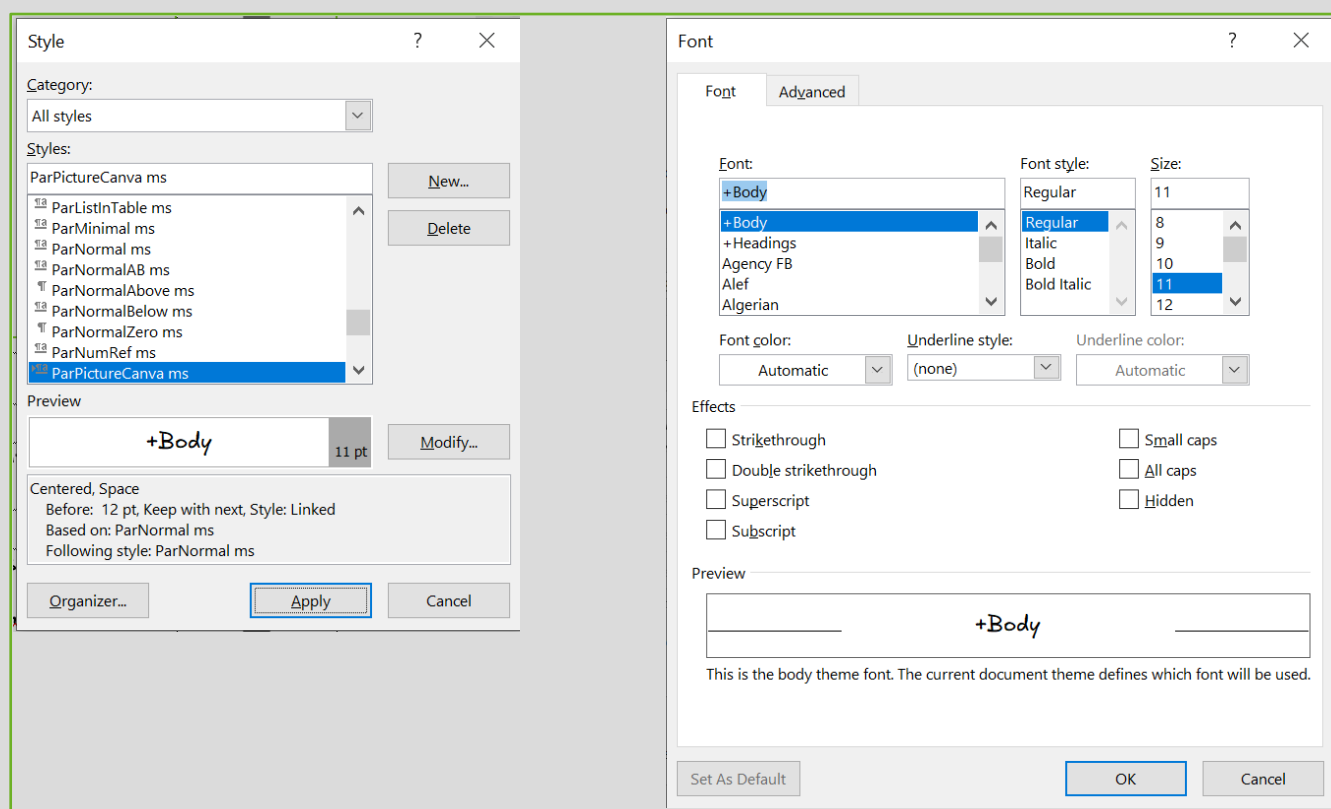
Pic. 44. Theme features.

Tab. 5. Theme_ms.thmx colors.

No.	Color	RGB Hex value	comment
1.	Text/Background - Dark 1	#000000	
2.	Text/Background - Light 1	#FFFFFF	
3.	Text/Background - Dark 2	#44546A	
4.	Text/Background - Light 2	#E7E6E6	
5.	Accent 1	#6DB126	
6.	Accent 2	#51841C	
7.	Accent 3	#C4EA9D	
8.	Accent 4	#FFFFFF	not used
9.	Accent 5	#FFFFFF	not used
10.	Accent 6	#FFFFFF	not used
11.	Hyperlink	#0563C1	
12.	Followed Hyperlink	#954F72	

The fonts selected in your theme can be assigned to styles definitions. Their names (*Heading* and *Body*) can be used in styles definitions, but are not mandatory. Instead of using *Heading* or *Body*, you can specify any other style. Thanks to these two categories, you can add another layer of abstraction to your styles definitions.

Example: style of this paragraph is *ParNormal ms*. If I wish to edit its setting, I get its preview shown as *+Body*. If I wish to change its font, the first two proposals from top are *+Body* and *+Headings*, the same options which are assigned to the theme concept.



Pic. 45. Settings of font related to concept of Theme.

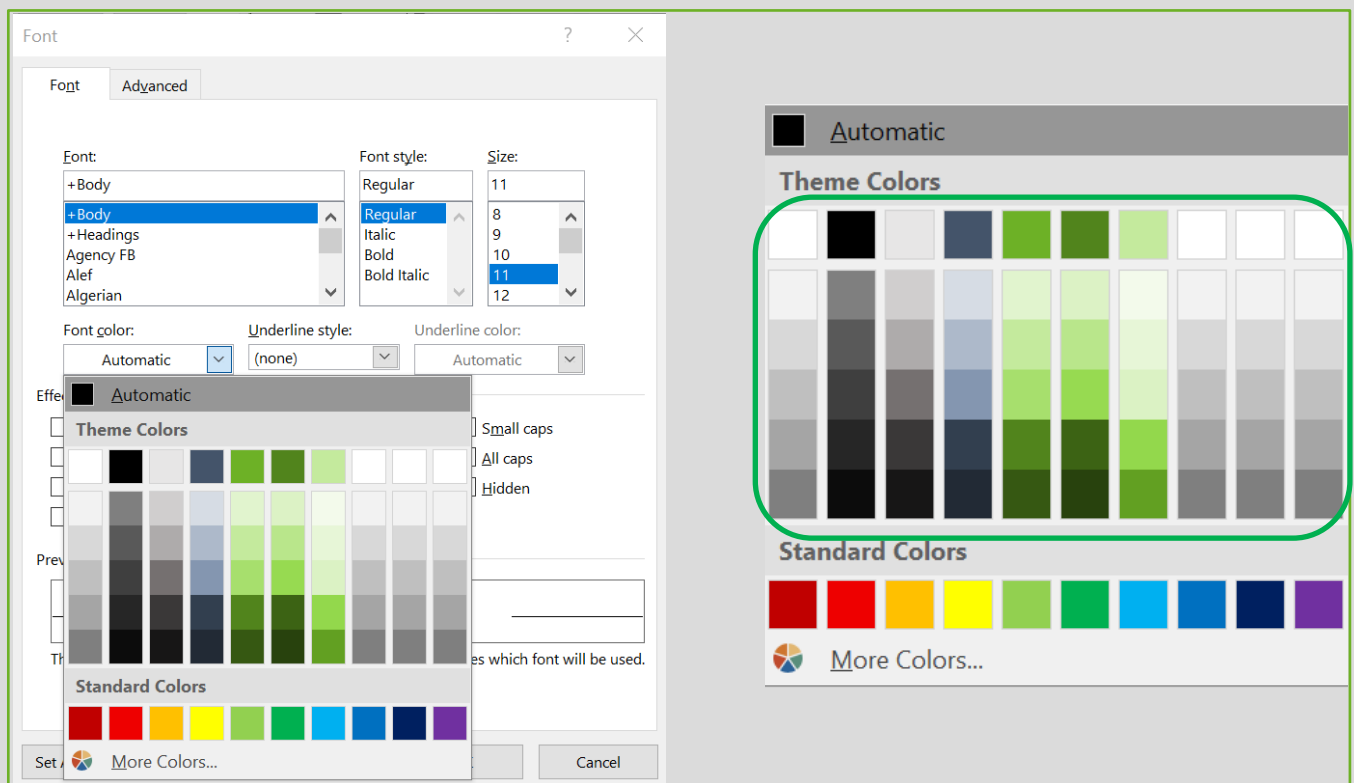
If many or all styles are based on *+Body* and *+Headings* definitions, then if you change your theme, definitions of the styles are adjusted as well. In other words, you can change look and feel of your document in a blink of an eye just by changing your theme.

From my point of view the main benefit coming from application of theme concept are colors. To explain it better, please see specific choices made in design of this document. For example, tables use specific color pattern: header of the table is a bit other shade of green then each second row of the table. Those colors come from my theme.

No.	column 1	column 2	column 3
1.			
2.			
3.			

Pic. 46. Colors of a table.

Thanks to that they are present in theme color palette. For example, if I wish to change color of the font (left picture below), the matrix displayed as *Theme Colors* comes from definition of the colors in my theme. In other words, it is easier to select one of those pre-defined colors, they have a priority (right picture, below). It comes handy more often, then I thought.



Pic. 47. Example color selection.

Bottom line question is, is it worth to keep a separate file to store those settings? In my opinion yes, it does. Thanks to that you can share these settings with others.

For purpose of this book I've designed and defined the theme file *Theme_ms.thmx*.

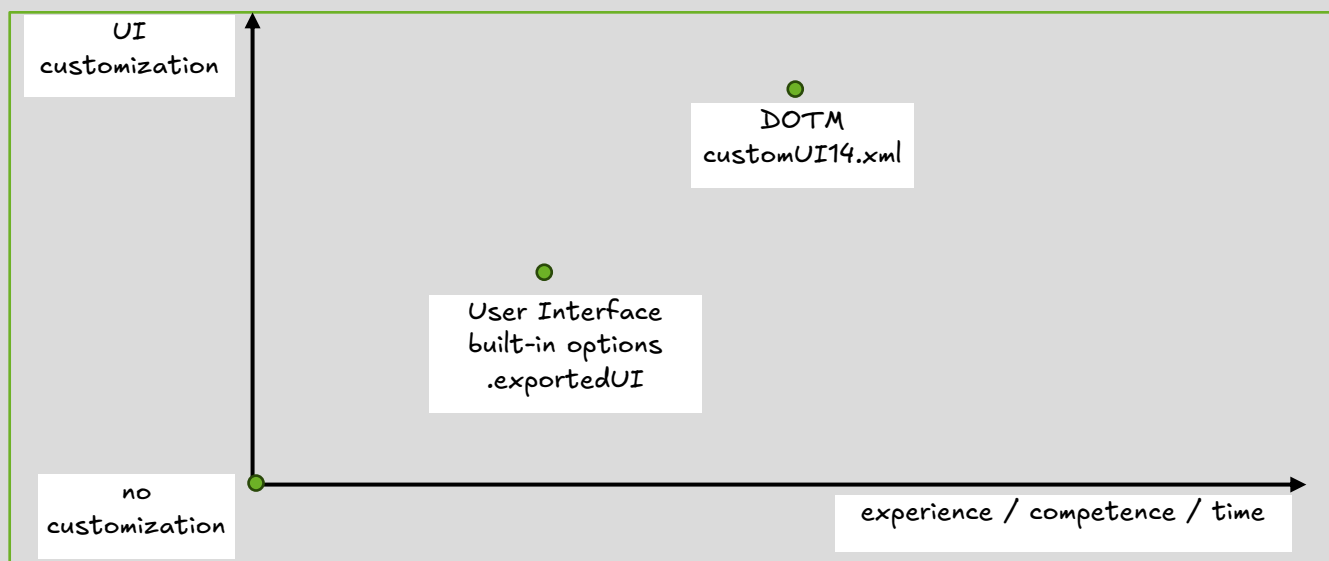
To see newly defined theme file you need to save it in specific location:

C:\Users\\AppData\Roaming\Microsoft\Templates\Document Themes

What can be confusing is that font itself is not stored in theme file. Instead, just name of the font is stored within that file. Theme files are binary.

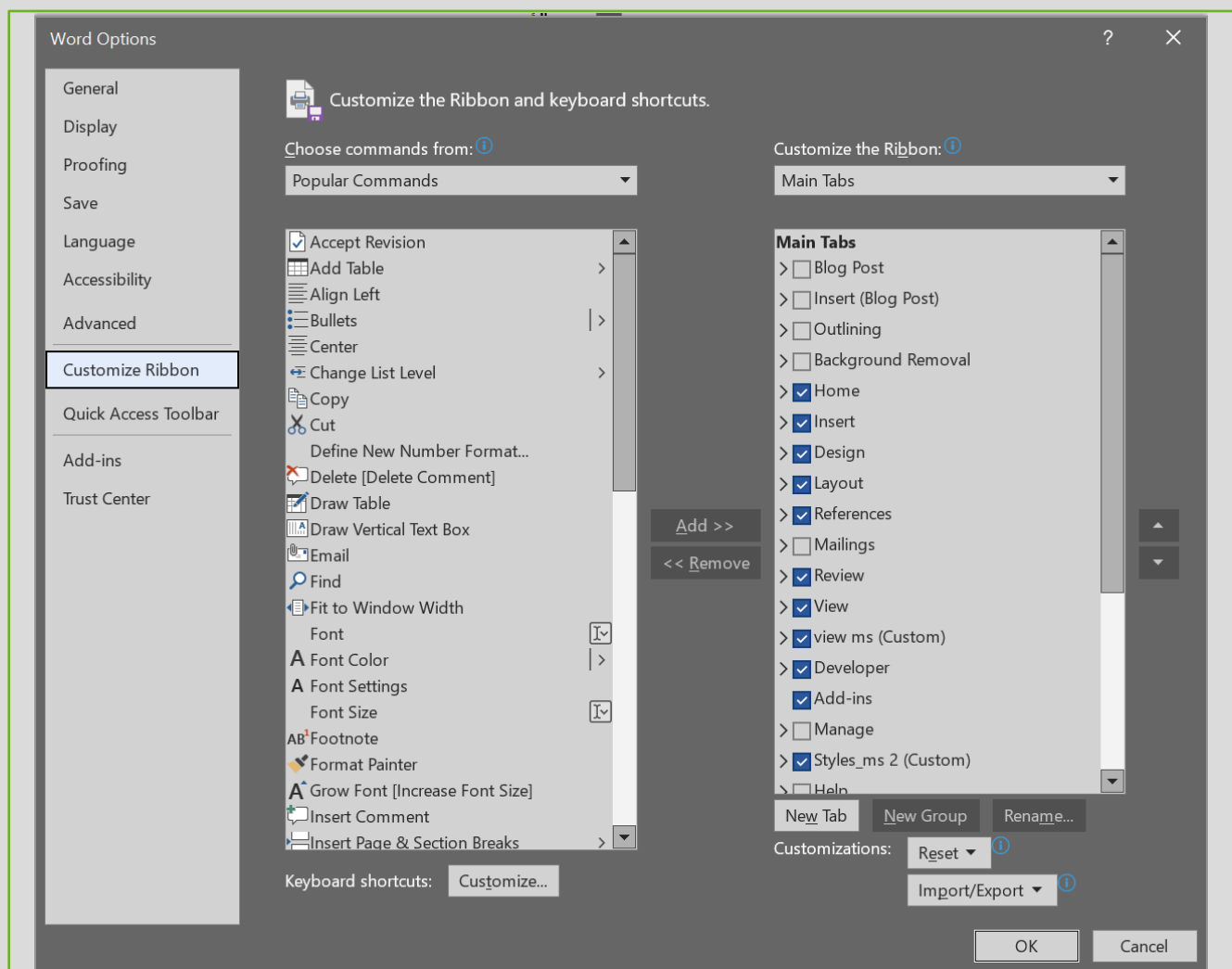
8. What are the menu files?

Menu files (.exportedUI) are text (XML) files which store settings of the Microsoft Word menu, which is often referenced as ribbon. There are available different levels of User Interface customization in Microsoft Word.



Pic. 48. Microsoft Word User Interface customization levels.

User Interface of Microsoft Word comes with quick and handy way to alter ribbon menu to your liking: *File > Options > Customize Ribbon*.



Pic. 49. Customize Ribbon.

Feel free to experiment here. You can restore initial settings with *Reset* button. In bottom right corner there is *Import/Export* button, which makes possible saving your settings within a file (.exportedUI). Thanks to that you can share your settings with others.

My advices:

- » Leave default tabs of Microsoft Word as they are. If you wish to change something in existing menus, copy them and name them to distinguish from original just by name, e.g. *Home > MyHome*.
- » Import or export should be a separate action from editing. For example, if you edit your menus, click *OK* when you are done. Then enter *Customization Ribbon* again and just use *Import/Export* button.
- » From my experience the best option available within this window is assignment of custom-made macros to custom made tab, group and buttons.

Soon you will discover some limitations of this level of customization:

- » limited number of icons,
- » no way to define your own icons
- » no way to display screen tips informing user about nature of buttons
- » only buttons can be added this way

If you wish to overcome those limitations, you need to learn about editing of your add-in files (DOTM), to prepare customUI4.xml. In other words, solution exists, see chapter [12.5](#).

Going back to menu files, they are quick and easy way to customize your environment. I strongly recommend you to experiment with them. It is low hanging fruit which enable you sharing your customization with others.

I store them personally in

c:\Users\<UserName>\AppData\Roaming\Microsoft\Word\STARTUP\

9. What are the fonts files?

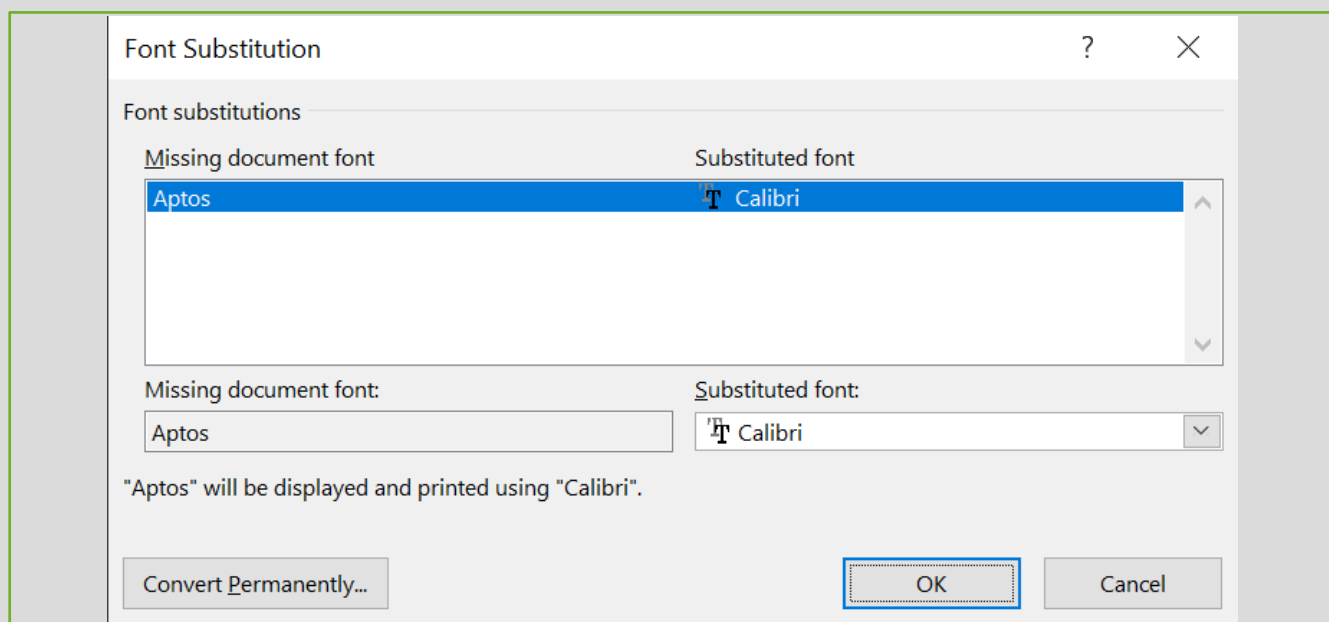
Font files are definitions of fonts. Fonts define look and feel of your documents.

Relationship between fonts and how Microsoft Word displays documents is not so obvious. By default, operating system, such as Microsoft Windows, comes with predefined set of fonts. Additional fonts can be installed without knowledge of computer user e.g. on time of different applications installation. It is possible that set of fonts installed on your computer differs from set of fonts installed on computer of your customer.

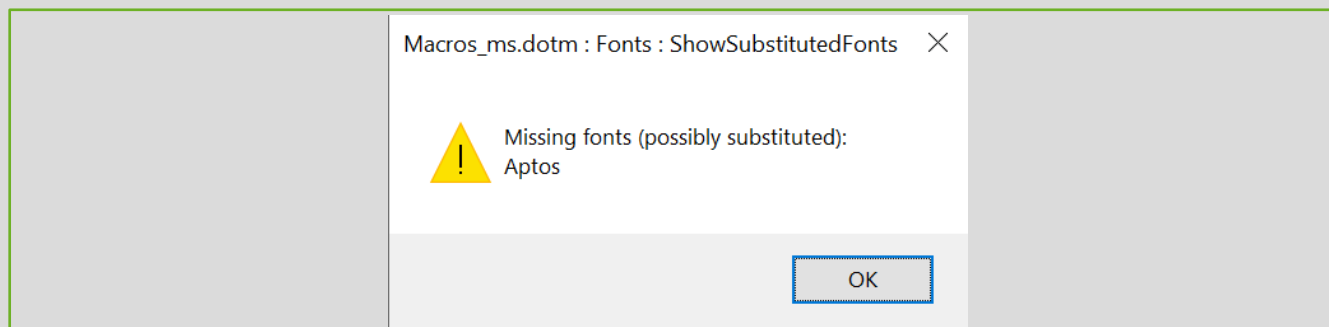
What happens, if you then wish to share your Microsoft Word content file with applied font which is not present on computer of your customer? Such font will be silently, without informing user, substituted by Microsoft Word. With all consequences, like shifted paragraphs, inconsistent view. Lesson here is it is better to take control and consciously deal with it.

Site note. Microsoft Word is known as WYSIWYG tool (What You See Is What You Get). This long acronym represents idea that it shows on the screen content exactly in the same way as it will be printed out on paper. Well, not quite, taking into consideration, that my file on my screen will not look exactly the same as your file on your screen, because of the different sets of fonts.

To discover this silent font substitution you will need to either look in *File > Options > Advanced > Show document content > Font Substitution...* button, or apply a macro, which will warn you about font substitution when document is opened for the first time. The [Pic. 50](#) shows example of result after *Font substitution...* button was pressed. The [Pic. 51](#) shows example of result when file was opened and my macro *ShowSubstitutedFonts* was active.



Pic. 50. Font substitution.



Pic. 51. Macro *ShowSubstitutedFonts*.

There is a workaround to this issue, which is named font embedding. As it was already sad, Microsoft Word files work as containers. You may put in it just another box, your specific fonts. From one hand, the specific font files aren't necessary to work with content files, from the other hand, if you wish to exactly represent the content, you need to embed them.

This chapter is dedicated to font files. We will dive deeper into configuration of Microsoft Word related to fonts in one of the next chapters, [11.6](#). Now notice where font files can be found:

- » system fonts:
C:\Windows\Fonts
- » user specific fonts:
C:\Users\<UserName>\AppData\Local\Microsoft\Windows\Fonts

Please also notice, that their size may vary, in my case from 35 kB up to 21 MB. Imagine that embedding few font files in your content file may increase its volume significantly, what can make sharing of such file cumbersome.

10. What are the content files DOCX and DOCM?

Again, let me use the same analogy which is valid for DOTM or DOTX files. From my perspective I imagine them are like [containers](#) or even [intermodal containers](#) in abstract form. They are like big boxes for holding different small boxes aka categories of information inside.

The technical difference between *template* or *add-in* files (DOTM or DOTX) and *content files* (DOCX or DOCM) is that the latter cannot be "run": user cannot double click them (or press enter upon filename in file manager) and thanks to that new file (child), by default named *Document1* will be created, with the same suite of styles as in parent file.

10.1. What kind of information categories may be stored within?

Please see in [Fig. 52](#) the information categories which may be stored within a *content file*.

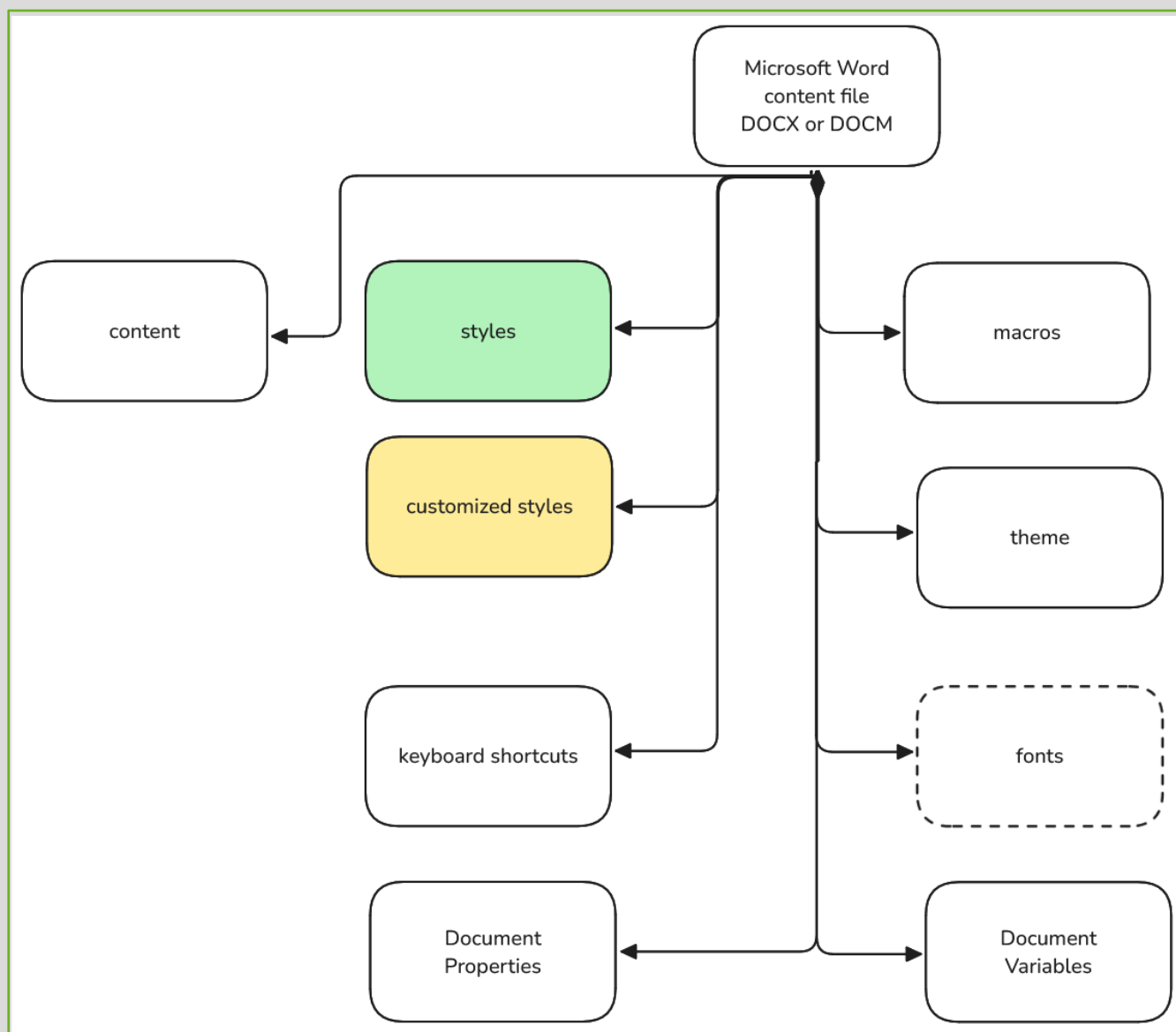


Fig. 52. Information categories, content file.

Remarks:

- » The green color represents default and mandatory information category, *styles* from *Normal.dotm*.
- » The yellow color represents optional customized styles.

- » Macros can be stored only in DOCM files. The type of content file must be changed manually from DOCX to DOCM to save macros in a content file. Such files are potentially a security threat, so security settings can be important in such cases. I don't recommend storing macros within content files.
- » Fonts are stored within content files only on purpose. It can be necessary if you want to copy your content file to another computer which does not have access to your set of fonts. Embedding fonts was discussed in the chapter [11.6.1](#).
- » Keyboard shortcuts. I store them always within content files.

10.2. Recommended information categories

I do not recommend to store macros within DOCM content files. This is potential security risk.

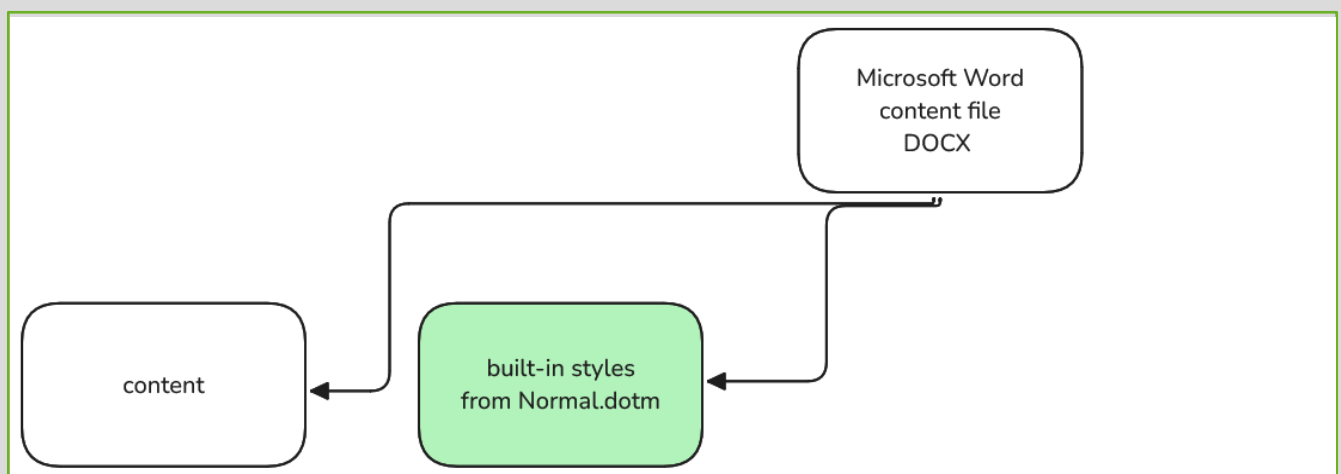
Imagine example content file *Document1* in two phases:

1. Before running macros from *Macros.ms.dotm*.
Such file contains only *content* and *built-in styles* from *Normal.dotm*.
2. After running macros from *Macros.ms.dotm*.
All the rest of information categories, including customized styles.

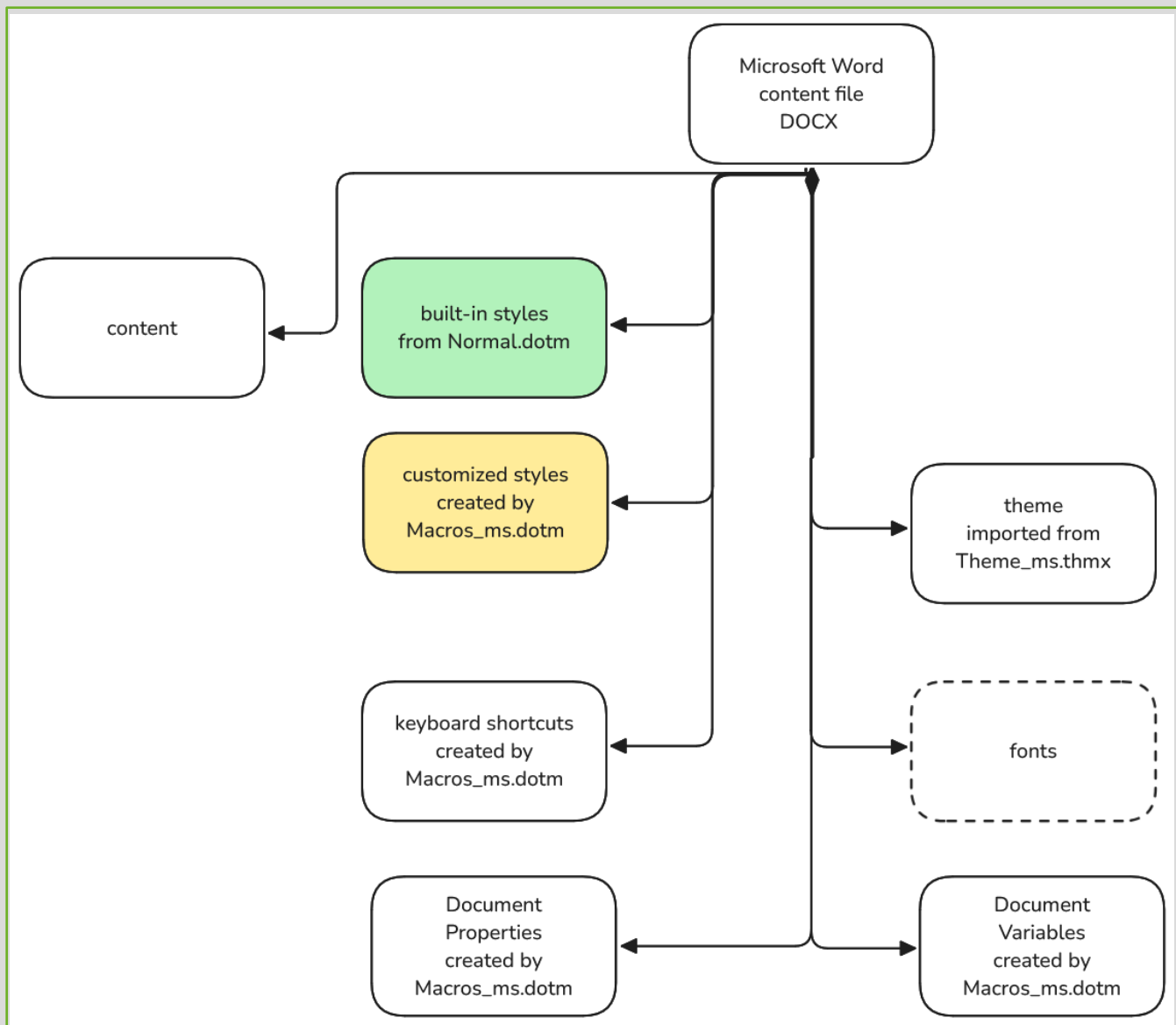
Remarks:

- » The green color represents default and mandatory information category from *Normal.dotm*.
- » The yellow color represents customized styles from *Macros.ms.dotm*.
- » Vast majority of categories are created or imported from external files: *Macros.ms.dotm* and *Theme.ms.thmx*.
- » Fonts are optional and most of the time are not stored within a content file. Embedding fonts is discussed in the chapter [11.6.1](#).

As one can see there is much more than just content itself within such a file.



Pic. 53. Before running macros from *Macros.ms.dotm*.



Pic. 54. After running macros from *Macros_ms.dotm*.

11. Information categories. Working with Microsoft Word

In this chapter I'll try to describe how I work with prepared set of macros. Some chapters refer to information categories, introduced in the previous chapter.

11.1. Microsoft Word workflows

There are two workflows:

1. content edition,
2. content formatting.

There is never ending debate about which workflow is better for content creation, and how is one obstructing the other etc. [5]. For many users on time of content edition, it is good to keep content unobstructed with any unprinted characters, sizes, color of page background etc. Opposite to that on time of content formatting it is good to see every supporting character or text processor feature which could support process of content formatting.

To make it possible I've prepared macro *ToggleSpecificFormatting* in module *Tools* which is by default connected to F4 keyboard key. It works as a toggle between the following modes:

1. Visible all unprinted characters (formatting marks), hidden text, field codes, bookmarks, text boundaries, crop marks.
2. Background color toggled to grey color (RGB = 219, 219, 219), which is much easier to my eyes than just white.
3. Invisible all printed characters (formatting marks), hidden text, field codes, bookmarks, text boundaries, crop marks.
4. Background color toggled to default one (no color = white).

11.2. Keyboard shortcuts

Keyboard shortcuts, especially connected with the possibility to apply styles, make Microsoft Word still unparalleled tool in terms of speed content formatting. In this chapter I did my best to convince you to apply shortcuts in your everyday practice. Generally working with mouse is one of the most important factors in terms of efficiency: those of us who most of the time work with mouses are much less efficient.

11.2.1. What are the keyboard shortcuts?

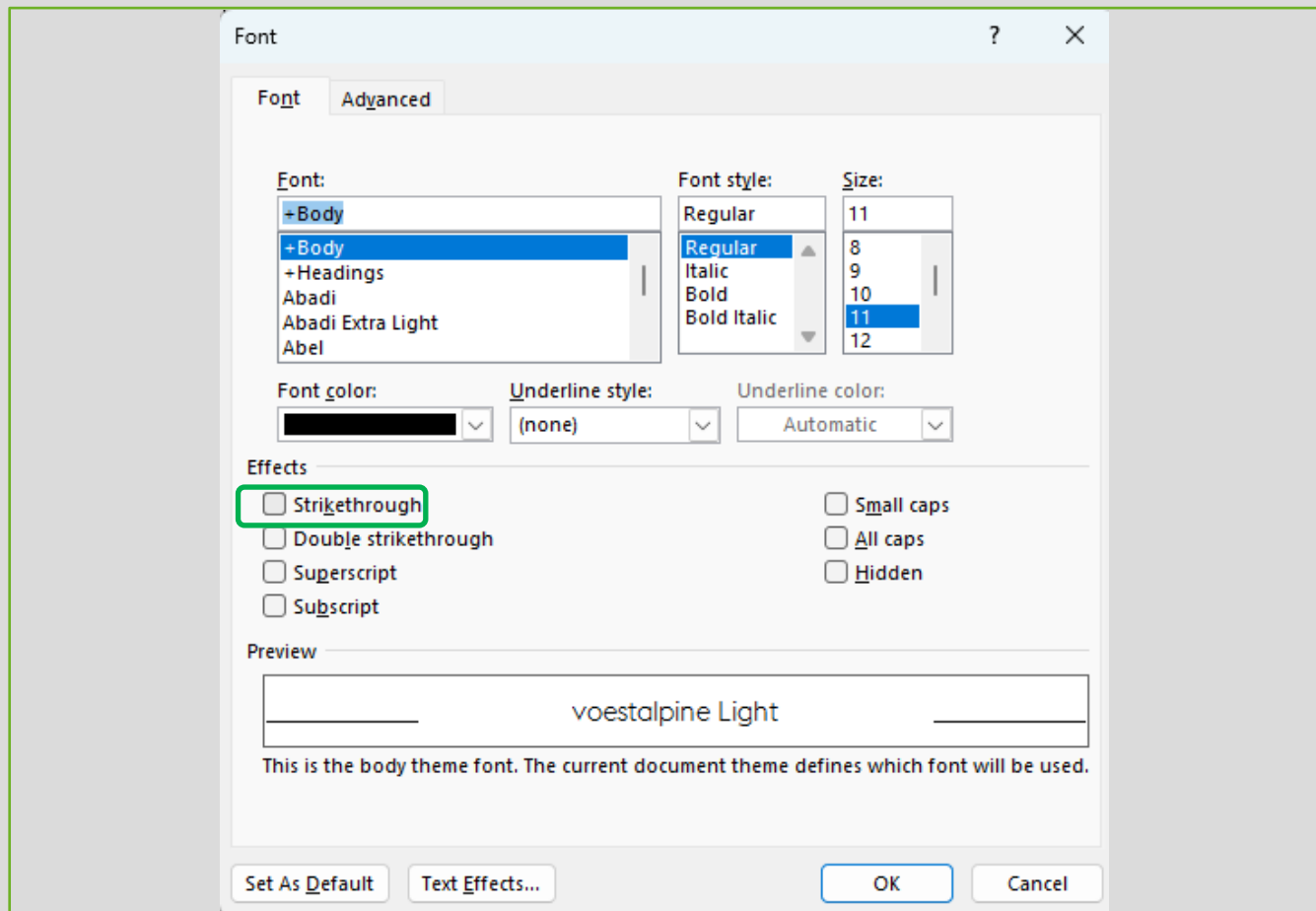
Computer keyboard is used most of the time to type characters serially. For comparison a music keyboard is used to press its keys in parallel, concurrently. This principle stays behind a concept of music chords or sound combinations. Combination of specific sounds make listening to music more enjoyable, contrary to serial sounds.

Also, computer keyboard let us to enter characters in parallel. By analogue also concurrently pressed characters sometimes are referred as chords, but most of the time as computer (keyboard) shortcuts or hotkeys. You can imagine that pressing of a shortcut could produce a pleasant sound.

11.2.2. Why keyboard shortcuts in Microsoft Word?

Microsoft Word could be seen as a huge container of many functions. Only small fractions of those functions are available easily in the User Interface. It is true that User Interface is configurable to some degree, but still finding and applying some functions with mouse navi-

gation is time consuming. The alternative approach is to define keyboard shortcuts at least for some most time-consuming functions. The idea of shortcuts is based on reusability of existing keys and combining them to call specific functions.



Pic. 55. Text strikethrough.

My favorite example is striking through text. Personally, I find it very useful to strike through edited content, without removing it. It mimics natural edition habits acquired on time of working with pen and paper. How many time one need to click with a mouse to strike through a specific word in a sentence?

Let's test:

1. Rich with a mouse specific word in a specific sentence.
2. Select it, by pressing down left mouse key and drag.
3. Click in the Microsoft Word menu at *Home*.
4. Click in a *Font* group bottom right corner of ribbon section: ↘. (It is hard to precisely click such a small icon, isn't it?)
5. Click the *Strikethrough* effect.
6. Click *OK*.

The same task, if accomplished with keyboard:

1. Rich specific word in a specific sentence (e.g. Ctrl + → / ↑ / ↓ / ←) × repeated.
2. Select it (e.g. Ctrl + Shift + → or Ctrl + Shift + ←).
3. Call a toggle macro: Ctrl + Shift + X.

What would be easier and quicker?

11.2.3. Is there an alternative approach?

There is an alternative approach. Imagine that each required function is assigned to a dedicated keyboard key. Usually, desktop keyboard has around 100 keys. How many keys should then be reserved to Microsoft Word? In my personal case about 70. There is a good chance that your office keyboard already has 12 keys ready to be assigned to special purposes. These are so called "f-keys" or "function keys", e.g. F1 ... F12. But what could I do with the rest of 58 keys if not apply the shortcuts?

Let's compare our working desk to an airplane cockpit. The idea "one key, one function" was met there. Pilot has all over her / his body access to keys, switches and other manipulators. Just to let her / him for immediate access to any desired function.



Pic. 56. Airplane cockpit [6].

Can we organize work in a similar matter and add e.g. dedicated keyboard just to call keyboard shortcuts and macros? The answer is yes but this is a subject outside of this book. You can do it yourself with relatively low cost and effort. In the provided example I use two keyboard: primary one, for the purpose of entering content and secondary one, 'macro keyboard', just to have immediate access to some functions.



Pic. 57. Microsoft Word oriented macro keyboard made by author of this book [7].

11.2.4. What are advantages and disadvantages of keyboard shortcuts?

Tab. 6. Keyboard shortcuts advantages and disadvantages.

advantages	disadvantages	comment
easier access to specific functions	need to memorize shortcuts to really push the limits	dedicated print-outs or even keeping windows containing shortcuts on a screen
quicker access to specific functions	misses could run undesired functions	undo function

11.2.5. How to start working with shortcuts

I have created plenty of keyboard shortcuts, which are stored in content files (DOCX). In other words, they are not active until not added to specific file.

There are 3x categories of shortcuts in Microsoft Word, dedicated to specific:

- » styles
- » macros
- » building blocks

Those dedicated to specific styles become active after they are added to content file (DOCX) by macro: menu > *Styles_ms* > *Manage 'ms' styles* > *Add 'ms' styles*.

Those dedicated to specific macros become active after shortcuts are added to content file (DOCX) by macro: menu > *Shortcuts_ms* > *Add* > *Add Macro Shortcuts*.

Those dedicated to specific building blocks are actually... hotstrings. Shortcuts are sometimes referred to as hotkeys. The wider, but related concept, is calling sequences of actions upon entering sequence of characters. Sequences of characters are often referred in IT as strings, so by analogue the full name of such a sequence would be a hotstring. Hotstrings are named in Microsoft Word as *AutoText*. *AutoText* can be combined with building blocks. Specific set of *AutoText* hotstrings are active if *BB_ms.dotx* is active as add-in.

Let's try. Open any newly created DOCX file and from ribbon menu select

- » *Styles_ms* > *Manage 'ms' styles* > *Add 'ms' styles*
to enable specific styles shortcuts and add them to document
- » *Shortcuts_ms* > *Add* > *Add Macro Shortcuts*
to enable specific macro shortcuts and add them to document

11.2.6. What have I done to make our life easier?

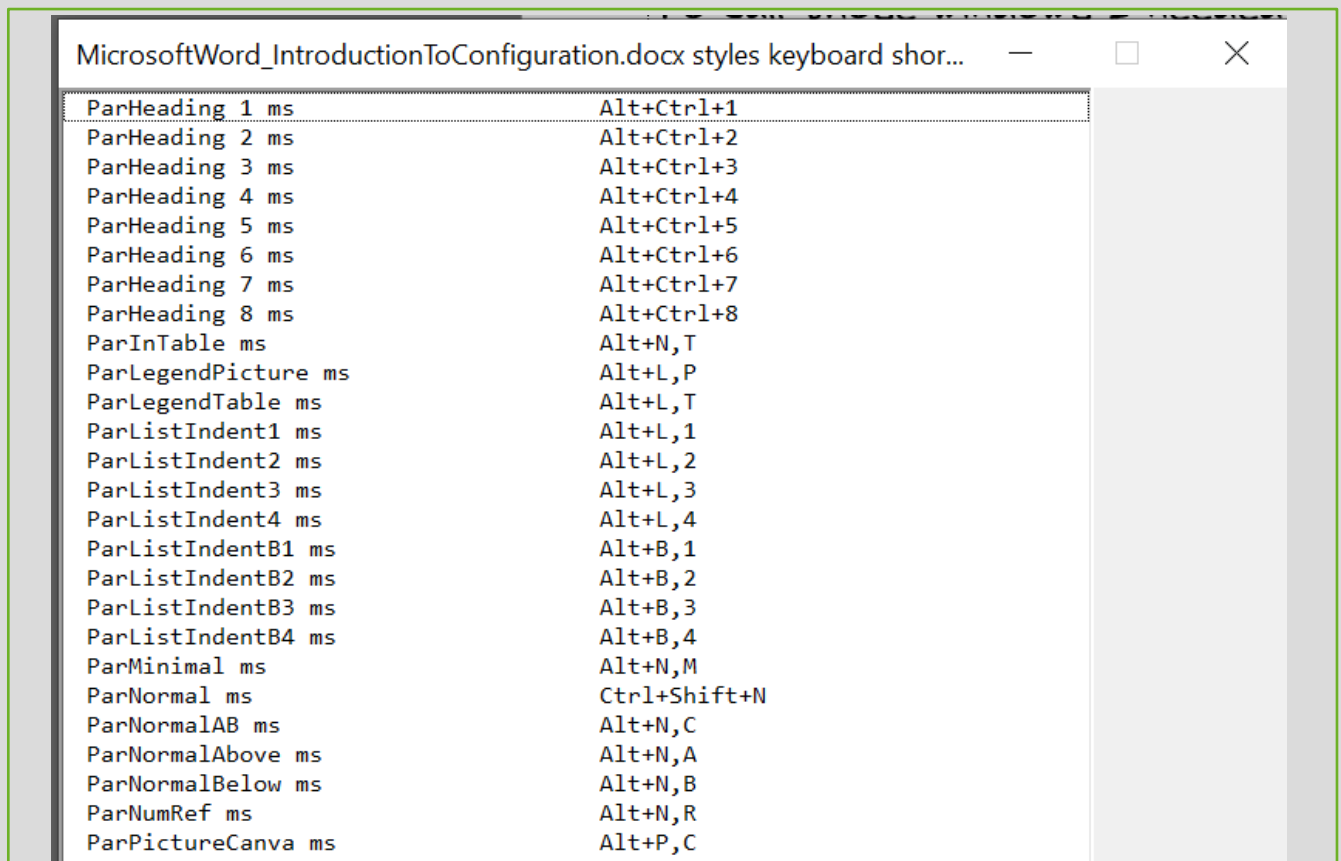
I decided to reinforce my memory with dedicated small windows, which show me keyboard shortcuts defined within my configuration files directly on my screen. Most of the time I work nowadays with at least 3x screens: laptop (14") + 2x office size (24" and 32"). Laptop screen is much smaller in size and sits a bit aside of two main screens. It makes almost perfect arrangement to keep windows with lists of keyboard shortcuts so close to my working place.

To call those windows I needed memorize just 3x keyboard shortcuts

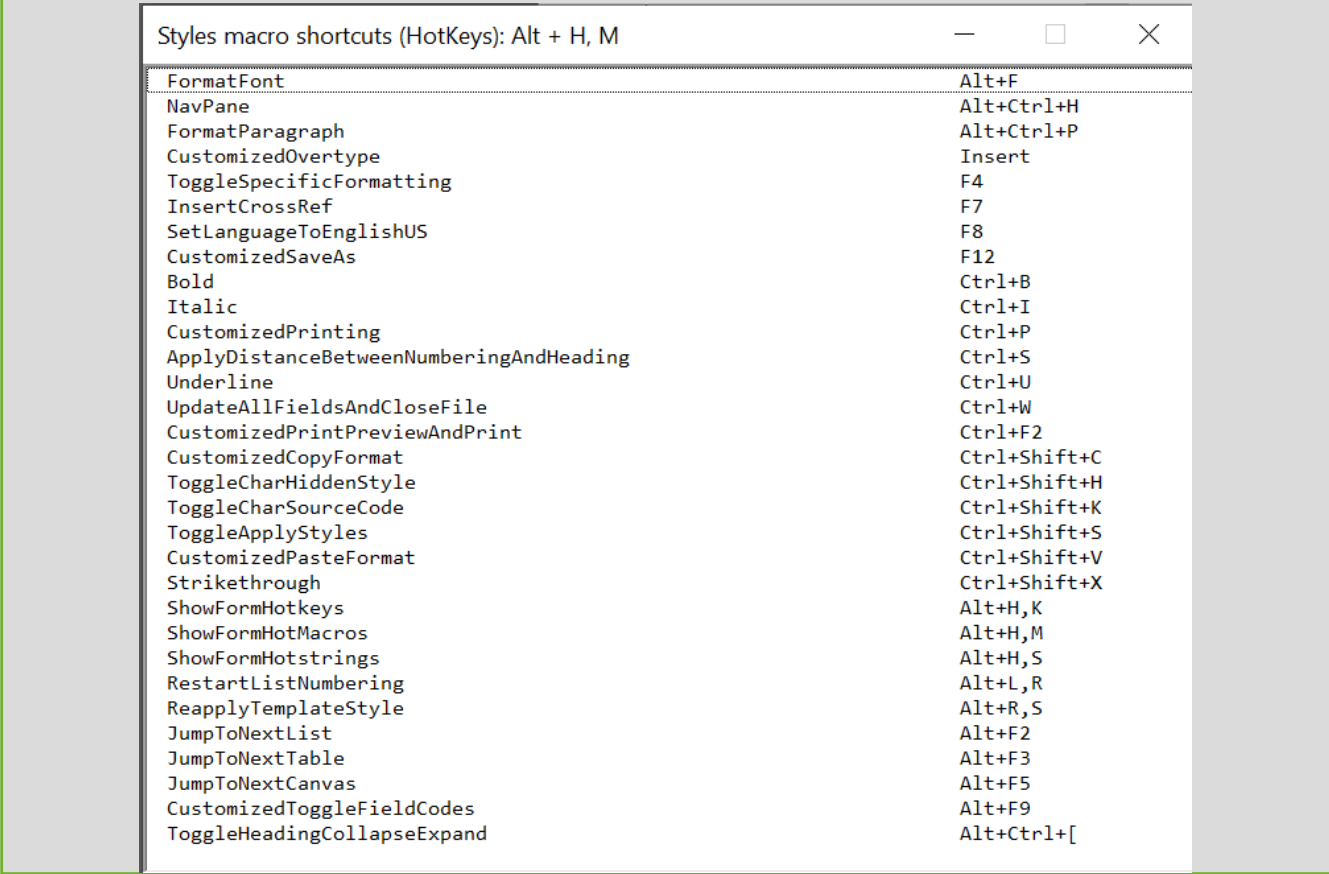
- » Alt + H, K for style shortcuts: Hot + Key
press down left Alt key and keep it pressed,
press down and release H key,
release left Alt key
press down and release K key

- » Alt + H, M for macro shortcuts: Hot + Macros
press down left Alt key and keep it pressed,
press down and release H key,
release left Alt key
press down and release M key
- » Alt + H, B for building blocks: Hot + Building Block
press down left Alt key and keep it pressed,
press down and release H key,
release left Alt key
press down and release B key

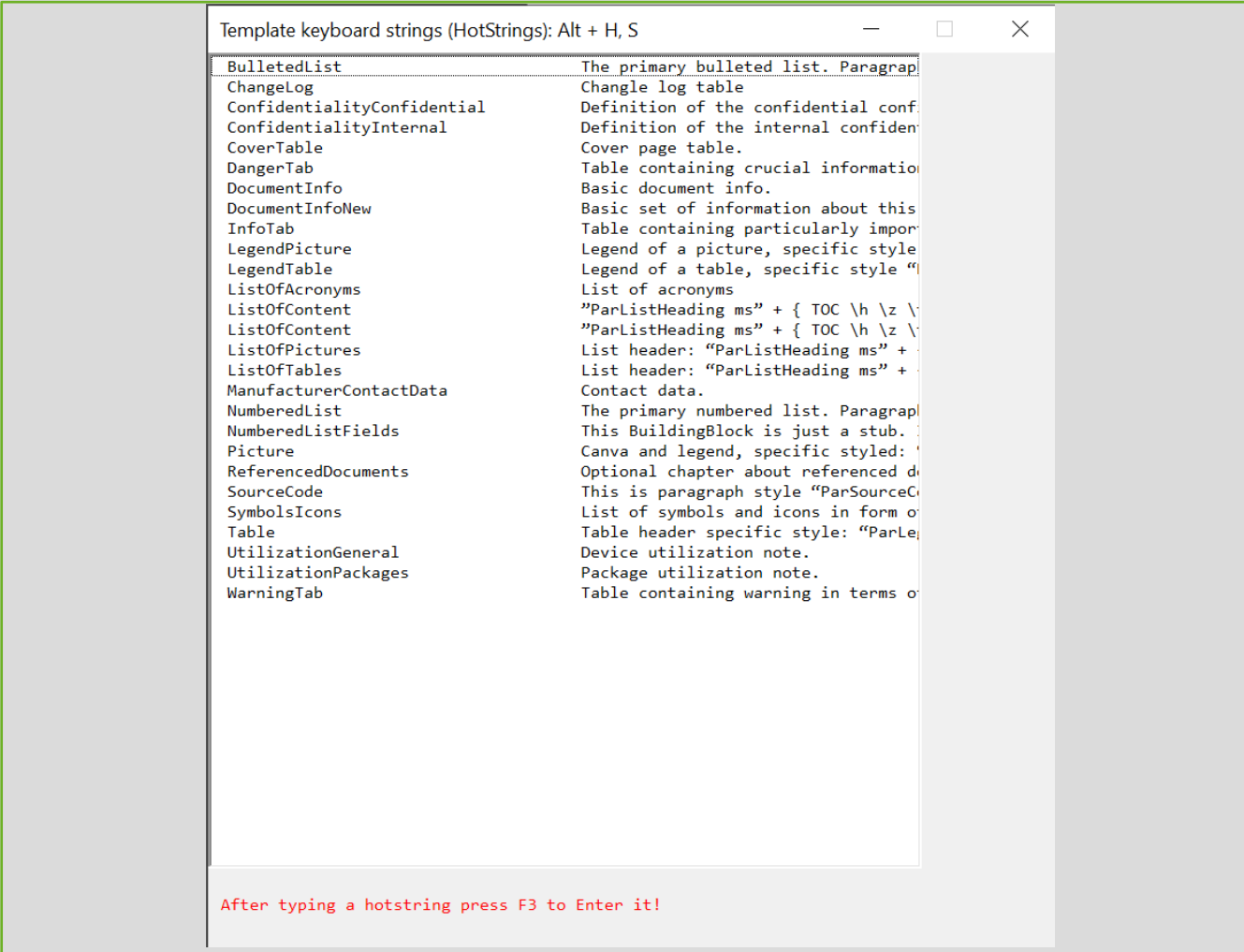
The macros can be also called from ribbon menu: *Shortcuts_ms > Show > Show windows.*



Pic. 58. Style hotkeys, Alt + H, K.



Pic. 59. Macro hotkeys.



Pic. 60. Building blocks *AutoText* or *Hotstrings*.

Instead of keeping occupied space on one of your screen you can printout the lists by selecting menu *Shortcuts_ms > List > Output content to .txt* and keep it in convenient place nearby your workplace.

11.3. Working with Content

Working with content is the ultimate goal of any text processor. Starting work with Microsoft Word, a user faces application User Interface. Variety of available functions and choices can be overwhelming. Adding time pressure to that, frequently met at workplaces, and poor training, and we get plenty of DOCX files, containing directly formatted content, what means without application of styles.

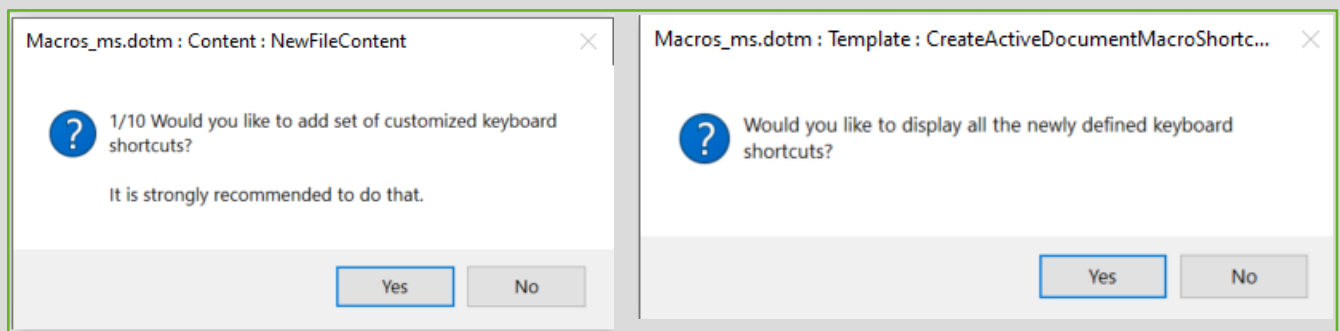
It is all about time and money, which must be invested into configuration of this advanced tool, which is Microsoft Word, to make every day work at least smooth.

In this chapter I want to show, how content creation looks like, if the investment in advanced configuration, was covered. I will start with automatic content creation, where based on previously prepared definitions of *information categories* and *configuration files* content stencil can be generated as a script, which runs one after another, series of procedures. Next, I'll describe chosen steps, which thanks to customized User Interface, make possible preparation of environment, where formatting of content is smooth and easy.

11.3.1. Automatic content creation

Let's start with completely new Microsoft Word document (Ctrl + N, or menu *File > New > Blank document*) and then select from customized menu *Content_ms > Content > New File Content*. You will walk through the 10 steps procedure. Pictures in this chapter, at the left show user oriented question, at the right show feedback when step is finished.

1. Add to newly created file customized keyboard shortcuts.

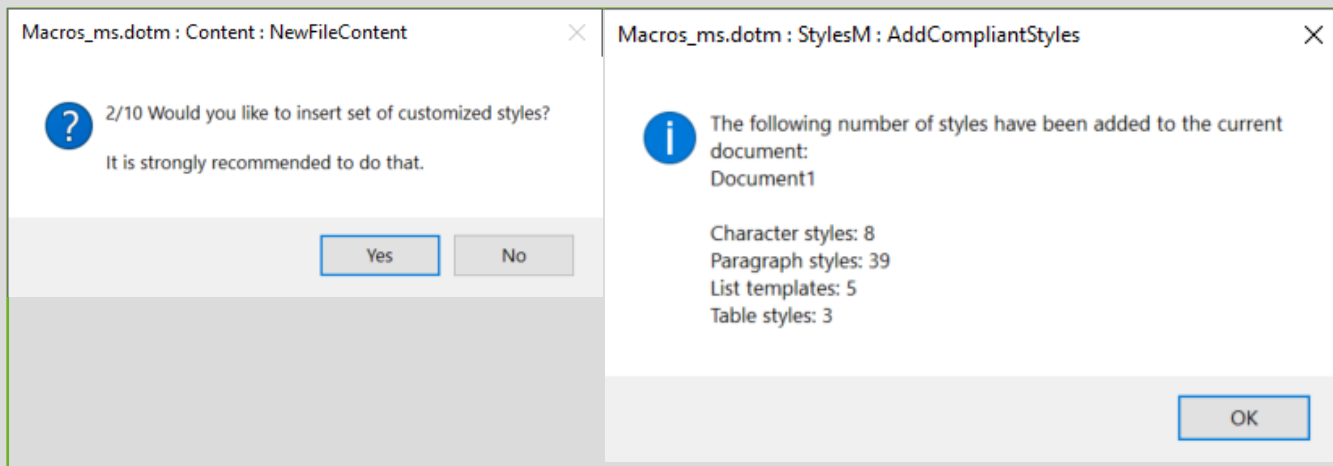


Pic. 61. Step 1: customized keyboard shortcuts.

If user answers positively, as a result she /he gets next question, if she / he wants newly defined keyboard shortcuts to be displayed on a screen in form of separate small window.

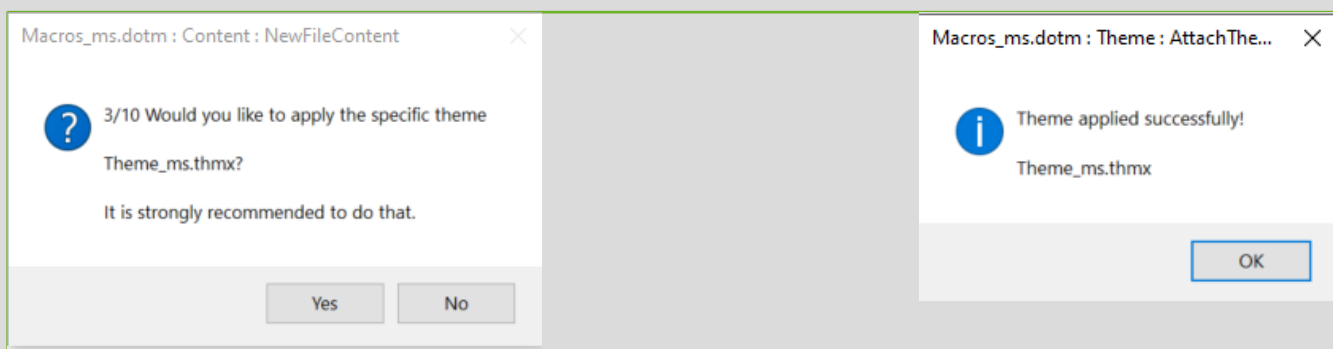
2. Add to newly created file customized / compliant keyboard shortcuts.

If user answers positively, as a result she / he gets summary showing numbers of styles belonging to 4 categories. Actually, to be precise, also 6 built-in styles are also modified (*Normal*, *TOC1*, *TOC2*, *TOC3*, *TOC7*, *TOC8*).



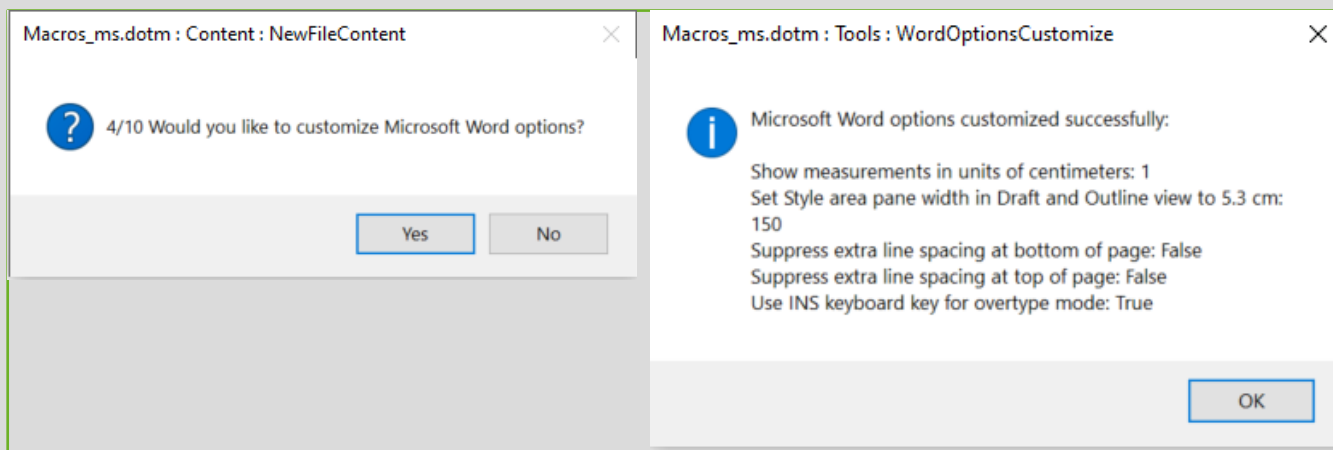
Pic. 62. Step 2: customized styles.

3. Apply theme Theme_ms.thmx



Pic. 63. Step 3: apply theme.

4. Customization of Microsoft Word options.



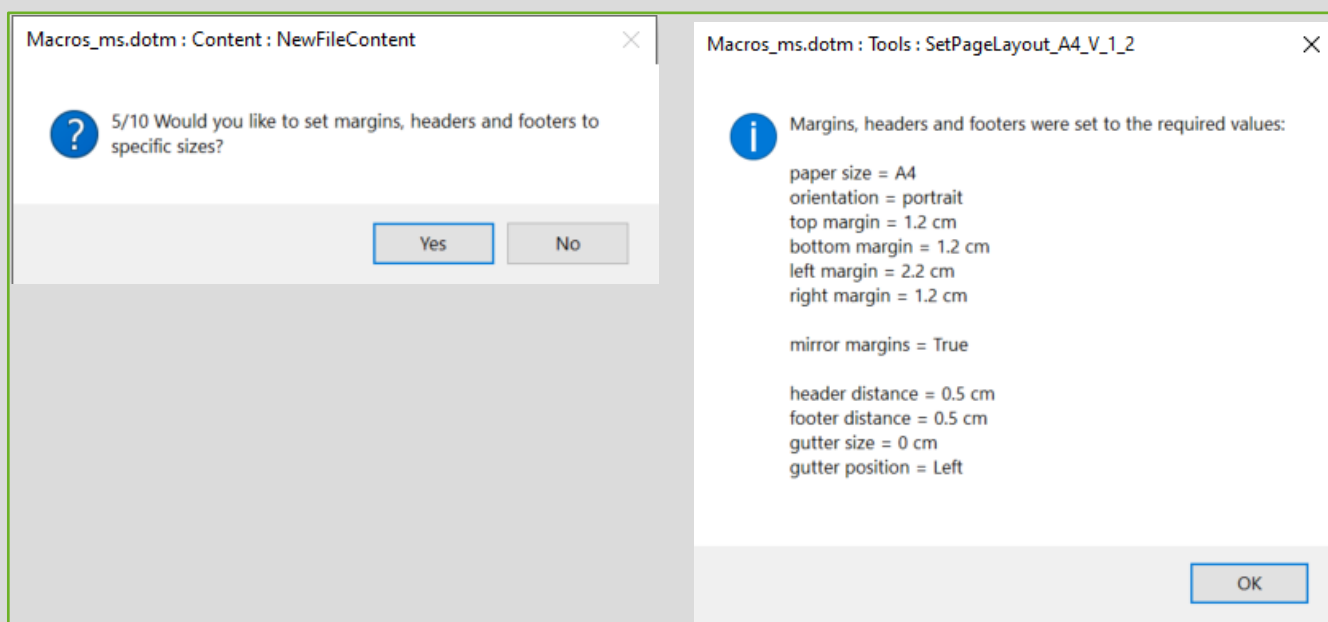
Pic. 64. Step 4: customization of Microsoft Word options.

Please note, that this step is not strongly recommended. These are subjective preferences of author:

- » show measurements in units of centimeters
- » set style area pane width in *Draft and Outline* view to 5.3 cm
- » do not suppress extra line spacing at bottom of page
- » do not suppress extra line spacing at top of page
- » use *INS* (insert) keyboard key for overtype mode

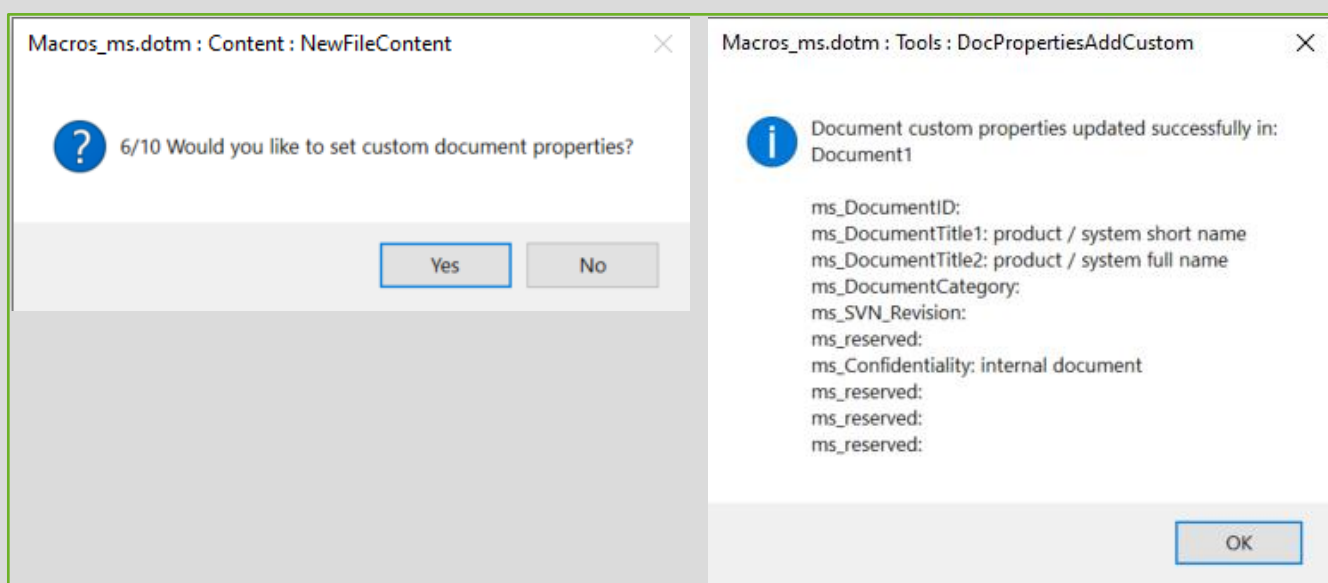
All these settings can be changed also manually in *File > Options > Advanced*.

5. Page layout (A4, portrait, mirrored margin 1.2 cm)



Pic. 65. Step 5: page layout.

6. Custom Document Properties



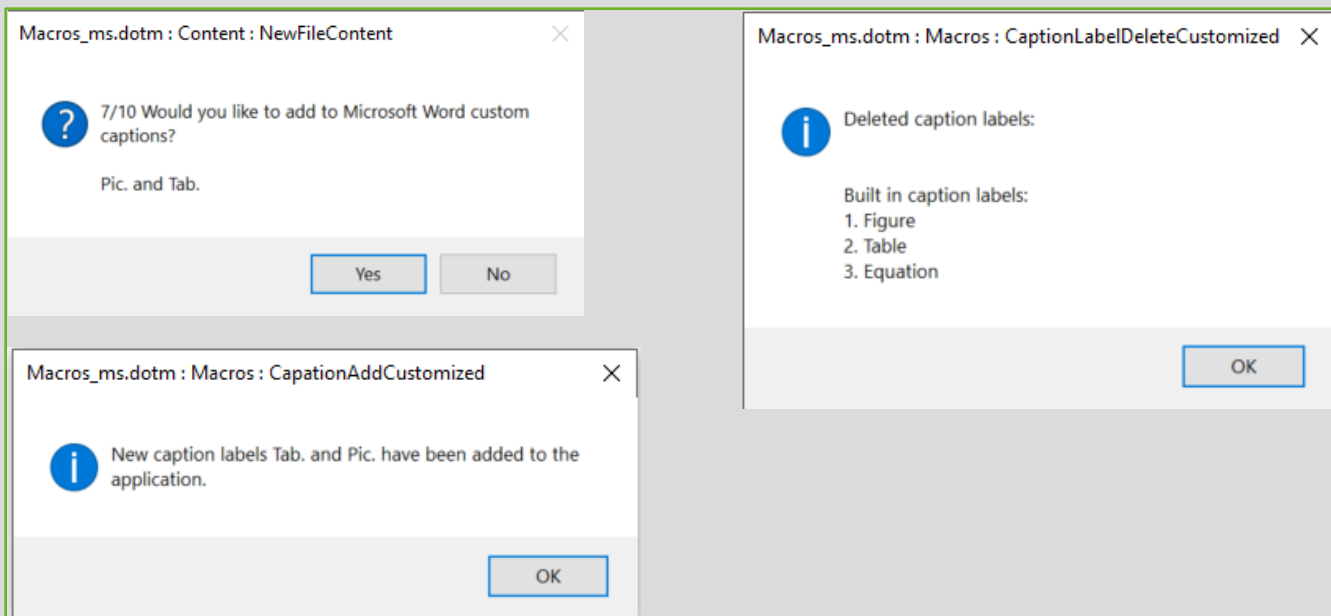
Pic. 66. Step 6: custom Document Properties.

There are in total 10 custom document properties. Out of those 10, the 7 are already named.

7. Custom captions

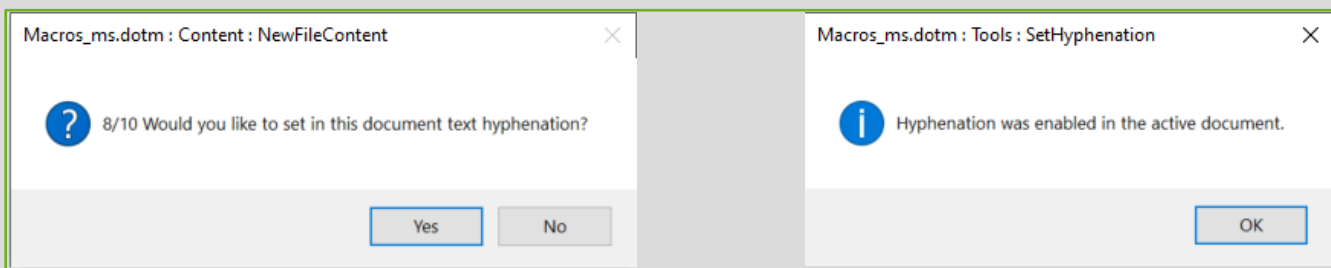
Captions are not stored within document, as one of the *information categories*. Instead, they are stored in *Normal.dotm*. My policy is to not interfere with *Normal.dotm*, if it is not necessary. As a compromise solution I prepared macro, which for every new document asks user if not define, at least temporarily, till this file is opened, the customized captions *Pic.* for pictures and *Tab.* for tables. If user answers positively, then existing customized captions are deleted and new captions are added.

Reminder: I recommend to set *Normal.dotm* as read only file. When you will close Microsoft Word and previously you have run this macro, Microsoft Word will ask you if to save changes to copy of *Normal.dotm*. I recommend you to not save those changes.



Pic. 67. Step 7: Captions.

8. Content hyphenation



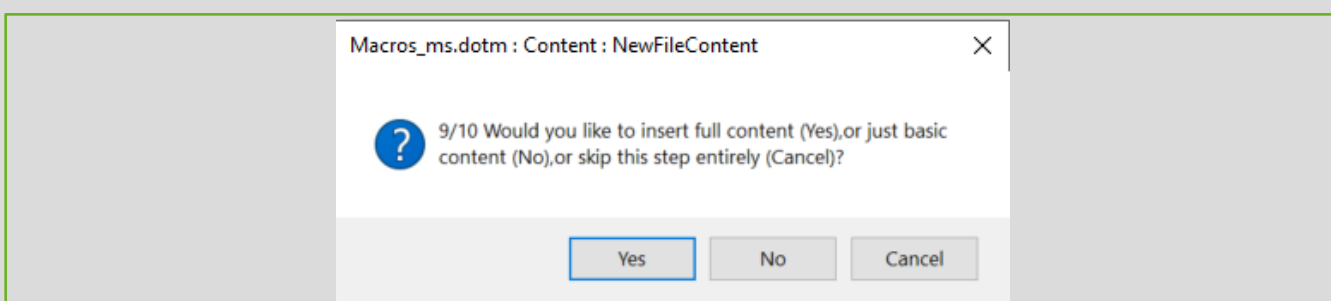
Pic. 68. Step 8: content hyphenation.

Reminder: the vast majority of customized styles ('ms') have already set hyphenation property to true. You can always add manually hyphenation in specific word by keyboard shortcut Ctrl + Alt + - (Optional Hyphen).

9. Insertion of content.

There are two examples of automatic content generation: *full content* and *basic content*.

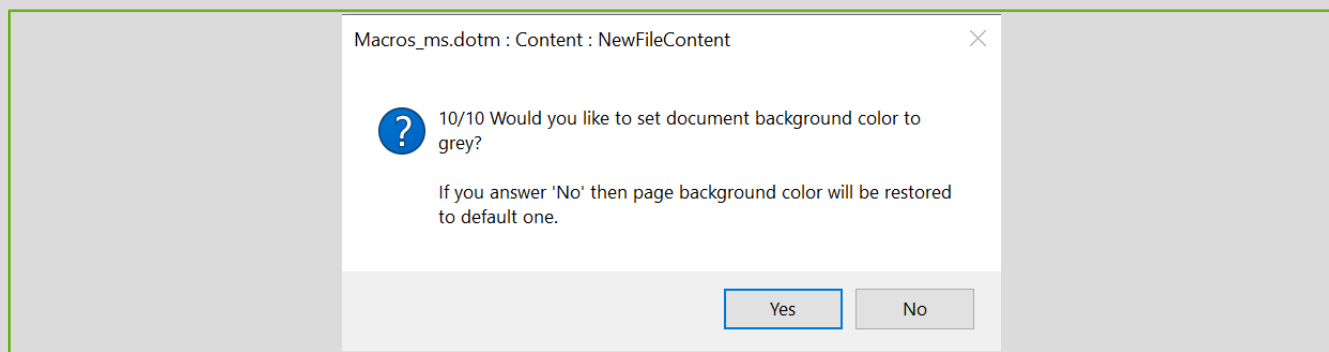
The difference between them is number of pages and building block which are inserted. After insertion of building blocks additional macros are run, which remove unnecessary tags.



Pic. 69. Step 9: Insertion of content.

10. Customized color of document background

After answering to the last question, your newly generated file is yours. Now you can save it under new name.

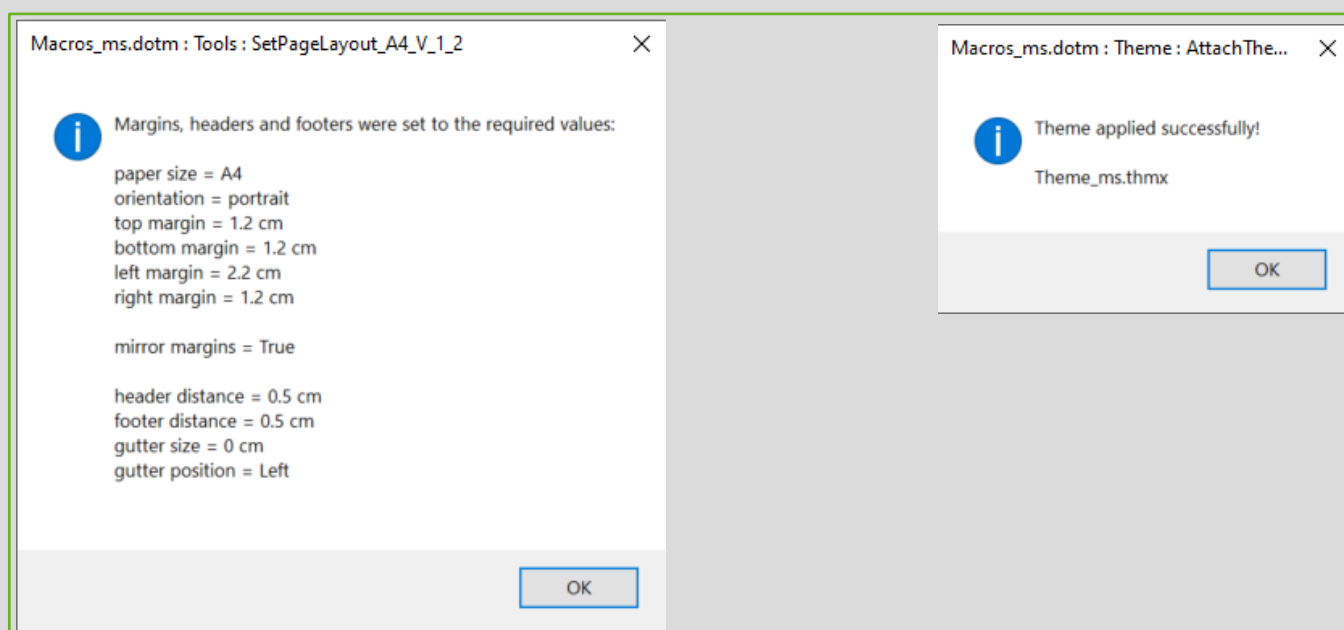


Pic. 70. Step 10: Customized color of document background.

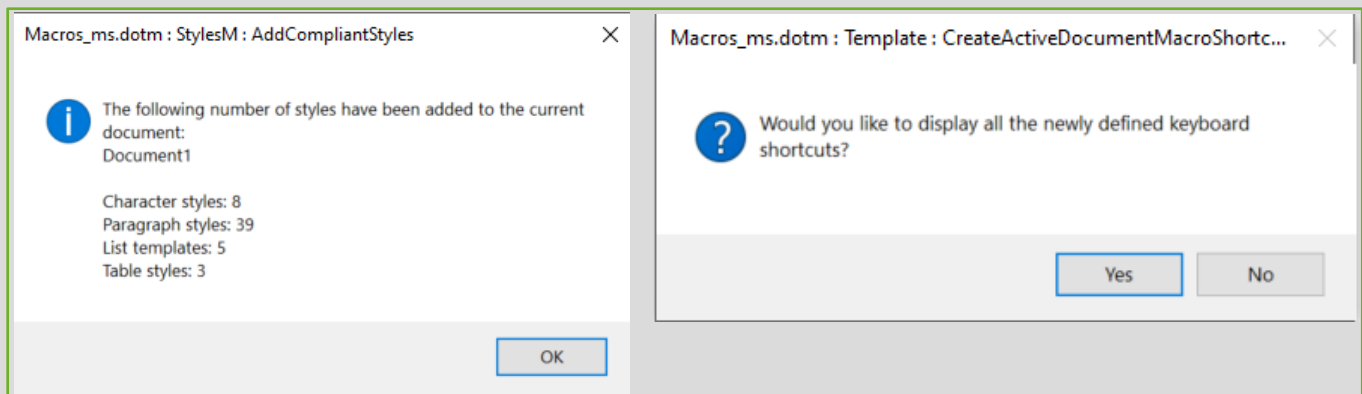
11.3.2. Manual content creation

Let's start with completely new Microsoft Word document (Ctrl + N, or menu *File > New > Blank document*), by following the steps:

1. Set document layout for the only one available section to A4 portrait, mirrored margins, margins set to 1.2 cm:
Menu > Tools_ms > Page Size > Page Size > A4 Portrait, mirrored margins 1.2 cm
2. Attach to a document theme *Theme_ms.thmx*:
Menu > Tools_ms > Theme > Attach Theme
3. Add to a document dedicated 'ms' styles:
Menu > Styles_ms > Manage 'ms' styles > Add 'ms' styles
4. Add to a document dedicated macro shortcuts:
Menu > Shortcuts_ms > Add > Add Macro Shortcuts

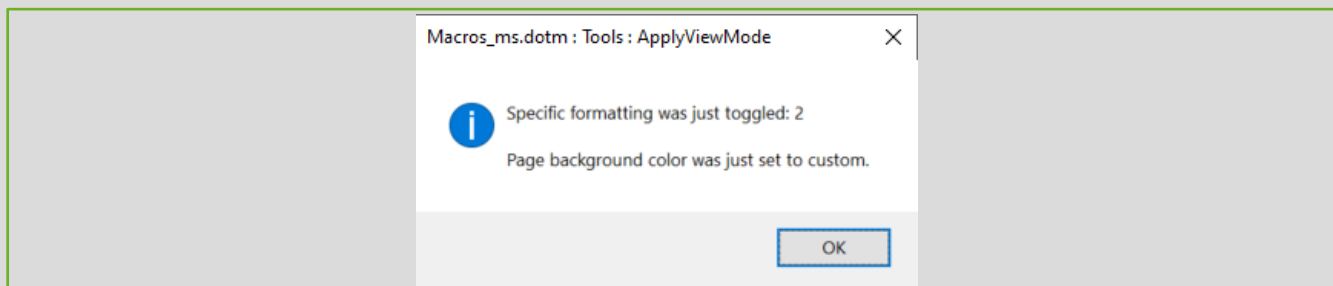


Pic. 71. Step 1 (left), step 2 (right).



Pic. 72. Step 3 (left), step 4 (right).

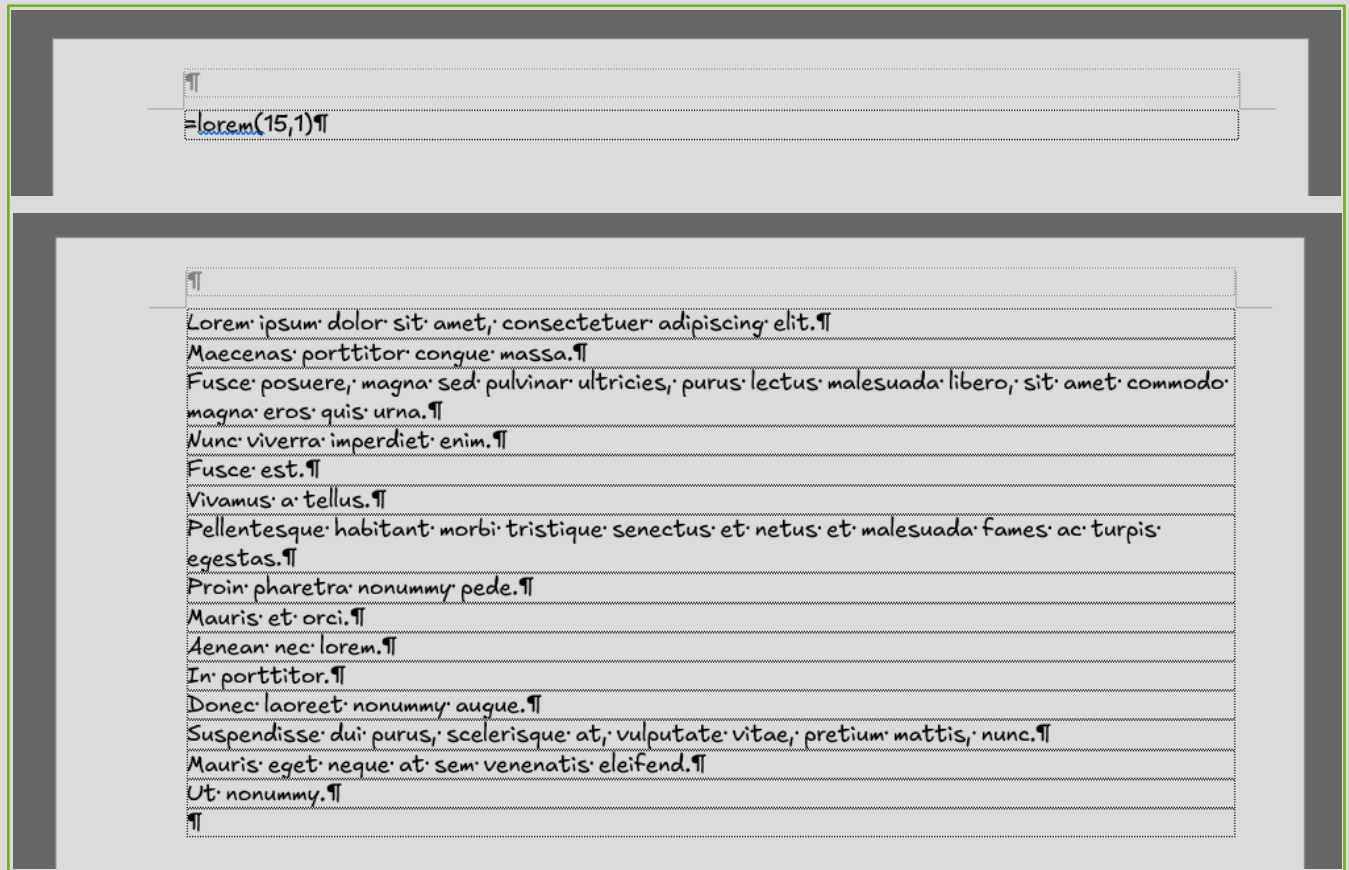
5. Enable unprinted characters and grey text background by pressing F4.



Pic. 73. Applied view mode no. 2.

11.3.3. Customized styles, keyboard shortcuts

Let's start and generate couple of paragraphs of text in Latin language. Microsoft Word has built-in generator. It is hidden under *File > Options > Proofing > AutoCorrect Options... > AutoCorrect > Replace text as you type*. This option must be active, at least temporarily. Enter in empty paragraph the text `=lorem(15,1)` and press enter at the end of line. You should get 15 paragraphs, each 1 sentence long.



Pic. 74. Some generated text.

Familiarize yourself with built-in keyboard shortcuts (see [Tab. 8](#)).

Let's apply style heading level 1 (*ParHeading 1 ms*) to the first paragraph. Put cursor there and press keyboard shortcut **Alt + Ctrl + 1**. Please note this paragraph was also automatically numbered. It took place, because 'underneath' there is a *List Template* which is assigned to heading styles.

Tab. 7. Built-in keyboard shortcuts.

Function	Keyboard shortcut	When is it helpful?
Restore default style to paragraph	Ctrl + Shift + N	If you wish apply <i>ParNormal ms</i> style to your current content selection
Restore default style to characters	Ctrl + Space	If you wish apply <i>CharDefault ms</i> style to your current content selection
Reveal Formatting pane	Shift + F1	If you wish to see all details related to formatting
Move entire paragraph, one after another	Shift + Alt + ↓ or Shift + Alt + ↑	If you wish to move entire paragraph.

1. → Lorem ipsum dolor sit amet, consectetur adipiscing elit.¶

Maecenas porttitor congue massa.¶

Fusce posuere, magna sed pulvinar ultricies, purus lectus malesuada libero, sit amet commodo magna eros quis urna.¶

Nunc viverra imperdiet enim.¶

Fusce est.¶

Vivamus a tellus.¶

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas.¶

Proin pharetra nonummy pede.¶

Mauris et orci.¶

Aenean nec lorem.¶

In porttitor.¶

Donec laoreet nonummy augue.¶

Suspendisse dui purus, scelerisque at, vulputate vitae, pretium mattis, nunc.¶

Mauris eget neque at sem venenatis eleifend.¶

Ut nonummy.¶

¶

Pic. 75. Heading 1.

Let's apply style heading level 3 (*ParHeading 3 ms*) to fourth paragraph. Put cursor there and press keyboard shortcut Alt + Ctrl + 3. Please note this paragraph was also automatically numbered.

1. → Lorem ipsum dolor sit amet, consectetur adipiscing elit.¶

Maecenas porttitor congue massa.¶

Fusce posuere, magna sed pulvinar ultricies, purus lectus malesuada libero, sit amet commodo magna eros quis urna.¶

1.1.1. → Nunc viverra imperdiet enim.¶

Fusce est.¶

Vivamus a tellus.¶

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas.¶

Proin pharetra nonummy pede.¶

Mauris et orci.¶

Aenean nec lorem.¶

In porttitor.¶

Donec laoreet nonummy augue.¶

Suspendisse dui purus, scelerisque at, vulputate vitae, pretium mattis, nunc.¶

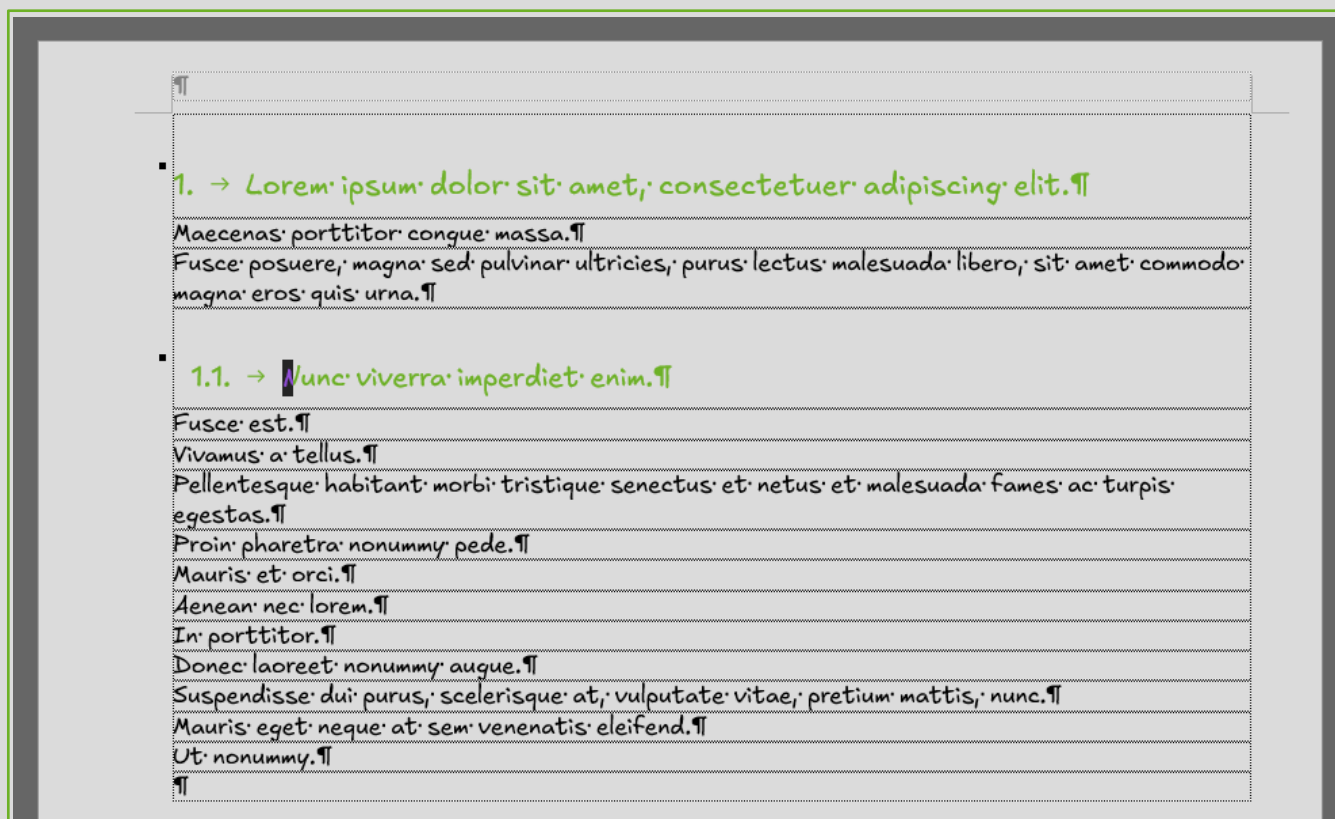
Mauris eget neque at sem venenatis eleifend.¶

Ut nonummy.¶

¶

Pic. 76. Heading 3.

Let's assume this was a mistake. We would like to have this paragraph at second, not third level. We can then press keyboard shortcut Alt + Ctrl + 2, but there is alternative, handy solution. Press *Home* keyboard key to place cursor just behind the → character, at the front of *N* character. Now press keyboard shortcut Shift + Tab.



Pic. 77. Heading 2.

Applying Tab or Shift + Tab you can easily change level of heading. Please note that numbering is adjusted accordingly and automatically.

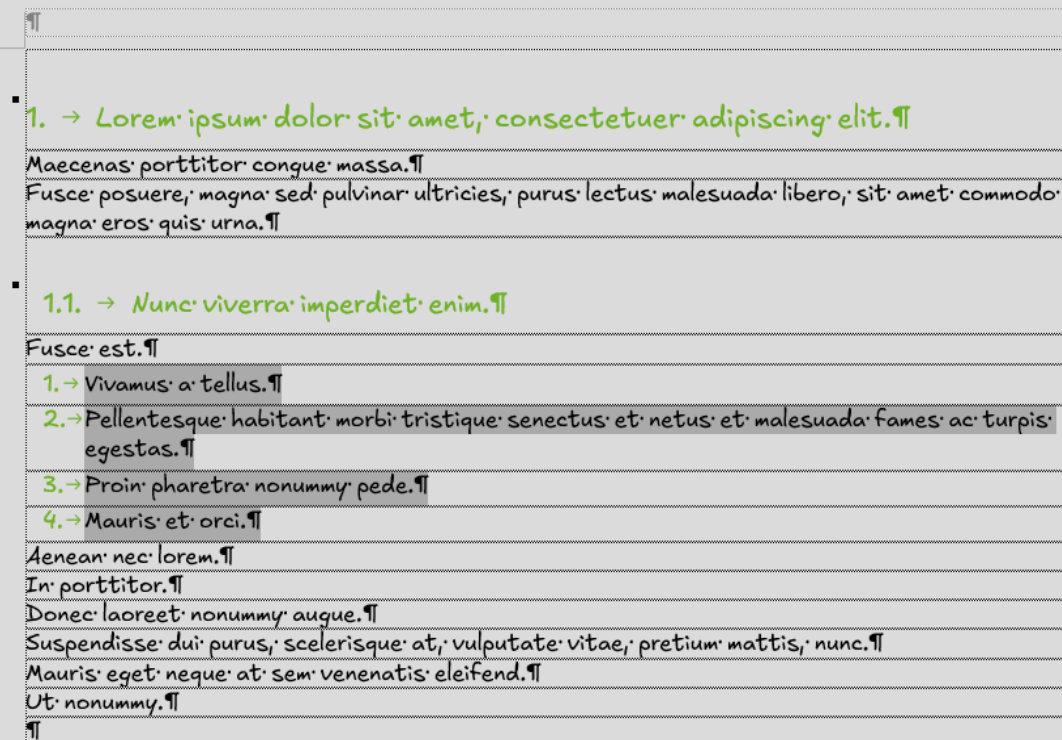
11.3.3.1. Lists

Now please select four paragraphs, one after another. Let's make a numbered list by pressing Alt + L, 1.

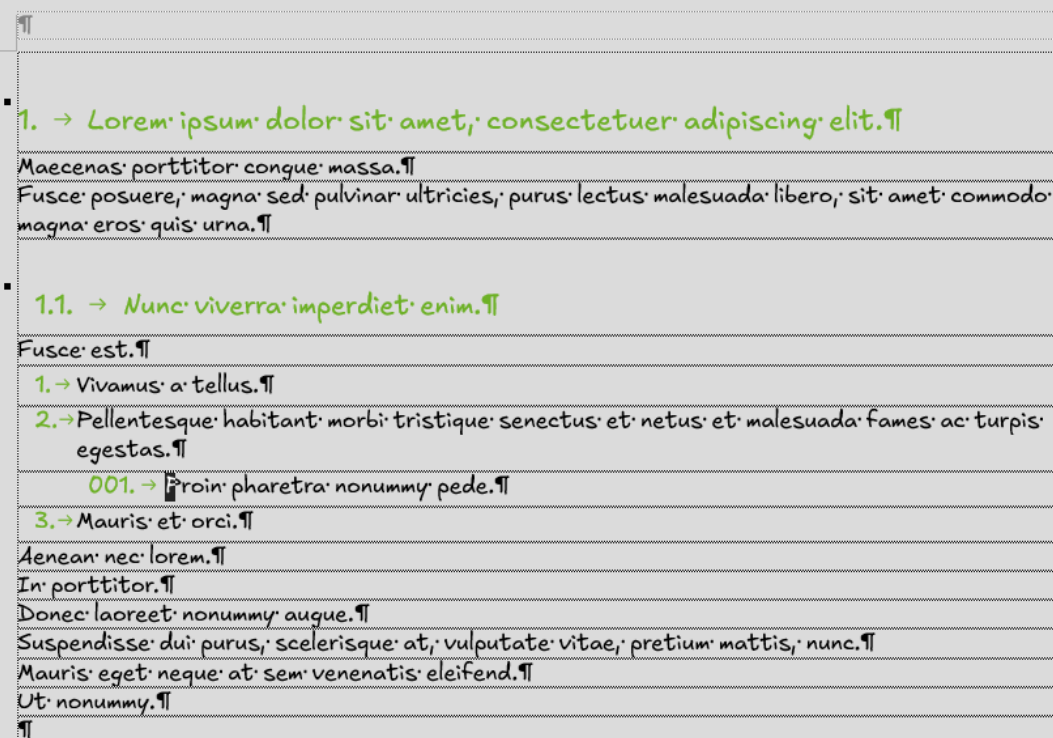
Please note that there is a way to remember list oriented keyboard shortcuts:

- » Numbered lists:
 - » there are four levels of that list: 1, ..., 4
 - » the numbered list is called with L as List
 - » all you need to remember is Alt; left Alt
 - » example: we want to apply third level numbered list: Alt + L, 3.
- » Bulleted lists:
 - » there are four levels of that list: 1, ..., 4
 - » the bulleted list is called with B as bullet
 - » all you need to remember is Alt; left Alt
 - » example: we want to apply second level bulleted list: Alt + B, 2.

Now let's use already known shortcut and move 3rd paragraph of the list 3 levels lower by pressing 3× Tab at the beginning of that line.



Pic. 78. Numbered list.



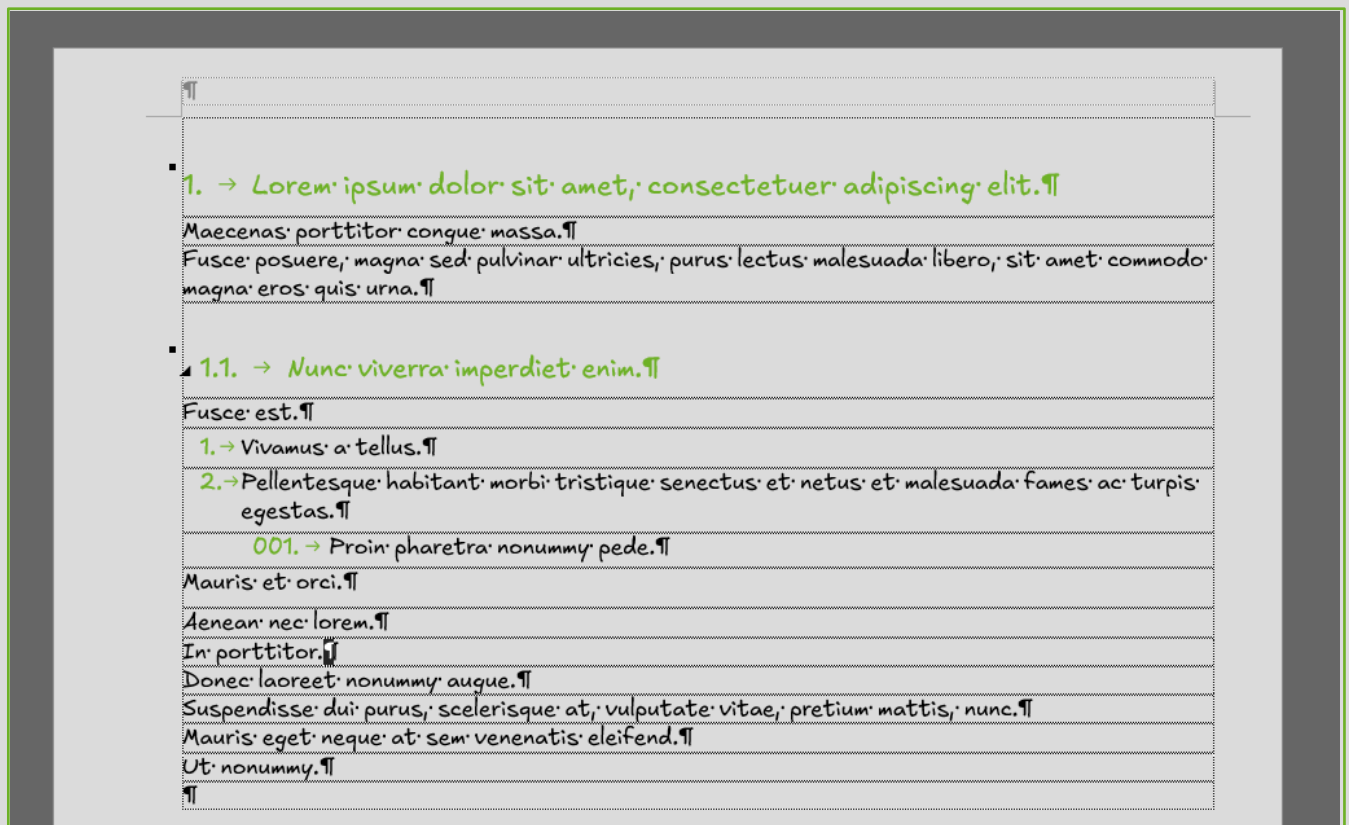
Pic. 79. Change of the numbered list level.

Again, please note, that numbering was adjusted accordingly and automatically.

In the same manner you can apply bulleted list with keyboard shortcut Alt + B, 1...4.

Now let's assume that last paragraph of the list should not be a part of this list anymore.

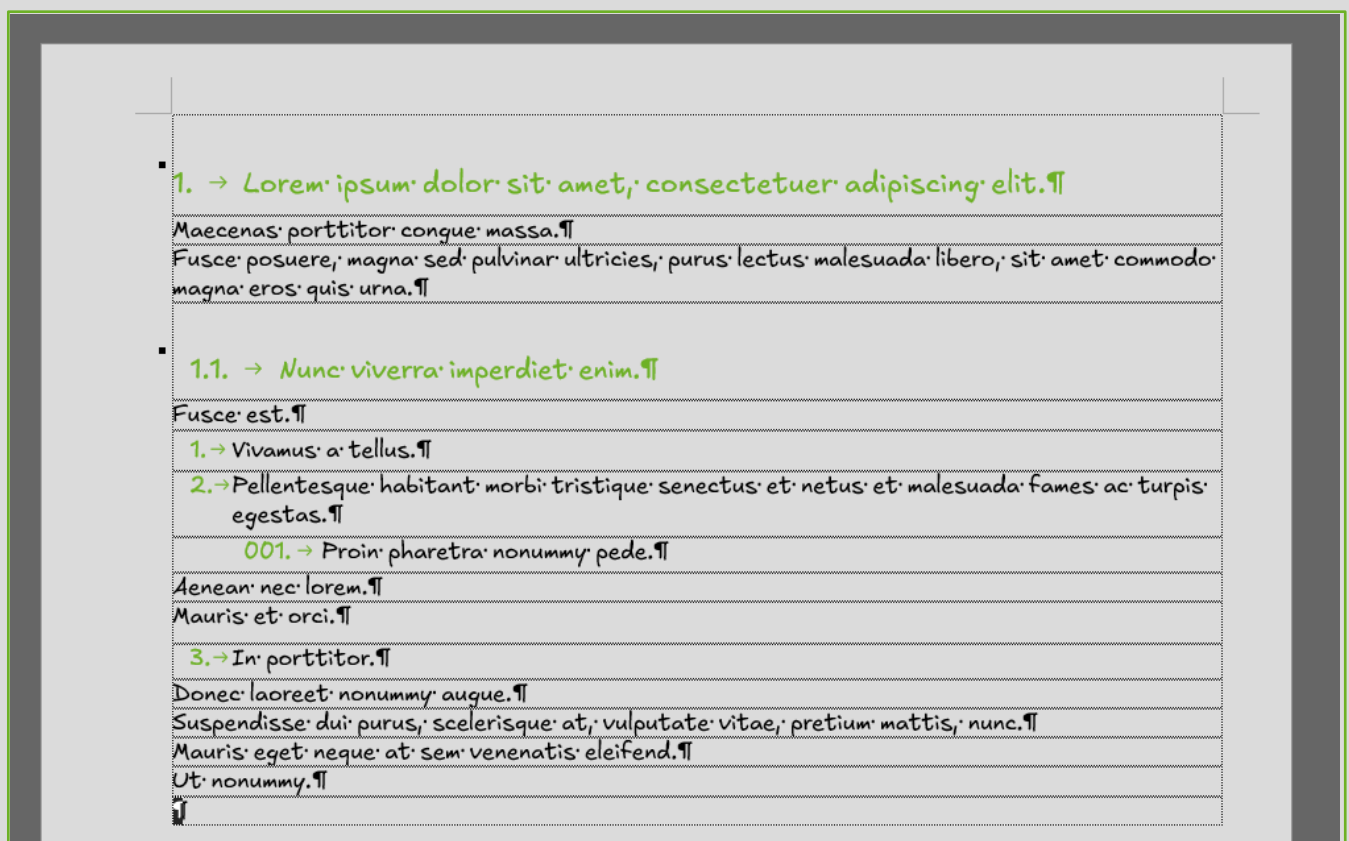
Place cursor there and press Shift + Ctrl + N.



Pic. 80. Restore default paragraph style.

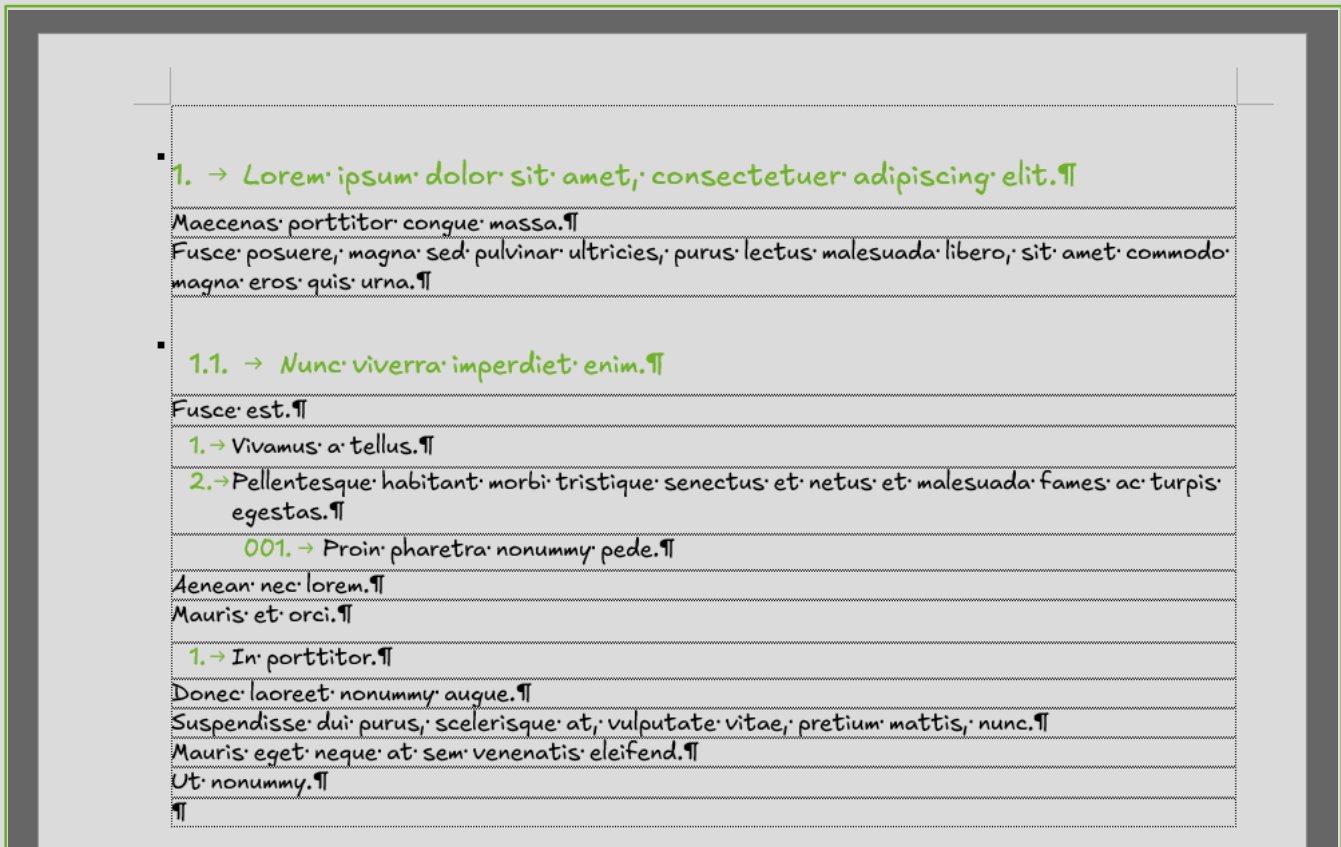
11.3.3.2. Reset of list numbering

One of the Microsoft Word features is keeping automatic incrementing of lists. Let's check, how it works. Put the cursor in the line 'In porttitor'. Apply numbered list at level 1 with keyboard shortcut Alt + L, 1.



Pic. 81. Added next numbered list.

Look what happened. For Microsoft Word this is not a beginning of a new list, but continuation of the previous one. Here I have another keyboard shortcut for you. Please cursor anywhere in this paragraph and press **Alt + L, R** to reset list numbering.

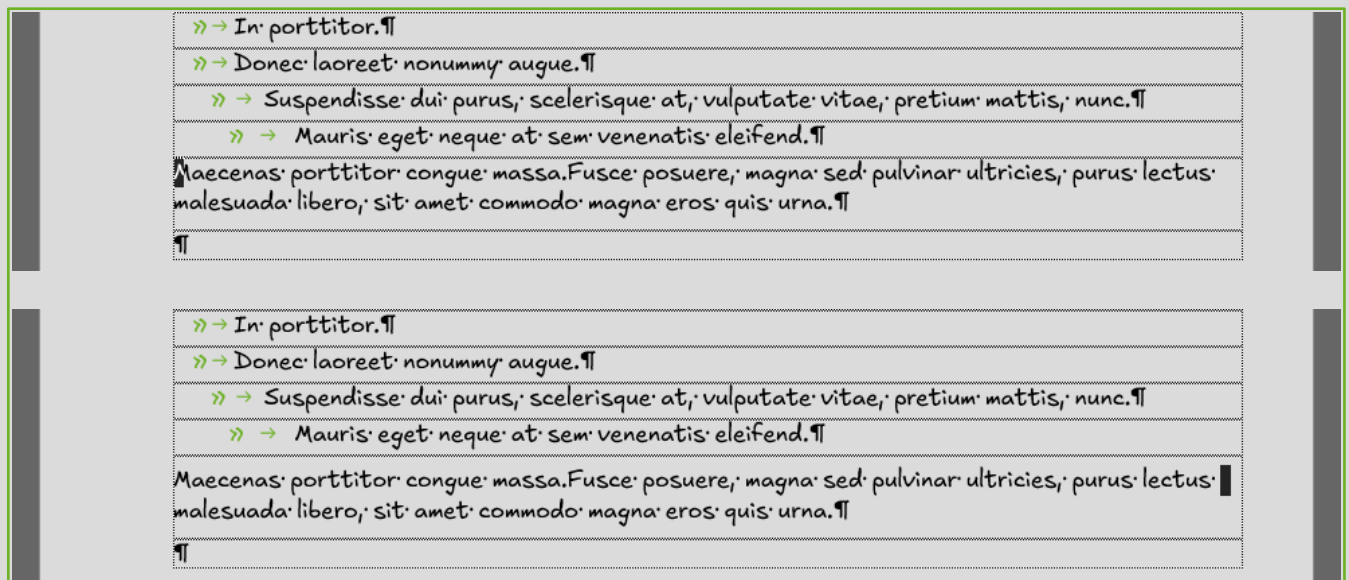


Pic. 82. Reset of numbering.

What if you would like to continue numbering? Then you can use context menu, which perhaps is available on your keyboard with hamburger menu icon (≡) to the right of spacebar key. If not, use your right mouse button and select from the context menu *Continue Numbering*.

11.3.3.3. The first text paragraph after list

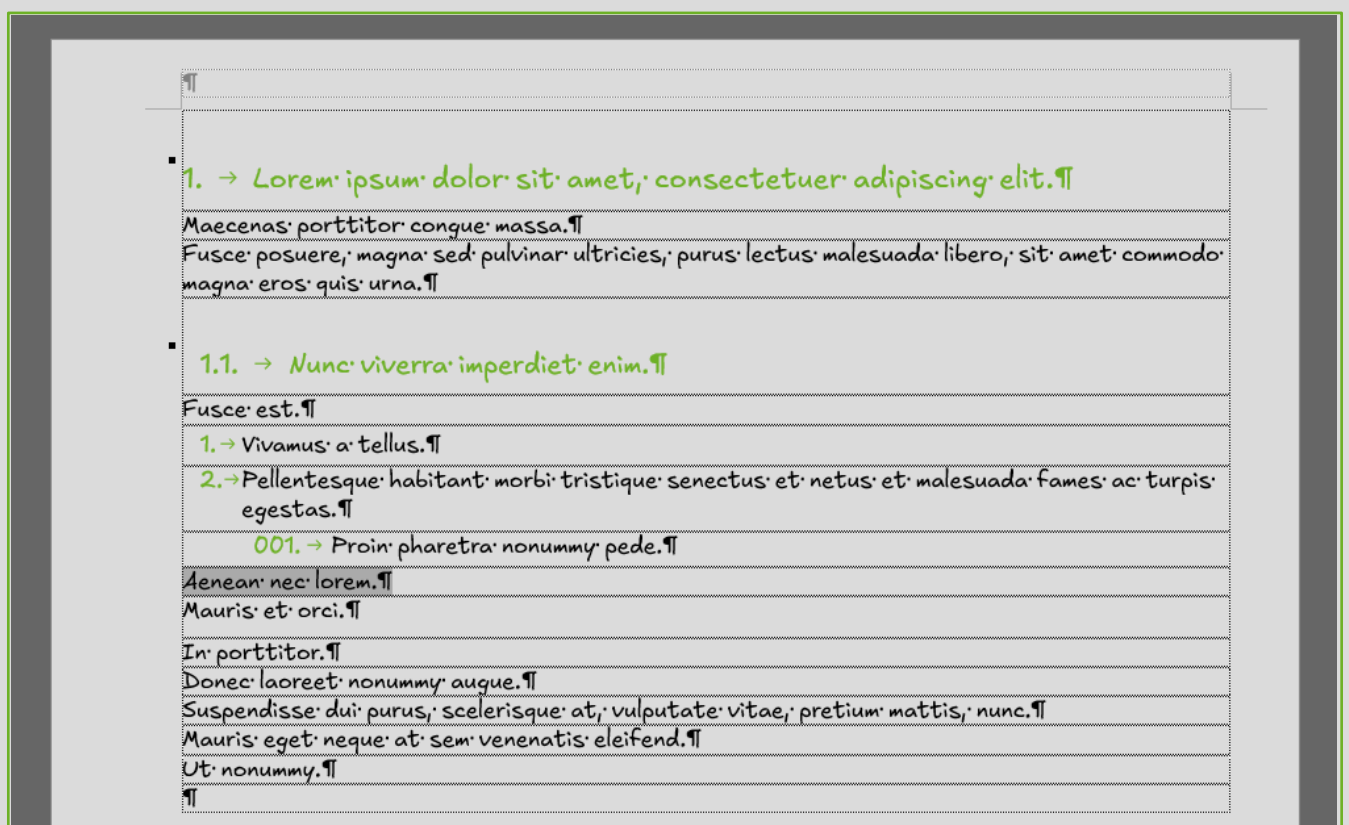
Please note what happens when after multiple level list comes paragraph of ordinary text. It seems to be unnaturally close. To compensate it, I designed additional style named *ParNormalAbove ms* (**Alt + N, A**). Compare before (top) and after (bottom).



Pic. 83. The first text paragraph after list.

11.3.4. Moving paragraph

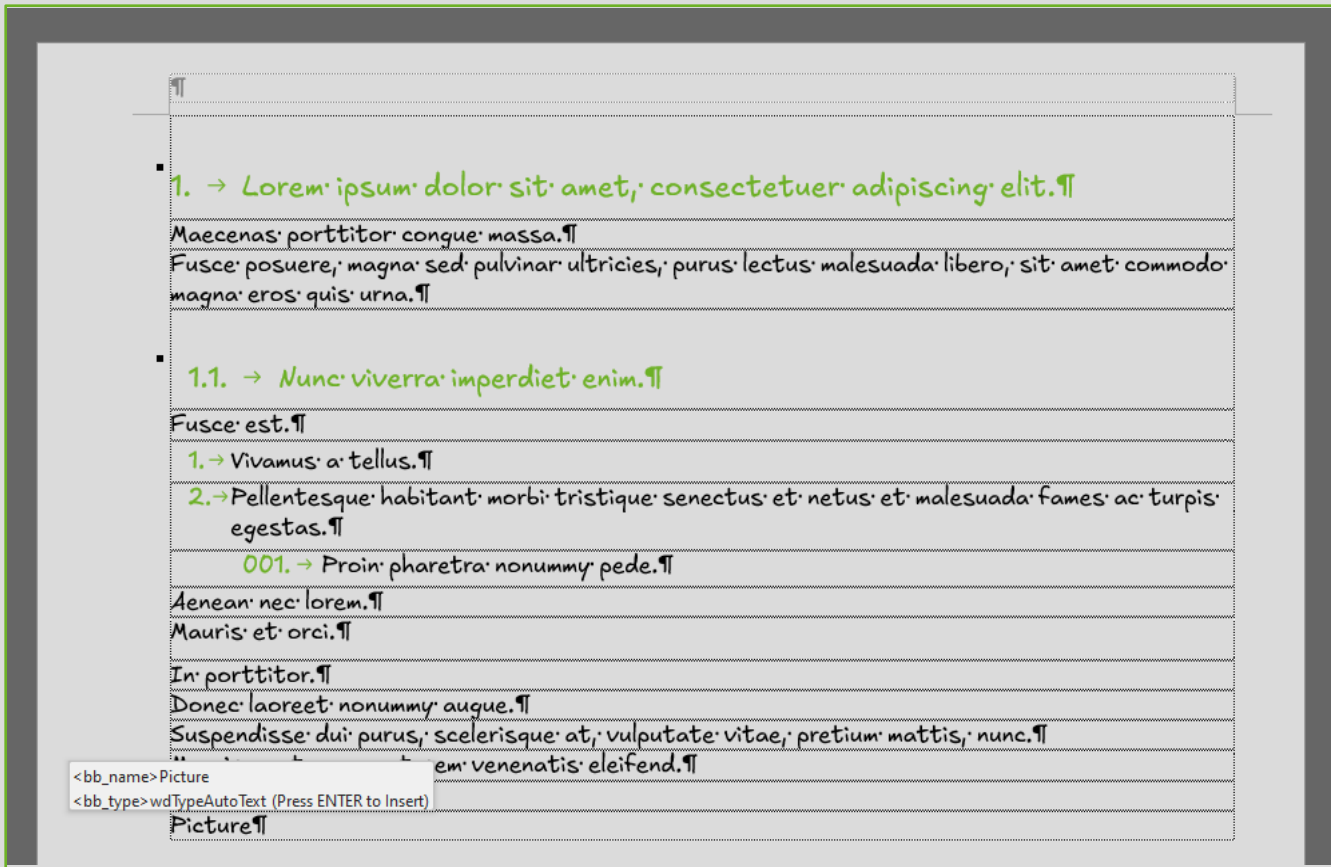
Let's move one of the paragraphs 'Aenean nec lorem.' above the previous one 'Mauris et orci.'. Put cursor in the second one, 'Aenean nec lorem.' and press keyboard shortcut Shift + Alt + ↑.



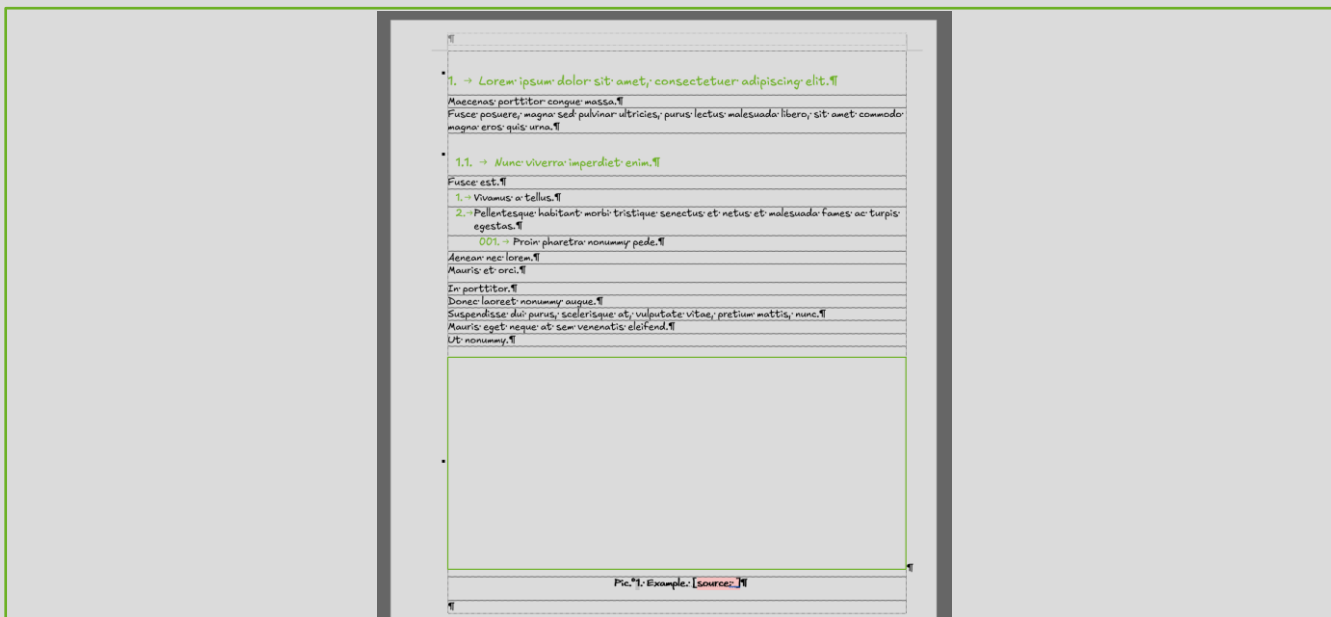
Pic. 84. Move entire paragraph, one after another.

11.3.5. Customized building blocks, keyboard shortcuts

Let's insert a picture. Set cursor in the last, empty paragraph. Type word picture and press F3.



Pic. 85. Write the word 'picture'.

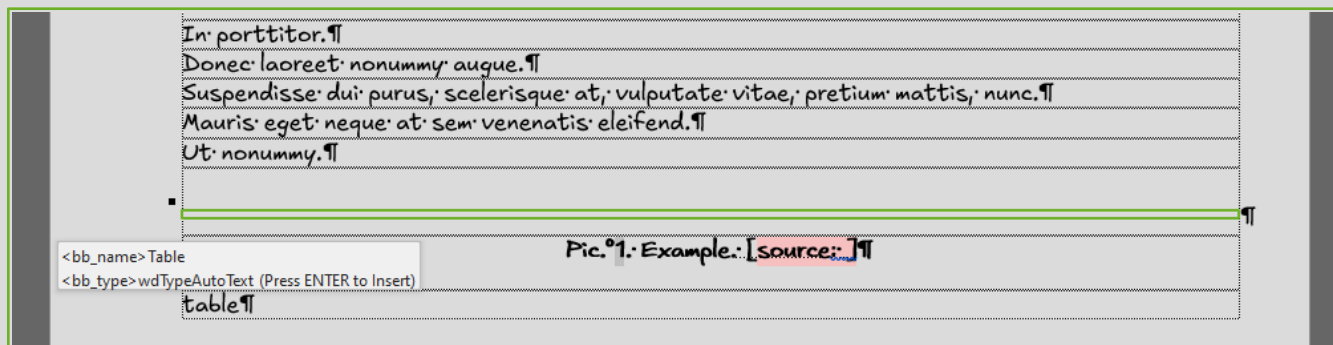


Pic. 86. The inserted picture building block.

Note that the picture building block was inserted, containing label, legend, correct styling, adjusted width. For clarity, I've removed building block header by running macro from menu > *BuildingBlocks_ms* > *Edit* > *Remove header*.

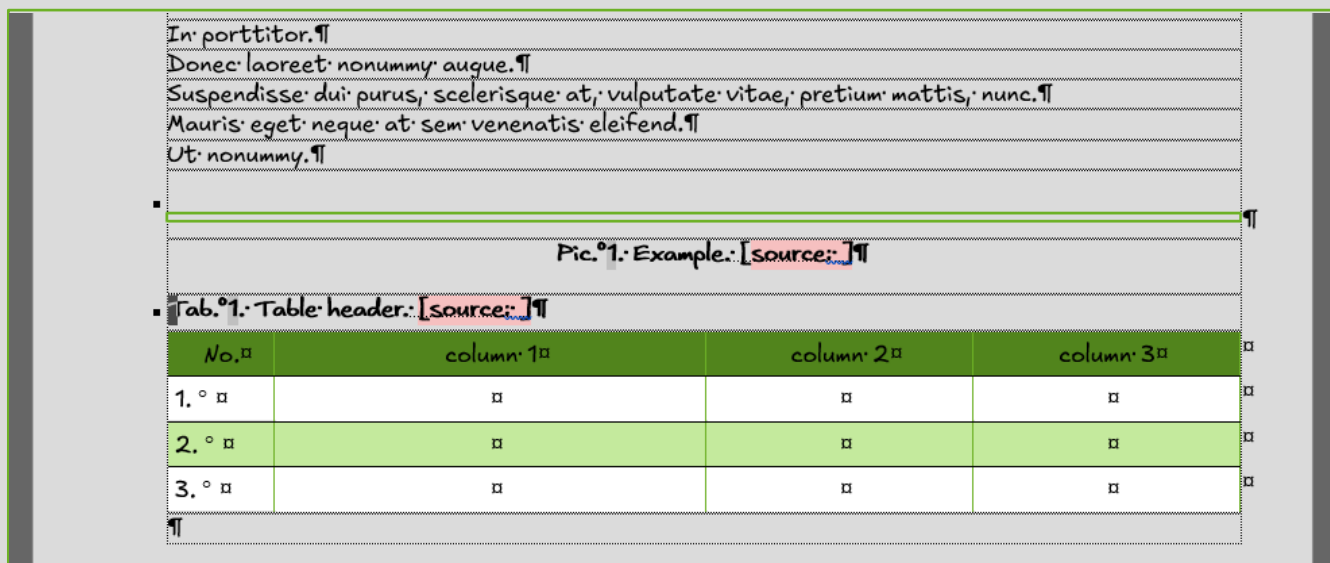
I'll reduce its height to minimum and add another frequently used building block for sake of an example, the table.

Let's insert a table, typing word table and pressing F3.



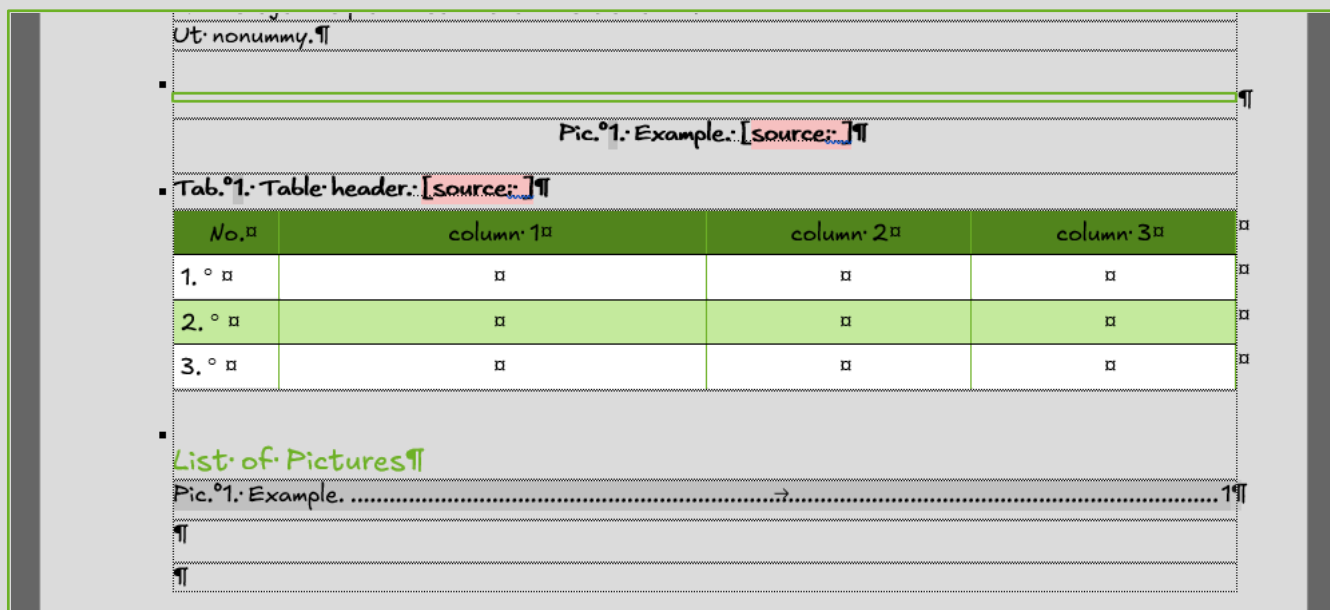
Pic. 87. Write the word 'table'.

I've run the *Remove header* macro again.



Pic. 88. The table building block.

Let's add in the last paragraph list of pictures. We will use *AutoText* feature again. Enter the word 'listofpictures' and press F3. Run the *Remove header* macro again.



Pic. 89. List of pictures.

In the same manner in the last paragraph enter the word 'listoftables' and press F3. Run the *Remove header* macro again.

Ut·nonumny.¶

¶

Pic.°1·Example·:[source:]¶

Tab.°1·Table·header·:[source:]¶

No.¶	column·1¶	column·2¶	column·3¶
1.°¶	¶	¶	¶
2.°¶	¶	¶	¶
3.°¶	¶	¶	¶

List·of·Pictures¶

Pic.°1·Example·.....→.....1¶

¶

List·of·Tables¶

Tab.°1·Table·header·.....→.....1¶

¶

¶

Pic. 90. List of tables.

Now set cursor in the first paragraph after 'List of Pictures' and use keyboard shortcut Alt + N, M to apply the style *ParMinimal* ms to this paragraph. Do the same at the paragraph second to last.

Ut·nonumny.¶

¶

Pic.°1·Example·:[source:]¶

Tab.°1·Table·header·:[source:]¶

No.¶	column·1¶	column·2¶	column·3¶
1.°¶	¶	¶	¶
2.°¶	¶	¶	¶
3.°¶	¶	¶	¶

List·of·Pictures¶

Pic.°1·Example·.....→.....1¶

¶

List·of·Tables¶

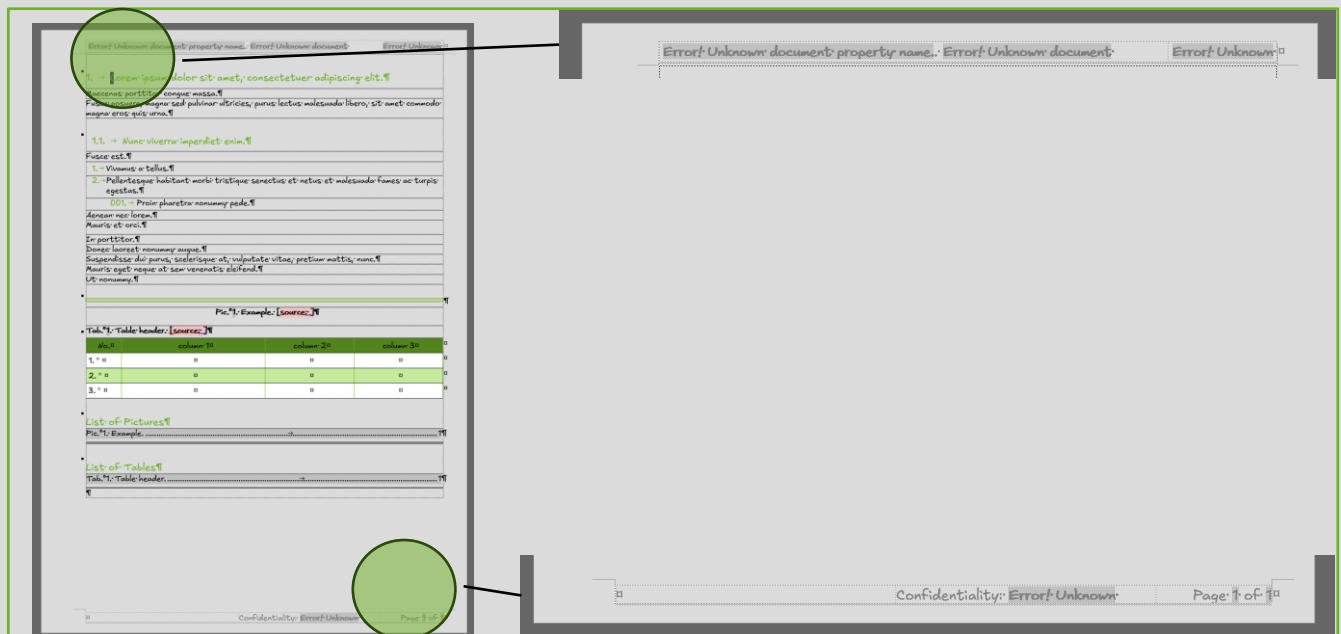
Tab.°1·Table·header·.....→.....1¶

¶

¶

Pic. 91. Applied style *ParMinimal* ms.

As the last example, let's add ordinary header and footer. Menu > *Insert* > *Header* > *Header-Ordinary*. Menu > *Insert* > *Footer* > *FooterOrdinary*. Run the *Remove header* macro again. Press Esc key to exit header and footer.



Pic. 92. Added HeaderOrdinary and FooterOrdinary.

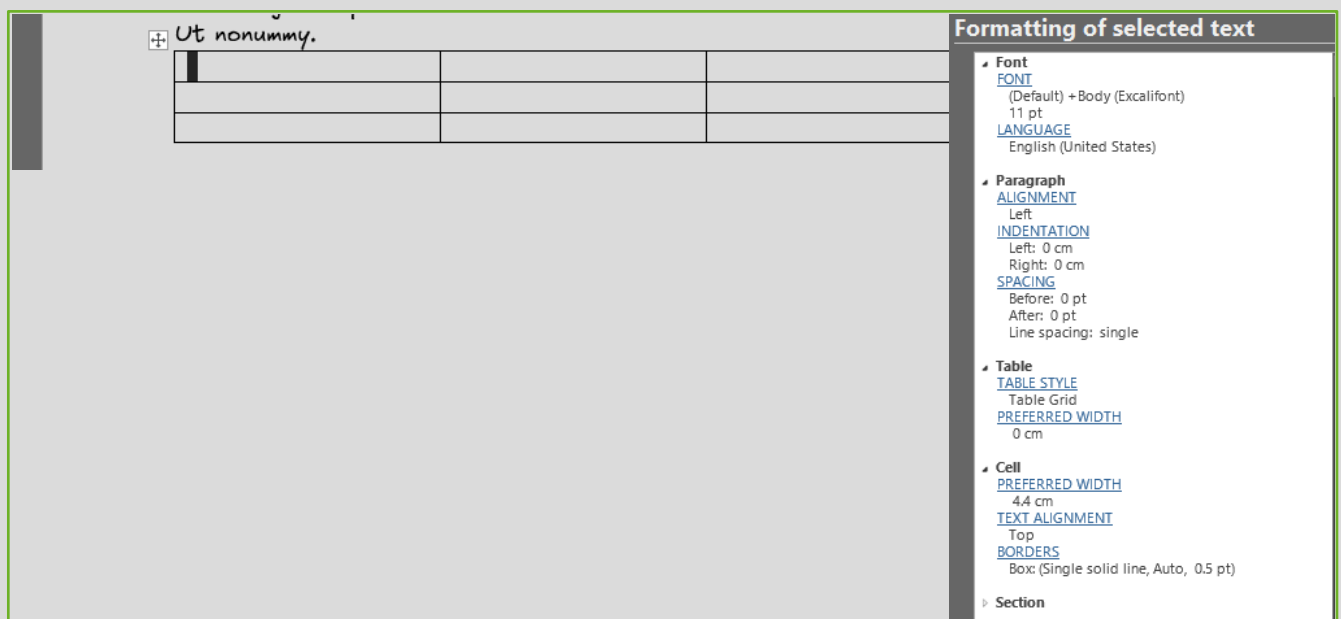
Mind the gray field area with message *Error! Unknown document property name.* We will fix this issue in the next chapter.

11.3.5.1. Working with tables

There are predefined just 3× styles of tables:

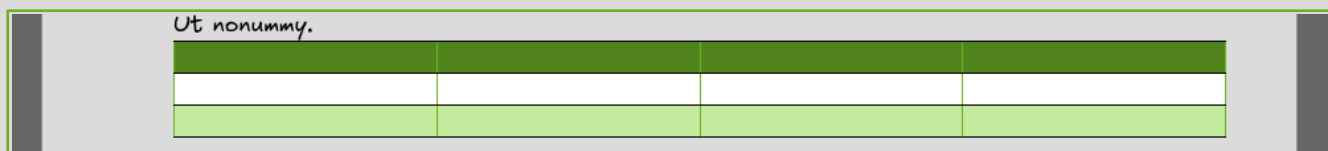
1. *TabTable ms*
2. *TabTableNoGrid ms*
3. *TabTableNoPadding ms*

Let's practice them. From menu > *Insert > Tables > Table > 3 × 4*. Switch display options to see just a content (F4). Check *Reveal Formatting* pane when cursor is set inside this table. Note that by default new table is styled as *Table Grid* and indeed it has visible grid.



Pic. 93. Newly crated table.

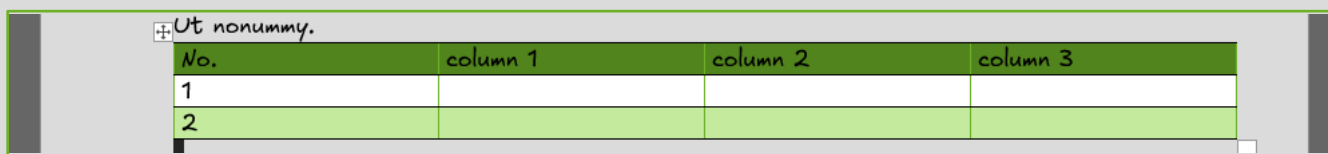
Select entire table (top left icon, two perpendicular double arrows) and *Apply Style* (Ctrl + Shift + S) *TabTable ms*.



Ut nonummy.			

Pic. 94. Style *TabTable.ms* applied to the example table.

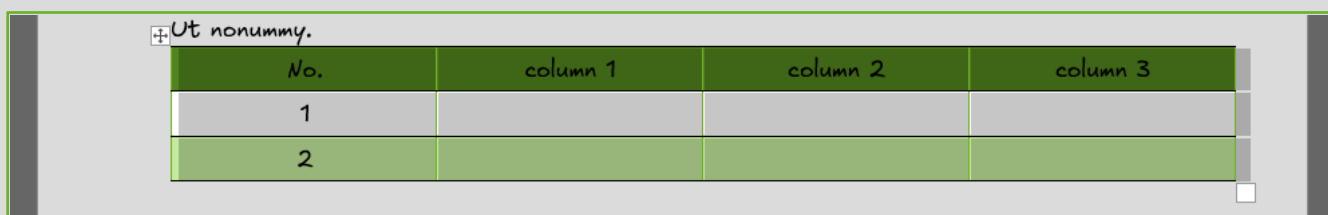
Add to the header the same text as in the table building block. Please note in *Reveal Formatting* pane that added text has no styling yet.



Ut nonummy.			
No.	column 1	column 2	column 3
1			
2			

Pic. 95. Added text.

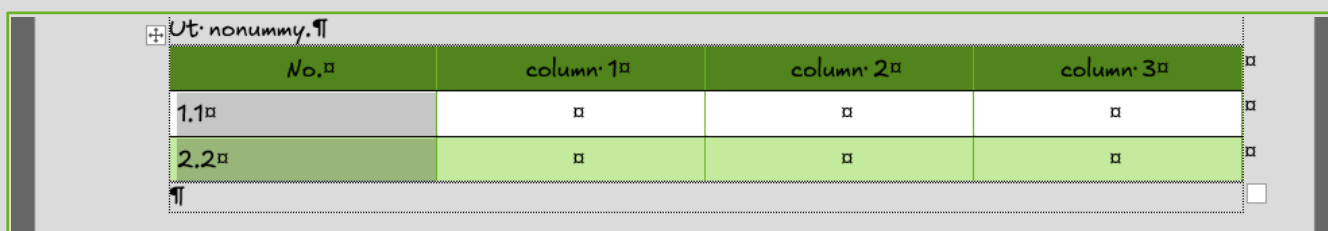
Select the entire table and apply the style *ParInTable.ms*.



Ut nonummy.			
No.	column 1	column 2	column 3
1			
2			

Pic. 96. Style *ParInTable.ms* applied to the example table.

Select just the numbers and apply the style *ParListInTable.ms*.



Ut nonummy.			
No.	column 1	column 2	column 3
1.1			
2.2			

Pic. 97. Style *ParListInTable.ms* applied to the numbered cells.

Slowly, but we are approaching to the state where all cells of the table have assigned paragraph style and the table starts to look similarly to building block. All we need now is to delete redundant numbering, mark two cells and center them (Ctrl + E).

The point is, that table styles mentioned at the beginning of this chapter do not format styles in a table cells. You need to make it manually or use building block. There is one more way: we can apply table specific macro. We have such macros in menu > *Tools.ms* > *Tables* > *Customize Formatting*. Insert another table of size 3x3. Select it. Apply *TabTable.ms* and then apply the macro *Customize Formatting*. Check again information in *Reveal Formatting* pane.

Another useful macro is *Keep on one page*. In technically oriented documents it is good to keep tables on one page. This macro takes care of that. Of course, if table is longer than text span on one sheet of paper, such macro has no use.

No.	column 1	column 2	column 3
1.1			
2.2			

Pic. 98. Style *TabTable ms* + *Customize Formatting* macro.

Table oriented macros can be found also in menu > *Validation_ms* > *Tables*:

- » *Format*
- » *Check Nesting*

The *Format* macro starts from the beginning of your document. Each time it finds a table, it is formatted according to defined rules: aligned to border lines of text, formatted as expected.

The *Check Nesting* macro starts from the beginning of your document. Each time it finds a nested table (table placed inside one of the cells of another table), it stops with warning. Table nesting from my perspective is harmful practice and should be avoided. Potentially it can ruin XML structure of the DOCX file.

11.3.5.2. The first text paragraph after table

Please note what happens when after a table comes paragraph of ordinary text. It seems to be unnaturally close to bottom edge of the table above it. To compensate it, similarly to lists, I designed style named *ParNormalAbove ms* (Alt + N, A). Compare before (top) and after (bottom).

Tab.°1. Table header: [source:]

No.	column 1	column 2	column 3
1. °			
2. °			
3. °			

Maecenas porttitor congue massa.

Tab.°1. Table header: [source:]

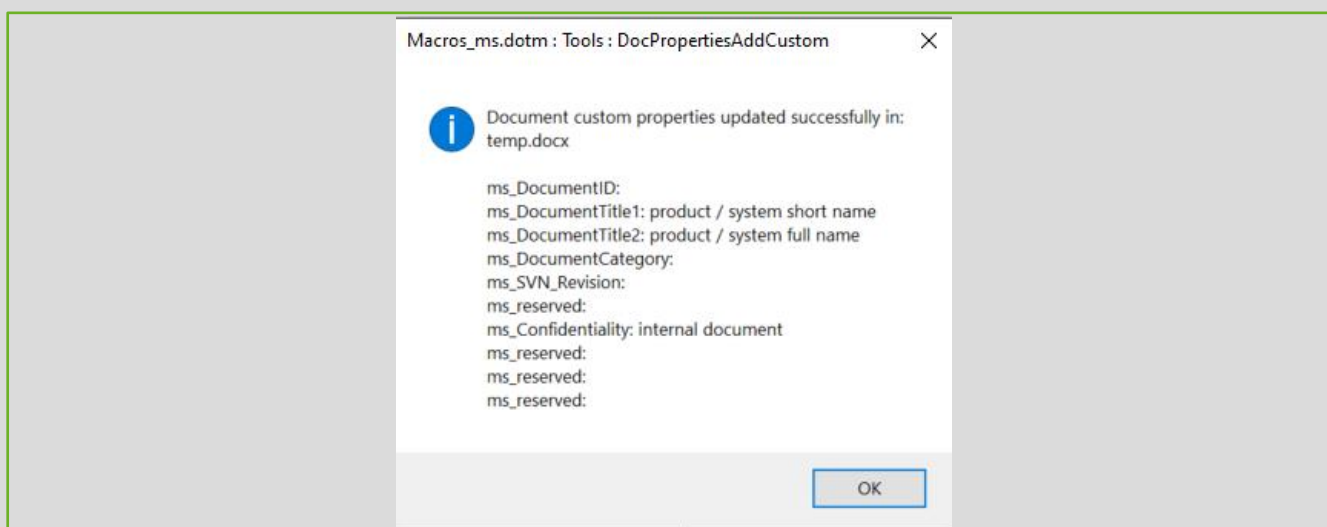
No.	column 1	column 2	column 3
1. °			
2. °			
3. °			

Maecenas porttitor congue massa.

Pic. 99. The first text paragraph after table.

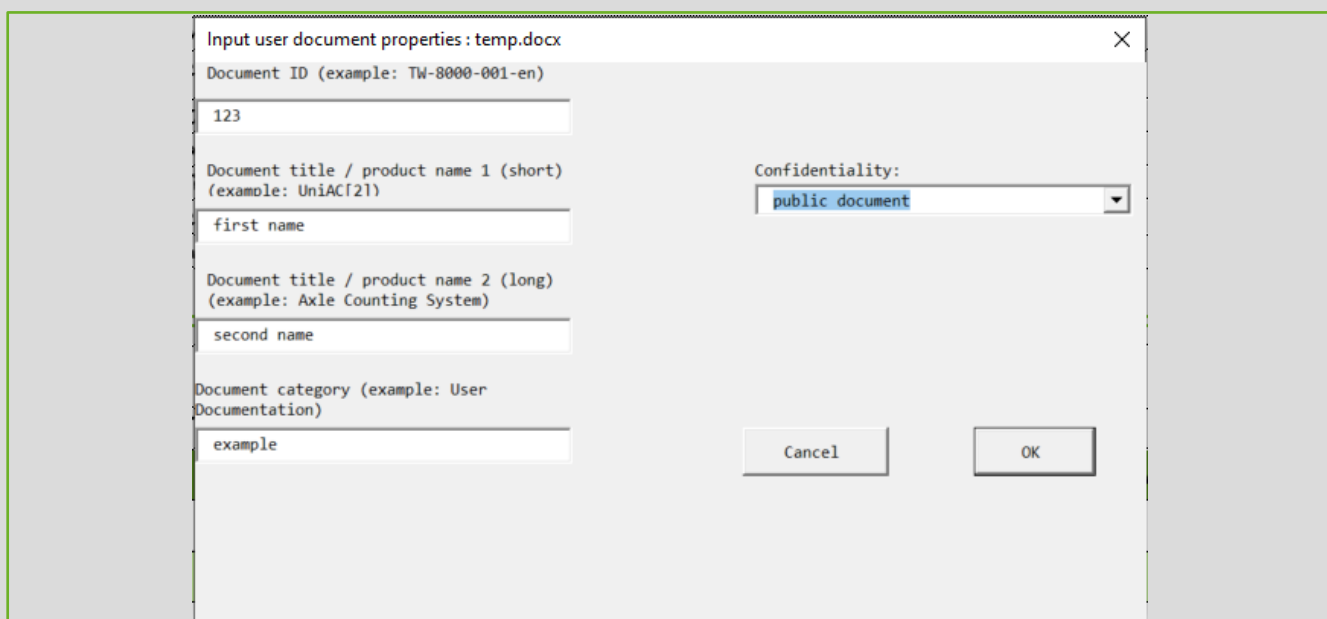
11.4. Customized document properties

Let's add customized document properties. Menu > *Tools_ms* > *DocProperties* > *DocProperties* > *Add*.



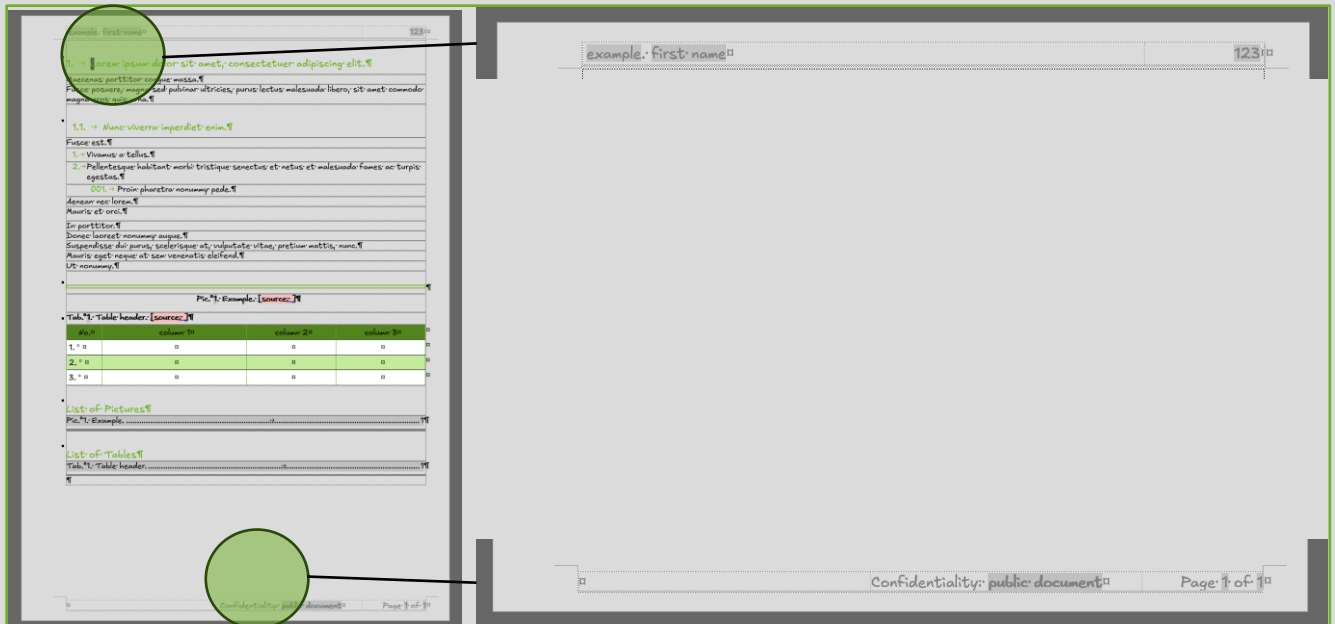
Pic. 100. Added customized document properties.

Now let's edit newly added document properties. Menu > *Tools_ms* > *DocProperties* > *DocProperties* > *Edit*.



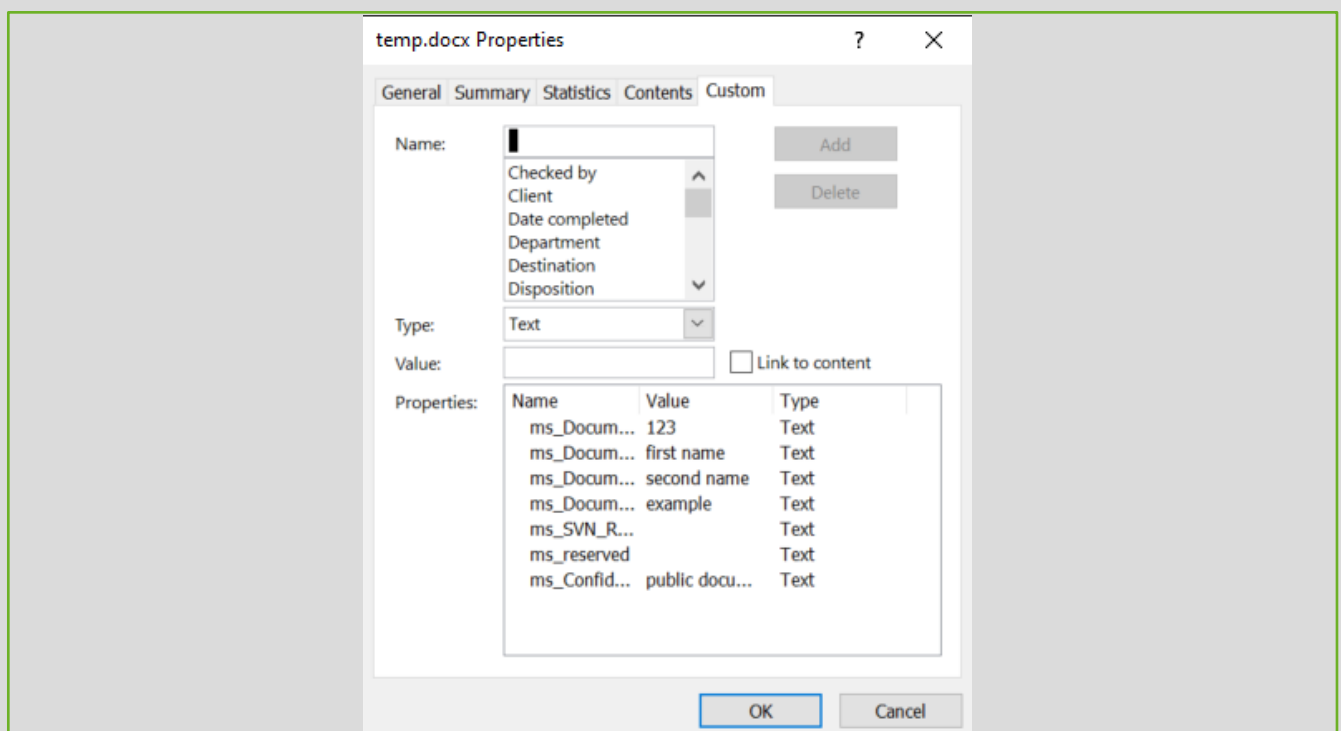
Pic. 101. Input user document properties.

After clicking OK, the fields in header and footer were updated.



Pic. 102. Updated fields.

You can check document properties manually in User Interface. Menu > File > Info > Properties > Advanced Properties > Custom tab.

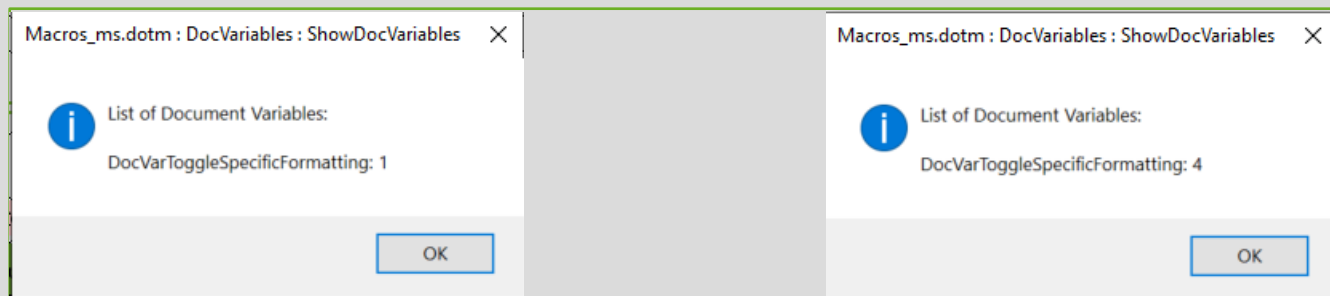


Pic. 103. Microsoft Word User Interface to custom properties.

11.5. Customized Document Variables

For Document Variables there is no User Interface. The Document Variables can store whichever information is necessary. I use them to store information about state of user interface formatting, connected with F4 keyboard shortcut.

To view if there are any Document Variables in currently opened document select menu > Tools_ms > DocVariables > DocVariables > Show DocVariables.



Pic. 104. Show DocVariables.

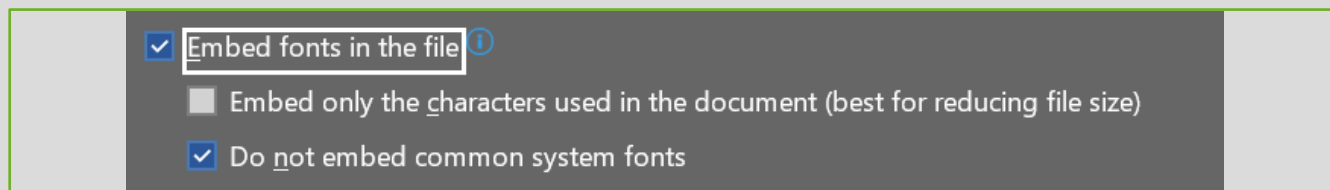
Press 2-3 times F4 keyboard key and try again. Compare with previous result. Upon file opening state of the variable `DocVarToggleSpecificFormatting` is applied automatically by macro `ClsAppEvents.AppWord_DocumentOpen` which calls the macro `Macros_ms.Tools.UpdateDisplayStateOnFileOpen`.

11.6. Fonts

Upon file opening it is checked automatically if there are any substituted fonts by macro `ClsAppEvents.AppWord_DocumentOpen` which calls the macro `Macros_ms.Fonts.ShowSubstitutedFonts`, compare chapter 9 for details.

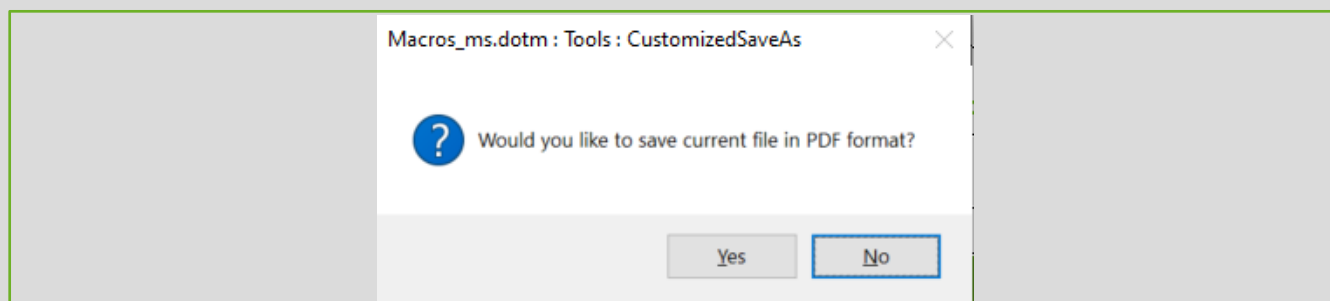
11.6.1. How to embed fonts within your document?

Menu: *File > Options > Save > Embed fonts in the file* and tick it on ☒.



Pic. 105. Embed fonts in the file.

Let's check how font embedding influences size of our file which was used in previous sub-chapters. At first let's save it applying F12 keyboard shortcut (by default it is *Save As* function of Microsoft Word). I have overloaded the default function with my macro. As a result, you will see on the screen the question.

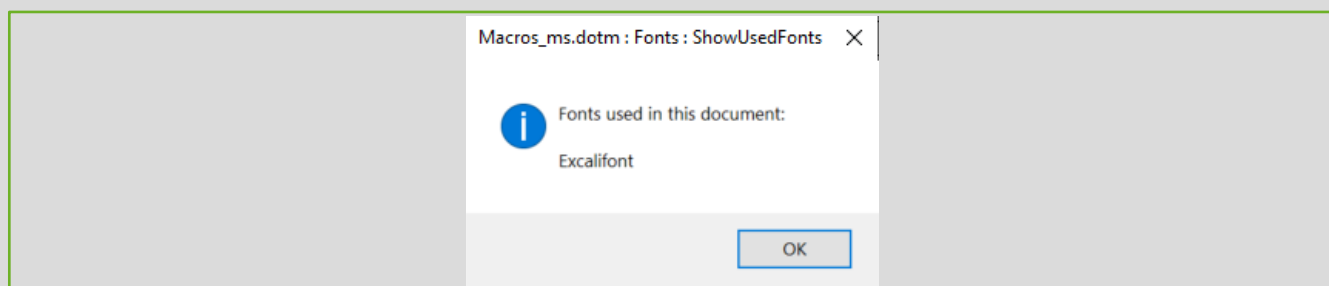


Pic. 106. Save as question.

Please answer *No* for a moment and save that file under the name `temp.docx`. Check in file manager size of that file. Mine is 42 610 bytes. Now embed fonts in the file, save it and check its size again. Mine is now 541 932 bytes. For comparison the, TTF file itself is 191 960 bytes.

11.6.2. How to check which fonts will be embedded in my DOCX file?

There is no User Interface tool to answer that question. I have prepared macro, which shows which fonts are applied in currently open document. Menu *Tools_ms > Fonts > Show Used Fonts*.



Pic. 107. Fonts applied in example file.

However, it doesn't answer to a question if actually all fonts displayed in this window will be embedded in a document. To answer that question you need to know by heart which fonts belong to system font category, as those will be not embedded by default.

11.6.3. Is font embedding process reversable?

In other words, is it possible to remove previously embedded fonts from your file? Yes, fonts embedding process is reversable. All you need to do are the same steps as before, but this time untick ☒ → ☐ the *Embed fonts in the file* option.

11.6.4. Can I check which fonts are embedded in my DOCX file?

There is no way to list embedded fonts by VBA in Microsoft Word.

To check which fonts are embedded in your Microsoft Word document, you can follow these steps:

1. Save your DOCX document.
2. Change the file extension from .docx to .zip.
3. Extract the contents of the .zip file.
4. Open the extracted folder and navigate to the word folder.
5. Look for a file named *fontTable.xml* and open it with a text editor.
6. In the *fontTable.xml* file, you will see a list of all the fonts used in the document. Embedded fonts will have an attribute indicating that they are embedded.

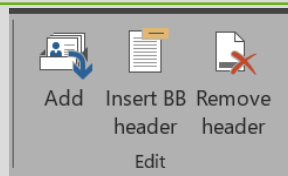
To recognize which fonts are embedded in your document by looking at the *fontTable.xml* file, you need to look for the `<w:embedRegular>`, `<w:embedBold>`, `<w:embedItalic>`, and similar elements within each `<w:font>` element. These elements indicate that the font is embedded.

11.7. Customized building blocks

In this chapter I introduce you to my predefined collection of building block.

11.7.1. How to store and manage collection of building blocks without using the Building Block Organizer

Assumption: you have already installed and configured my set of macros from *Macros_ms.dotm*.



Pic. 108. Menu BuildingBlocks.ms > Edit.

The idea is to apply macros for that purpose. Whenever a new building block is defined, add in the front of it the following section of content paragraphs, named as *BB header*. It can be inserted by applying *BuildingBlocks.ms > Edit > Insert BB header*.

<bb_name>MyBuildingBlockTemplate
<bb_type>wdTypeAutoText wdTypeFooters wdTypeHeaders wdTypeTableOfContents wdTypeCustom1
<bb_category>General Custom 1
<bb_description>
<bb_insertoptions>wdInsertContent wdInsertParagraph wdInsertPage

Let's assume you would like to define a building block related to your home address. Section related to this building block is marked as code:

John Smith the Second
Sun Boulevard 18/3
P.O. 19 Countryside

Now you can combine content of your future building block with header, choosing just one out of few specific options, which are separated with vertical bar |.

<bb_name>MyHomeAddress
<bb_type>wdTypeAutoText
<bb_category>General
<bb_description>My home address.
<bb_insertoptions>wdInsertContent
John Smith the Second
Sun Boulevard 18/3
P.O. 19 Countryside

Select all paragraphs in the frame above and apply *BuildingBlocks.ms > Edit > Add*. The newly defined building block will be added to *BB.ms.dotx*. Without opening the *Building Block Organizer* window.

The downside of this method is that whenever you insert building block prepared this way, its header is inserted as well. You will need to delete it either manually either automatically by applying *BuildingBlocks.ms > Edit > Remove header*. The Remove header macro tries to find and delete all headers from entire document, what can be not efficient for long documents.

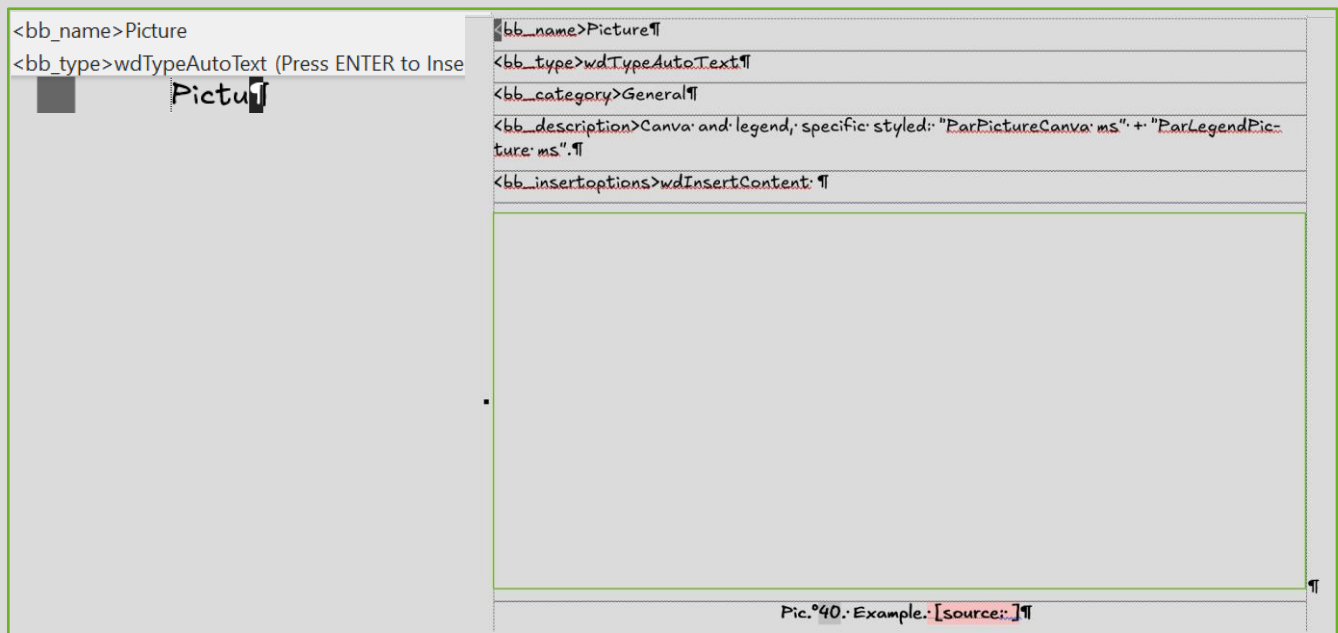
11.7.1. Calling of the building blocks with *AutoText* feature

Inserting building block into document content can be improved by applying *AutoText* feature of Microsoft Word.

Almost everybody is familiar with concept of keyboard combinations, where you press several keyboard keys concurrently or in specific sequence, and as feedback a function assigned to those keyboard keys is called. The same idea can be used for sequence of letters, or text strings. You can define specific text strings and link them logically to building blocks. This concept is sometimes called *Hotstrings* to distinguish it from *Hotkeys*. In Microsoft Word there is implemented similar mechanism under name *AutoText*.

My personal comment. The idea of *AutoText* is great, but for some unclear to me reasons it is implemented locally and in quite constrained way only in Microsoft Word. Potentially it could be useful system wide, in every application of Microsoft Windows where text is involved. This idea of system wide implementation is possible thanks to [AutoHotkey \[12\]](#) language and its scripts, but this is subject for separate book.

Whenever possible, assign building block to the type `wdTypeAutoText`. See previous chapter for details. As a result, when you start to type name of your building block, e.g. picture, small window will popup informing you about active *AutoText* definition. Now you can press Enter, or better F3, to call this *AutoText* expansion. Compare result in [Pic. 109](#). The issue is, that this small window tip does not always appear. If it does not, still, if you know that your definition exists, you can call *AutoText* expansion by pressing F3.



Pic. 109. AutoText and its expansion.

The downside of this method is that you need to remember definitions of your available *AutoText* building block. There is however a remedy to that, at least partially.

I have prepared macro: *Shortcuts.ms > Show > Show Windows > BuildingBlocks Hotstrings*, which brings to a screen small window as a form of on screen reminder about available *AutoText* aka *Hotstrings* definitions within the *BB.ms.dotx*, compare [Pic. 60](#).

11.7.2. Entire collection of building blocks in one file

It can be handy to manage entire collection of building blocks from one DOCX file. The *Building Block Organizer* window has a preview, but it small and doesn't let you see all the details.

Assumption: you have already installed and configured my set of macros from *Macros.ms.dotm*.

1. Open the *BB.ms.dotx*.
2. Run from the menu *BuildingBlocks.ms > Export All*.

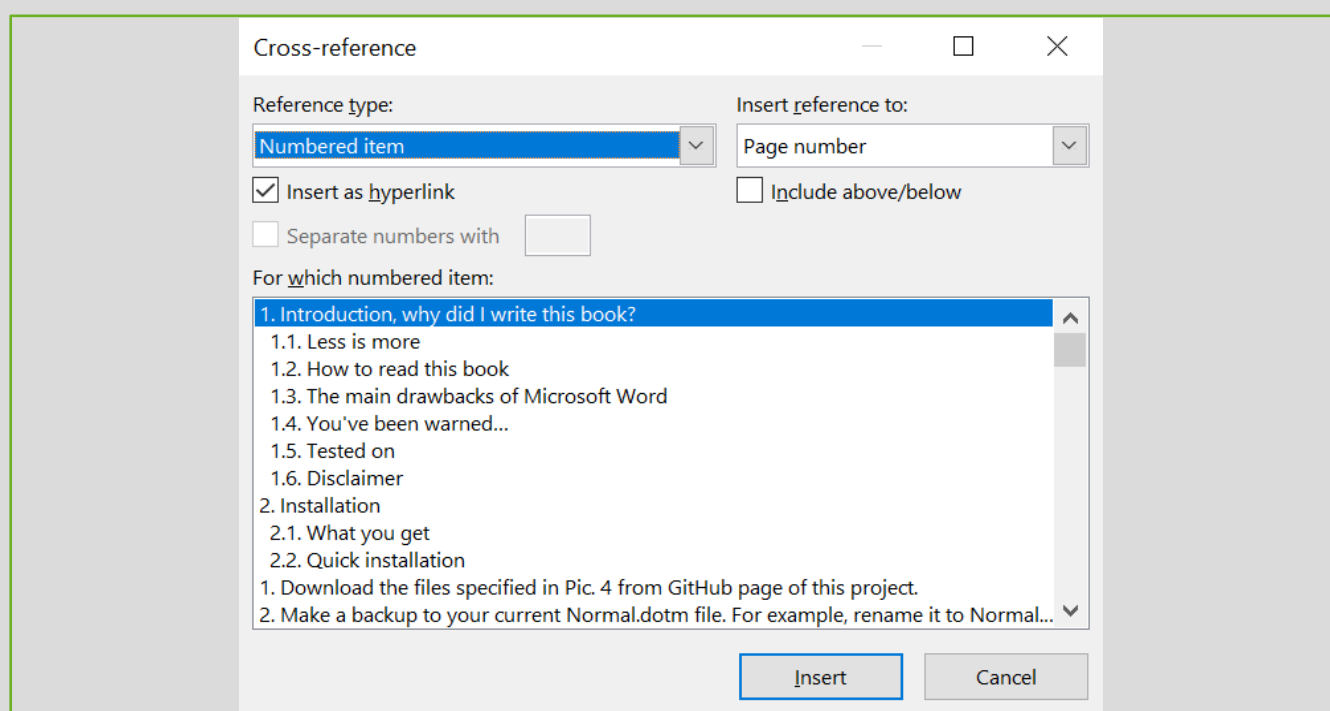
Expected result: file named *BB.ms_BB_Content.docx* saved in your current working directory. Each building block is exported, together with its header and styling. You can watch all of them and apply changes, if necessary, following procedure from chapter [11.7.1](#).

11.8. Customized captions

The *Captions* (menu *References > Captions > Insert Caption*) are information category which is a bit aside of my classification. They are not stored within add-ins, and not in the document itself. It seems that captions are stored in *Normal.dotm* and only there, at least in Microsoft Word versions older than 2010. Lack of customized captions embedded in the file may be the reason why your file, when opened somewhere else, on other computer, and edited, do not insert next captions correctly. To me the only remedy is to write dedicated macros and store information about customized captions in form of VBA code. This approach is similar to dealing with customized page layout, which cannot be easily saved without applying VBA code.

11.9. Cross-References

The default cross-references, available from menu *> Insert > Links > Cross-reference* require few choices, few mouse clicks.



Pic. 110. Cross-reference window.

I have adopted macro code, which works for numbered items, specific captions *Tab.* and *Pic.* and for referenced list.

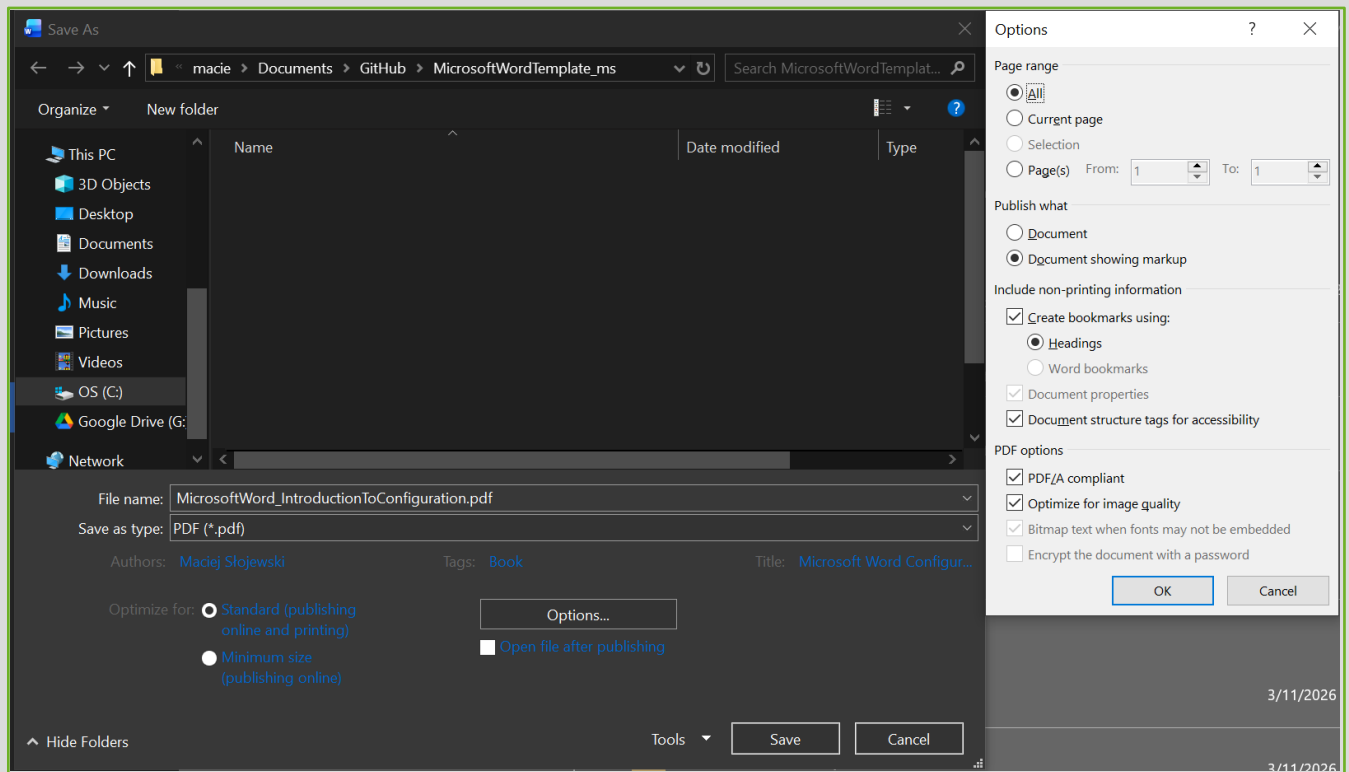
For example, let's assume to put here in the text cross-reference to chapter 11.8. Then all I have to do is to select all the digits and then press F7. As a result, I get 11.8: underlined hyperlink to chapter no. 11.8.

Another example, let's assume to put here in the text cross-reference to picture no. 110. Then all I have to do is to select caption, dot and digits, *Pic. 110* and the press F7. As a result, I get Pic. 111: underlined, hyperlink with nonbreaking space between caption and the digits to picture no. 110.

Last example, let's assume to put here the link to document from chapter 17 (References), no. 15 in the list of references. Then all I have to do is to put here square brackets and reference digits [15] and press F7. As a result, I get [15]: underlined hyperlink to 15th position in chapter References.

11.10. PDF as an output

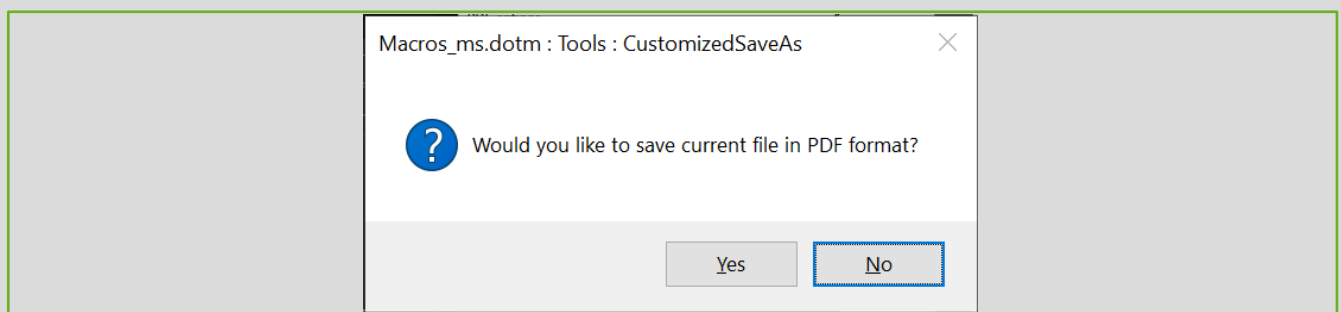
There is a macro assigned to F12 keyboard shortcut, which by default is used to call *Save As* function. The same dialog box is used by default in Microsoft Word to save document in PDF format.



Pic. 111. Default Save As dialog box, Options dialog box.

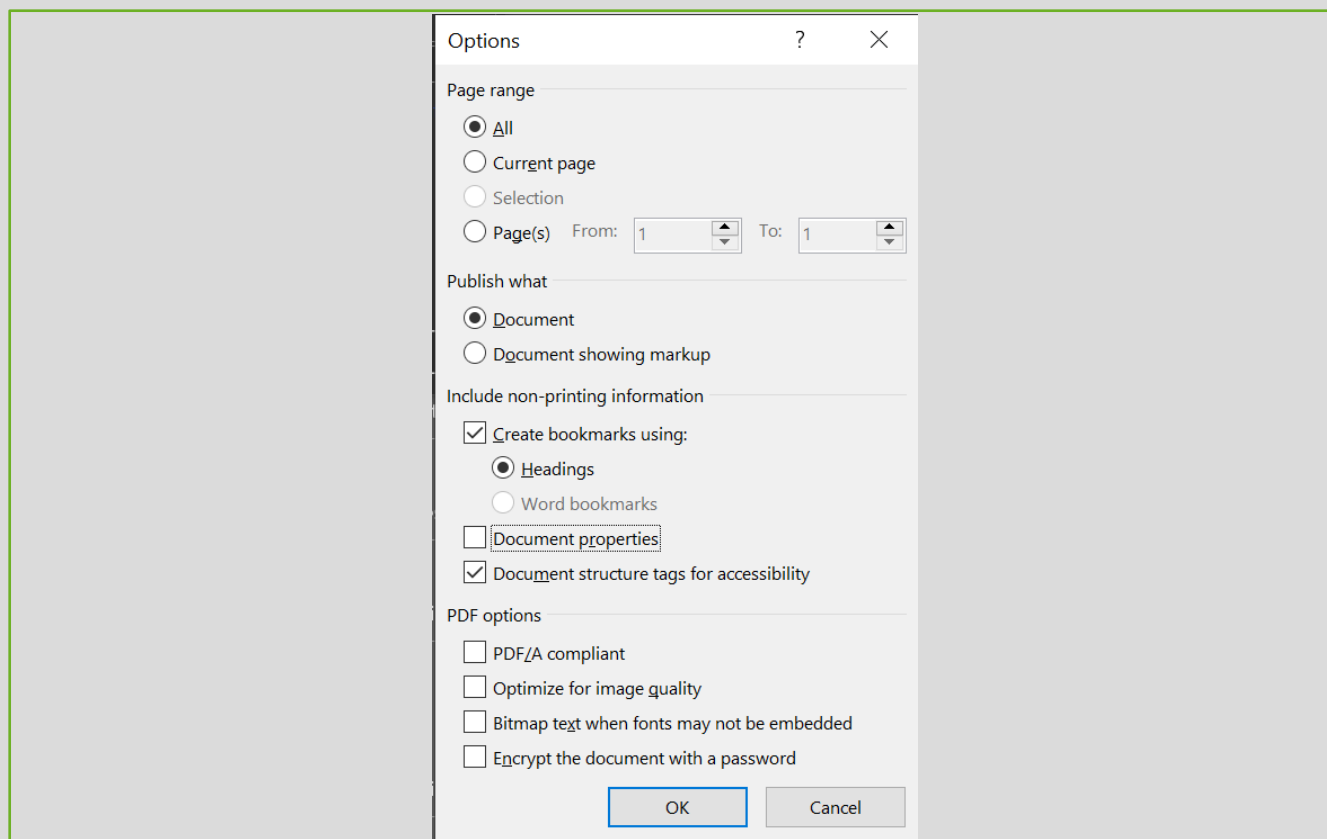
You could also use *File > Print* option and choose PDF as output format.

My goal is to minimize number of steps necessary to have PDF file and also use standardized set of parameters each time I want to save PDF file.



Pic. 112. Customized Save As (F12).

If user answers *No* (default choice), then standard dialog is displayed, where user may specify location and name of the file. If user answers *Yes*, then PDF file is saved in the default user file location with predefined set of parameters.



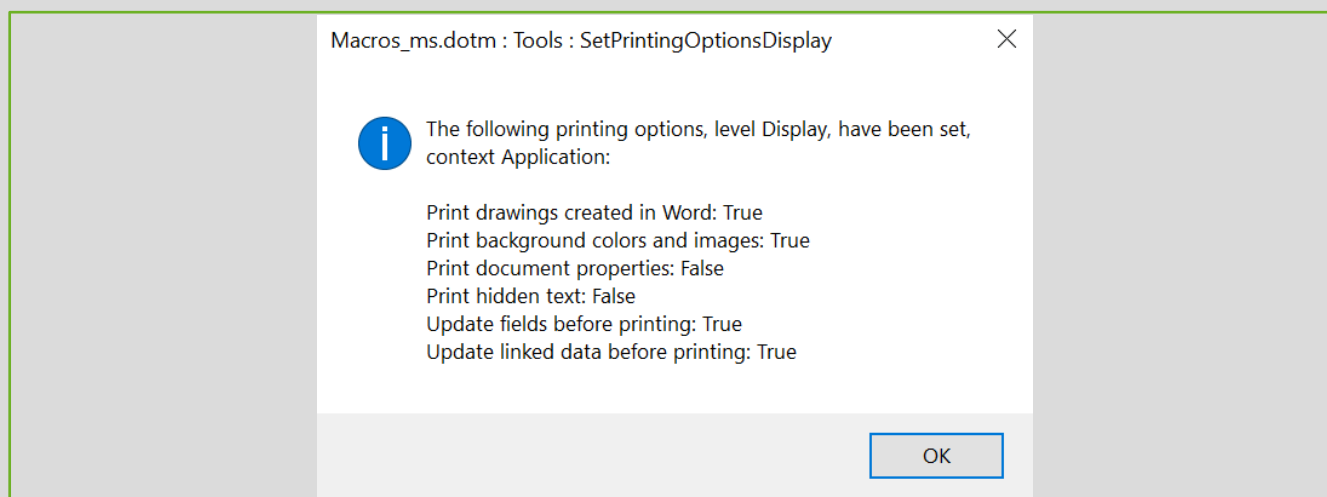
Pic. 113. Predefined set of parameters.

11.11. Printing enhancements

There is a macro assigned to Ctrl + P keyboard shortcut, which by default is used to call *Print* function. The same dialog box is used by default in Microsoft Word to print document.

Enhancements:

- » All document fields (including Table of Contents) are automatically updated prior to printing.
- » All references are checked if they are not broken prior to printing.
- » Default set of printing options is checked or set, if differs.
- » If document background color is set to custom grey color, user is asked if not restore white color before printing.



Pic. 114. Default set of printing options.

11.12. Closing file enhancements

There is a macro assigned to Ctrl + W keyboard shortcut, which by default is used to call Microsoft Word *C*lose function.

Enhancements:

- » All document fields (including Table of Contents) are automatically updated before file is closed.
- » All references are checked if they are not broken before file is closed.

Thanks to that macro you know that your content was refreshed and do not contain broken references.

If you don't want to run this macro, just close Microsoft Word window of your document with top right X button.

12. Configuration of Microsoft Word User Interface

In this chapter I focus on specific aspects of Graphical User Interface, such as:

1. ribbon menu
2. status bar
3. panes.

12.1. Introduction, Microsoft Word and size of 24 inches display

One of the unique features of Microsoft Word is hidden under horrible WYSIWYG acronym, which means *What You See Is What You Get*. In other words, Microsoft Word text or content processor can show on a screen the actual size, look and feel of a document, as it would look like upon printing.

The basic size of a paper sheet, at least in Europe, is rectangular, A4:

$$a_{A4} = 210 \text{ mm}$$

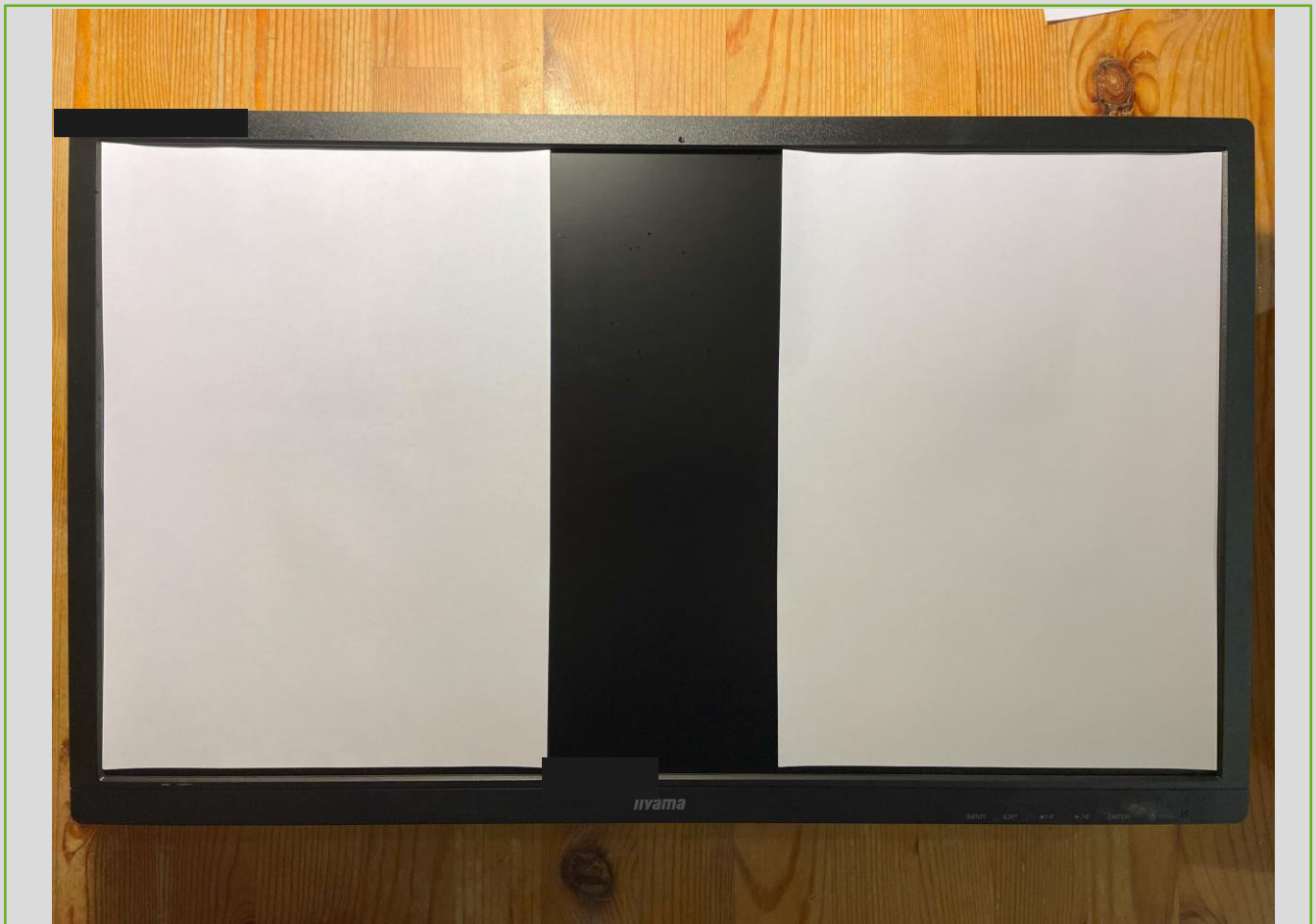
$$b_{A4} = 297 \text{ mm}.$$

Side note: the A... sized sheets of paper follow the interesting rule: paper in the A series format has an aspect ratio of $\sqrt{2}$ (≈ 1.414 , when rounded). A0 is defined so that it has an area of 1 m² before rounding to the nearest 1 millimeter. Successive paper sizes in the series (A1, A2, A3, etc.) are defined by halving the area of the preceding paper size and rounding down, so that the long side of A(n + 1) is the same length as the short side of A_n. Hence, each next size is nearly exactly half the area of the prior size. So two A2 pages (in landscape orientation) fit together over an A1 page (in portrait orientation), an A3 page is half an A2 page, A4 is half an A3 and so on [17].

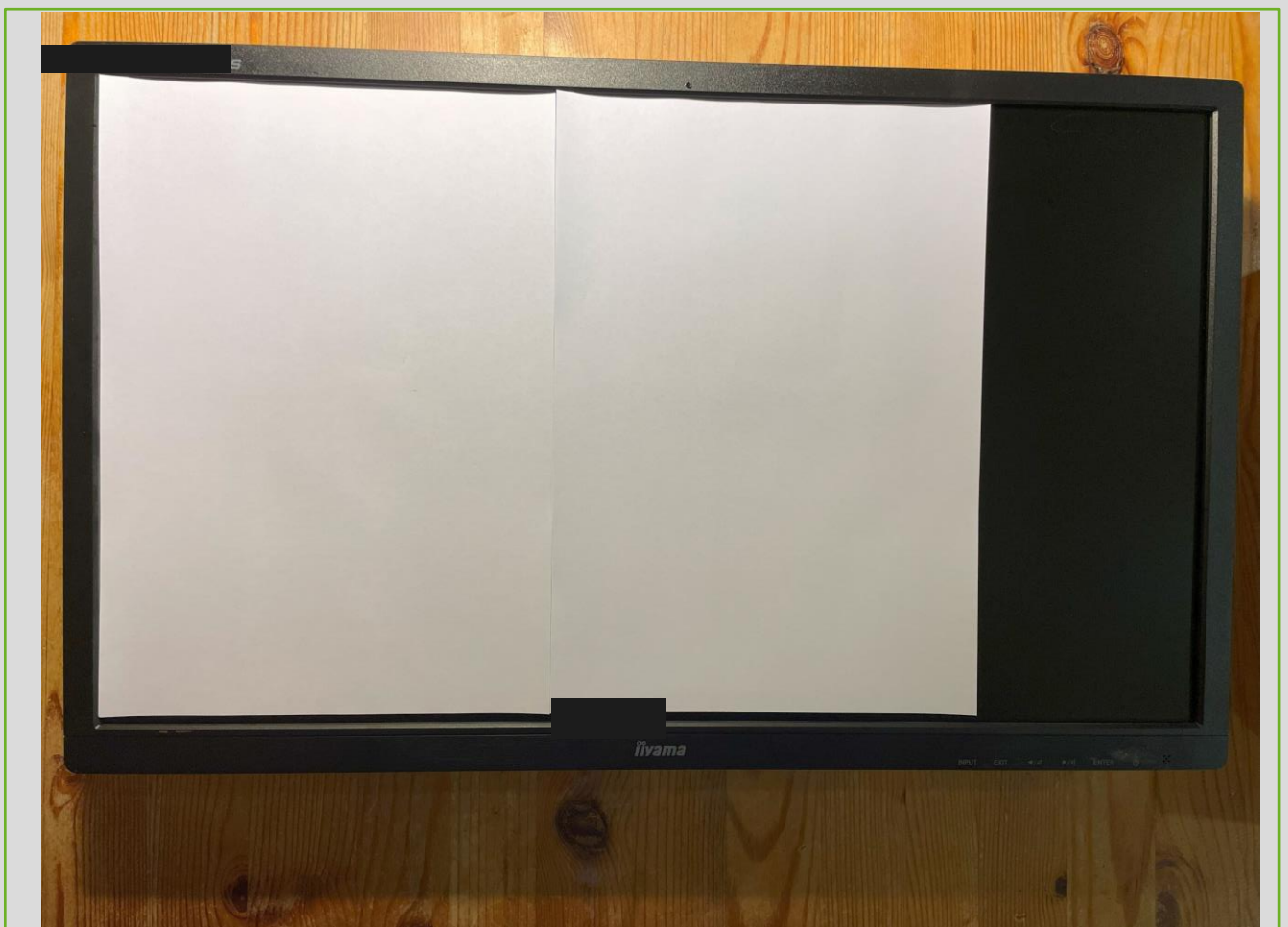
The very first size of computer display, which fits to this size, is 24" (inches), what means that its diagonal is equal to

$$\text{diagonal} = 24'' \approx 24 \cdot 25.4 \text{ mm} \approx 609.6 \text{ mm} = c$$

By strange coincidence I have in my office exactly 24" physical display [18]. For sake of experiment, I placed 2× sheets of A4 paper on a display, compare [Pic. 115](#) and [Pic. 116](#).



Pic. 115. 24" Display and 2× A4 blank pages, side to side.



Pic. 116. 24" Display and 2× A4 blank pages, moved to the left.

Following my 24" display specification:

$$a_{\text{display}} = 301 \text{ mm} > b_{\text{A4}} \text{ by } 4 \text{ mm}$$

$$b_{\text{display}} = 533.5 \text{ mm} > 2 \cdot b_{\text{A4}} \text{ by } 113.5 \text{ mm:}$$

$$533.5 \text{ mm} - 2 \cdot b_{\text{A4}} = 533.5 \text{ mm} - 2 \cdot 210 \text{ mm} = 113.5 \text{ mm}$$

Results:

- » It is possible to keep 2× A4 sheets of paper (A3) on the 24" display.
- » If you wish then to keep any menu "traditionally", I mean horizontally at the top, you are left with just 4 mm.
- » But you have 113.5 mm which is 38.2 % of horizontal for vertical menu(s).

So maybe it would be more reasonable to keep all menus, e.g. operating system and application, like Microsoft Word vertically? Then why can't a Microsoft Word ribbon menu be moved from its top horizontal position? Hopefully now you see more clearly that drawback.

12.2. Ribbon menu

The name *ribbon* for top menu was introduced by Microsoft. It has a big advantage that can be quite widely configured. Unfortunately, it cannot be unlinked from the top part of a screen, neither moved to either screen side.

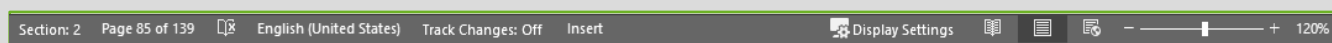
I strongly recommend you to learn how to configure your own ribbon menu and save output of your work to .exportedUI file. If you wish to learn more about this process, you can easily find detailed manuals in the Internet / YouTube. Please also visit chapter 8 of this book.

Once such file is imported, it is no longer updated upon changes added by user. You've been warned.

To save space on my screen personally I keep ribbon menu collapsed with built-in keyboard shortcut Ctrl + F1. This shortcut works as a toggle: pressed again restores position of ribbon menu.

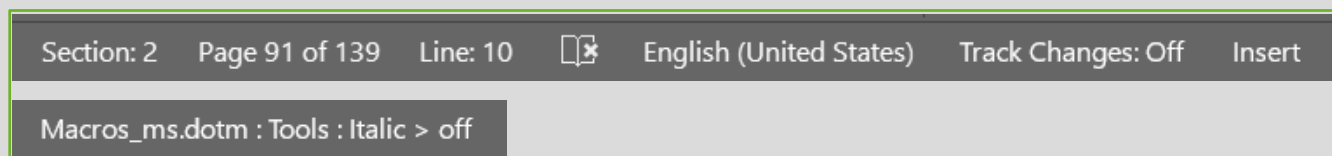
12.3. Status bar

By *status bar* is understood bottom part of Microsoft Word screen.



Pic. 117. Example of *status bar*.

Settings of *status bar* are not available from VBA code. They are not saved besides application context. But VBA macros can output messages to status bar area. Many of my macros which are called with keyboard shortcuts output their messages this way.



Pic. 118. Example, information displayed in the left part of *status bar* by macro.

To enter settings of the *status bar*, move mouse pointer over bottom left part of Microsoft Word screen and right click.

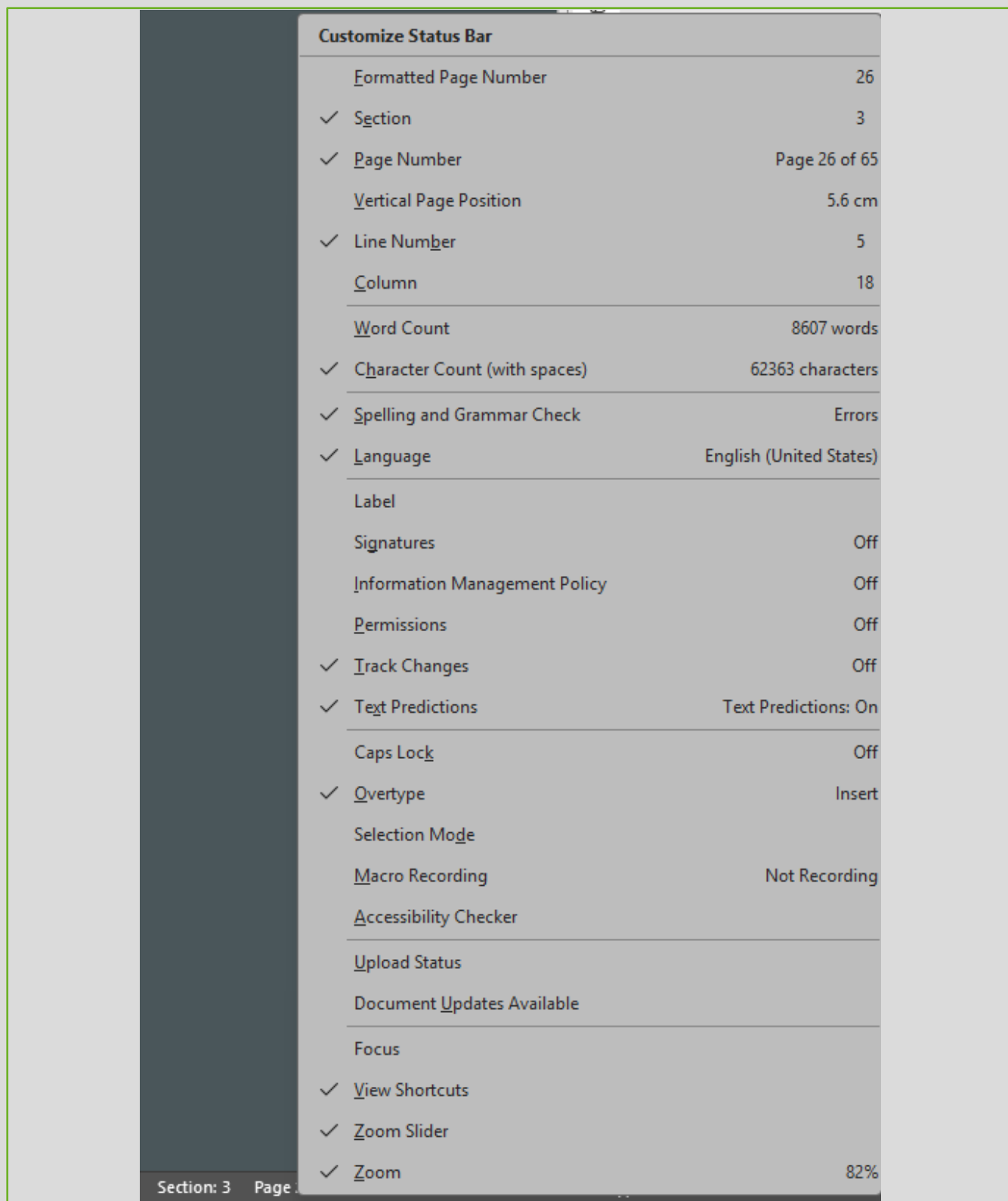
There are two handy keyboard shortcuts to toggle specific options, which state is shown in status bar:

- » Insert (INS): toggles insertion mode, aka text replacement mode of typing status bar: Overtyping.
- » Ctrl + Shift + E for toggling tracking changes status bar: Track Changes

The amount and availability of status bar parameters depend on version of Microsoft Word. I strongly recommend enable at least parameter Section.

My settings of status bar take into account:

- » Section
- » Page Number
- » Line Number
- » Character Count
- » Spelling and Grammar Check
- » Language
- » Track Changes
- » Text Predictions
- » Overtyping
- » View Shortcuts
- » Zoom Slider
- » Zoom



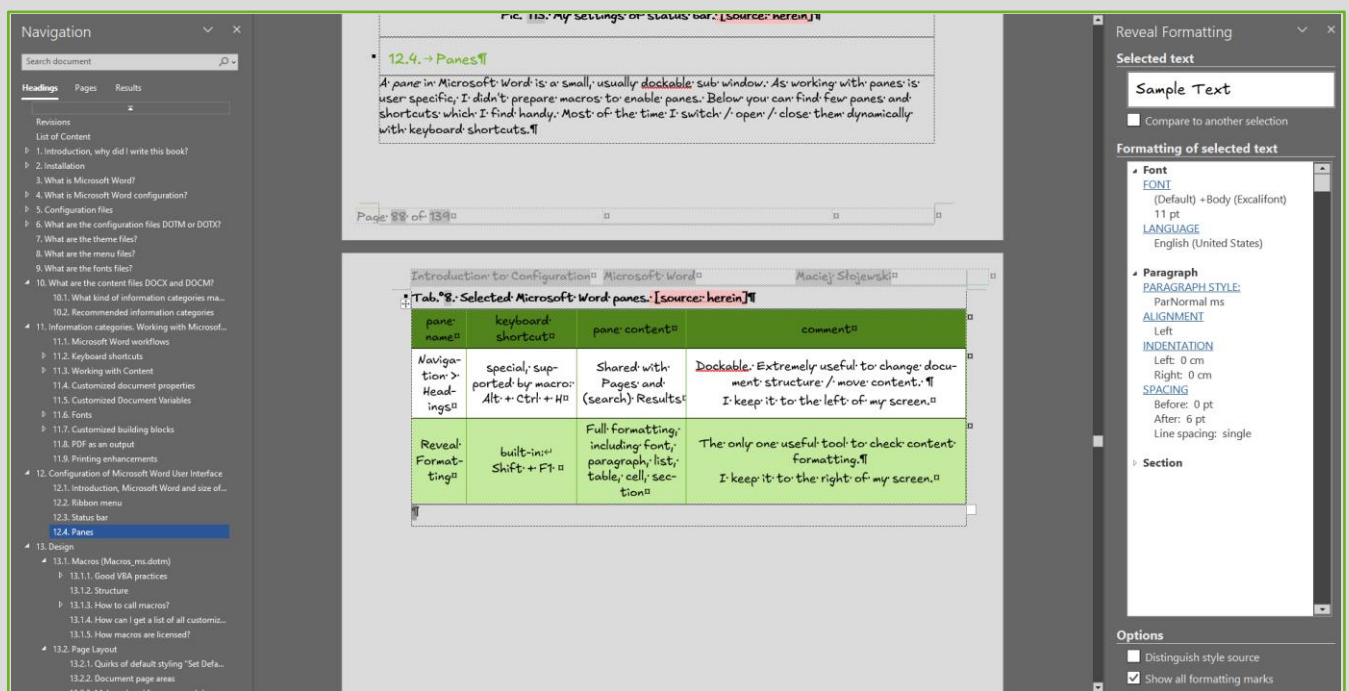
Pic. 119. My settings of status bar.

12.4. Panes

A pane in Microsoft Word is a small, usually dockable, sub window. By dockable I mean it can be moved outside of the parent window or glued to left or right edge of parent window. As working with panes is user specific, I didn't prepare macros to enable panes. Below you can find few panes and shortcuts which I find handy. Most of the time I switch / open / close them dynamically with keyboard shortcuts.

Tab. 8. Selected Microsoft Word panes.

pane name	keyboard shortcut	pane content	comment
Navigation > Headings	special, supported by macro: Alt + Ctrl + H	Shared with Pages and (search) Results	Extremely useful to change document structure / move content. I keep it to the left of my screen.
Reveal Formatting	built-in: Shift + F1	Full formatting, including font, paragraph, list, table, cell, section	The only one useful tool to check content formatting. I keep it to the right of my screen.



Pic. 120. Both panes opened and docked to left and right edge of parent window.

12.5. CustomUI14.xml

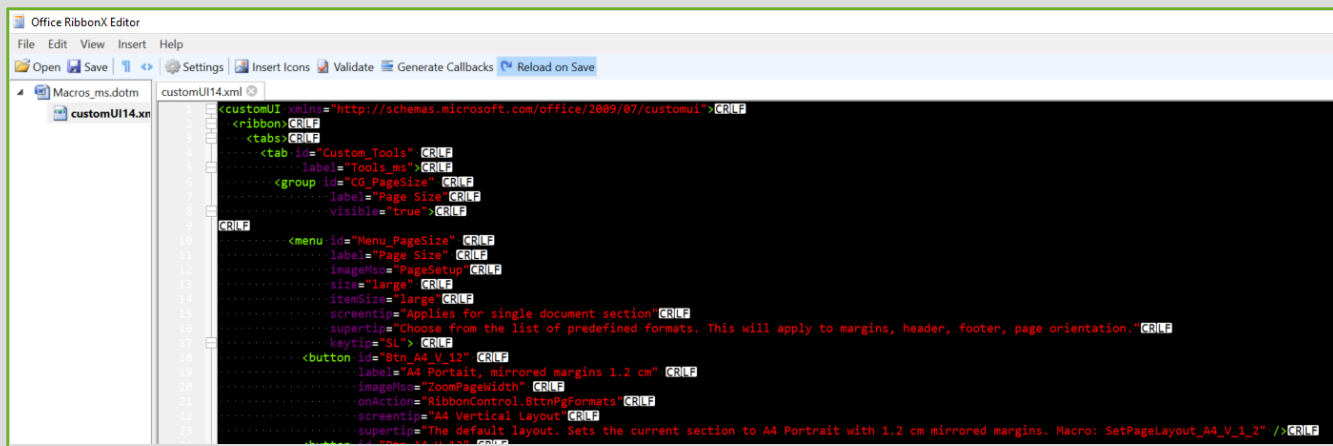
You can get more customization options in terms of User Interface, if you are willing to accept XML format of configuration files. Resource to gain some technical background and information about Ribbon customization is available in a series of Microsoft articles:

- » [Customizing the 2007 Office Fluent Ribbon for Developers \(Part 1 of 3\) \[19\]](#)
- » [Customizing the 2007 Office Fluent Ribbon for Developers \(Part 2 of 3\)](#)
- » [Customizing the 2007 Office Fluent Ribbon for Developers \(Part 3 of 3\)](#)

To start your adventure with this level of customization I recommend you to download the [Office RibbonX Editor \[8\]](#) and any Microsoft Office set of icons [\[9\]](#). You can find excellent introduction in the article [Customize the Word Ribbon User Interface \(It Doesn't Take Rocket Science\)](#) by Greg Maxey [\[11\]](#).

In this book repository [\[15\]](#) you can find example of [CustomUI14.xml](#) file which is embedded in the *Macros.ms.dotm* and contains all customized ribbon menus described in this book.

The *Office RibbonX Editor* do not need to be installed. It can work as a standalone application. Run the *OfficeRibbonXEditor.exe* and open the *Macros.ms.dotm*.



Pic. 121. OfficeRibbonXEditor and Macros_ms.dotm.

You not only need a dedicated XML file. Second step is dedicated VBA module, which works as a bridge between XML identifiers and your macros. This module is named *RibbonControl*. Example code snippet:

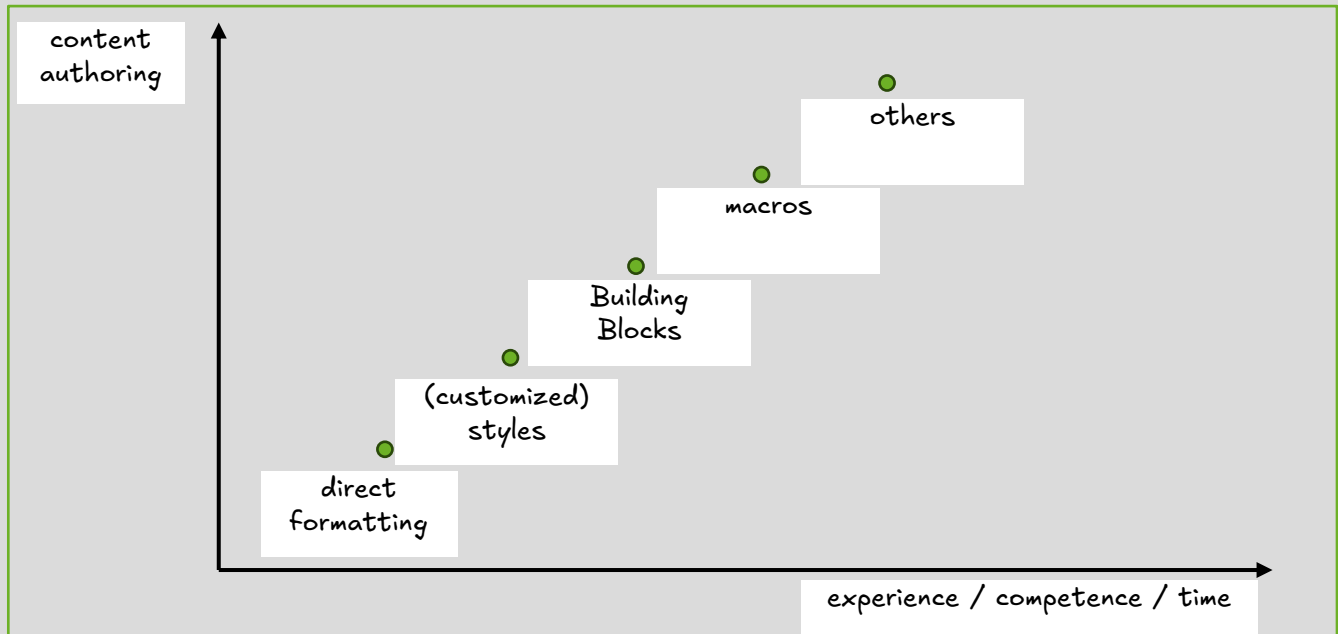
```
' tab: Macros_ms
' 2026-03-09 by ms
Sub BtnMacros(control As IRibbonControl)
    Select Case control.ID
        Case "Btn_MacrosImport"
            Call Macros_ms.Macros.ImportAllVBAModules
        Case "Btn_MacrosExport"
            Call Macros_ms.Macros.ExportAllVBAModules
        Case "Btn_MacrosDelExcept"
            Call Macros_ms.Macros.DeleteAllVBAModulesExceptMacros
        Case "Btn_MacrosDelAll"
            Call Macros_ms.Macros.DeleteAllVBAModules

        Case "Btn_MacrosCounter"
            Call Macros_ms.Macros.ShowMacrosCounter
        Case "Btn_MacrosList"
            Call Macros_ms.Macros.ListMacros
        Case "Btn_MacrosNonAscii"
            Call Macros_ms.Macros.ScanProjectForNonAscii
    End Select
End Sub
```

From one hand the *CustomUI14.xml*/added value are bigger flexibility and access to whole spectrum of ribbon options. From the other hand it requires much more work to prepare it. The choice is yours. From my perspective it was important to limit amount of files which need to be installed by user and keeping User Interface in one file with macros.

13. Design

Less is more.



Pic. 122. Relationship between experience / competence / time and content authoring.

13.1. Macros (Macros.ms.dotm)

Microsoft Word scripts (macros) are prepared in specific language named VBA (Visual Basic for Applications).

The VBA is from one hand compiled language, from the other it is interpreted, classic "best of both worlds" situation. In other words, it is a hybrid language. It uses a two-stage process that allows it to be fast enough for heavy spreadsheets while remaining flexible enough for you to edit code on the fly.

When you finish writing a macro and hit 'Run' (or when the project compiles in the background), VBA does not stay as a plain text. It is converted into P-Code (Packed Code or Pseudo-code). The VBA editor analyzes your syntax and transforms those English-like commands into an intermediate, binary-like format. This step catches 'Compile Errors' (like missing End If) before the code even starts. Because it is already pre-digested into P-Code, the computer does not have to read and spell-check every line of text while the macro is actually running.

Even though the code is pre-compiled into P-Code, your computer's processor cannot read P-Code directly. It needs a translator. When you execute the macro, a component called the VBA Virtual Machine (part of the DLLs like VBE7.DLL) 'interprets' that P-Code line-by-line into machine instructions the CPU understands. Unlike a compiled language, which turns into a standalone .exe file, VBA requires the host application (e.g. Microsoft Word) to act as an interpreter during runtime. This is also why you can 'pause' code, change a variable, and keep going - a feature that is very difficult in purely compiled language.

I decided to start my journey with VBA even that language itself as well as its native editor VBE (Visual Basic Editor) are constrained and lack many features easily available in modern SDKs.

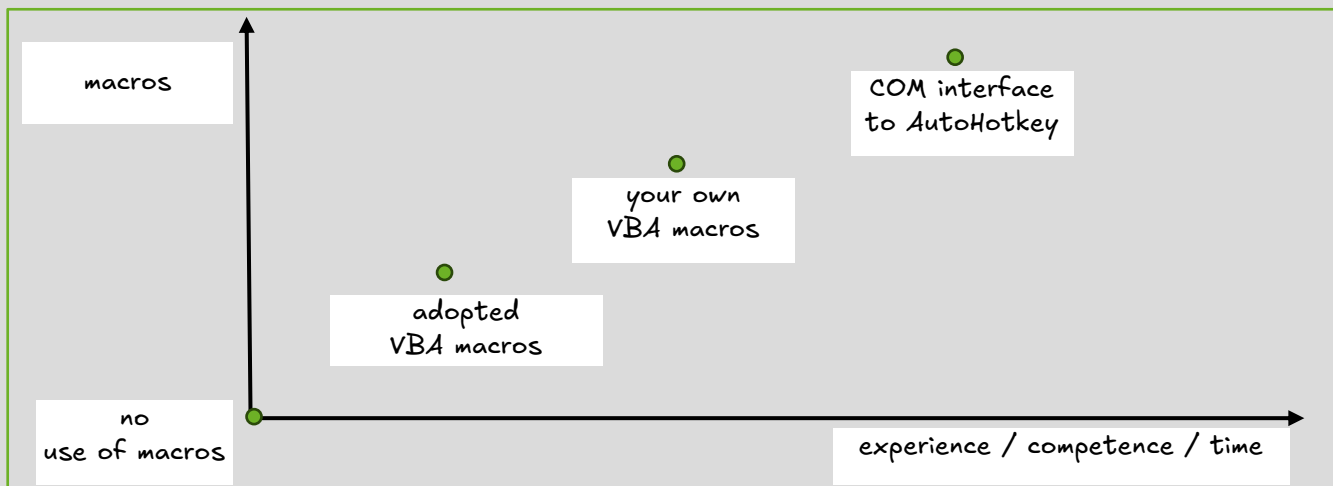
The main added value coming from VBA from my perspective:

- a. I could replace graphical representation of styles with text representation. As a consequence, I could use the same constants to be used with many styles at the same time.

- b. Some actions cannot be done without VBA application. For example, access to *Document Variables* is possible only from VBA.
- c. Some actions can be linked together. For example, the built-in *Save* function can be wrapped before or after with custom VBA macros. Thanks to that I can before saving refresh all fields in document body.
- d. Some actions help to minimize or even cancel quirks of Microsoft Word. For example, switching fields view with **Alt + F9** keyboard shortcuts natively causes Microsoft Word to jump to random place in a document. With this wrapping technique I can force it to scroll document content to the place from which it was called.
- e. I can add missing functions, like
 - i. checking for double spaces, double dots, lonely dots etc.
 - ii. save page layout parameters (margin distances, header / footer distances) in form of VBA macro
- f. and many, many more...

Preparing so many macros would not be possible without progress of Artificial Intelligence (AI). Help coming from chats and other dedicated tools significantly speeded up my learning.

Use of macros in Microsoft Word from my perspective is one of the most important competences, which you can gain, and main way to improve your everyday life and work quality.



Pic. 123. Microsoft Word User Interface customization levels.

Please note, that VBA macros are not at the very top of available competences. Above there is COM (Component Object Model, a Microsoft software interface technology) interface, which enables for example calling of Microsoft Word functions from e.g. [AutoHotkey \[12\]](#) scripts. For example, it is possible to build your own menus in separate windows, which you can move to any of your screens. But this subject is beyond of what I wanted to say in this book.

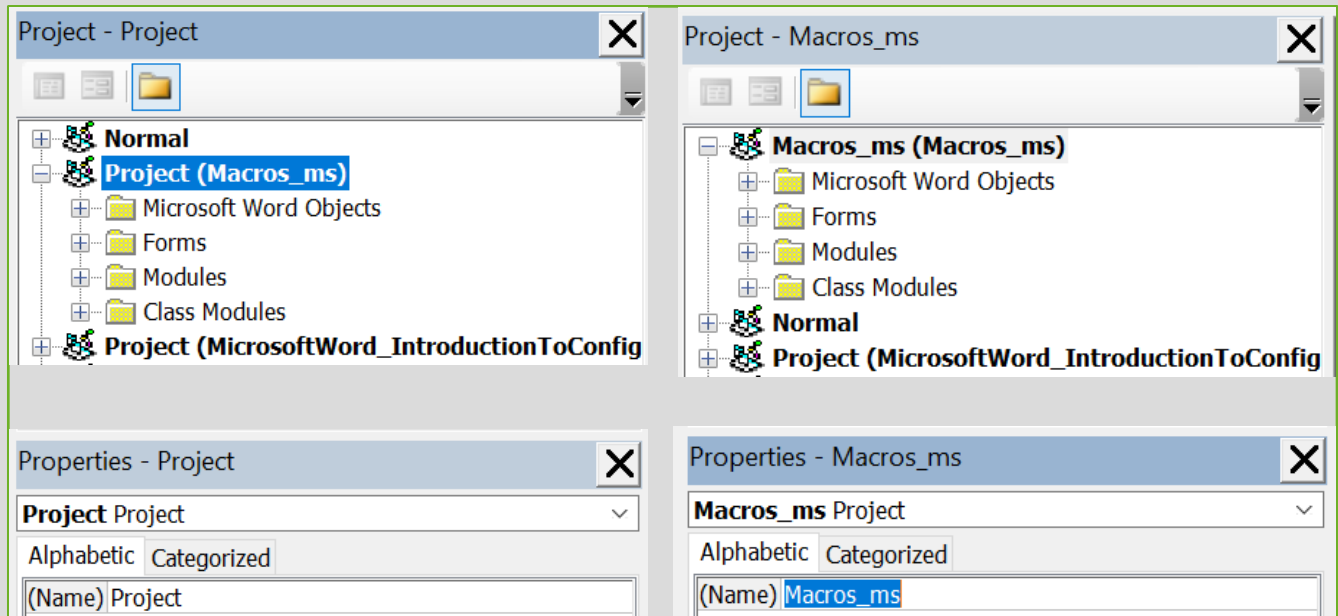
13.1.1. My VBA practices

In this chapter I share with few techniques that I applied in my code. By far this list is not enough to master VBA:

- » name your project
- » apply sub headers
- » frequently compile your code

13.1.1.1. Name your project

By default, each new DOTM file is named *Project*. It is possible to give it specific name. When you will have hundreds of macros in different files, it can be a case you will repeat the same macro name. To minimize consequences of such an event, you can add unique name to your project.



Pic. 124. Change project name, *Project* (left) → *Macros_ms* (right).

Use your project name to precisely determine subroutine calls.

Example. Instead generic subroutine call:

```
Call Set_CommandShortcuts(IfMsgBox:="quiet")
```

Use precise subroutine call:

```
Call Macros_ms.Shortcuts.Set_CommandShortcuts(IfMsgBox:="quiet")
```

13.1.1.2. Apply sub headers

Whenever you use MsgBox for communication with user, it is a chance to inform her / him which macro actually displays it. For that purpose, I add to every sub (macro) the following sequence of variables.

```
Dim FileName As String:      FileName = C_F_Macros
Dim ModuleName As String:    ModuleName = C_M_Macros
Dim MacroName As String:     MacroName = "YourMacroName"
Dim MsgBoxTitle As String:   MsgBoxTitle = FileName & " : " & ModuleName & " : " & MacroName
```

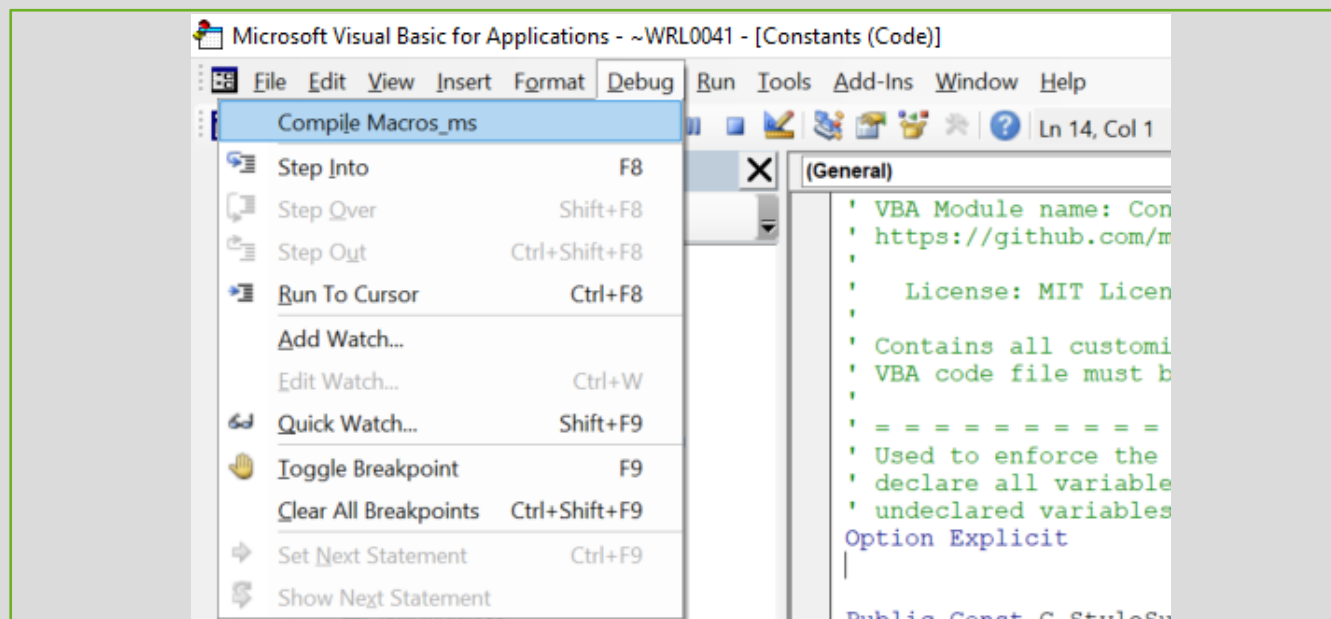
For MsgBox I use at least 3× arguments: Prompt, Buttons and Title.

```
MsgBox _
    Prompt:="Line 1" & vbNewLine & _
    "Line 2", _
    Buttons:=vbCritical + vbOKOnly, _
    Title:=MsgBoxTitle
```

13.1.1.3. Frequently compile your code

It is never enough to remind myself about code compilation. For me it is by far the quickest way to spot my errors. Especially if I assume that my change in code is 'minor'.

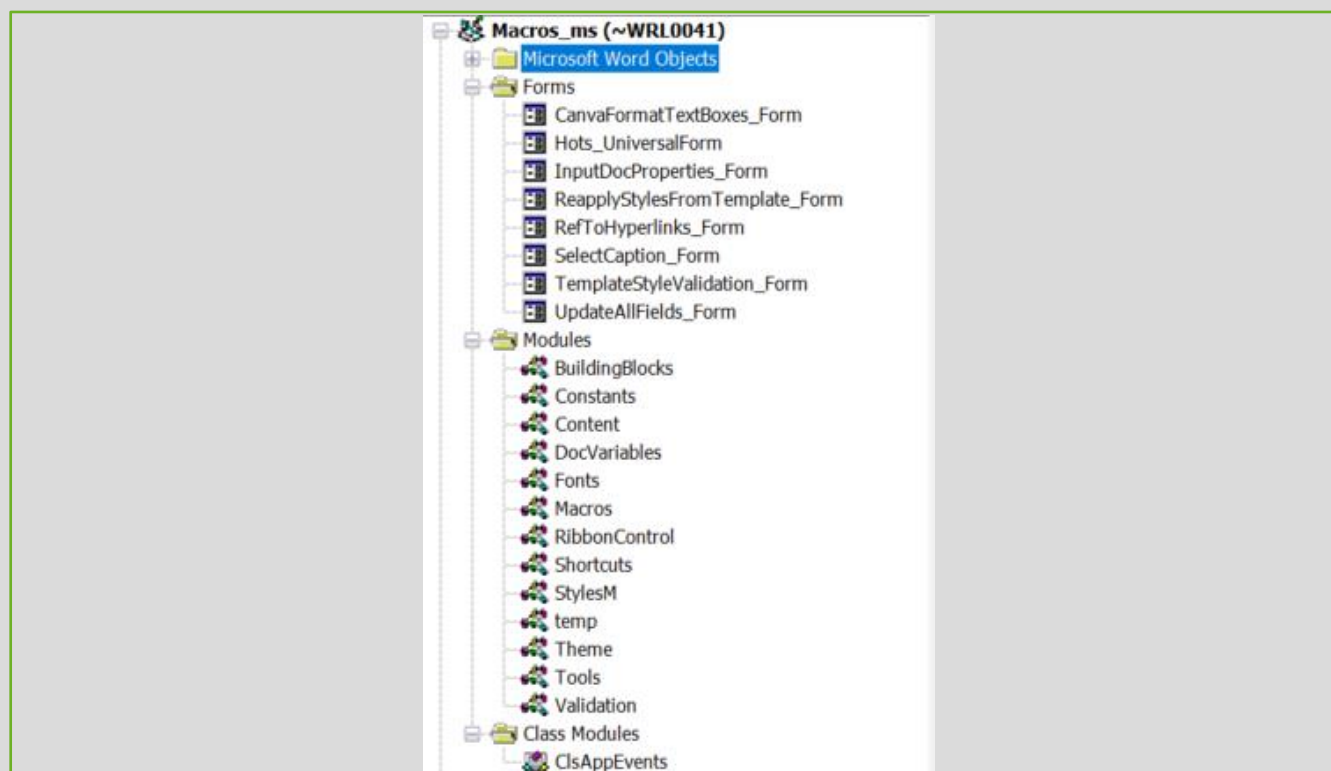
Unfortunately, there is no keyboard shortcut to this function.



Pic. 125. Debug > Compile function.

13.1.2. Structure

All my macros are stored in single file, *Macros.ms.dotm*.



Pic. 126. Structure of *Macros.ms.dotm*.

Each module starts with a commented-out header informing about the most important macros available to be called from User Interface. For example, header of the module Macros:

THIS PAGE IS LEFT INTENTIONALLY BLANK

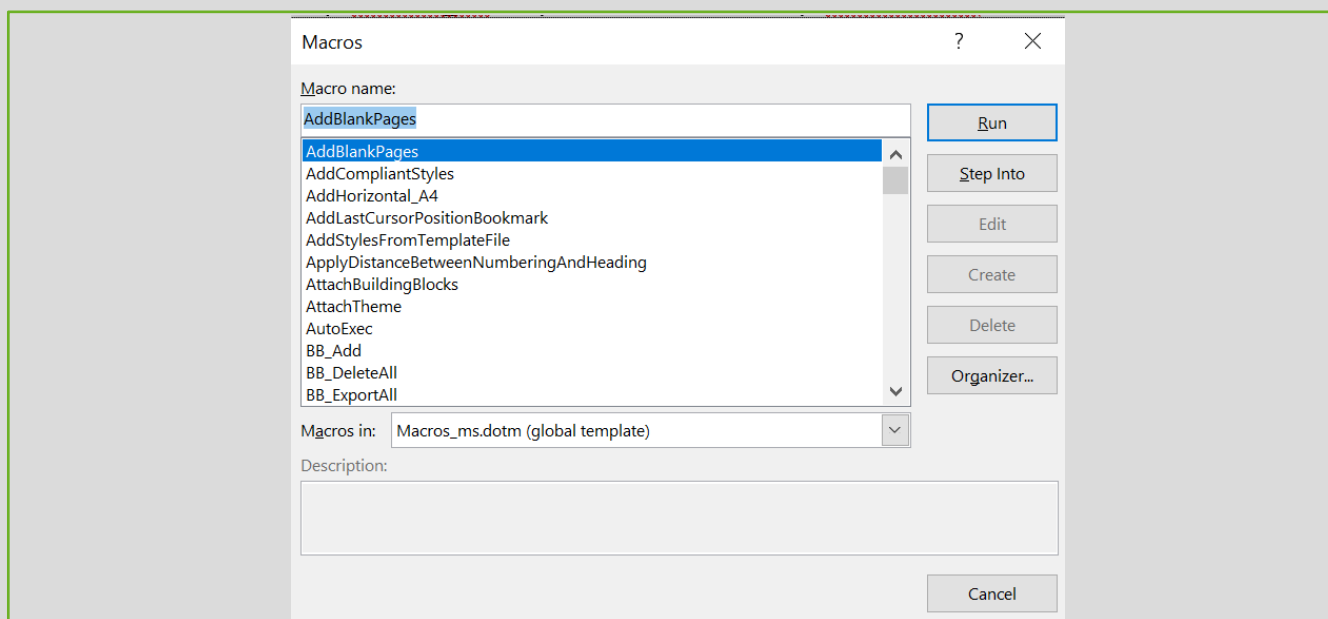
13.1.3. How to call macros?

There are several ways to call custom made macros:

- » general, built-in method
- » dedicated ribbon menu
- » keyboard shortcuts

13.1.3.1. General, built-in method

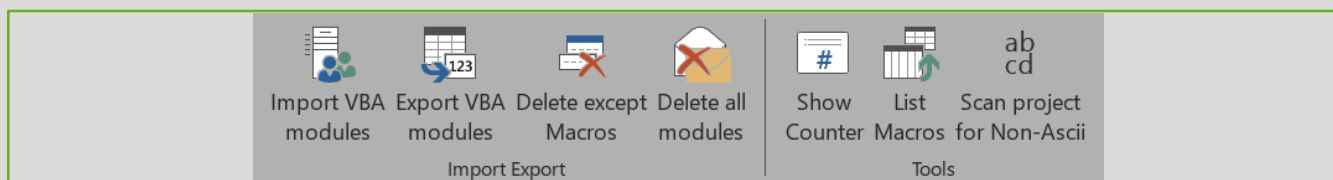
Just use Alt + F8 keyboard shortcut. Knowing the exact macro name enables precise calling it.



Pic. 127. User Interface, Macros, Alt + F8.

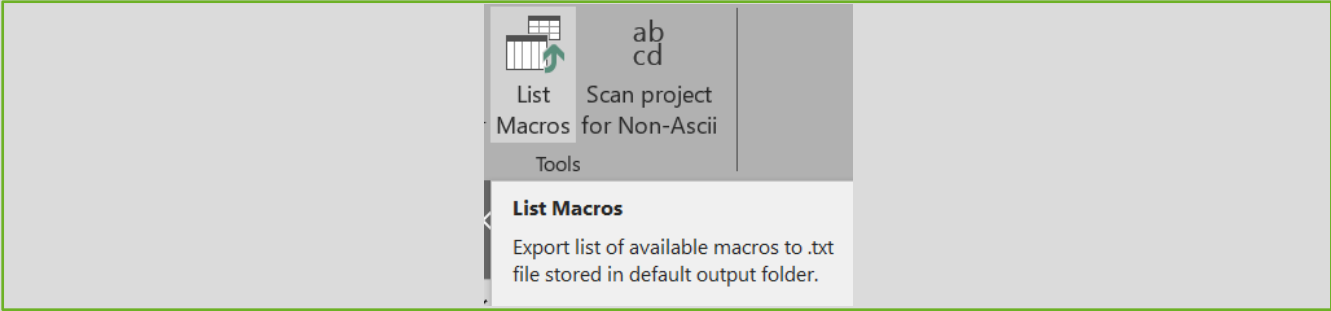
13.1.3.2. Dedicated ribbon menu

For the most of macros, at least those important to everyday tasks, I've prepared User Interface in form of dedicated ribbon menus. Those ribbon menus are embedded in *Macros_ms.dotm* in form of the file *customizeUI14.xml*. Code of this file is available in project repository [\[15\]](#). Names of positions in menu more or less correspond to names of macros in the modules.

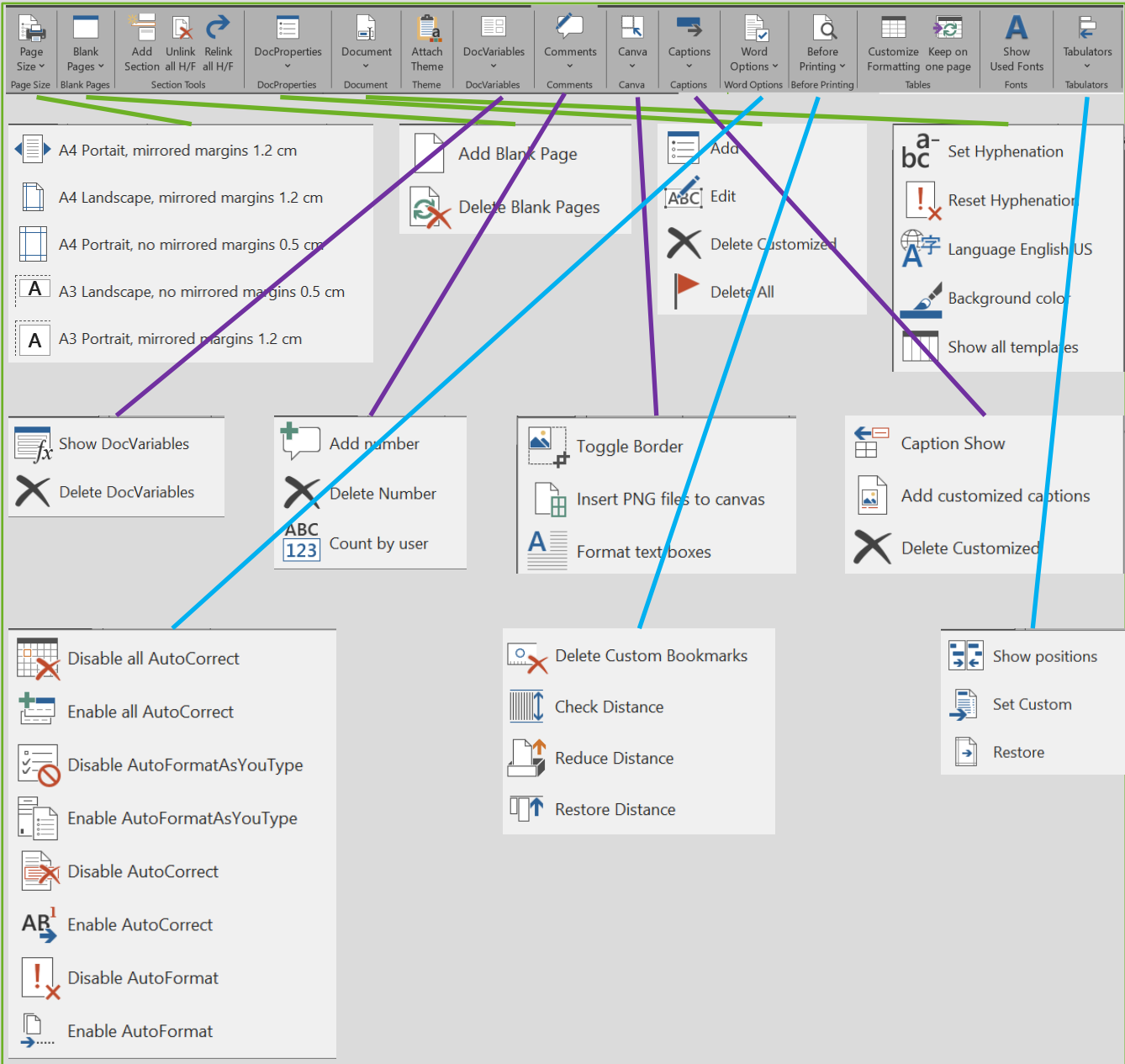


Pic. 128. Menu: Macros_ms.

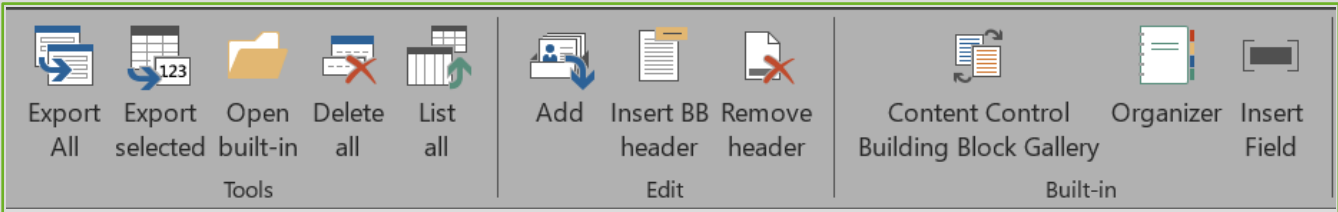
Whenever you move mouse cursor over specific icon, you get screen tip with short information about purpose of each macro.



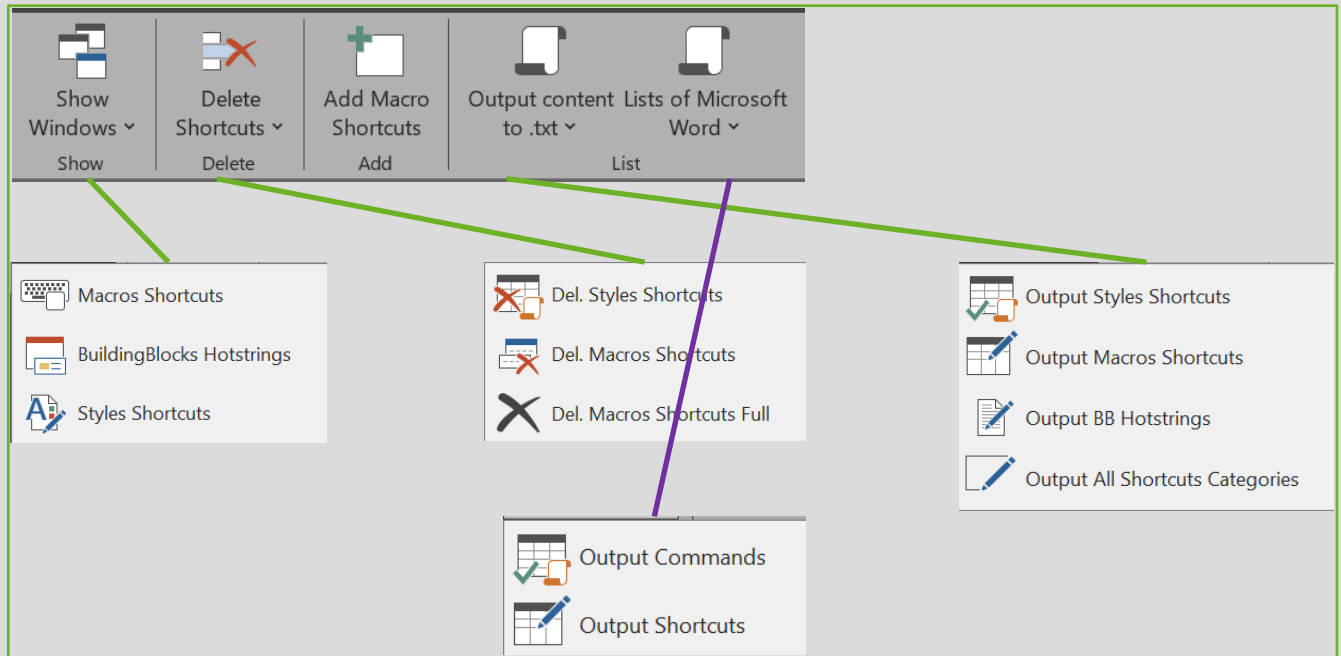
Pic. 129. Screen tip macro example.



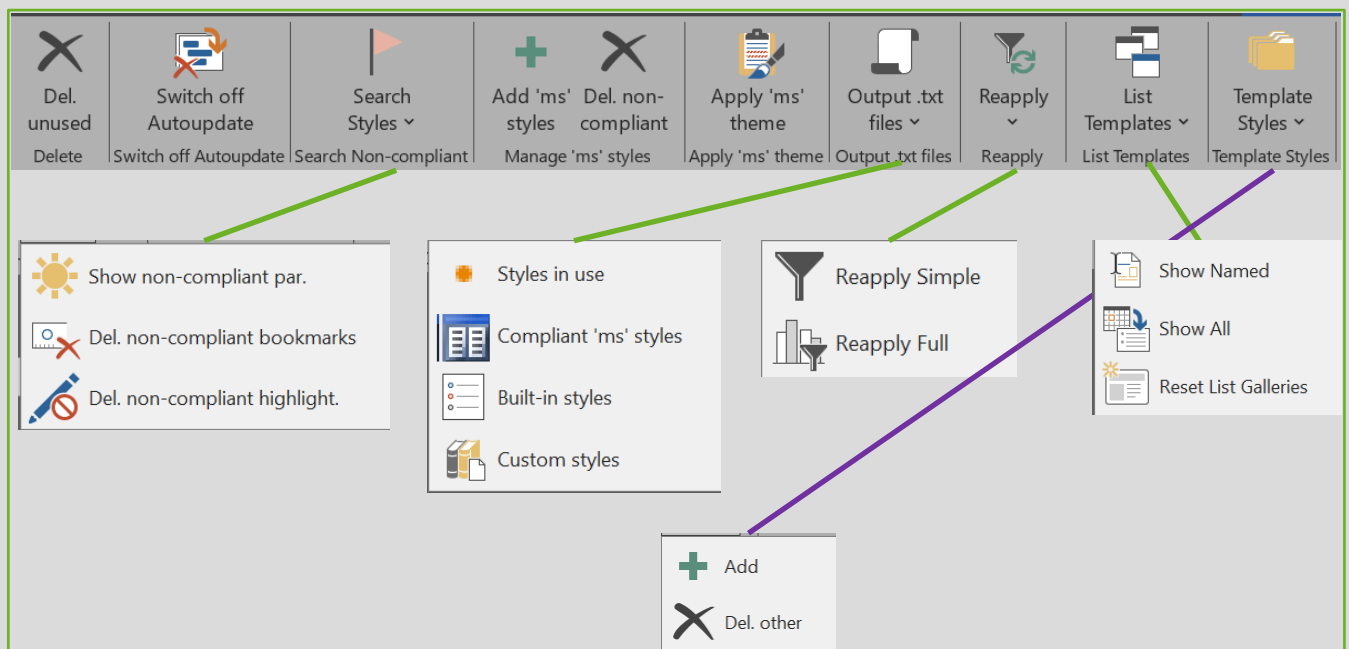
Pic. 130. Menu: Tools_ms.



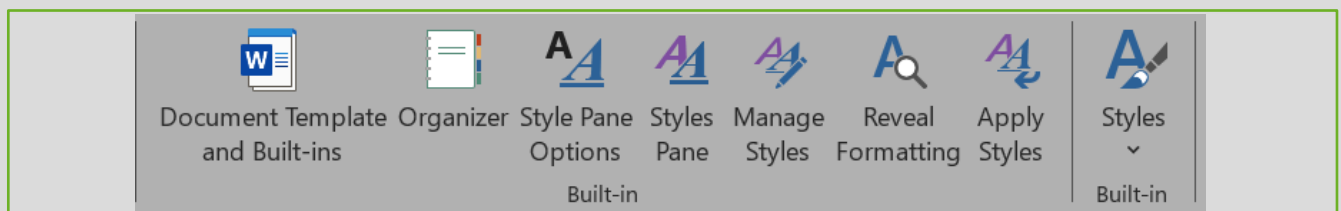
Pic. 131. Menu: BuildingBlocks_ms.



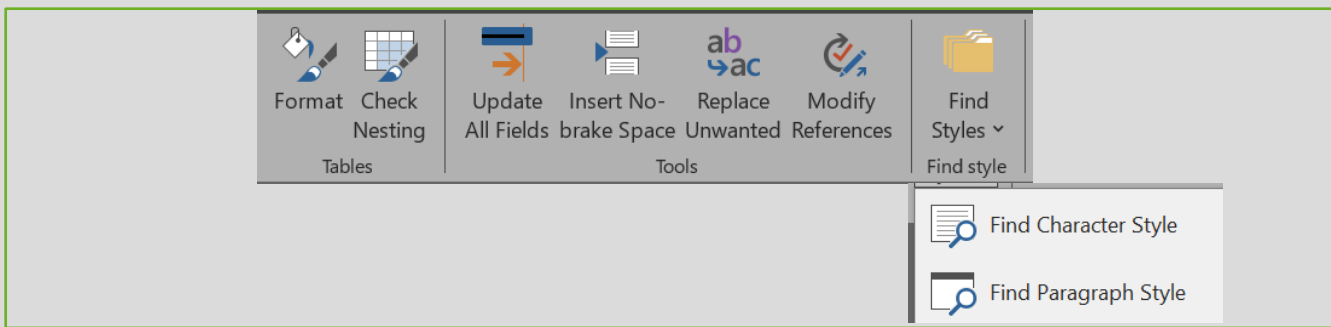
Pic. 132. Menu: Shortcuts_ms.



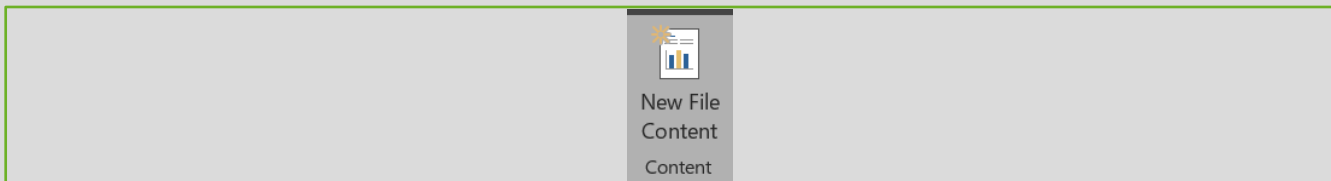
Pic. 133. Menu: Styles_ms.



Pic. 134. Menu: Styles_ms, built-in Microsoft Word functions.



Pic. 135. Menu: Validation.



Pic. 136. Menu: Content_ms.

13.1.3.3. Macro customized keyboard shortcuts

Some macros are not specified within User Interface menus. Those are macros called with keyboard shortcut. You can get a list of those shortcuts from menu > *Shortcuts_ms* > *List* > *Output content to .txt* > *Output Macros Shortcuts*.

Tab. 9. Macro customized keyboard shortcuts.

No.	macro name	shortcut	comment
1.	FormatFont	Alt+F	calls <i>Font</i> window
2.	NavPane	Alt+Ctrl+H	calls <i>Navigation</i> pane; unfortunately, there is no way in VBA to specify <i>Headings</i> tab
3.	FormatParagraph	Alt+Ctrl+P	calls <i>Paragraph</i> window
4.	CustomizedOvertyp	Insert	If you enable <i>File</i> > <i>Options</i> > <i>Advanced</i> > <i>Editing Options</i> > <i>Use the insert key to control overtype mode</i> and <i>Customize Status Bar</i> > <i>Overtyp</i> then each time you press keyboard key <i>Insert</i> , you get extra message box on the screen
5.	ToggleSpecificFormatting	F4	See chapter 11.1
6.	InsertCrossRef	F7	See chapter 11.9
7.	SetLanguageToEnglishUS	F8	Select the text and press F8 to switch language to English US
8.	CustomizedSaveAs	F12	Asks a question if not save document as PDF file or just DOCX
9.	Bold	Ctrl+B	Added value: prompt in <i>Status Bar</i>
10.	Italic	Ctrl+I	Added value: prompt in <i>Status Bar</i>

No.	macro name	shortcut	comment
11.	CustomizedPrinting	Ctrl+P	Updates all fields in document. Checks set of printing options of Microsoft Word. If they are not aligned with required pattern, asks a question about set up of internal printing options of Microsoft Word. Checks if background color of document is set to white. If not, asks a question about switching to white background. Displays old fashioned <i>Print</i> window, which is much easier to navigate to me.
12.	CustomizedFileSave	Ctrl+S	Removes unnecessary building block tags and saves document
13.	Underline	Ctrl+U	Added value: prompt in <i>Status Bar</i>
14.	UpdateAllFieldsAndCloseFile	Ctrl+W	Updates all fields, checks references and closes the file
15.	CustomizedPrintPreviewAndPrint	Ctrl+F2	Updates all fields, remembers from which page was called <i>Print Preview</i> ; when you close <i>Print Preview</i> with Esc keyboard key, you may continue from where you have finished
16.	CustomizedCopyFormat	Ctrl+Shift+C	Added value: restored old shortcut of <i>Format Painter</i>
17.	ToggleCharHiddenStyle	Ctrl+Shift+H	Toggles for selected text the <i>CharHidden</i> ms style; added value: for better visibility background color of the hidden text is set to red
18.	ToggleCharSourceCode	Ctrl+Shift+K	Toggles for selected text the <i>CharSourceCode</i> ms style
19.	ToggleApplyStyles	Ctrl+Shift+S	Calls window <i>Apply Styles</i> . When this keyboard shortcut is pressed again outside of the <i>Apply Styles</i> window, it is closed
20.	CustomizedPasteFormat	Ctrl+Shift+V	Added value: restored old shortcut of <i>Format Painter</i>
21.	Strikethrough	Ctrl+Shift+X	Toggles for selected text the <i>CharCrossout</i> ms style
22.	ShowFormHotkeys	Alt+H,K	See chapter 11.2.6
23.	ShowFormHotMacros	Alt+H,M	See chapter 11.2.6
24.	ShowFormHotstrings	Alt+H,S	See chapter 11.2.6
25.	RestartListNumbering	Alt+L,R	See chapter 11.3.3.2
26.	ReapplyTemplateStyle	Alt+R,S	Reapply previously set style for that paragraph
27.	JumpToNextList	Alt+F2	Jumps to next list or next list position
28.	JumpToNextTable	Alt+F3	Jumps to next table in document
29.	JumpToNextCanvas	Alt+F5	Jumps to next canvas in document
30.	CustomizedToggleFieldCodes	Alt+F9	Added value: cursor stays where it was on time of calling <i>ViewFieldCodes</i>

No.	macro name	shortcut	comment
31.	ToggleHeadingCollapseExpand	Alt+Ctrl+[	If cursor is in heading and this keyboard shortcut is pressed, it is toggled between states collapsed and expanded

13.1.4. How can I get a list of all customized macros in this project?

It is possible to generate a list of all macros in this project. Run the following macro when add-in file containing macros is the active document: menu > *Macros_ms* > *Tools* > *List Macros*.

13.1.5. How macros are licensed?

All code of macros is open sourced, according to [MIT license](#) [1].

13.1.6. How to run certain macro each time document is opened?

By default, Microsoft Word uses special macros for that purpose which must be run from *ThisDocument* module in *Normal.dotm* template, because this template is loaded by default when Microsoft Word starts. There is however bypass of this obstacle. In the *Macros_ms.dotm* you can find class module named *ClsAppEvents* and sub *AppWord_DocumentOpen*. You can use it to add any of your macros to be run automatically whenever Microsoft Word document is opened.

13.2. Page Layout

The concept of page layout takes into consideration:

- » size and orientation of page
e.g. A4 page sheet, Portrait
- » margins,
e.g. mirrored, gutter issue
- » header and footer area

13.2.1. Document page areas

You can set up plenty of options, to get just simple page of text to your liking. I'm going to show you, how to do that, step by step.

Let's assume the page area is just divided into the following areas:

- » document body
- » header
- » footer
- » margins (left, right, top, down)

The header is the area above document body. The footer is the area below document body. Margins are to the left, right, top and down of the document body.

So wait, where does the top margin start, and where does a header? We will answer that question in a minute.

Soon we will see that there are more than those areas on the page. Probably even now some of you think about comments and endnotes.

13.2.2. Make colored frame around document content area

In this chapter we will set up a wide frame around document body area, to better visualize it, as shown in [Fig. 137](#).

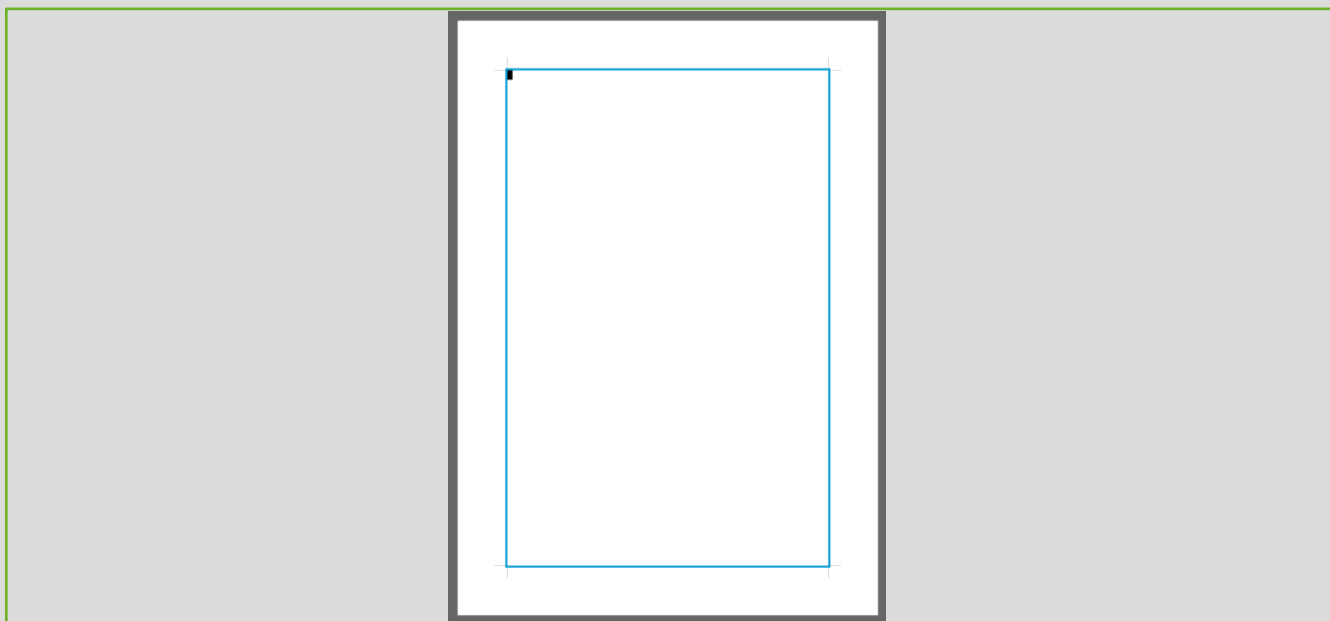


Fig. 137. Shape of our future page overview.

Let's start from the new document and default page settings. Open completely new document (e.g. `Ctrl + N`, or *File > New*) and set default page margins: *Layout > Page Setup > Margins > Normal*. We get single section, one page document with default size of margins, header and footer, all symmetric.

(My quiet assumption is, that your template file is the default *Normal.dotm*).

Now select *Design > Page Background > Page Borders*. There is plenty of settings in the *Borders and Shading* window which should pop up. We will not discuss them one by one, this is not our purpose. Jump directly to the tab *Page Border* and select the following settings: box, color blue, width to 3 pt. Now a crucial point, click *Options* button.

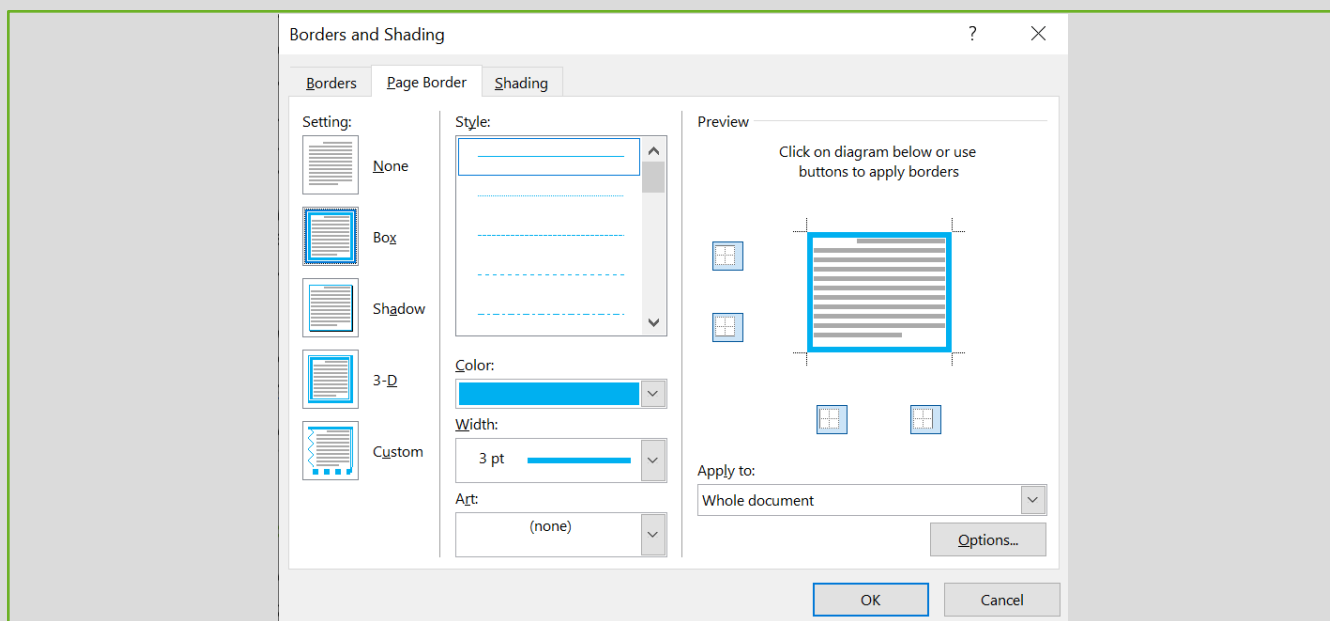
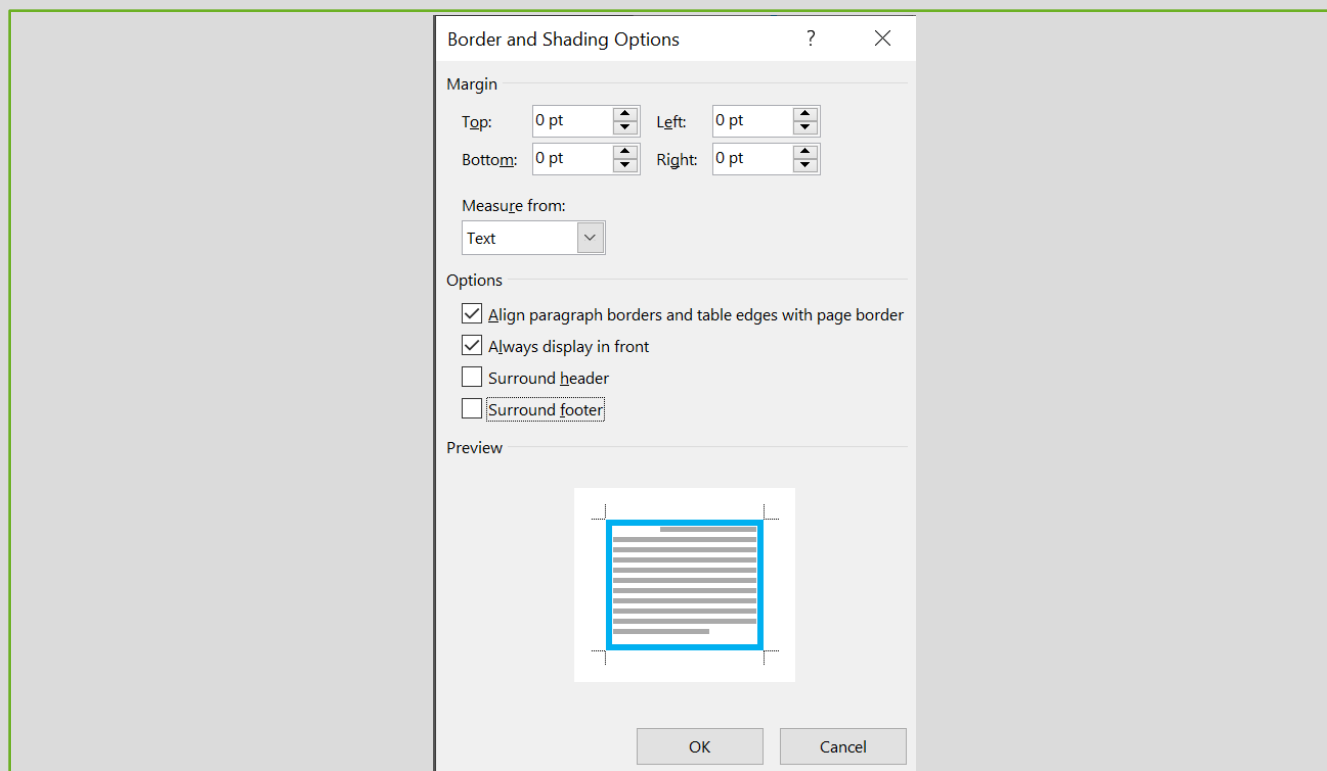


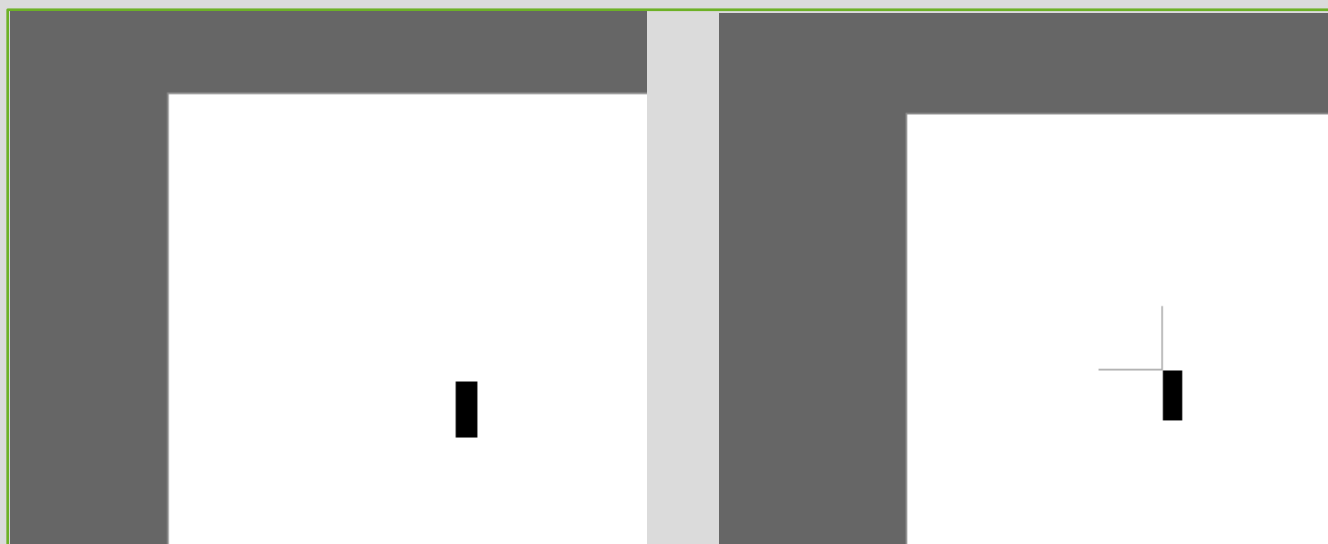
Fig. 138. Borders and Shading.

Set up all the options exactly as shown below and click *OK* when you're ready.



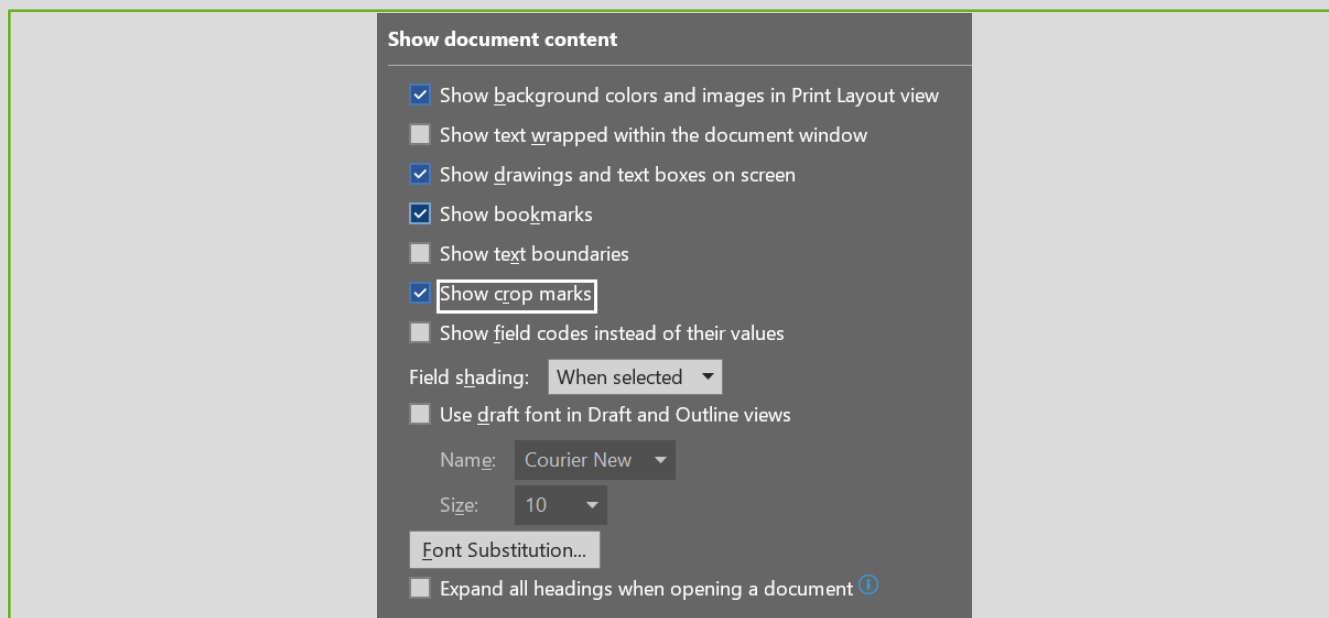
Pic. 139. Border and Shading Options.

We are almost done. There is one more setting which will help us with navigating and design of different areas of a page, named *crop marks*. The crop marks are small, L-shaped symbols that appear at the corners of your document's margins. They are designed to visually indicate the boundaries of the text area, which is particularly helpful when working with custom paper sizes or preparing files for professional printing.



Pic. 140. Top-left document corner, crop mark disabled (left) and enabled (right).

To enable them for time being, select *File > Options > Advanced*, and then scroll down to section named *Show document content*, and put a tick mark ☒ to *Show crop marks*. Click *OK* when ready.

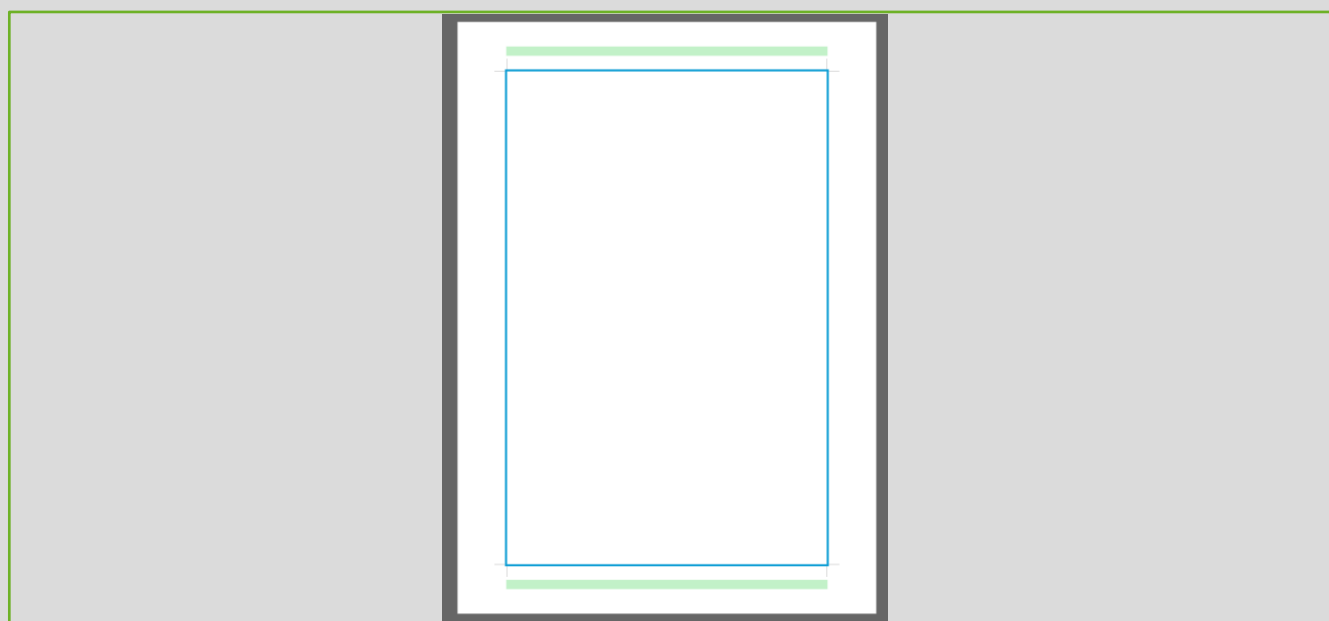


Pic. 141. Crop marks setting.

Now, as we have established visible lines around document body, we can sort out margins, header and footer.

13.2.3. Make colored background of header and footer area

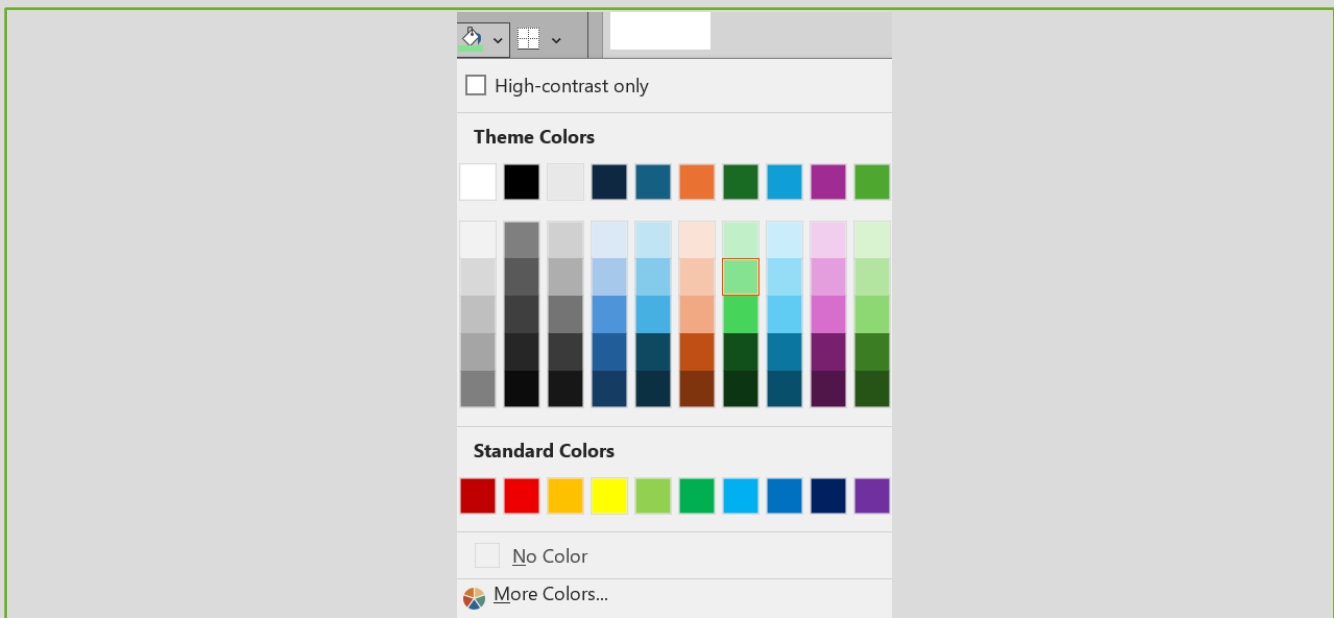
Again, we will not dive deep into all available settings dedicated to margins, header and footer, but focus on our goal. The goal of this subchapter is to set header and footer area visually visible, compare [Pic. 142](#).



Pic. 142. Colored background of header and footer area.

The header and footer areas may contain just another paragraph of content, e.g. text. We will set background for those paragraphs to specific color, to distinguish it from document body area.

From ribbon menu select *Insert > Header & Footer > Header > Edit Header*. Now your cursor should blink in the top area of the page. From the ribbon menu select with a mouse *Home > Paragraph* and search for *Shading* button drop down window. Select light green color of your liking and click *OK*. Repeat the same steps for footer area.

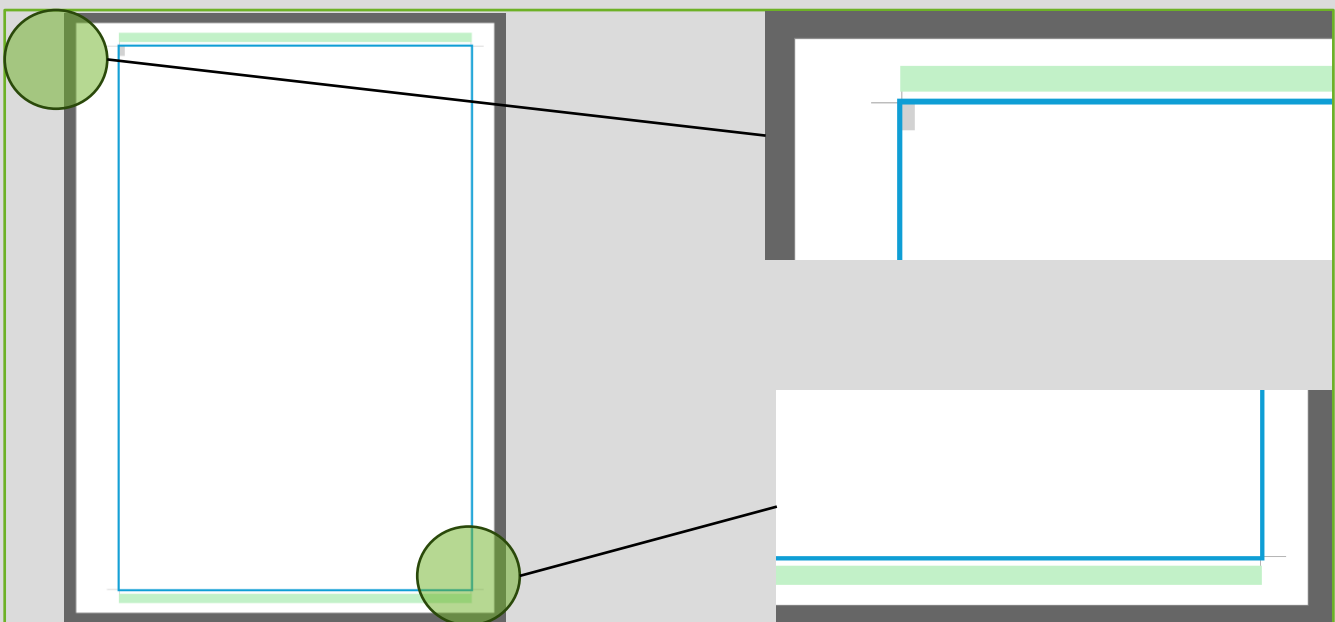


Pic. 143. See small red rectangle around selected color.

13.2.4. Design of customized page layout

Now as we're able to see where document body is and where are located header and footer area, let's learn what is necessary to get the following effect (compare [Pic. 144](#)):

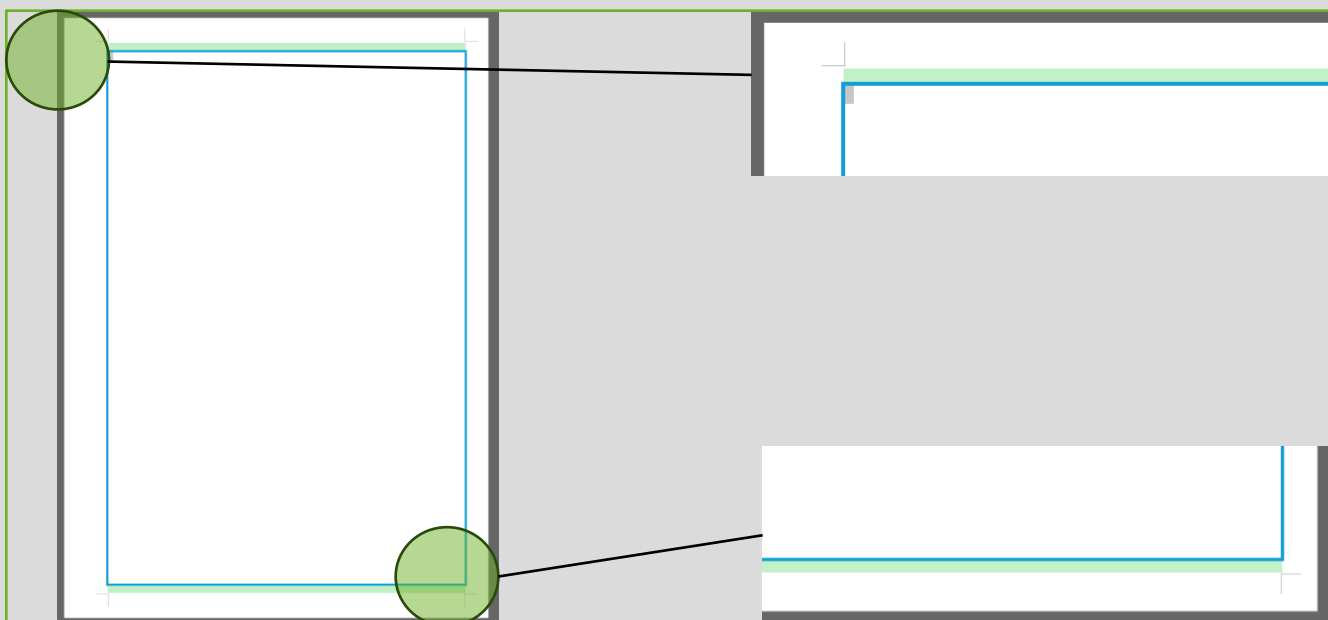
- » size of margins reduced from 2.5 cm to 1.2 cm,
- » mirrored margins, to keep extra 1 cm space for purpose of page binding
- » set header and footer as small as possible, just to keep inside a single line of text set to 11 pt, styled with default paragraph style (*Normal*).



Pic. 144. Expected result.

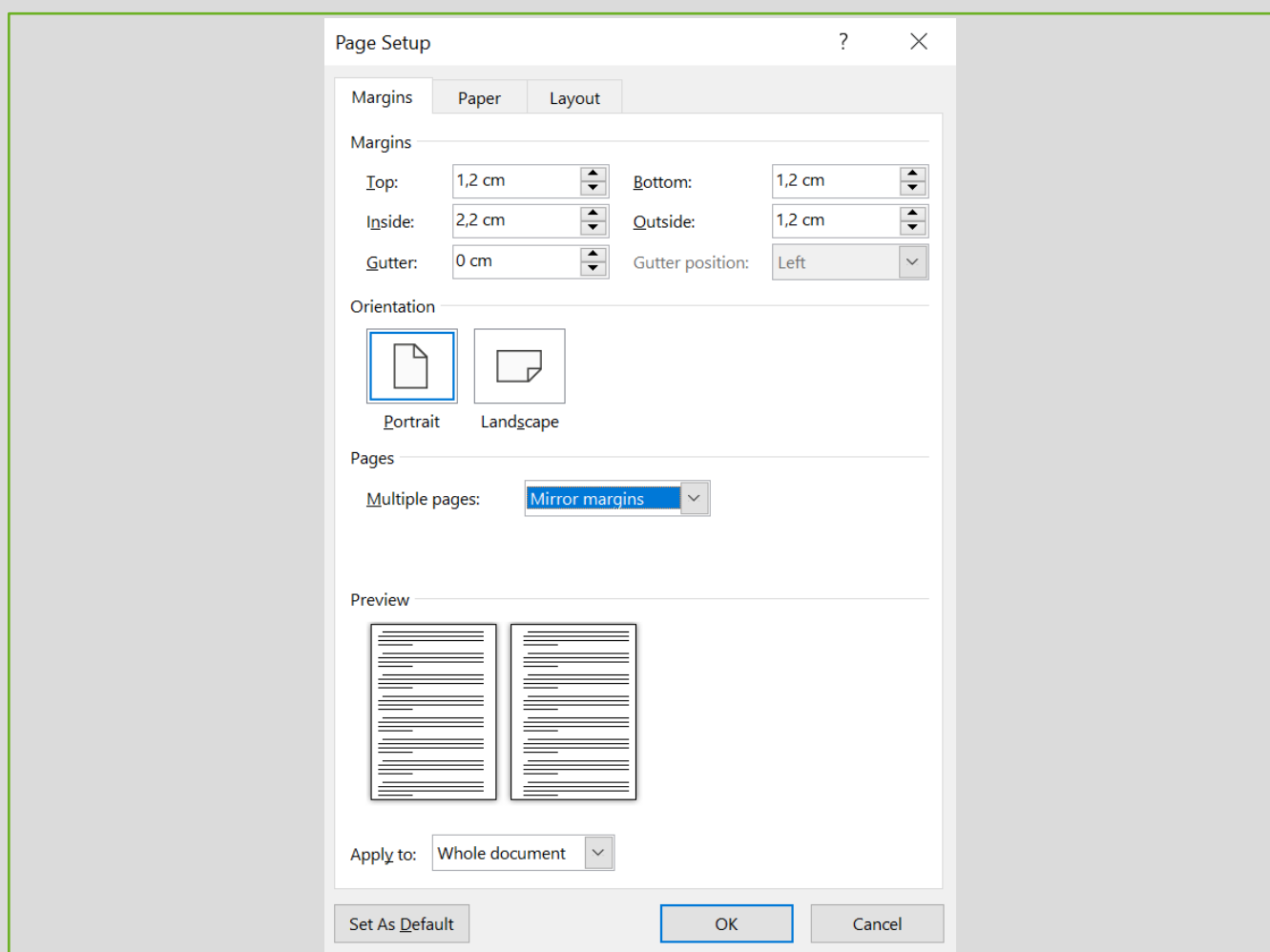
13.2.5. Setting the customized margins

We will start with margins. Our future result is shown in [Pic. 145](#). (Please note it will be slightly different from the result shown in [Pic. 144](#)).



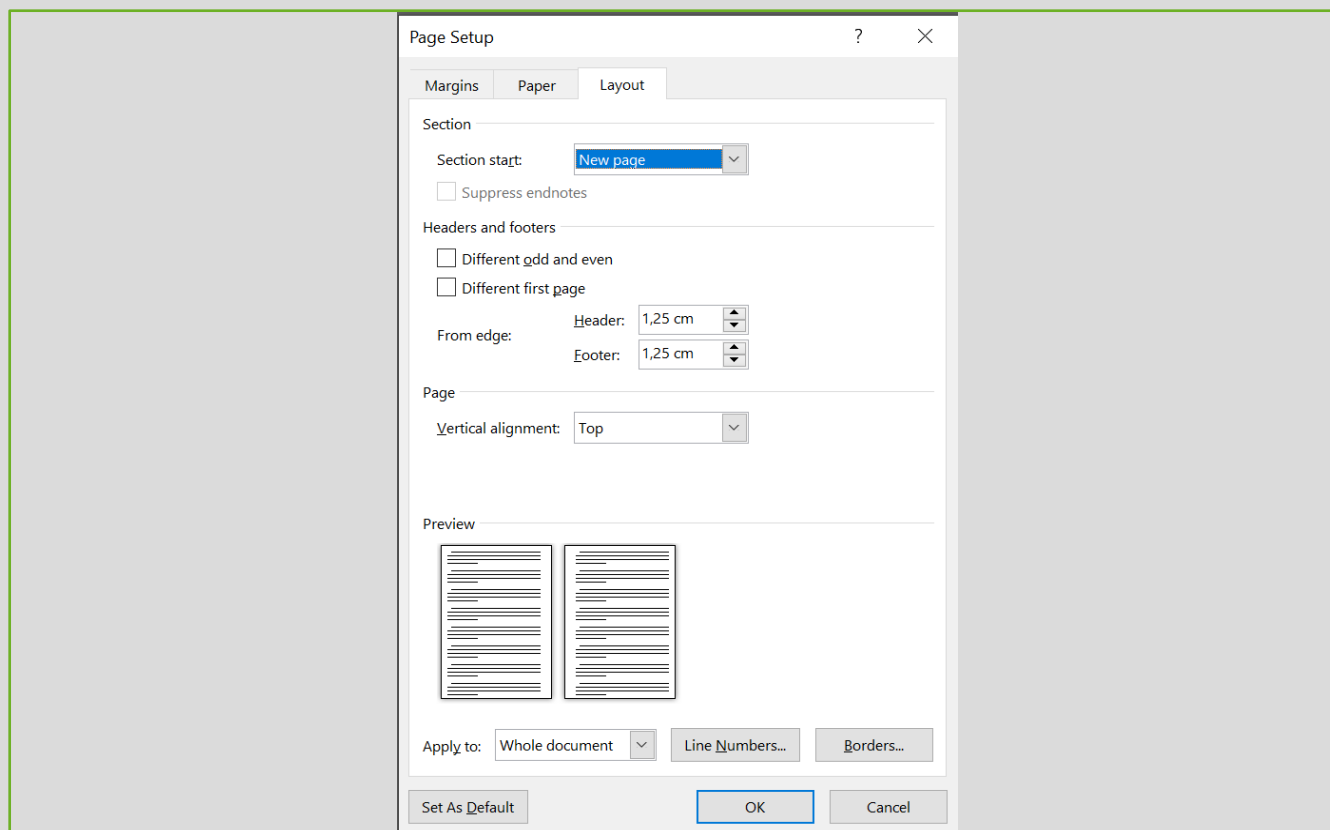
Pic. 145. New values of margins.

Select from the ribbon menu *Layout > Page Setup > small diagonal arrow in the bottom-right corner of this section*: \. You should see the *Page Setup* window. Copy the settings from the picture to tab *Margins*. Before clicking *OK* please switch to *Layout* tab.



Pic. 146. Margins values.

In the *Layout* tab you can see the current values of Footer and Header. For time being just remember the values are equal to 1.25 cm, compare [Pic. 147](#). Now click *OK* button.



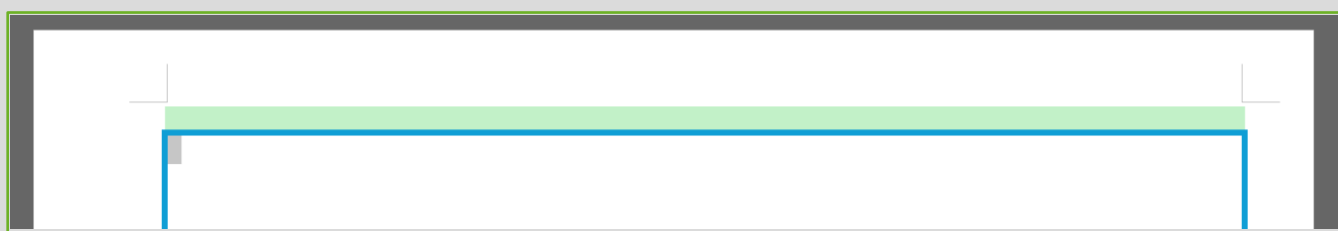
Pic. 147. Header and footer values.

Please note, that for the very top of your page layout:

- » size of header (SH): 1.25 cm
- » size of top margin (STM): 1.2 cm

Watch that $SH > STM$. The green area occupied by paragraph belonging to the header starts slightly below the crop marks! The text body area is tangent to the green area.

This shouldn't happen in correctly designed page layout. Microsoft Word algorithm silently compensates our mistake: size of document body was shrunk to compensate area occupied by header, without any feedback to designer.

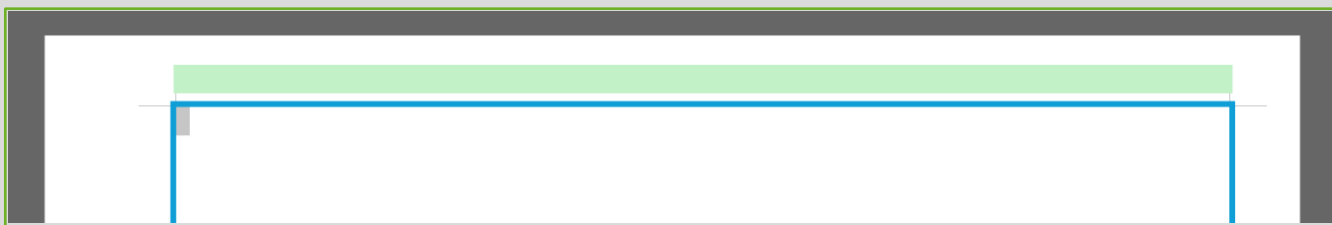


Pic. 148. Size of header > size of top margin.

Let's fix that issue in the next subchapter.

13.2.6. Setting the header and footer height

Our future result is shown in Pic. 149.

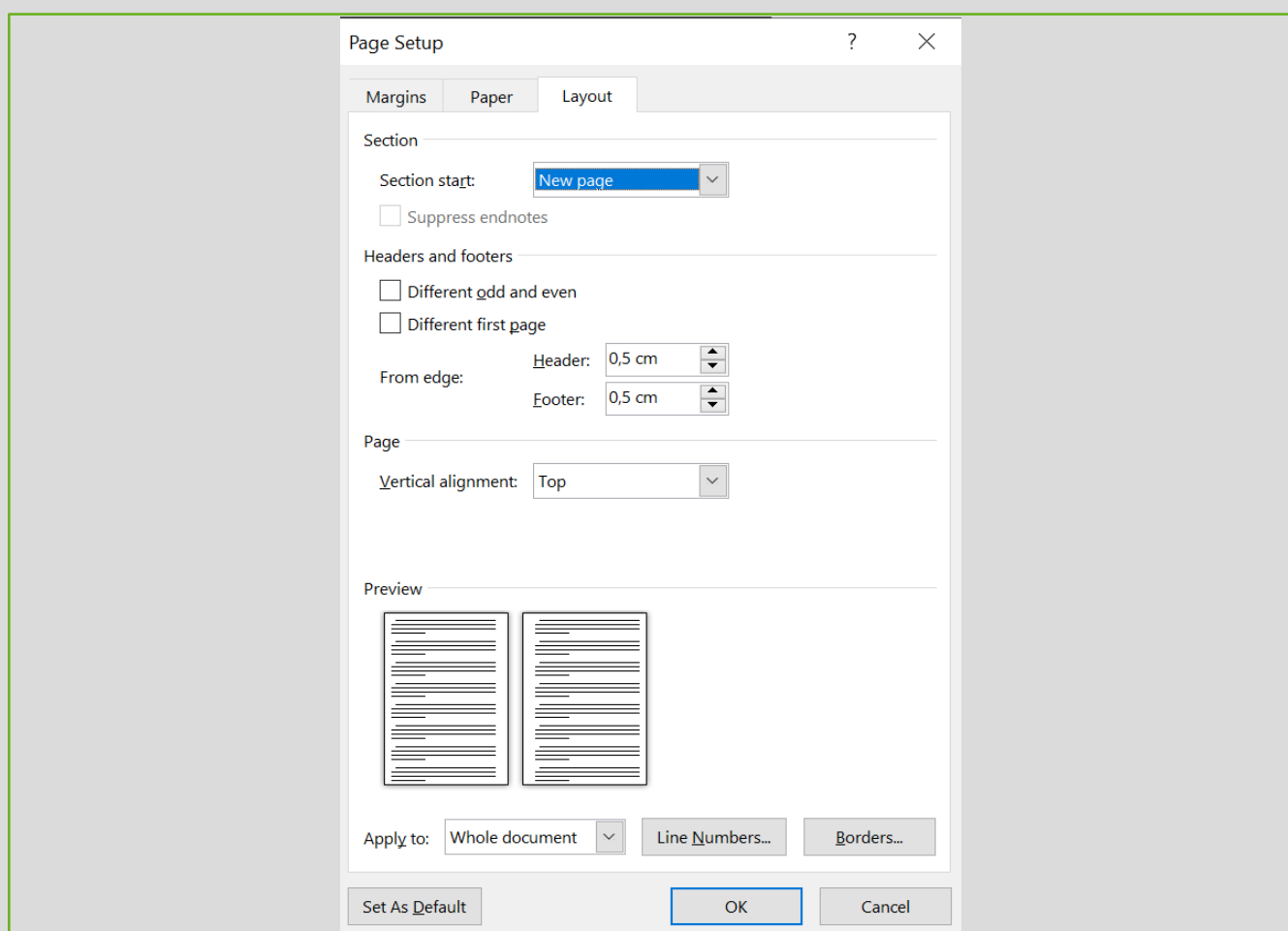


Pic. 149. Expected result of header.

Again, select from the ribbon menu *Layout > Page Setup >* small diagonal arrow in the bottom-right corner of this section: ↘. You should see the *Page Setup* window, compare. Set the header and footer to 0.5 cm.

Why just 0.5 cm and not less? The answer is linked to technical features of office grade laser printers, which I use most of the time for my print outs. The issue is, printer itself holds the sheet of paper on time of printing, creating extra margin. In other words, it is not possible to cover completely with text the whole sheet of paper, there is always a technological margin caused by printer limits. Those limits are usually set to something like 0.4 cm or even less. For exact details please consult your printer data sheet or... prepare in Microsoft Word by yourself set of pages with different frame sizes, to check it. As a side effect of this experiment you may discover problems with mechanical calibration of your printer, e.g. one margin slightly shifted from page border.

To have covered 100 % of paper sheet area you may then make a trick: print your content to a bigger text sheet (e.g. A3 instead of A4, or special format A4+), taking advantage of printing the entire available A4 area, and after printing to cut it down to your required size (A4).



Pic. 150. Sizes of header and footer.

13.2.7. The first conclusions

Conclusion is that *Header distance* parameter available in the *Page Setup* window, in the *Layout* tab, defines distance from the top edge of paper sheet to the beginning of header first paragraph. The height of that strip is automatically adjusted to size of the style and font of that paragraph.

What is actual height of the header? Where is it finished? In my case style of the header paragraph is *Normal*. Its default font is Calibri, 11 pt. You can reveal that data by pressing keyboard shortcut Shift + F1, what brings to the screen the *Reveal Formatting* pane. Now enter again area of the header (right mouse double click in that area of page or ribbon menu > *Insert* > *Header & Footer* > *Header* > *Edit Header*). The 11 pt is equal to 0.388 mm. And we know that main document body starts at 1.2 cm from page top edge. Additional 3 pt are occupied by edge of the frame limiting printed area.

The exact values aren't important to us, except the relation $SH < STM$ should be kept. This condition is met this time, as area of the header (green) is above of the top crop marks.

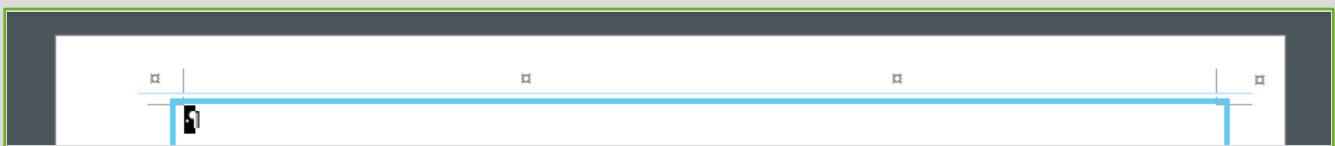
As you can see (compare e.g. [Pic. 149](#)), there is now a white gap between green area representing area of the header and blue frame representing text body. That is a good sign, as now text in the header will be visually separated with this extra margin. We will soon add another form of visual separation to the header and footer with single horizontal line.

Unfortunately, the ruler, which is part of Microsoft Word User Interface (ribbon menu > *View* > *Show* > *Ruler*), is not very helpful on time of distances set up.

13.2.8. Tables in header and footer

Now, as we have header and footer, and have some knowledge about relationships between distances, let's finish with formatting of the content. Our goal is

- » to put tables in both header and footer
- » to move the borders of the tables outside to the right and left of document body area, to occupy all printable paper area
- » to add the bottom of the header and top of the footer single, solid, horizontal line.



Pic. 151. Expected result.

I recommend you to insert tables in header and footer. As content of those tables use document variables or Microsoft Word variables (*fields*). It is appealing visually and creates professional outlook.

13.2.9. Setup of header extra width

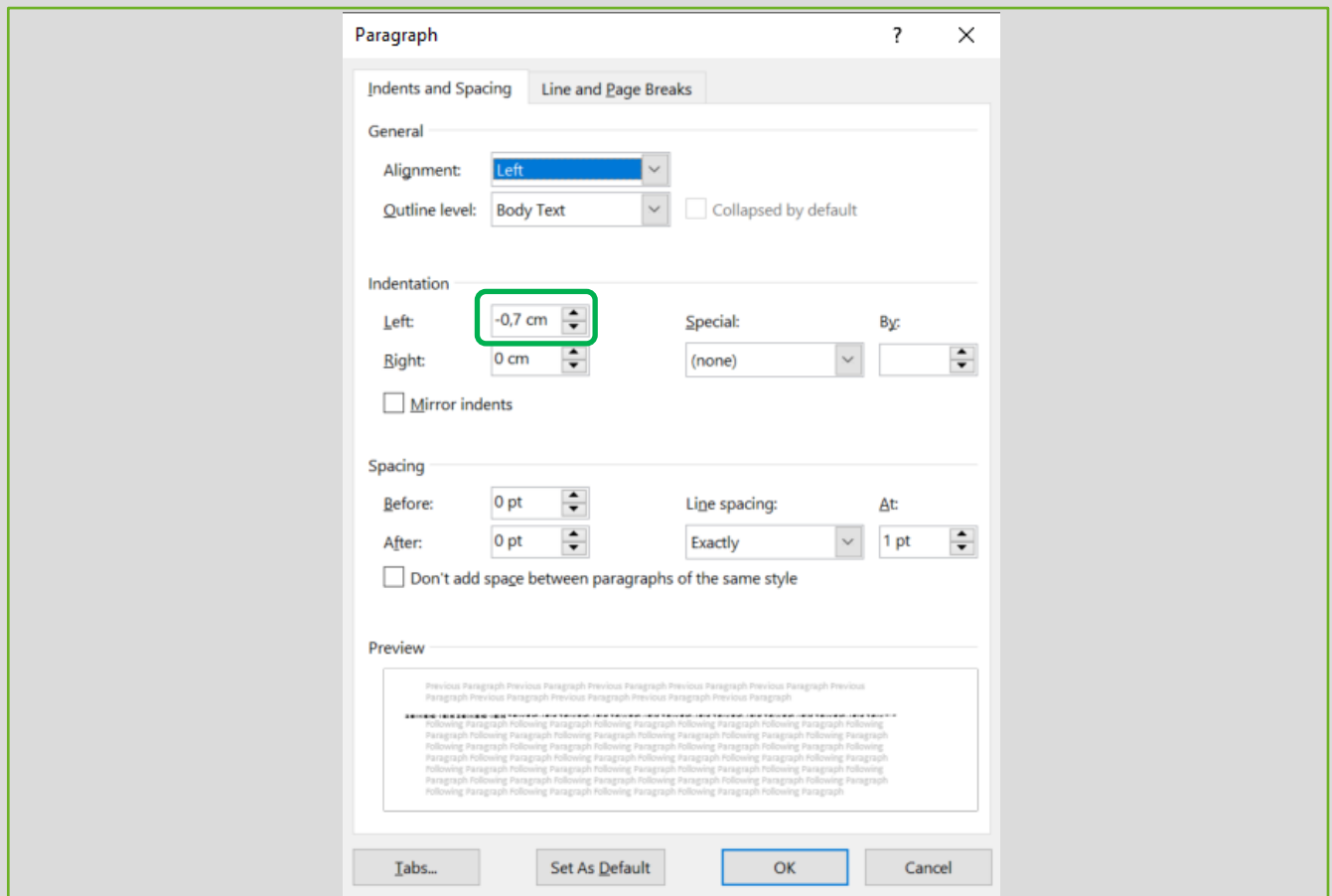
We want to move paragraphs outside of text body limits, to keep it just 0.5 cm apart from physical page borders. Knowing width of the narrower margin it is 1.2 cm, we search for the x value, to reduce width of paragraph:

$$0.5 \text{ cm} = 1.2 \text{ cm} - x$$

$$x = 0.7 \text{ cm}$$

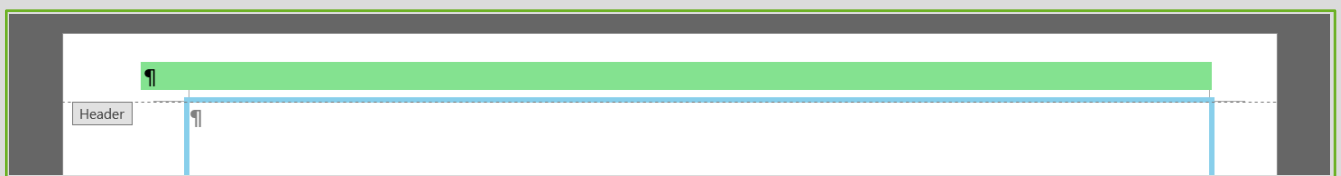
We need to move indentation by $x = 0.7 \text{ cm}$, so we set indentation to -0.7 cm .

Enable unprinted characters (ribbon menu > *Home* > *Paragraph* > *Show/Hide*, or click the icon ¶). Enter the heading for editing. Change settings of the paragraph in the header, to move it outside of the document body, by setting negative left indentation to -0.7 cm: ribbon menu *Home* > *Paragraph* and select diagonal arrow ↘ in the bottom-right corner of the section *Paragraph* (compare [Pic. 152](#)).



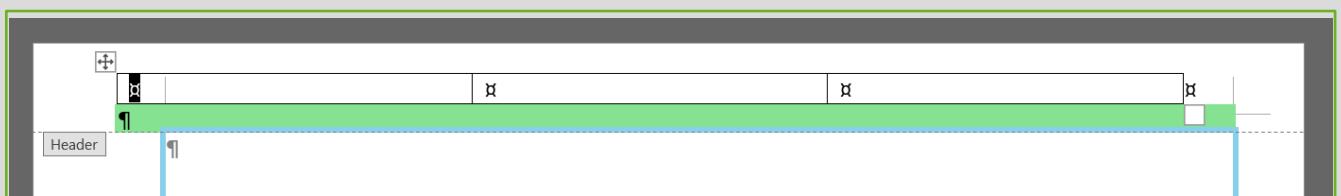
Pic. 152. Set negative left indentation.

After applying new negative indentation, header should look like below.



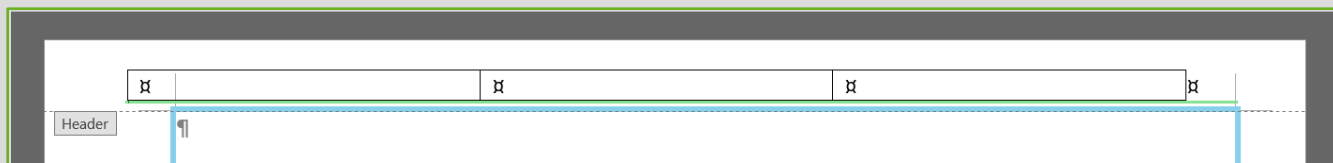
Pic. 153. Header paragraph with negative indentation.

Insert table in the header: ribbon menu > *Insert* > *Tables* > *Table* > 3× columns, 1× row.



Pic. 154. Table in the header.

Reduce size of font in the paragraph which follows the table to minimum, 1 pt: Ctrl + Shift + P.



Pic. 155. Reduced size of font in the empty paragraph below the table.

Change width of the table, to occupy all page width except last 0.5 cm from the right (all printable width of page sheet).

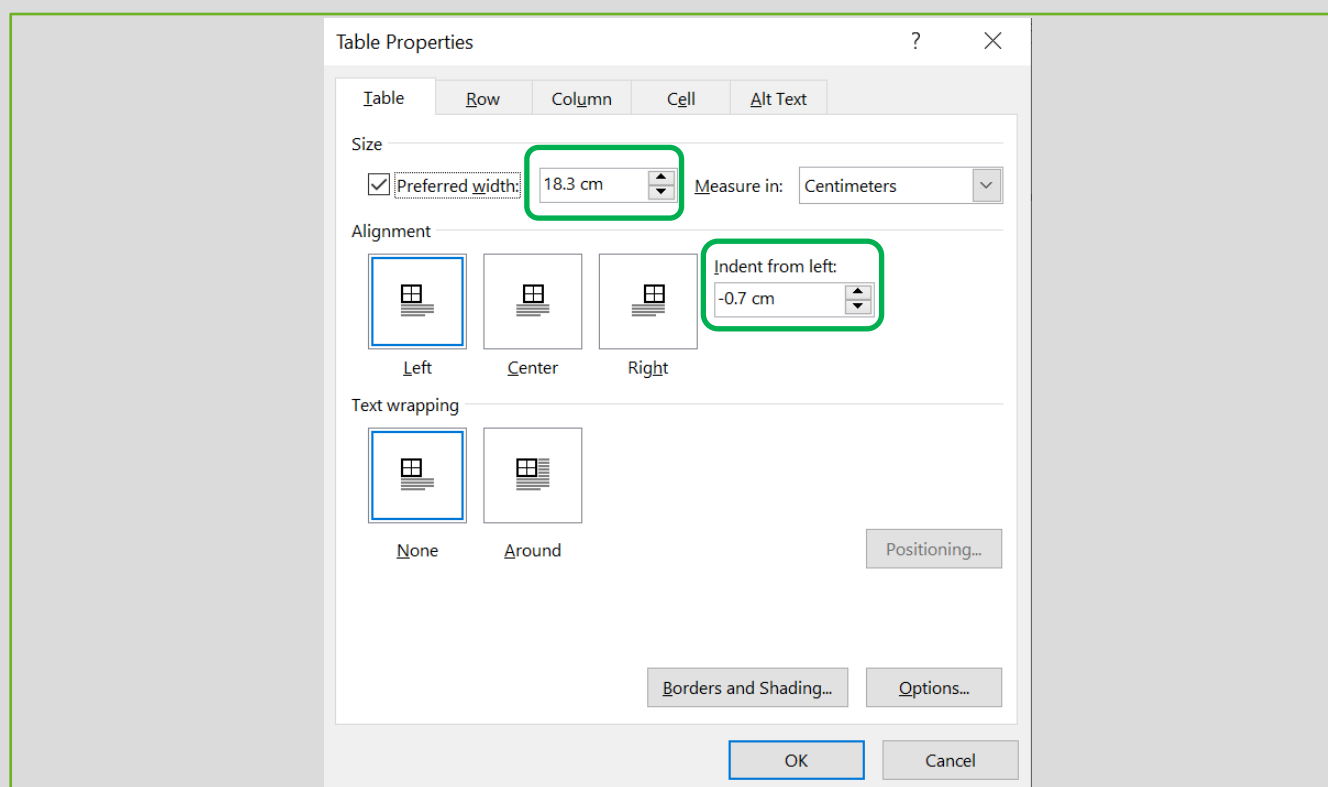
Page width (PW) is 21 cm (A4). Current width of mirrored margins (MM):

$$2.2 \text{ cm (inside margin)} + 1.2 \text{ cm (outside margin)} = 3.4 \text{ cm.}$$

If content of the table would start just 0.5 cm from the page border, and currently it starts at 1.2 cm, then it must be shifted by 0.7 cm.

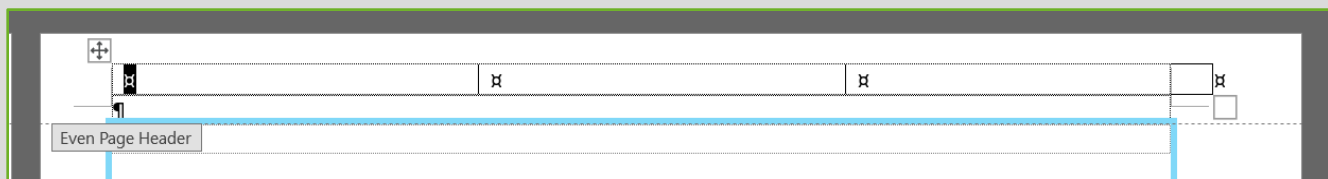
Table width would be $PW - MM + 0.7 \text{ cm} = 21 \text{ cm} - 3.4 \text{ cm} + 0.7 \text{ cm} = 18.3 \text{ cm}$.

Ribbon menu > *Table Layout* > *Table* > *Properties* and set preferred width to 18.3 cm and indent from left to -0.7 cm.



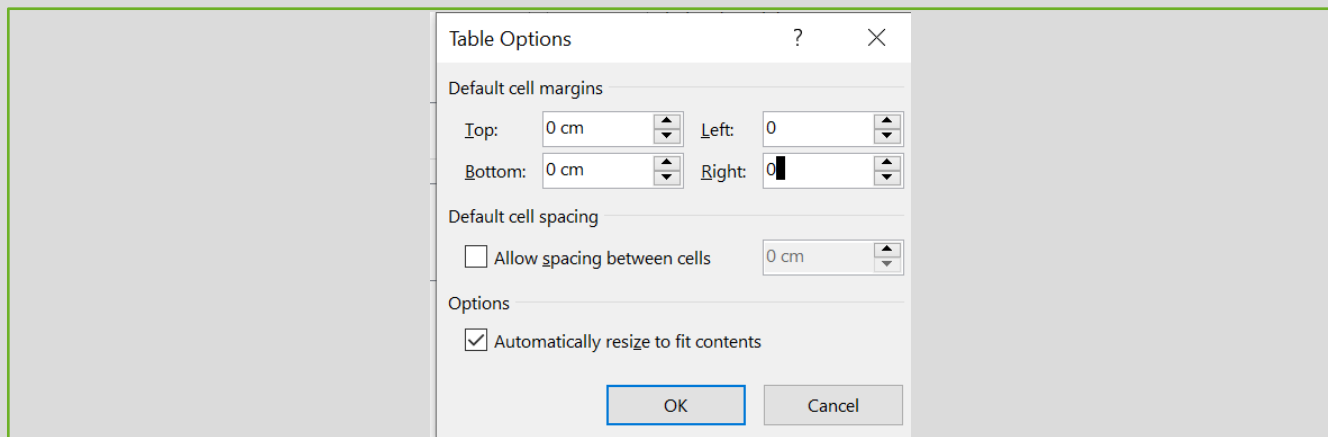
Pic. 156. Width of header table.

The new table is visible in Pic. 157.



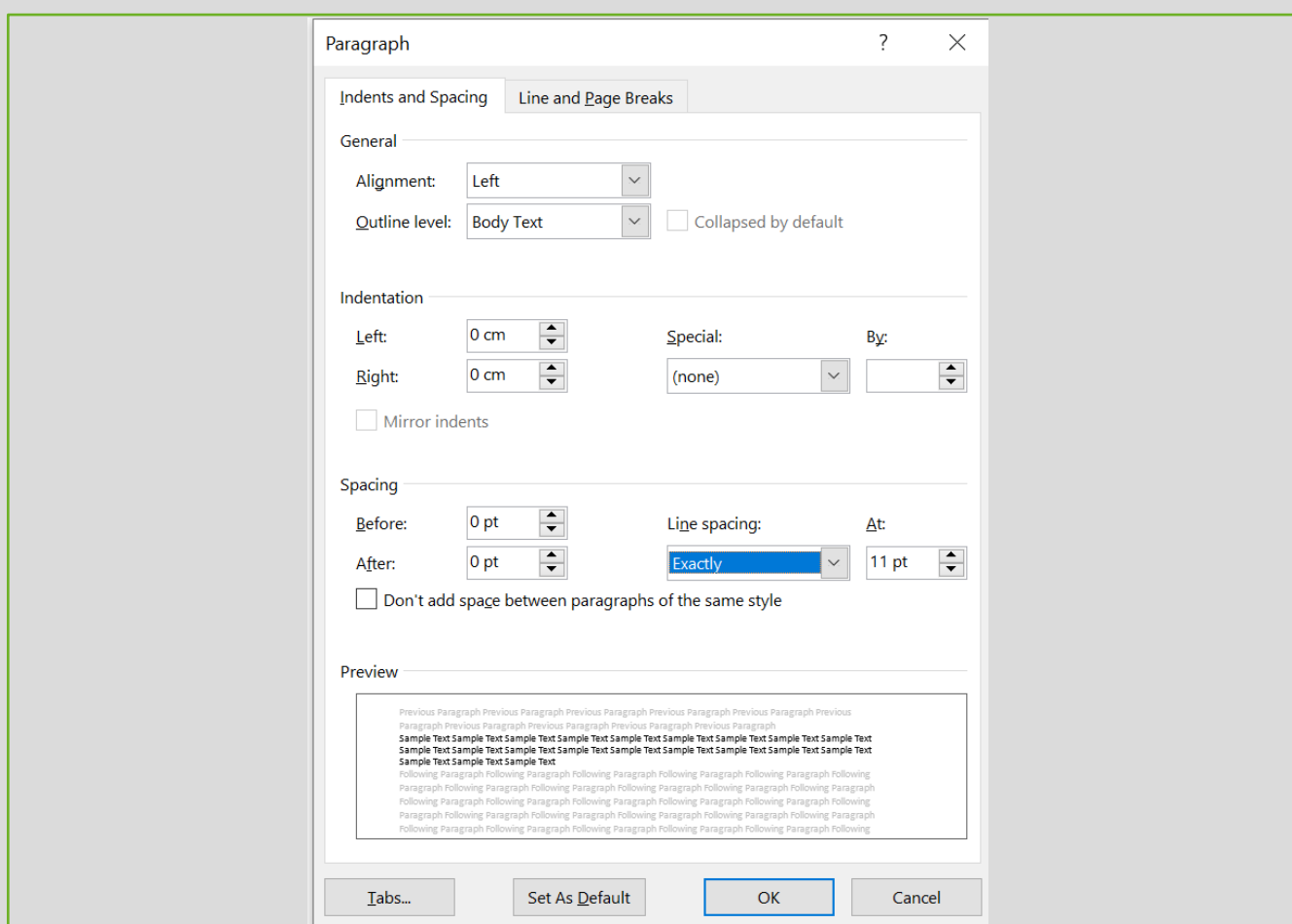
Pic. 157. New table width.

We want to set internal margins of cells in this table to 0 cm. Put cursor inside of the table then select from the ribbon menu > *Table Layout* > *Table* > *Properties*, then select button *Options* and enter 0 cm values in all cell margins, compare [Pic. 158](#).



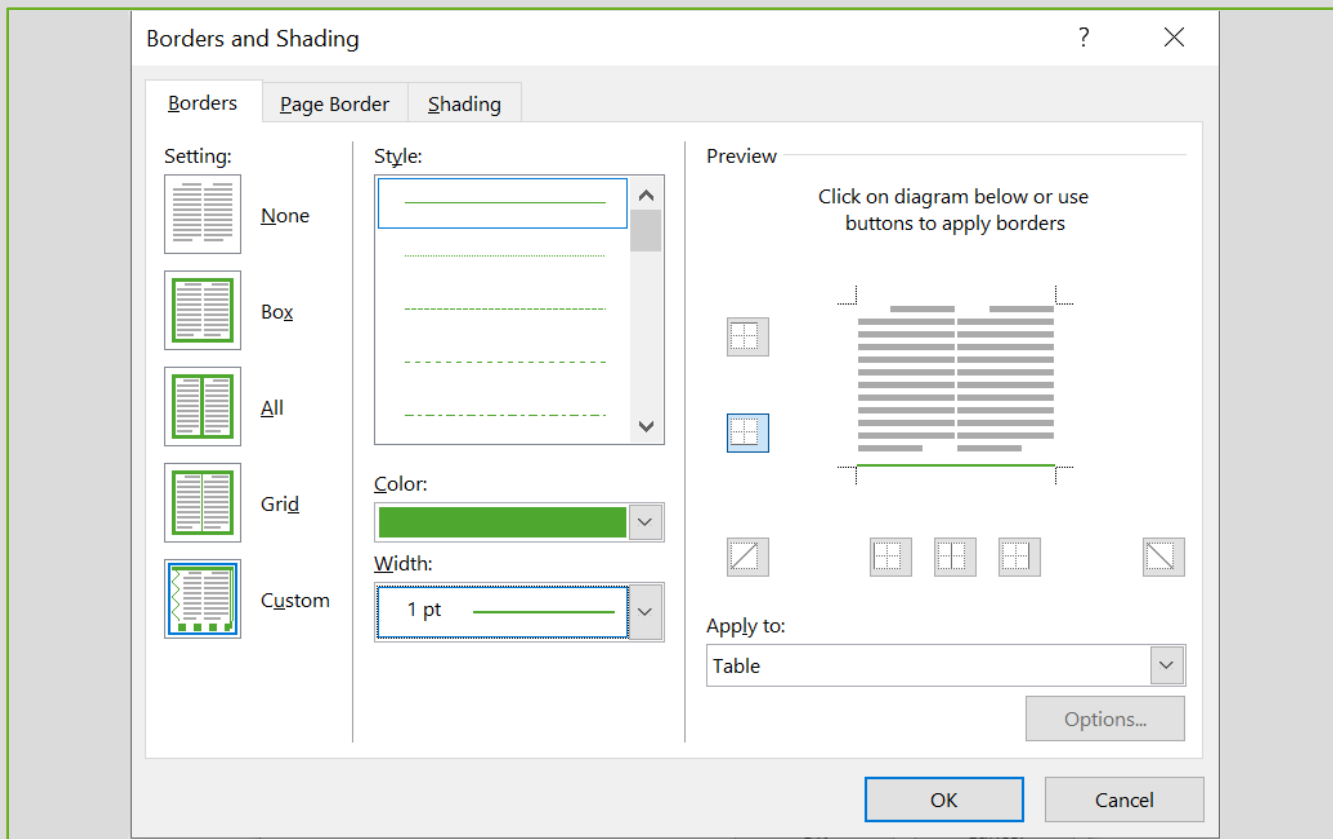
Pic. 158. Reduced margins of the table.

Set height of text paragraph inside table cells to exactly 11 pt (0.388 cm $\approx$ 0.4 cm). Put cursor inside the table then select entire table and from ribbon menu select *Home > Paragraph* and set line spacing to 11 pt, compare [Pic. 159](#).



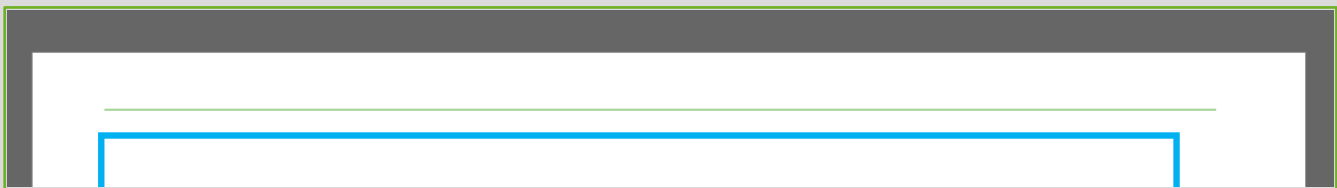
Pic. 159. Changed line spacing of table cells.

Next step of header preparation is change of bottom line of header table. Its width will be set to 1 pt and green color. Put cursor within table body and select from ribbon menu *> Table Design > Borders > Borders*. In the window *Borders and Shading*, in the tab *Borders*, set parameters as shown in [Pic. 160](#). Note that only bottom line is changed.



Pic. 160. Table bottom line set to green.

After pressing OK the result is like in [Pic. 161](#). For clarity unprinted characters were switched off.

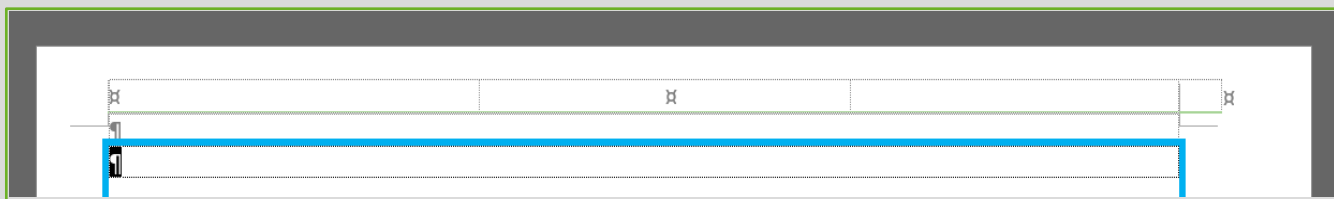


Pic. 161. Bottom line of header table.

Final touches are related to:

- » enabling table grid lines:
put cursor within the header table and then select from ribbon menu > *Table Layout* > *Table* > *View Gridlines*
- » switching off paragraph background for empty paragraph which follows header table:
put cursor within the header, in paragraph that follows table,
ribbon menu > *Home* > *Paragraph* > *Shading* > no color.
- » setting content alignment of the header row cells: left to upper-left, right to upper-right:
put cursor within the table, in the left cell of that table
ribbon menu > *Table Design* > *Alignment* > *Align Top Left*,
put cursor within the table, in the middle cell of that table
ribbon menu > *Table Design* > *Alignment* > *Align Top Center*
put cursor within the table, in the right cell of that table
ribbon menu > *Table Design* > *Alignment* > *Align Top Right*

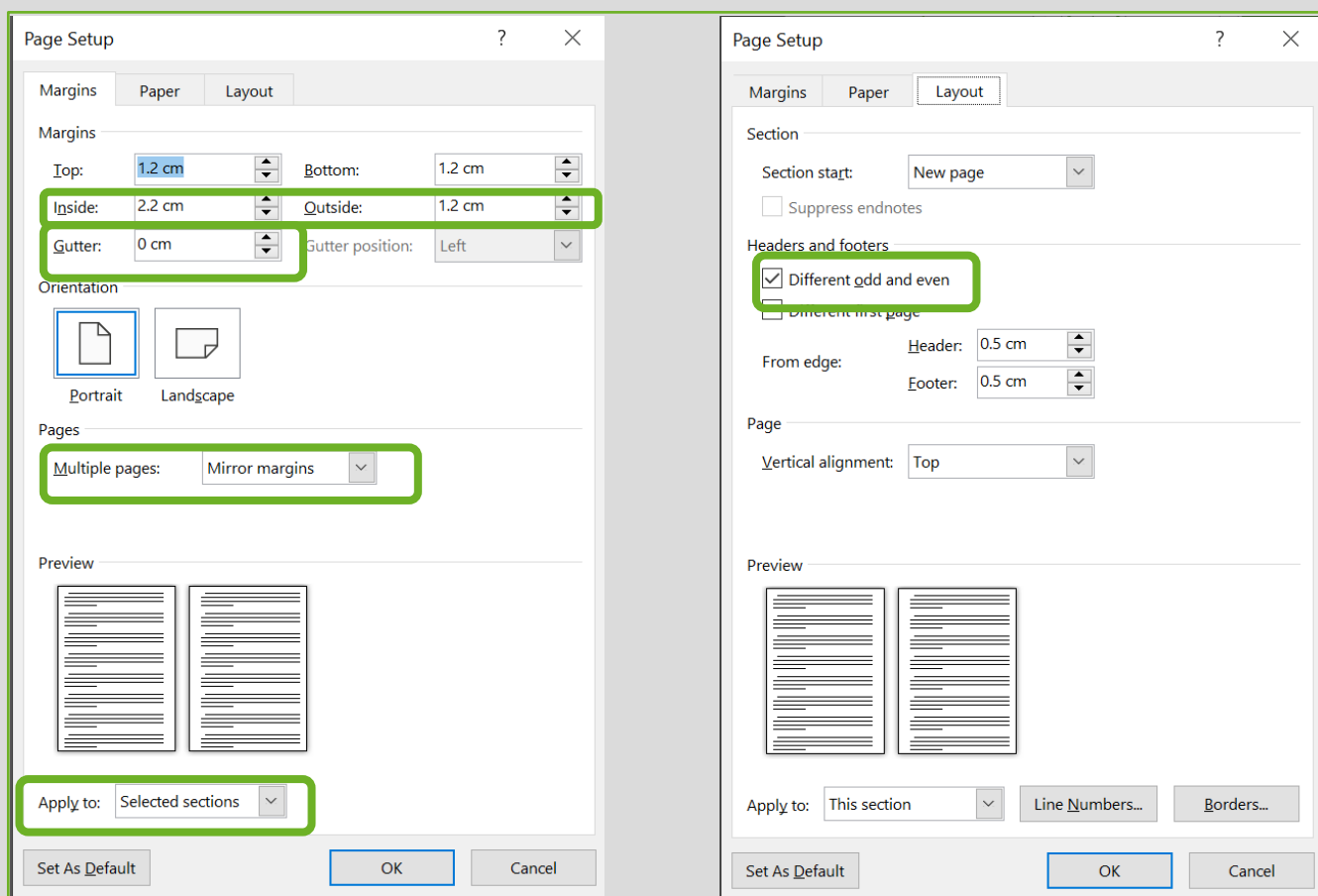
You can see current header layout after applying above settings in [Pic. 162](#).



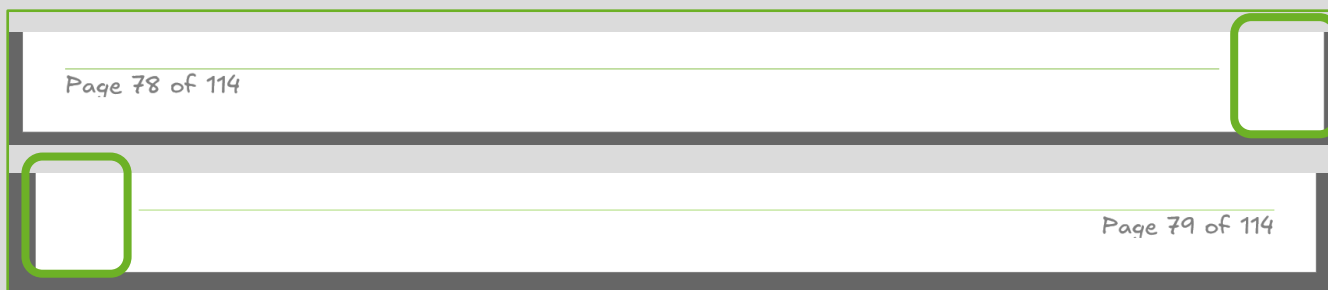
Pic. 162. Header layout, enabled visibility of unprinted characters.

13.2.10. Next conclusions

- » **Gutter.** Don't use *Gutter* (set it to 0 cm) if your document has more than just a single section. The caveat is that *Gutter* property is set for the entire document. If you wish to have mirrored margins, you need to simulate this setting with *Inside* and *Outside* margin settings. Compare [Pic. 163](#). There is a "virtual gutter" between *Inside* and *Outside* equal to 1 cm, even that *Gutter* itself is set to 0 cm.
- » **Mirrored margins.** If you plan to work with mirrored margins, it is a good practice to keep also different odd and even headers and footers. This practice is visible in books where the page numbers are in the bottom outer corners. They are mirrored, meaning they are always on the edge opposite the gutter, compare [Pic. 164](#). Consequently, for mirrored margins you need to design and store more building blocks.



Pic. 163. Example.



Pic. 164. Page number on mirrored footers, opposite to gutter, marked in red.

- » There is no easy method in Microsoft Word to store newly defined page layout for future references. It cannot be added to User Interface. The only way which I know to preserve your settings is to define VBA macro and store it e.g. in your macro oriented add-in file.

13.2.11. Header and footer follow-up

Now you may, as a follow up:

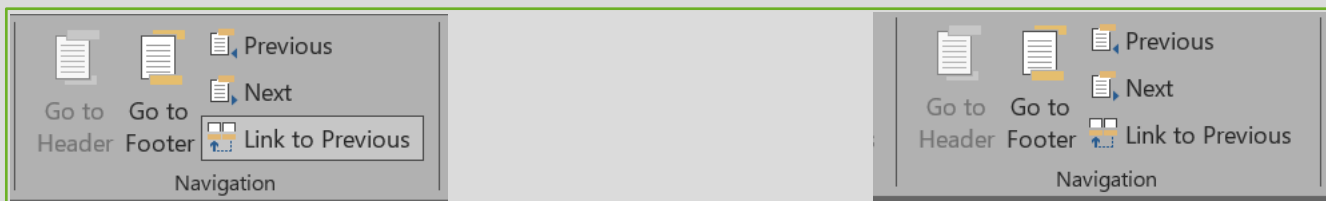
- » apply **styles** to
 - » table itself (*TabTableNoGrid ms*)
 - » paragraphs of the table (*ParInTable ms*)
 - » paragraph below the table (*ParMinimal ms*)
- » put content in form of the Microsoft Word variables (**fields**) to cells of the table (e.g. ribbon menu > *Insert* > *Text* > *Quick Parts* > *Fields* > *DocProperty* > *ms_DocumentTitle1*, *ms_DocumentTitle2*, *Author*)
- » create a **building block** for whole heading and save it to add-in, e.g. *BB_ms.dotx*
- » save for future reference parameters of page layout in form of **macro** (*SetPageLayout_44_V_1_2*)
- » repeat above steps for the second mirrored header, taking care of mirroring content accordingly

You may continue with the setup of the footer in similar manner.

13.2.12. Additional section of document, linked headers and footers

In technical documents it is sometime a case that body content of the vertical A4 paper sheet is too small to conveniently present e.g. table. To keep differently oriented pages in one document, you need to add more sections.

When you use mirrored margins and change page orientation, by default Microsoft Word keeps headers and footers linked between sections. Consequently, headers and footers on horizontal page will look like constrained. But they look just like on vertical page. You need to break the link between headers and footers of different sections. You can make it manually, separately for each section, and separately for each header and footer, by pressing *Link to Previous* button in the *Header & Footer* > *Navigation*, compare [Pic. 165](#). It might be some manual work to do.



Pic. 165. Left: header linked to previous section, right: unlinked.

I've developed macros to handle both cases:

- » `UnlinkAllHeadersFooters`
breaks links between headers and footers in all sections
- » `AddSectionAndKillLinkToPrevious`
adds additional section and unlinks its header and footer, as well as previous section and the next section

13.2.13. Blank pages

There is one more caveat if you plan to change page orientation. A single physical sheet of paper cannot be *Portrait* on one side and *Landscape* on the other. Word ensures that the "back" of a landscape page is also treated as part of that landscape section or creates a blank buffer to reset the orientation for the next sheet. You can ignore this behavior or add even pages to your document with content, e.g. "this page is left blank on purpose". I've prepared two macros for that purpose:

- » `AddBlankPages`
adds blank pages whenever section has odd number of pages
- » `DeleteTempBlankPages`
delete previously inserted blank pages

13.3. Design of content, requirements

My subjective rules:

- » **Content must be separated from macros.**
Content: DOCX files.
Macros: dedicated DOTM file, `Macros_ms.dotm`.
- » **Content should be stored within DOCX files.**
The configuration files (DOTX or DOTM) may contain content. It could be potentially useful to a user to get a kind of *ReadMe* content within such a file. For example, I can imagine that after opening e.g. `Macros_ms.dotm` reader would see information explaining how and where to store such a file.
- » **Content files should store keyboard shortcuts.**
Keyboard shortcuts should not be stored within configuration files (DOTX or DOTM)
- » **Content can be generated by VBA macros.**
E.g. `Macros_ms.dotm` can generate content from existing building blocks.
- » **Content must use only defined set of styles.**
In other words, styles from outside of the predefined set are prohibited. Only this way you can keep the same look and feel of your content. The predefined styles must not be edited. How to limit to user editing of existing styles is not covered in this book.
- » **Security of content should be clearly stated.**
In modern world setting the security boundaries is an obligation.

In this book I design content in the same way as if I prepare technical documents, taking into consideration the following additional rules:

- » One dedicated theme: *Theme_ms.thmx*.
- » Dense, compacted content.
Minimum space between lines of the same paragraphs.
Automatic [hyphenation](#) [4].
- » Technical documents based on those rules will be read mainly on screens of electronic devices. In consequence text will not be centered (justified), but aligned to the left.
- » Main categories of content: text, pictures (including diagrams, schemas), tables, lists.
- » References, e.g. internal document links, are important and should be handled.
- » Endnotes will not be used.
They distract process of reading.
- » Pictures must:
 - » be always placed on canvases
 - » have legends below canvases
- » Mandatory list of pictures in the document, at the end of document
- » Tables:
 - » must have heading
 - » content must be addressable, e.g. each row with a number, to easily reference to specific cell of a table
 - » if possible, keep table on the same page
- » Mandatory list of tables in the document, at the end of document
- » Lists:
 - » numbered, down to 4th level
 - » bulleted, down to 4th level
 - » dedicated list for references
- » Table of Content, at the beginning of the document
 - » down to third level
 - » with header

13.4. Fonts

Subjective classification of fonts:

- » criteria: space occupied by single glyph:
 - » monospace
 - » proportional
- » criteria: ease of reading
 - » serif
 - » sans-serif
- » criteria: distinguishing of glyphs:
 - » [homoglyphs](#) [3]
 - » no homoglyphs
- » criteria: availability
 - » available in Operating System
 - » custom

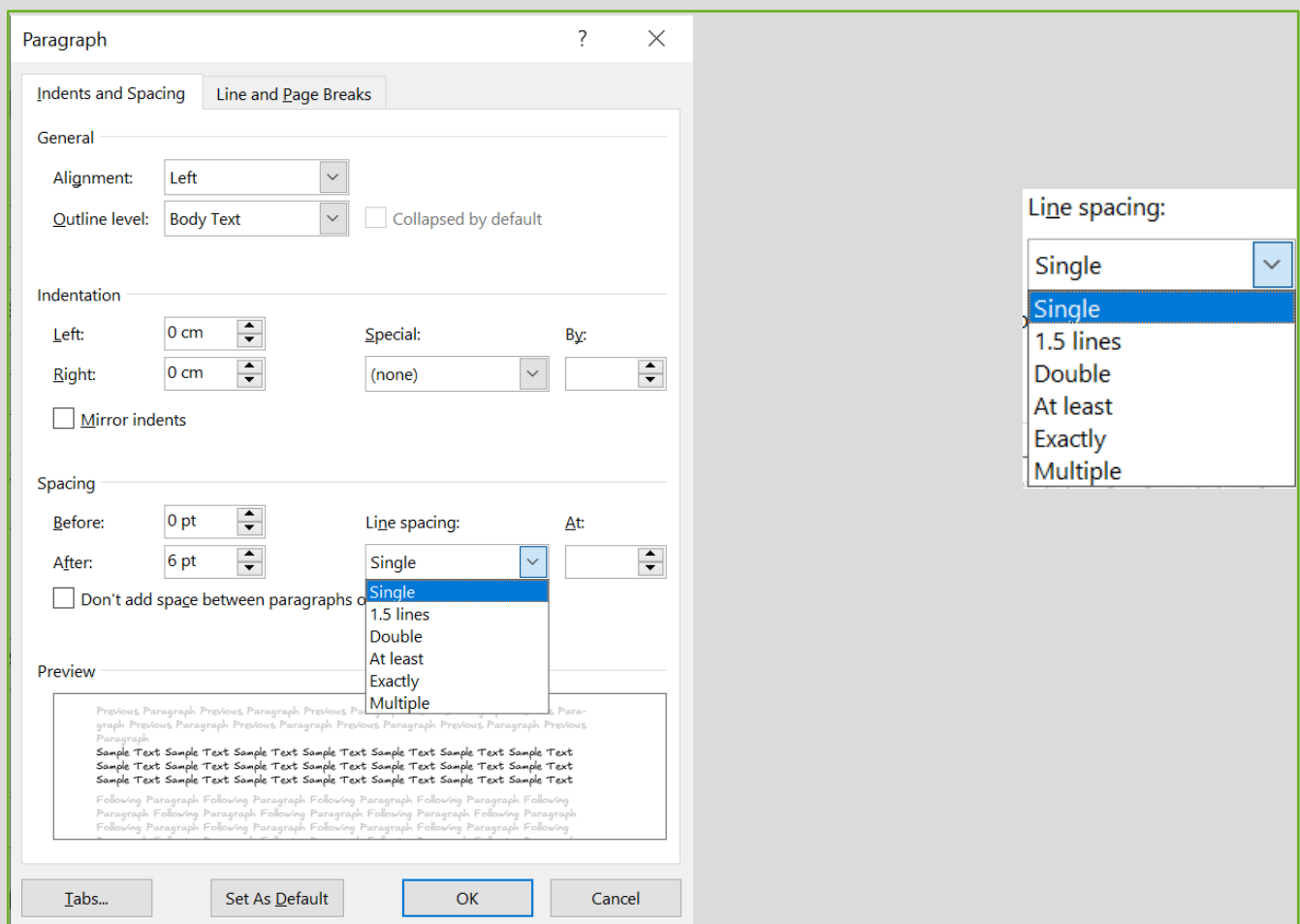
Two fonts should satisfy all styling:

1. Main font for body and headers: Excalifont-Regular.ttf.
 - » can be proportional
 - » can be serif
 - » can have homoglyphs, e.g. O and 0
 - » can be custom
2. Auxiliary font for technical content: source code, special values: Consolas.
 - » cannot be proportional
 - » can be serif
 - » cannot have homoglyphs, e.g. 0 and 0
 - » cannot be custom, to be readable even if file was sent out in source form to another computer

13.5. Dense, compacted content

Microsoft Word comes with several options, which can be set on purpose.

13.5.1. Minimum space between lines of the same paragraphs



Pic. 166. Paragraph line spacing options.

In Microsoft Word, *Single* does not mean "Font Size + 0." If you have an 11 pt font, *Single* spacing might actually be 13.2 pt or 14 pt depending on the font's internal metadata.

Every font file contains three hidden values defined by the type designer:

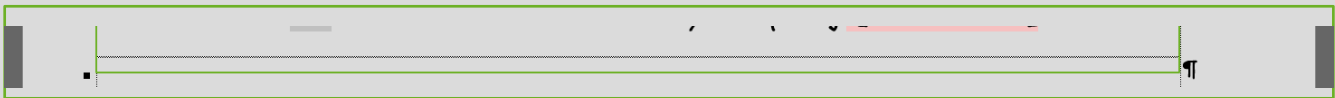
- a. **Ascent:**
How far the highest character (like Ö) goes up.
- b. **Descent:**
How far the lowest character (like the ogonek in q or the tail of a g) goes down.
- c. **Line Gap:**
The extra "breathing room" intended by the designer.

When you select *Single*, Microsoft Word adds these three together. If a font has a high or low diacritics (e.g. capital O with two dots above: Ö), the *Single* spacing will be taller than a font with short ones, even if both are 11 pt.

Anything lower than *Single* can cut off letters. This happens when you use the *Exactly* setting.

If you set a font to *Exactly* 11 pt, you are effectively deleting the *Line Gap* and potentially part of the *Descent*. Clipping: Word treats the line height as a physical "box". If the glyph is bigger than the box, Microsoft Word simply stops rendering the pixels outside of it.

There is one more issue, which I've found extremely annoying. If your specific paragraph is dedicated just to canvas and its line spacing feature is set to *Exactly*, then it will disappear. The canvas itself will be visible just in this one paragraph.



Pic. 167. Cut down canvas with *Exactly* line spacing.

If you want a specific minimum distance but want to prevent clipping, you may use the *At Least* setting. If you set it to *At Least* 11 pt, Microsoft Word will try to use 11 pt. However, if you type a special character (like a mathematical symbol or a large building block) that requires 15 pt, Word will automatically expand that specific line to prevent the letter from being cut off. This can lead to "uneven" line spacing within a single paragraph, which looks unprofessional.

Conclusion: Most of the paragraph styles have common setting of line spacing to *Single*.

13.5.2. Hyphenation

In Microsoft Word hyphenation can be set at three different levels:

1. Document-Wide

Location: *Layout > Page Setup > Hyphenation*

If this is set to *None*, Word will not hyphenate anything, regardless of what your styles say.

» Manual vs. Automatic:

You decide if Word should insert hyphens on the fly or if you want to approve each one.

» Hyphenation Zone:

You can define how close to the right margin a word must be before it triggers a hyphen.

» Limit Consecutive Hyphens:

A vital setting for professional layout to avoid "ladders" (too many hyphens in a row at the edge of the page).

2. The Paragraph Level

Location: *Home > Paragraph > ¶ > Line and Page Breaks tab > Don't hyphenate*

This is where your classification is most accurate. Even if the document-Wide hyphenation is turned on, you can tell specific paragraphs to stay out of it.

- » This is a binary toggle. You aren't setting hyphenation here; you are essentially granting an exemption to that specific paragraph.
3. Word-Specific Level
- Location: Direct insertion via keyboard (Ctrl + Alt + -)
- Optional Hyphen.
- » If you press Ctrl + Alt + - inside a word, you create a "hidden" hyphen (shown as not printable character ~).
 - » Word will only show this hyphen if the word needs to break at the end of a line. This is the detailed way to fix a single awkward line in your 100+ page document without changing global or paragraph settings.

Independently, hyphenation can be set as a feature for paragraph styles.

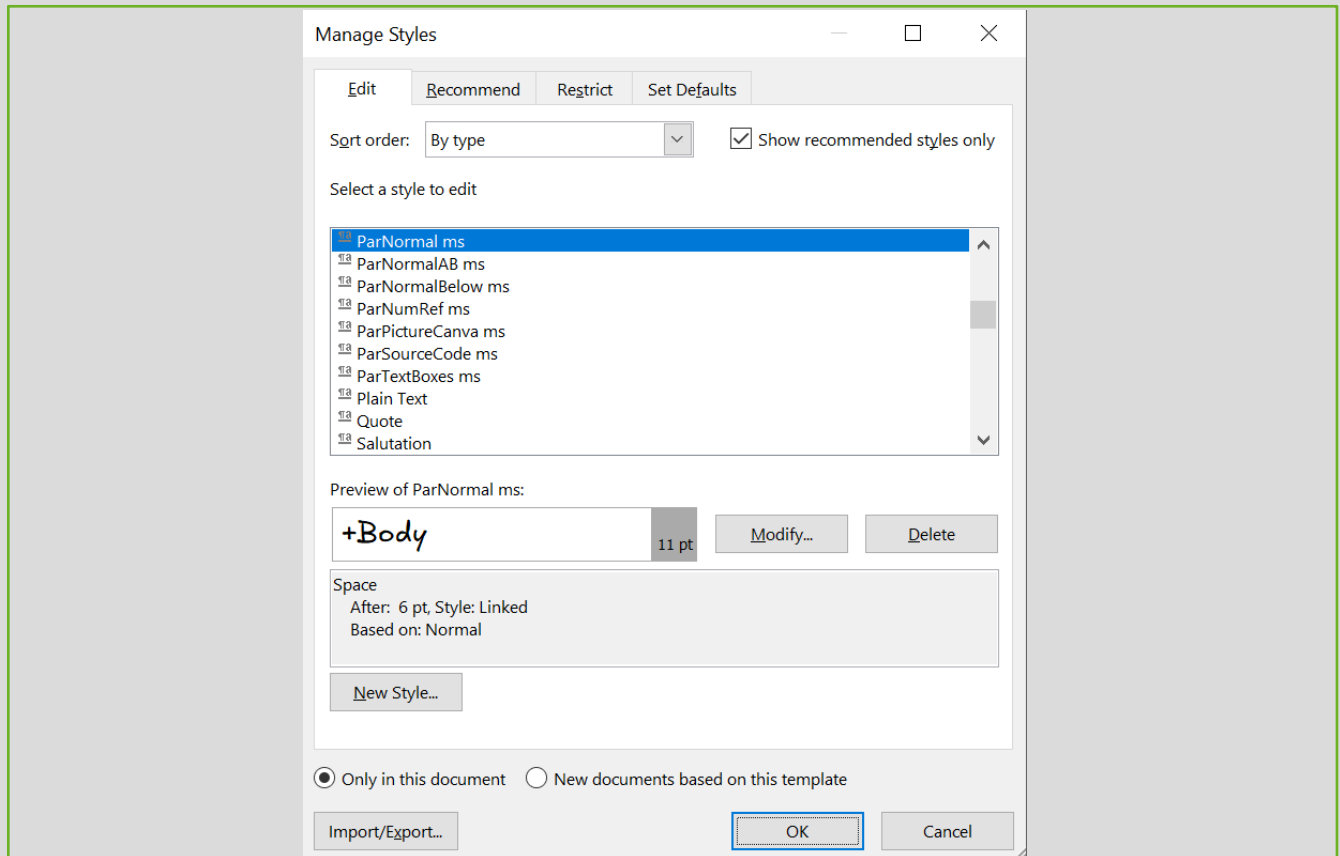
Conclusion: Most of the paragraph styles have common setting of hyphenation set to *True*.

13.6. Styles

My general rule is to use customized styles instead of the built-in, whenever possible.

In my version of Microsoft Word there are 373 built-in styles. So why to define customized styles? Answer to this question is: to get control over what is Microsoft Word doing with styles.

There is no way to delete built-in styles. You like or not, all 373 built-in styles could be found in your document. What is worse, there is no way to easily filter them out in *Manage Styles* window.



Pic. 168. *Manage Styles* window.

Some styles in Microsoft Word are linked with User Interface. For example, the tables of contents are based on *TOC1*... *TOC9* built-in styles. You use them and then you can benefit from fields, or you define your own styles, but then fields do not work anymore.

There are just few examples of built-in styles which I alter for my purposes:

- » *Normal*,
just to be sure that all customized styles have always the same base
- » *TOC1*, *TOC2*, *TOC3*
for purpose of list of content
- » *TOC8* and *TOC9*
for list of tables and list of pictures.

For existing documents, which were formatted without strict rules (e.g. direct formatting, outdated or not supervised styles) it is essential to me to quickly filter out my set of styles from others. The only way which I've found is named suffix 'ms'. Some of my macros are used to separate my styles from built-in styles. Then I can exchange some of the other styles with mine.

Naming convention of customized styles:

- » character type styles:
 - » prefix: 'Char'
 - » suffix: 'ms'
 - » example: *CharBold ms*
- » paragraph type styles:
 - » prefix: 'Par'
 - » suffix: 'ms'
 - » example: *ParNormal ms*
- » table type styles:
 - » prefix: 'Tab'
 - » suffix: 'ms'
 - » example: *TabTable ms*

The most important design rule is to keep as small number of customized styles as possible. Have I succeeded? To finish this book, I have designed in total about 50 styles. The majority, about 40, are paragraph styles. But only 3 styles dedicated to tables.

It was my conscious decision to keep styles definitions in VBA code. This is the only way to easily compare and scale up style definitions easily. Finally, I have a tool to share style definitions with others reliably, in text form.

In *Macros_ms.dotm* add-in style definitions are stored in the module *StylesM*.

13.7. Lists

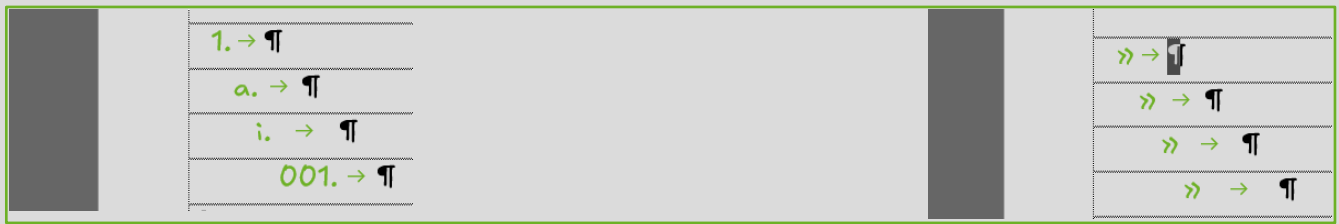
The most important Microsoft Word design rule applying to lists is: one paragraph style can be assigned just to one *ListTemplate*. For example, if you plan to have numbered list and bulleted list and both lists would use the same look and feel of paragraph styles, you need to double paragraph styles.

For example, I have *ListTemplate* dedicated for multilevel numbered list and for multilevel bulleted list. Both look almost identical (except list markers), but each of them must use different set of 4x paragraph styles.

- a. multilevel numbered list:
ParListIndent1 ms, *ParListIndent2 ms*, *ParListIndent3 ms*, *ParListIndent4 ms*

b. multilevel bulleted list:

ParListIndentB1 ms, ParListIndentB2 ms, ParListIndentB3 ms, ParListIndentB4 ms



Pic. 169. Two types of multilevel lists.

From VBA perspective the data model which is used to control lists are *ListTemplate*. They link indentation, enumeration and paragraph styles.

In *Macros_ms.dotm* add-in lists (*ListTemplate*) all 6× definitions are stored in the module *StylesM*.

13.8. Tables

From my perspective I need just 3× tables:

- » Table where header is colored, each second row is colored.
- » Plain table, without any frames.
- » Plain table, without any frames and without any internal margins in cells.

13.9. Security issues

My suggestion is to prepare yourself a list of questions to be answered on time of preparing a document stencil.

13.9.1. How do you inform user that content of this document is protected?

You can answer to this question putting in header or footer **confidentiality** level and add somewhere, e.g. in legal statement chapter, the explanation, how many levels there are and what do they mean.

Confidentiality levels, example:

Public document: This means that content of this document is intended for business partners, without prior written agreements. The English (-en) version of this document is the original version. All non-English editions of this document are translations of the original.

Internal document: This means that the content of this document is intended for individual business partners, based on individual agreements, e.g. non-disclosure agreements (NDA) or provisions between business partners (B2B). Not for public use without written permission. The English (-en) version of this document is the original version. All non-English editions of this document are translations of the original. This document is intended for the exclusive use of persons operating the system. Without written consent, this document must not be made available to third parties.

Confidential document: This means that the content of this document it is dedicated only to business partners, based on individual agreements, e.g. Non-Disclosure Agreement (NDA) or provisions between business partners (B2B). Not for public use without written permission and it should be specially protected against any unauthorized persons, in particular: copies should be left in locked places, mailing with this information should be encrypted and the list of persons with access to this information should be known. The English (-en)

version of this document is the original one. All non-English language editions of this document are translations of the original one. This document is intended for the exclusive use of persons operating the system. Without written agreement, this document must not be made available to third parties.

Suggestion: you can prepare building block for that purpose.

13.9.2. Which confidentiality level should be assigned to particular document?

Follow internal rules of your organization to answer this question.

13.9.3. In which form document should be sent to external party?

It depends a lot on other factors: do you plan your customer to prepare just a review, or work with provided content?

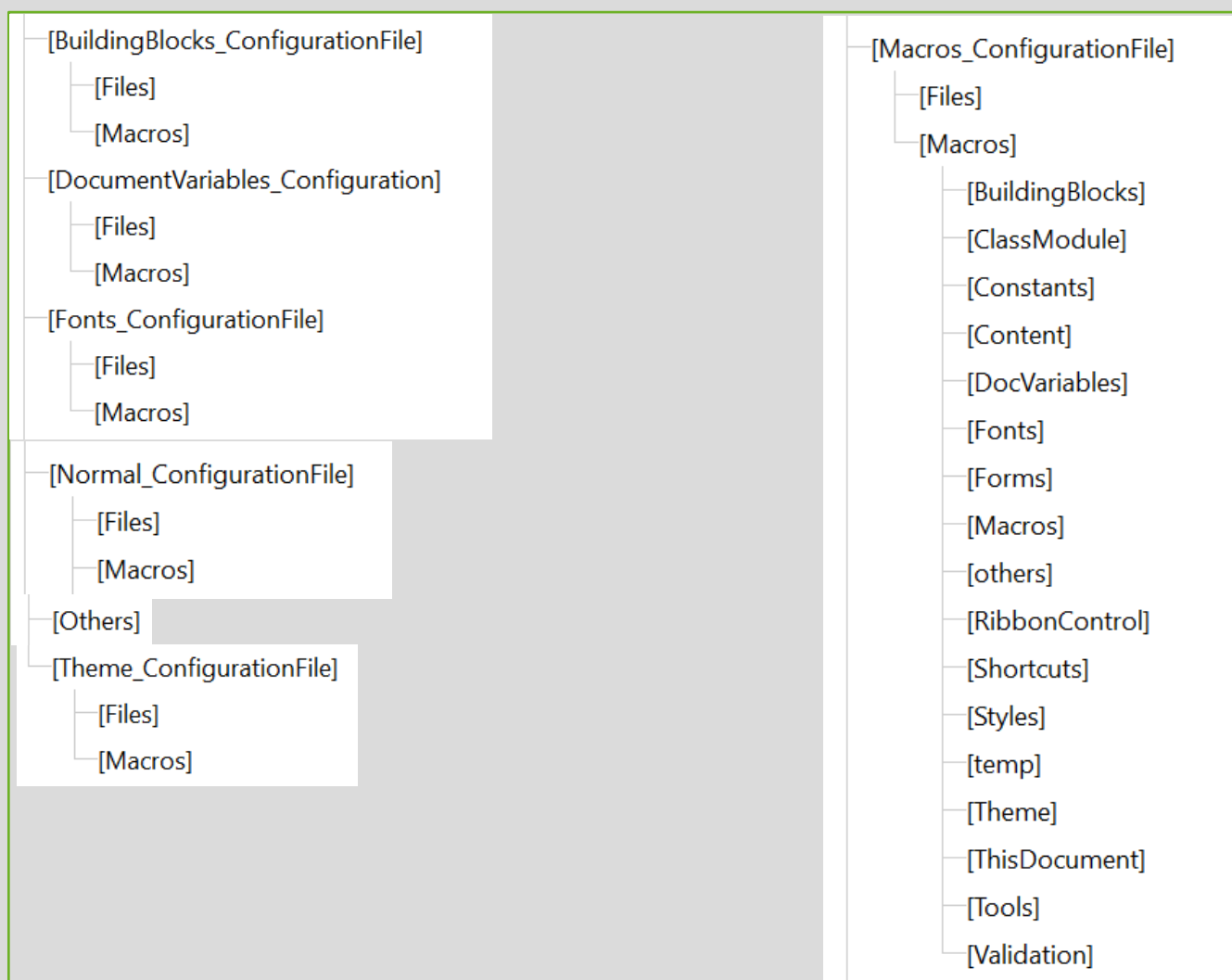
The most popular way is to prepare PDF, but even then, there are still plenty of variants. PDF file can be secured against copying of a content. Or it can be used just a content container, to keep it together in specific form.

If you wish to provide DOCX file for future editing, take precautions to be sure that content on remote computer will look the same as at yours. For example, consider embedding of specific fonts. The fonts are not embedded by default. Take into consideration if actually you can share fonts, as they can be copyright protected.

It can be even more subtle, if you wish your customer to run specific macros. Then consider sharing them as well.

14. This book GitHub repository

<https://github.com/mslonik/Microsoft-Word-Configuration> [15]



Pic. 170. Repository structure.

If you wish to download complete files, you can find them in [Files] branches. Please note, that sometimes in single folder there are more than just one file.

Macros source code is stored in [Macros_ConfigurationFile] > [Macros]. For majority of folders I've prepared dedicated *ReadMe* files.

You can easily find in Internet manuals how to download GitHub repository to your local computer.

15. How to search for official support?

Official method for official support are described in the following chapters.

15.1. Official Microsoft online help

<https://support.microsoft.com/en-US/word>

15.2. Official Microsoft forum

<https://answers.microsoft.com/en-us/msoffice/forum>

Example of challenges, where I was involved:

[How to run automatically Document_New macro with /t command line - Microsoft Community](#)

15.3. How to issue a problem?

<https://feedbackportal.microsoft.com/feedback>

15.4. Official releases of Microsoft Word and known bugs

[Fixes or workarounds for recent issues in Word for Windows - Microsoft Support](#)

[Update history for Microsoft 365 Apps \(listed by date\) - Office release notes | Microsoft Learn](#)

16. Web pages dedicated to Microsoft Word

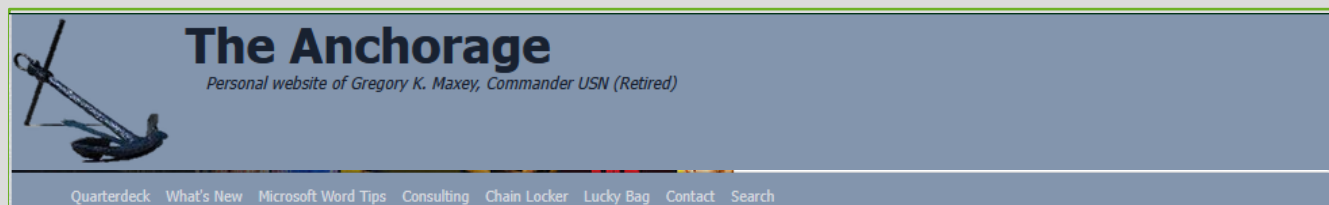
Below is my personal list of the best web pages dedicated to Microsoft Word. Order of appearance matters. These are hidden gems, which over the years helped me to dive in into nuances of Microsoft Word.

16.1. Short URL list

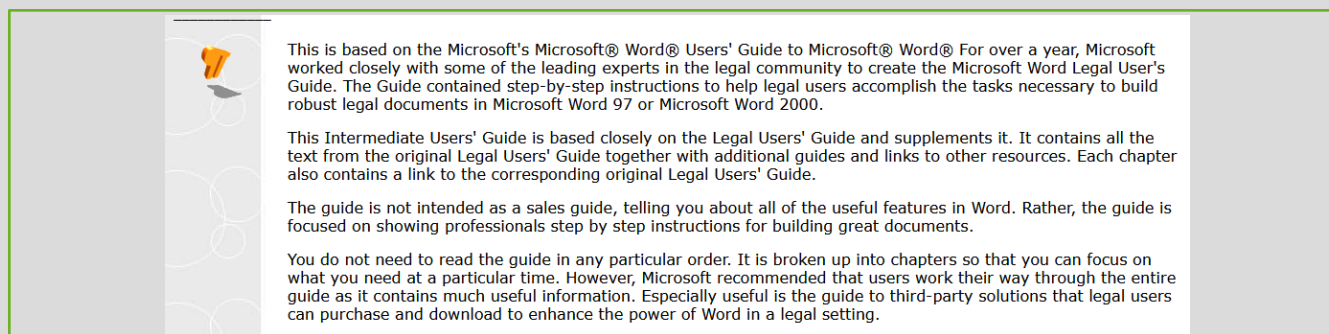
- » https://gregmaxey.com/word_tips.html
- » <https://addbalance.com/usersguide/>
- » <https://wordribbon.tips.net/index.html>
- » <https://bettersolutions.com/word/styles/index.htm>
- » <https://www.clausebase.com/msword/numbering>
- » <https://wordmvp.com/>

16.2. List with illustrations

The following pictures show headers of web pages on time of writing this book. All of them work as Internet links upon clicking.



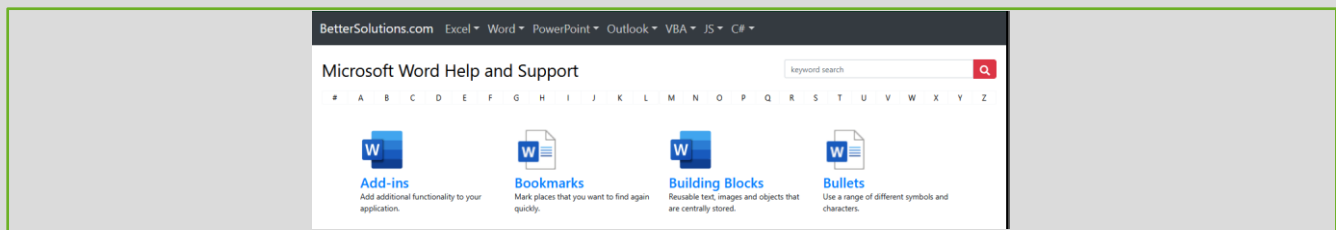
Pic. 171. [The Anchorage](https://gregmaxey.com/word_tips.html), personal website of Gregory K. Maxey.



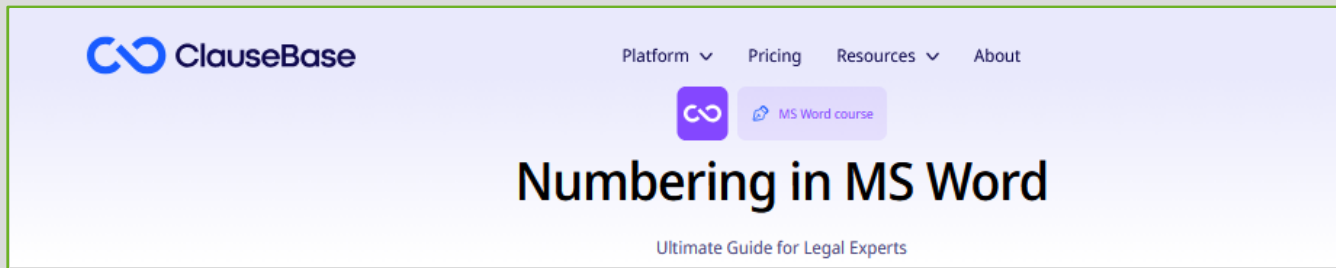
Pic. 172. [Add balance](https://addbalance.com/usersguide/).



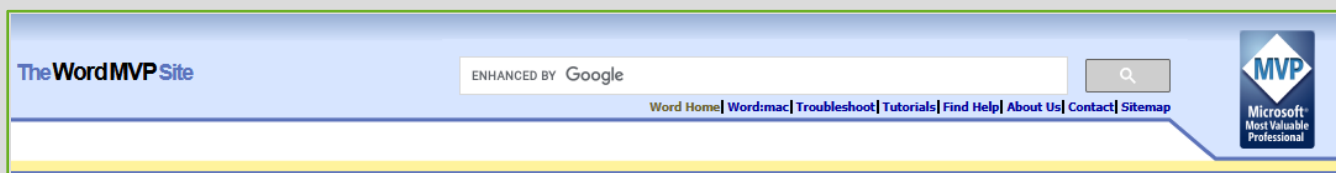
Pic. 173. [Word ribbon tips](https://wordribbon.tips.net/index.html).



Pic. 174. Better solutions.



Pic. 175. ClauseBase, Numbering in MS Word.



Pic. 176. The Word MVP Site.

17. References

The chapter shows a list of documents referred to or quoted in the content of this document. They are numbered continuously. Whenever a position from this list is referred to, a short title in human readable form and italics should be inserted, followed by the consecutive number in square brackets which can be used as internal hyperlink to a full document title, e.g. *Short Document Name* [hyperlink_number].

The documents are divided into two categories: the first are external documents, e.g. technical standards, legal documents, and the second is manufacturer documentation, delivered by manufacturer, e.g. user documentation, technical documentation.

- [1]. MIT (Massachusetts Institute of Technology) license
https://en.wikipedia.org/wiki/MIT_License
- [2]. Screen shot comes from Computer History Archives Project ("CHAP")
<https://youtu.be/P2mjSUR3QLo?si=1Fz6Z2ULTqXGsl-I>
- [3]. Homoglyphs, Wikipedia
<https://en.wikipedia.org/wiki/Homoglyph>
- [4]. Hyphenation / syllabification, Wikipedia
<https://en.wikipedia.org/wiki/Syllabification>
- [5]. Word Processors Are Evil And Should Not Exist!
<https://youtu.be/pOnCEGTlTqM?si=F9DMyYZLGYN2GtIS>
- [6]. Plane cockpit, Wikipedia
https://en.wikipedia.org/wiki/Glass_cockpit
- [7]. Macro keyboard made by author of this book.
<https://mslonik.pl/zycie/blog/1094-klawiatura-do-makr>
- [8]. Office RibbonX Editor
<https://github.com/fernandreu/office-ribbonx-editor/releases>
- [9]. Microsoft Office 2016/365 - imageMso Gallery
https://codekabinett.com/download/Microsoft-Office-2016_365-imageMso-Gallery.pdf
- [10]. Excalidraw project home page
<https://excalidraw.com/>
- [11]. Customize the Word Ribbon User Interface (It Doesn't Take Rocket Science) by Greg Maxey
https://gregmaxey.com/word_tip_pages/customize_ribbon_main.html
- [12]. AutoHotkey
<https://www.autohotkey.com/>
- [13]. DITA, Wikipedia
https://en.wikipedia.org/wiki/Darwin_Information_Typing_Architecture
- [14]. CCMS, Wikipedia
https://en.wikipedia.org/wiki/Component_content_management_system
- [15]. This book GitHub repository
<https://github.com/mslonik/Microsoft-Word-Configuration/tree/main>
- [16]. Office Open XML, Wikipedia
https://en.wikipedia.org/wiki/Office_Open_XML
- [17]. International standard paper sizes, Wikipedia
https://en.wikipedia.org/wiki/International_standard_paper_sizes#A_series
- [18]. Display specification
<https://www.displayspecifications.com/en/model/f79ee0e>

[19]. Customizing the 2007 Office Fluent Ribbon for Developers (Part 1 of 3)

[https://learn.microsoft.com/en-us/previous-versions/office/developer/office-2007/aa338202\(v=office.12\)?redirectedfrom=MSDN](https://learn.microsoft.com/en-us/previous-versions/office/developer/office-2007/aa338202(v=office.12)?redirectedfrom=MSDN)

18. Epilog

Microsoft Word is far from being perfect tool for authoring. But it is de facto standard in many offices throughout the world. Instead of thinking of it as an enemy, it is better to harness it. Of course, as usual, the question is, how much time do you need, or how much time spending is justified, to be comfortable with text authoring tool? And as always, the easiest answer to this question is, it depends.

For example, it depends on your needs. There are ideas such as DITA (Darwin Information Typing Architecture) [13] or CCMS (Component Content Management System) [14], which are far beyond context of this book. Microsoft Word itself, without expansions, is not enough for these purposes. My thought here is, that I miss very much easy way to conditionally format my files content in Microsoft Word.

In PDF files you can embed even animations. From the other hand, it is not possible to collapse headers in PDF files. On purpose I did not make comparison between different content authoring tools, as this book is dedicated to Microsoft Word.

It took me years to prepare VBA code, which is granted to you in GitHub repository. Hopefully, you will find some inspiration, or even help in your everyday work. That was my intention on time of writing this book.

By far this work is neither complete nor finished. For me this project is finished, so I release it.

No one has yet learned to sail on calm waters. Good luck!

Maciej Słojewski, Gdańsk, 2026-03-15

List of Pictures

Pic. 1. Example of small dialog box.....	10
Pic. 2. Microsoft Windows: <i>Control Panel > System > About > Windows specifications</i>	11
Pic. 3. Microsoft Word: <i>File > Account > About Word</i>	11
Pic. 4. Files necessary to be downloaded.....	13
Pic. 5. New menus.....	13
Pic. 6. Set up of add-ins.....	13
Pic. 7. Task Manager and Microsoft Word.....	15
Pic. 8. <i>Normal.dotm</i> , information categories.....	15
Pic. 9. VBE with imported all source files.....	16
Pic. 10. VBE, enlarged structure of <i>Macros_ms</i>	17
Pic. 11. <i>Macros_ms.dotm</i> , information categories.....	17
Pic. 12. <i>BB_ms.dotx</i> , information categories.....	18
Pic. 13. Hardware oriented word processor devices [2].....	19
Pic. 14. Microsoft Word application setting example, default resolution of images.....	21
Pic. 15. Microsoft Word configuration files.....	22
Pic. 16. Folders where Microsoft Word configuration is stored.....	24
Pic. 17. Document Template button.....	24
Pic. 18. Templates and add-ins ver. folder A and folder B.....	25
Pic. 19. Location of theme file.....	26
Pic. 20. Workgroup templates.....	27
Pic. 21. Access to workgroup templates.....	27
Pic. 22. <i>File > Option > Save</i>	28
Pic. 23. Find OneDrive icon in SystemTray.....	29
Pic. 24. Select settings (symbol).....	29
Pic. 25. Select settings again (symbol).....	30
Pic. 26. Select Manage backup button.....	30
Pic. 27. Consider switching off.....	31
Pic. 28. DOTM or DOTX as containers.....	32
Pic. 29. Configuration categories in DOTM file.....	33
Pic. 30. Example.....	34
Pic. 31. Attachment relationship.....	34
Pic. 32. Attachment: <i>Document1</i> has access to <i>Normal.dotm</i> containers.....	35
Pic. 33. Tick <i>Automatically update document styles</i>	36
Pic. 34. Attachment: <i>Document1</i> has got styles from <i>MyCompanyTemplateFile.dotm</i>	36
Pic. 35. Manage Styles button, <i>Set Defaults</i> tab.....	38
Pic. 36. Recommended <i>Set Defaults</i>	39
Pic. 37. My setup.....	40
Pic. 38. Attachment: <i>Document1</i> has got styles and content from <i>MyCompanyTemplateFile.dotm</i>	41
Pic. 39. <i>Building Blocks Organizer</i> window.....	42
Pic. 40. Modularization of building blocks (BB files).....	43
Pic. 41. <i>BuildingBlocks_ms</i> buttons.....	44
Pic. 42. Settings of building block related files.....	44
Pic. 43. Empty <i>Building Block Organizer</i>	44
Pic. 44. Theme features.....	45
Pic. 45. Settings of font related to concept of Theme.....	46
Pic. 46. Colors of a table.....	47
Pic. 47. Example color selection.....	47
Pic. 48. Microsoft Word User Interface customization levels.....	48
Pic. 49. Customize Ribbon.....	48
Pic. 50. Font substitution.....	50
Pic. 51. Macro <i>ShowSubstitutedFonts</i>	50
Pic. 52. Information categories, content file.....	52
Pic. 53. Before running macros from <i>Macros_ms.dotm</i>	53

Pic. 54. After running macros from <i>Macros_ms.dotm</i>	54
Pic. 55. Text strikethrough.....	56
Pic. 56. Airplane cockpit [6].....	57
Pic. 57. Microsoft Word oriented macro keyboard made by author of this book [7].	57
Pic. 58. Style hotkeys, Alt + H, K.....	59
Pic. 59. Macro hotkeys.....	60
Pic. 60. Building blocks <i>AutoText</i> or <i>Hotstrings</i>	60
Pic. 61. Step 1: customized keyboard shortcuts.....	61
Pic. 62. Step 2: customized styles.....	62
Pic. 63. Step 3: apply theme.....	62
Pic. 64. Step 4: customization of Microsoft Word options.....	62
Pic. 65. Step 5: page layout.....	63
Pic. 66. Step 6: custom Document Properties.....	63
Pic. 67. Step 7: Captions.	64
Pic. 68. Step 8: content hyphenation.....	64
Pic. 69. Step 9: Insertion of content.....	64
Pic. 70. Step 10: Customized color of document background.....	65
Pic. 71. Step 1 (left), step 2 (right).....	65
Pic. 72. Step 3 (left), step 4 (right).....	66
Pic. 73. Applied view mode no. 2.....	66
Pic. 74. Some generated text.....	67
Pic. 75. Heading 1.....	68
Pic. 76. Heading 3.	68
Pic. 77. Heading 2.....	69
Pic. 78. Numbered list.	70
Pic. 79. Change of the numbered list level.....	70
Pic. 80. Restore default paragraph style.....	71
Pic. 81. Added next numbered list.	71
Pic. 82. Reset of numbering.	72
Pic. 83. The first text paragraph after list.	73
Pic. 84. Move entire paragraph, one after another.....	73
Pic. 85. Write the word 'picture'.....	74
Pic. 86. The inserted picture building block.	74
Pic. 87. Write the word 'table'.	75
Pic. 88. The table building block.....	75
Pic. 89. List of pictures.....	75
Pic. 90. List of tables.....	76
Pic. 91. Applied style <i>ParMinimal ms</i>	76
Pic. 92. Added <i>HeaderOrdinary</i> and <i>FooterOrdinary</i>	77
Pic. 93. Newly crated table.....	77
Pic. 94. Style <i>TabTable ms</i> applied to the example table.	78
Pic. 95. Added text.	78
Pic. 96. Style <i>ParInTable ms</i> applied to the example table.	78
Pic. 97. Style <i>ParListInTable ms</i> applied to the numbered cells.	78
Pic. 98. Style <i>TabTable ms</i> + <i>Customize Formatting</i> macro.....	79
Pic. 99. The first text paragraph after table.....	79
Pic. 100. Added customized document properties.	80
Pic. 101. Input user document properties.	80
Pic. 102. Updated fields.....	81
Pic. 103. Microsoft Word User Interface to custom properties.	81
Pic. 104. Show <i>DocVariables</i>	82
Pic. 105. Embed fonts in the file.....	82
Pic. 106. Save as question.....	82
Pic. 107. Fonts applied in example file.	83
Pic. 108. Menu <i>BuildingBlocks_ms</i> > <i>Edit</i>	84

Pic. 109. AutoText and its expansion.....	85
Pic. 110. Cross-reference window.....	86
Pic. 111. Default Save As dialog box, Options dialog box.	87
Pic. 112. Customized Save As (F12).	87
Pic. 113. Predefined set of parameters.....	88
Pic. 114. Default set of printing options.....	88
Pic. 115. 24" Display and 2x A4 blank pages, side to side.....	91
Pic. 116. 24" Display and 2x A4 blank pages, moved to the left.....	91
Pic. 117. Example of <i>status bar</i>	92
Pic. 118. Example, information displayed in the left part of <i>status bar</i> by macro.....	92
Pic. 119. My settings of <i>status bar</i>	94
Pic. 120. Both panes opened and docked to left and right edge of parent window.	95
Pic. 121. OfficeRibbonXEditor and <i>Macros.ms.dotm</i>	96
Pic. 122. Relationship between experience / competence / time and content authoring.	97
Pic. 123. Microsoft Word User Interface customization levels.....	98
Pic. 124. Change project name, <i>Project</i> (left) → <i>Macros.ms</i> (right).	99
Pic. 125. Debug > Compile function.	100
Pic. 126. Structure of <i>Macros.ms.dotm</i>	100
Pic. 127. User Interface, <i>Macros</i> , Alt + F8.....	103
Pic. 128. Menu: <i>Macros.ms</i>	103
Pic. 129. Screen tip macro example.	104
Pic. 130. Menu: <i>Tools.ms</i>	104
Pic. 131. Menu: <i>BuildingBlocks.ms</i>	104
Pic. 132. Menu: <i>Shortcuts.ms</i>	105
Pic. 133. Menu: <i>Styles.ms</i>	105
Pic. 134. Menu: <i>Styles.ms</i> , built-in Microsoft Word functions.	105
Pic. 135. Menu: <i>Validation</i>	106
Pic. 136. Menu: <i>Content.ms</i>	106
Pic. 137. Shape of our future page overview.	109
Pic. 138. Borders and Shading.....	109
Pic. 139. Border and Shading Options.....	110
Pic. 140. Top-left document corner, crop mark disabled (left) and enabled (right).	110
Pic. 141. Crop marks setting.....	111
Pic. 142. Colored background of header and footer area.	111
Pic. 143. See small red rectangle around selected color.	112
Pic. 144. Expected result.....	112
Pic. 145. New values of margins.	113
Pic. 146. Margins values.	113
Pic. 147. Header and footer values.	114
Pic. 148. Size of header > size of top margin.	114
Pic. 149. Expected result of header.....	115
Pic. 150. Sizes of header and footer.....	115
Pic. 151. Expected result.....	116
Pic. 152. Set negative left indentation.	117
Pic. 153. Header paragraph with negative indentation.	117
Pic. 154. Table in the header.....	117
Pic. 155. Reduced size of font in the empty paragraph below the table.	118
Pic. 156. Width of header table.	118
Pic. 157. New table width.....	118
Pic. 158. Reduced margins of the table.....	119
Pic. 159. Changed line spacing of table cells.....	119
Pic. 160. Table bottom line set to green.....	120
Pic. 161. Bottom line of header table.	120
Pic. 162. Header layout, enabled visibility of unprinted characters.....	121
Pic. 163. Example.....	121

Pic. 164. Page number on mirrored footers, opposite to gutter, marked in red.	122
Pic. 165. Left: header linked to previous section, right: unlinked.	123
Pic. 166. Paragraph kine spacing options.	125
Pic. 167. Cut down canvas with <i>Exactly</i> line spacing.	126
Pic. 168. <i>Manage Styles</i> window.	127
Pic. 169. Two types of multilevel lists.	129
Pic. 170. Repository structure.	131
Pic. 171. The Anchorage, personal website of Gregory K. Maxey.	133
Pic. 172. Add balance.	133
Pic. 173. Word ribbon tips.	133
Pic. 174. Better solutions.	134
Pic. 175. ClauseBase, Numbering in MS Word.	134
Pic. 176. The Word MVP Site.	134

List of Tables

Tab. 1. First tested setup.	10
Tab. 2. Second tested setup.	11
Tab. 3. The location of configuration files.	14
Tab. 4. The default location of configuration files.	23
Tab. 5. Theme_ms.thmx colors.	46
Tab. 6. Keyboard shortcuts advantages and disadvantages.	58
Tab. 7. Built-in keyboard shortcuts.	67
Tab. 8. Selected Microsoft Word panes.	95
Tab. 9. Macro customized keyboard shortcuts.	106

THIS PAGE IS LEFT INTENTIONALLY BLANK

THIS PAGE IS LEFT INTENTIONALLY BLANK

xyz@123.com

Patreon 👍: <https://www.patreon.com/user?u=18185391>
GitHub 📄: <https://github.com/mslonik>
YouTube 📺: <https://www.youtube.com/@mslonik>
Web 🌐: <https://mslonik.pl/>
LinkedIn 💡: <https://www.linkedin.com/in/maciejslojewski/>